

Automa

Documentacion completa del sistema

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de los 20 Markdown de docs/system-documentation/ — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Documentación del sistema — Automa

Portada e índice navegable de la documentación técnica, funcional, arquitectónica y operativa del repositorio `automa-pc`.

1. Identificación

Campo	Valor
Sistema	Automa
Nombre del paquete	<code>automa-pc</code> (<code>pyproject.toml</code> , <code>[project].name</code>)
Versión analizada	<code>0.3.0</code> (<code>pyproject.toml</code> , <code>[project].version</code>)
Commit analizado	<code>ff246abb4d6f5235afb4dcc74032a50645db954c</code> (rama <code>main</code>)
Fecha del análisis	2026-08-27
Lenguaje principal	Python 3.10+ (CI corre 3.10, 3.11 y 3.12)
Plataforma objetivo	Windows 10 / 11. La CI también corre <code>ubuntu-latest</code>
Licencia	MIT (<code>LICENSE</code>)
Repositorio	<code>https://github.com/vladimiracunadev-create/automa-pc</code>
Sitio del proyecto	<code>https://vladimiracunadev-create.github.io/automa-pc/</code>

Tamaño medido en el commit analizado (`git ls-files`, `wc -l`):

Métrica	Valor
Archivos versionados	210
Archivos <code>.py</code> de producción (sin <code>tests/</code>)	47
Archivos <code>.py</code> de pruebas	17
Líneas de Python de producción	6 126
Líneas de Python de pruebas	1 525
Flows declarativos (<code>flows/*/manifest.json</code>)	27
Acciones registradas (<code>engine/action_registry.py</code>)	36
Pruebas que pasan (<code>pytest</code>)	150

2. Descripción breve

Automa es un **orquestador local de automatización (RPA) para el escritorio Windows**, escrito en Python puro. Cada caso de uso se declara en un `manifest.json` con pasos, condiciones y transiciones; un motor determinista los ejecuta, aplica una política de sandbox por flow y deja trazabilidad completa en SQLite, JSONL y archivos de salida.

No hay ningún modelo de lenguaje en el camino de ejecución. El **"agente" es el manifest declarativo, no un LLM**. El repositorio contiene un adaptador opcional de visión multimodal (`plugins/analyzers/vision_model_analyzer.py`) que ningún flow del catálogo usa; el detalle y la evidencia de esa afirmación están en [09 · APIs e integraciones](#).

3. Propósito de esta documentación

Permitir que:

- 1. Un desarrollador nuevo se incorpore al proyecto sin depender de conocimiento tácito.
- 2. Una persona no técnica entienda qué hace el sistema y para qué sirve.
- 3. Un desarrollador experimentado consulte detalles de acciones, motor y persistencia.
- 4. Un auditor revise arquitectura, base de datos, dependencias, seguridad y deuda técnica.
- 5. Otro agente de IA use estos documentos como contexto verificable del repositorio.

4. Público destinatario por documento

Documento	Usuario final	Dev nuevo	Dev senior	Auditor	Dirección
01 · Descripción general	[OK]	[OK]	o	o	[OK]
02 · Instalación y ejecución	[OK]	[OK]	[OK]	o	o
03 · Arquitectura	o	[OK]	[OK]	[OK]	o
04 · Mapa del código	o	[OK]	[OK]	[OK]	o
05 · Referencia técnica	o	o	[OK]	[OK]	o
06 · Explicación profunda	o	[OK]	[OK]	[OK]	o
07 · Base de datos	o	[OK]	[OK]	[OK]	o
08 · Flujo de datos	o	[OK]	[OK]	[OK]	o
09 · APIs e integraciones	o	[OK]	[OK]	[OK]	o
10 · Configuración	[OK]	[OK]	[OK]	[OK]	o
11 · Seguridad	o	o	[OK]	[OK]	[OK]
12 · Pruebas y calidad	o	[OK]	[OK]	[OK]	o
13 · Despliegue y operación	o	o	[OK]	[OK]	o
14 · Solución de problemas	[OK]	[OK]	[OK]	o	o
15 · Riesgos y deuda técnica	o	o	[OK]	[OK]	[OK]
16 · Glosario	[OK]	[OK]	o	[OK]	[OK]
17 · Resumen ejecutivo	[OK]	o	o	[OK]	[OK]
18 · Guía para nuevo desarrollador	o	[OK]	o	o	o

19 · Matriz de trazabilidad	o	[OK]	[OK]	[OK]	o
-----------------------------	---	------	------	------	---

[OK] destinatario principal · o lectura opcional

5. Tabla de contenidos

#	Documento	Contenido	Estado
—	README	Portada, índice y convenciones	[OK] Completo
01	Descripción general del sistema	Qué es, qué resuelve, casos de uso, explicación no técnica	[OK] Completo
02	Instalación y ejecución	Requisitos, instalación, ejecución, pruebas, errores frecuentes	[OK] Completo
03	Arquitectura	Estilo, capas, patrones, diagramas Mermaid	[OK] Completo
04	Mapa completo del código	Inventario jerárquico de directorios, módulos y funciones	[OK] Completo
05	Referencia técnica	Catálogo de acciones, funciones, endpoints, comandos y errores	[OK] Completo
06	Explicación profunda del código	Flujo interno módulo a módulo, decisiones y casos límite	[OK] Completo
07	Base de datos	Esquema SQLite, diccionario de datos, consultas, ERD	[OK] Completo
08	Flujo de datos	Origen, validación, transformación, almacenamiento, consumo	[OK] Completo
09	APIs e integraciones	Endpoints HTTP, webhooks, Playwright, OCR, IA opcional	[OK] Completo
10	Configuración	<code>configs/</code> , <code>context.example.json</code> , variables de entorno	[OK] Completo
11	Seguridad	Sandbox, auth del panel, superficie de ataque, controles ausentes	[OK] Completo
12	Pruebas y calidad	150 tests, cobertura medida, huecos priorizados	[OK] Completo
13	Despliegue y operación	CI/CD, PyInstaller, Inno Setup, logs, respaldo y rollback	[OK] Completo
14	Solución de problemas	Síntoma → causa → diagnóstico → solución → riesgo	[OK] Completo
15	Riesgos y deuda técnica	Hallazgos clasificados por severidad e impacto	[OK] Completo
16	Glosario	Términos técnicos y de dominio en lenguaje claro	[OK] Completo

17	Resumen ejecutivo	Presentación del sistema para decisión	[OK] Completo
18	Guía para un nuevo desarrollador	Itinerario de incorporación y primeras tareas	[OK] Completo
19	Matriz de trazabilidad	Funcionalidad → módulo → función → persistencia → prueba	[OK] Completo

Recursos adicionales

- [assets/](#) — recursos gráficos de esta documentación. Actualmente vacío: los diagramas se declaran como código Mermaid dentro de los propios `.md` y se rasterizan al generar el PDF. Las capturas del producto viven en [docs/manual_screenshots/](#) y la portada en [docs/assets/](#).
- [pdf/](#) — versión PDF de cada documento, más un consolidado, generados por script.

6. Cómo generar los PDF

Los Markdown de esta carpeta son la **fente única**. Los PDF se generan a partir de ellos:

```
python scripts/build_docs_pdf.py          # todos los documentos + consolidado
python scripts/build_docs_pdf.py --only 03 # solo el documento 03 (iteración rápida)
python scripts/build_docs_pdf.py --check   # comprueba dependencias sin generar
```

Requisitos, opciones y limitaciones conocidas en [13 · Despliegue y operación → Generación de la documentación en PDF](#).

7. Documentación añadida en el propio código

Este análisis revisó los 47 archivos `.py` de producción. **El repositorio ya tiene una densidad alta de comentarios de calidad**: los módulos con decisiones no obvias ([engine/paths.py](#), [engine/sandbox.py](#), [engine/secrets.py](#), [engine/cron.py](#), [engine/template.py](#), [engine/introspection.py](#), [actions/browser_extract.py](#), [actions/notify.py](#), [actions/system.py](#), [app/server.py](#), [installer/automa_entry.py](#)) llevan docstring de módulo explicando **por qué** existe el código, no qué hace la línea siguiente. Los bloques de seguridad de [app/server.py](#) documentan incluso el modelo de amenaza que cierran.

Siguiendo el criterio de no degradar un código bien comentado con ruido, **no se modificó ningún archivo de código fuente en este análisis**. Los archivos sin docstring de módulo ([engine/models.py](#), [engine/loader.py](#), [engine/conditions.py](#), [actions/filesystem.py](#), [actions/rules.py](#), [actions/ui.py](#), [actions/screen.py](#), [actions/http_actions.py](#)) son módulos cortos, con nombres explícitos y tipado completo; su explicación se aporta en [06 · Explicación profunda](#) en lugar de duplicarse en el código. La recomendación de añadirles docstring queda registrada como deuda de baja severidad en [15 · Riesgos y deuda técnica](#).

El único archivo Python nuevo que introduce este trabajo es [scripts/build_docs_pdf.py](#), la herramienta que genera los PDF. No participa en el runtime del producto.

8. Relación con la documentación previa del repositorio

Este conjunto **no reemplaza** la documentación que ya existía en [docs/](#); la complementa y la referencia. La documentación previa está orientada a producto, operación y contrato de extensión; esta está orientada a comprensión del sistema y auditoría.

Ya existía	Este conjunto aporta
docs/ARQUITECTURA.md — diseño técnico resumido	Arquitectura con diagramas, capas y flujos completos (03)
docs/CREAR_FLUJOS.md — contrato para escribir un flow	Explicación del motor que consume ese contrato, línea a línea (06)
docs/FAMILIAS_Y_CASOS.md — catálogo	Inventario verificado de los 27 flows con su política real (04, 19)
docs/SEGURIDAD.md — sandbox y modelo de confianza	Controles presentes y ausentes , medidos flow por flow (11)

<code>docs/VALIDACION.md</code> — schema, pytest, CI	Cobertura medida y huecos priorizados (12)
<code>docs/OPERACION.md</code> y <code>RUNBOOK.md</code> — día a día	Despliegue, CI/CD y procedimientos de recuperación (13)
<code>docs/TROUBLESHOOTING.md</code> — fallas comunes	Guía síntoma → causa → diagnóstico → solución → riesgo (14)
<code>docs/ROADMAP.md</code> — hoja de ruta	Estado verificado del código en el commit analizado (01, 17)

9. Convenciones utilizadas

9.1 Marcadores de confianza

Toda afirmación de estos documentos está anclada a evidencia del repositorio. Cuando algo no se pudo comprobar, se marca explícitamente:

Marcador	Significado
<i>(sin marcador)</i>	Hecho verificado: leído directamente en el código, la configuración o el historial del repositorio. Se cita archivo y, cuando aplica, símbolo o comando ejecutado.
INFERENCIA	Conclusión razonada a partir del código, no una afirmación literal del repositorio.
REQUIERE VALIDACIÓN	Depende de ejecución en Windows real, de servicios externos o de una decisión humana pendiente.
NO DOCUMENTADO EN EL REPOSITORIO	El repositorio no contiene la información.
NO IDENTIFICADO	Se buscó y no se encontró evidencia en ningún sentido.

9.2 Referencias a código

- Los archivos se citan con ruta relativa a la raíz del repositorio: `engine/orchestrator.py`.
- Los símbolos se citan con su nombre real en el código, sin traducir: `Orchestrator.run`, `SandboxPolicy.assert_paths_allowed`, `_BUILT_IN_ACTIONS`.
- Los nombres de columnas SQL, claves JSON de manifest y variables de entorno se citan en su forma original (`flow_folder`, `allowed_paths`, `AUTOMA_PANEL_TOKEN`).
- Las líneas citadas corresponden al commit analizado y pueden desplazarse en commits posteriores; el nombre del símbolo es la referencia estable.

9.3 Idioma

Documentación en español, igual que el resto del repositorio y que los comentarios del código. Los identificadores de código, los nombres de acción (`browser.extract_content`) y los nombres de columnas SQL se mantienen en su forma original para garantizar trazabilidad textual.

9.4 Datos sensibles

Ningún documento contiene credenciales, tokens ni claves reales. Los valores de ejemplo son ficticios y están marcados como tales; las URL de ejemplo usan dominios reservados (`example.com`). Los hallazgos de seguridad se describen sin publicar cadenas explotables.

10. Elementos pendientes de validar

Lista viva de lo que esta documentación **no** pudo verificar en este análisis. El detalle y la recomendación de cada punto están en [15 · Riesgos y deuda técnica](#).

#	Pendiente	Motivo	Documento
1	Comportamiento real de las acciones de UI (<code>ui.hotkey</code> , <code>ui.type_text</code> , <code>ui.click</code>) y de captura (<code>screen.*</code>)	Requieren una sesión Windows interactiva con escritorio gráfico. No se ejecutó ningún flow que mueva teclado o ratón durante el análisis, por ser una acción con efecto sobre la máquina del usuario	05, 06
2	Flows 02, 07 y 21–27 de punta a punta	Requieren <code>playwright</code> y el navegador Chromium descargado (<code>python -m playwright install chromium</code>), que no forman parte de las dependencias declaradas en <code>pyproject.toml</code>	02, 12
3	Flows 12 y 17 (OCR)	Requieren el binario externo <code>tesseract</code> , que el repositorio no instala. <code>OCRImageAnalyzer</code> degrada con <code>status: "unavailable"</code> en vez de fallar	09, 14
4	Funcionamiento del binario empaquetado (<code>Automa.exe</code>) y del instalador	Requiere PyInstaller + Inno Setup en Windows. No se compiló en este análisis. Hay un hallazgo abierto sobre <code>installer/automa.spec</code> (ver #5)	13, 15
5	Si los flows 21–27 funcionan dentro del binario empaquetado	<code>installer/automa.spec</code> NO lista <code>actions.browser_extract</code> en <code>hiddenimports</code> , y el registro lo carga por <code>import_module</code> dinámico. INFERENCIA: PyInstaller no lo detectaría y esos siete flows fallarían con <code>ModuleNotFoundError</code> en el <code>.exe</code> . Sin build no se puede confirmar	13, 15
6	Vulnerabilidades conocidas de las dependencias	No se ejecutó <code>pip-audit</code> ni consulta a bases de CVE en este análisis. La CI sí lo hace (<code>security.yml</code>)	11
7	Publicación efectiva del release y del sitio GitHub Pages	Depende de tags y de la configuración del repositorio en GitHub, fuera del árbol de trabajo	13

Documentación generada por análisis estático y ejecución de las pruebas del repositorio en el commit `ff246ab`. Si el código cambia, actualizar primero los Markdown de esta carpeta y regenerar los PDF con `python scripts/build_docs_pdf.py`.

01 · Descripción general del sistema

Qué es Automa, qué problema resuelve, quién lo usa, qué entra y qué sale. Al final hay una sección escrita para alguien sin formación técnica.

1. Qué es

Automa es un orquestador local de automatización (RPA) para el escritorio Windows.

Cada tarea automatizable se describe en un archivo JSON llamado *manifest* (`flows/<carpeta>/manifest.json`). Ese archivo declara una lista de pasos, cada paso invoca una **acción** con nombre (`screen.capture_screenshot`, `browser.fill_form`, `system.run_powershell...`), y opcionalmente declara condiciones, reintentos y transiciones entre pasos. Un motor determinista (`engine/orchestrator.py`) lee el manifest, aplica una política de seguridad, ejecuta los pasos en orden y persiste todo lo que ocurrió.

Tres propiedades definen el producto y conviene protegerlas:

- 1. **Local-first.** Todo corre en `127.0.0.1`. No hay backend remoto, no hay cuenta, no hay telemetría saliente. La única persistencia es un SQLite en disco.
- 2. **Determinista y sin LLM.** No hay ningún modelo de lenguaje en el camino de ejecución. La lógica de decisión es `engine/conditions.py` (13 operadores de comparación) y `actions/rules.py` (reglas declarativas). Ver §7.
- 3. **Declarativo.** Añadir un caso de uso es añadir una carpeta con un JSON. No se toca el motor. Los 27 flows actuales están contruidos con las 36 acciones existentes.

2. Qué problema resuelve

Un operador que trabaja sobre Windows repite tareas que ninguna herramienta única cubre: capturar el escritorio y dejar evidencia fechada, rellenar un formulario web con datos de un dataset, inventariar una carpeta, vigilar si una página cambió, auditar los enlaces de un sitio, tomar un snapshot de recursos del equipo.

Cada una de esas tareas es fácil de scriptear una vez. Lo difícil, y lo que Automa aporta, es lo que viene después:

Problema	Cómo lo resuelve Automa
Cada script vive en su propio archivo con su propia forma	Contrato único: <code>schemas/manifest.schema.json</code> , validado por <code>scripts/validate_project.py</code>
No queda registro de qué se ejecutó, cuándo y con qué resultado	Cada corrida escribe en SQLite (<code>runs</code> , <code>steps</code> , <code>events</code>), en JSONL (<code>logs/</code>) y en un snapshot JSON (<code>state/</code>)
Un script puede escribir donde quiera	<code>SandboxPolicy</code> puede restringir acciones, rutas y tiempo máximo por flow
Programar la tarea implica salir a Task Scheduler	Scheduler propio con intervalo o cron de 5 campos y lock en SQLite
Hay que abrir una terminal para lanzar nada	Panel local de 3 pestañas con atajos <code>Alt+N</code> , o ventana nativa <code>pywebview</code>

3. A quién está dirigido

Actor	Qué hace con el sistema	Evidencia
Operador local	Único rol con acceso. Ejecuta flows desde el panel, programa horarios, consulta el histórico	El panel escucha en <code>127.0.0.1:8787</code> (<code>app/server.py::run_server</code>) y no tiene usuarios ni roles

Autor de flows	Escribe <code>manifest.json</code> nuevos usando las acciones existentes	Contrato en <code>docs/CREAR_FLUJOS.md</code> y <code>schemas/manifest.schema.json</code>
Desarrollador de acciones	Añade funciones a <code>actions/</code> y las registra en <code>_BUILT_IN_ACTIONS</code>	<code>engine/action_registry.py</code>
Integrador externo	Dispara un flow por webhook <code>POST /api/hook/<folder></code>	Requiere <code>AUTOMA_WEBHOOK_TOKEN</code> ; deshabilitado si no está definido
Sistema de monitoreo	Consume <code>GET /metrics</code> en formato Prometheus	<code>engine/metrics.py::prometheus_text</code>

No existe multiusuario, no existe RBAC y no existe sesión de usuario. El propio README del repositorio lo declara: «Multiusuario / RBAC — [alto] No — un operador local».

4. Casos de uso: el catálogo verificado

27 flows en `flows/`, agrupados por la clave `family` de su manifest. Conteo real obtenido leyendo los 27 manifests:

Familia	Flows	Qué caracteriza al grupo
<code>sistema</code>	10	Interactúan con componentes nativos de Windows o leen el estado del equipo
<code>navegador</code>	9	Lanzan Chromium con Playwright: capturan, rellenan o <code>leen</code> el DOM
<code>pantalla</code>	6	Capturan píxeles del escritorio, con o sin OCR posterior
<code>filesystem</code>	1	Inventario de una carpeta
<code>documentos</code>	1	Resumen de archivos de texto de una carpeta

Catálogo completo, con sus pasos y su política de sandbox real, en [04 · Mapa del código](#) y [19 · Matriz de trazabilidad](#).

Cinco ejemplos que ilustran el rango del sistema:

- **07_browser_form_filler** — la operación más compleja del repositorio. Carga 100 registros de `data/seeds/form_seeds.json`, elige uno que no se haya usado antes (tracking persistente en `data/seeds/.used_indices.json`), lanza Chromium **visible** con `slow_mo=250 ms`, rellena 10 campos uno a uno, envía, lee la validación JavaScript de la página y persiste el payload. Un solo paso de manifest; toda la complejidad vive en `actions/browser_form.py::fill_form`.
- **23_web_change_detector** — extrae el texto de una página, calcula su SHA-256, lo compara con la corrida anterior y **solo si cambió** dispara `notify.send`. La condición está en el manifest (`"when": {"path": "decision.status", "operator": "eq", "value": "alerta"}`), no en el código.
- **18_powershell_audit** — ejecuta un comando PowerShell contra una allowlist de 13 verbos de solo lectura y guarda `stdout/stderr/exit_code`.
- **08_windows_lock_workstation** — un paso, `ui.hotkey` con `Win+L`. Con `allowed_actions` de un solo elemento y `max_runtime_seconds: 5`.
- **05_system_healthcheck** — snapshot de CPU/RAM/disco con `psutil`, evaluación por reglas y reporte JSON. Es el flow que usa el smoke test.

5. Flujo general

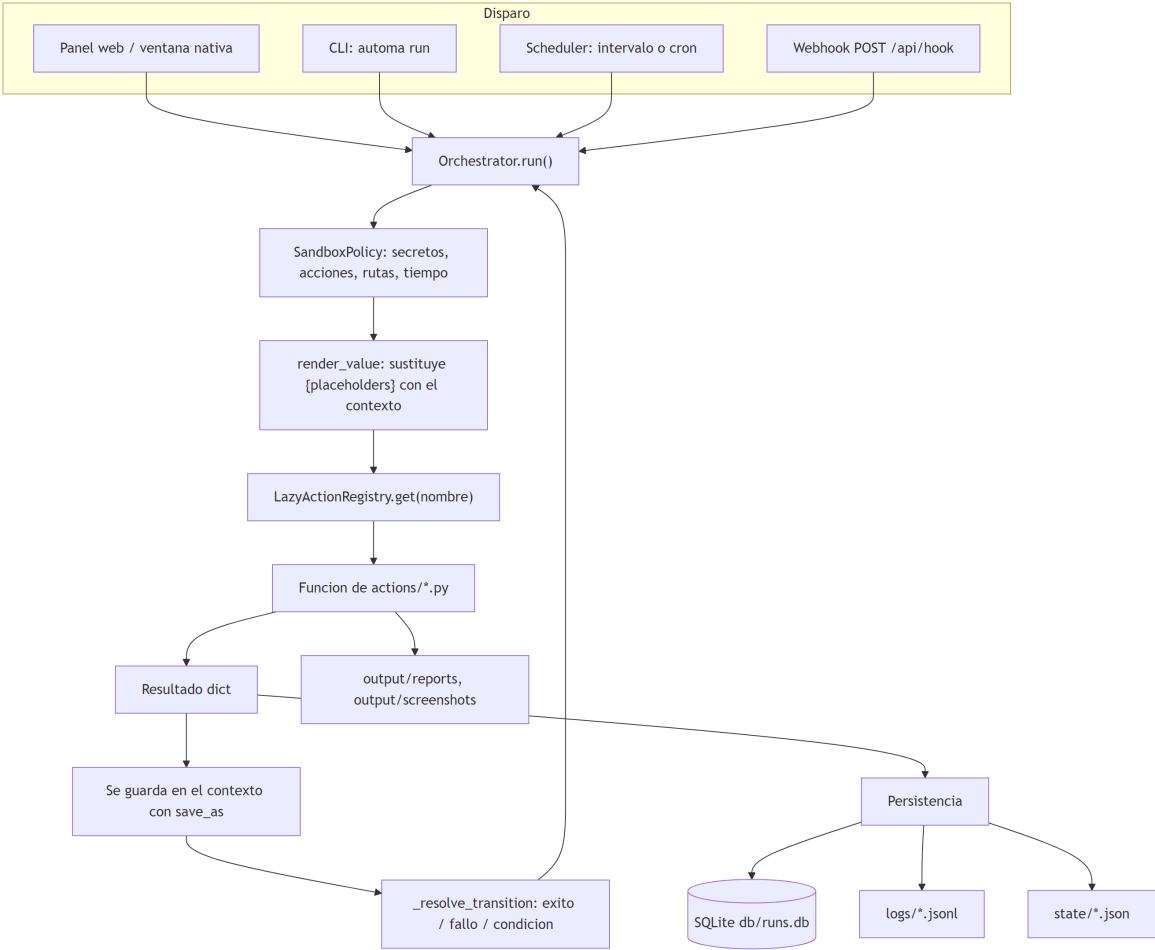


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: los cuatro disparadores convergen en el mismo **Orchestrator**, la política se aplica antes de cada acción, y cada paso realimenta el contexto del que se alimenta el siguiente. La persistencia es triple y simultánea.

Lo que el diagrama NO muestra: que el bucle es **síncrono y de un solo hilo dentro de una corrida** — un paso lento bloquea el flow entero, y **max_runtime_seconds** solo se comprueba *entre* pasos, nunca interrumpe uno en curso. Tampoco muestra que el panel lanza la corrida en un hilo aparte (**threading.Thread** en **do_POST**) para devolver el **run_id** de inmediato y dejar que el navegador haga polling.

6. Entradas y salidas

Entradas

Entrada	Origen	Módulo que la lee
manifest.json	flows/<carpeta>/	engine/loader.py::FlowLoader.load_manifest
Contexto del flow	configs/<carpeta>.json → flows/<c>/context.user.json → flows/<c>/context.example.json, en ese orden de prioridad	engine/loader.py::FlowLoader.load_context
context_overrides	Body JSON de POST /api/run/<folder>	app/server.py::do_POST → Orchestrator.__init__
Secretos	Variables de entorno, con respaldo en secrets/secrets.json	engine/secrets.py::get_secret
Estado del equipo	psutil, mss, pyperclip, PowerShell	actions/system.py, actions/screen.py

Páginas web	Chromium vía Playwright	<code>actions/browser_*.py</code>
-------------	-------------------------	-----------------------------------

Salidas

Salida	Ruta	Escrita por
Historial estructurado	<code>db/runs.db</code> (7 tablas)	<code>engine/database.py</code>
Log de eventos por corrida	<code>logs/<flow_id>_<run_id>.jsonl</code>	<code>engine/logger.py::JsonLogger</code>
Snapshot completo del estado	<code>state/<flow_id>_<run_id>.json</code>	<code>engine/state_store.py::StateStore</code>
Reportes de los flows	<code>output/reports/*.json, *.csv, *.md</code>	<code>actions/filesystem.py::write_json</code> , <code>actions/browser_extract.py</code>
Capturas	<code>output/screenshots/*.png</code>	<code>actions/screen.py</code> , <code>actions/browser_capture.py</code>
Métricas	<code>GET /metrics</code> (texto Prometheus)	<code>engine/metrics.py::prometheus_text</code>

Detalle no obvio: `engine/introspection.py::extract_existing_paths` decide qué cuenta como "output" de una corrida recorriendo el estado en busca de rutas de archivo existentes, pero **solo acepta las que están bajo `output/`**. Un comentario del propio archivo explica el porqué: antes contaba cualquier ruta existente y contaminaba la lista con los archivos que el flow solo había *leído*.

7. La cuestión de la IA: qué hay y qué no hay

El repositorio se presenta explícitamente como «sin IA». La verificación en el código confirma esa afirmación para el catálogo publicado, con un matiz que conviene registrar:

Hecho	Evidencia
Ningún flow del catálogo llama a un modelo de lenguaje	Las 26 acciones distintas usadas por los 27 manifests son de filesystem, pantalla, sistema, UI, reglas, HTTP, notificación y navegador
La toma de decisiones es determinista	<code>engine/conditions.py::matches</code> implementa 13 operadores (<code>eq</code> , <code>gt</code> , <code>contains</code> , <code>regex</code> ...). <code>actions/rules.py::evaluate_rules</code> evalúa reglas declaradas en el manifest y devuelve la primera que coincide
Existe un adaptador de visión multimodal	<code>plugins/analyzers/vision_model_analyzer.py::VisionModelAnalyzer</code> soporta <code>mock</code> , <code>openai_compatible</code> y <code>ollama</code>
Ese adaptador no es alcanzable desde ningún flow	Solo lo usa <code>actions/vision.py::inspect_screen_target</code> , y esa acción no aparece en ninguno de los 27 manifests. Verificado programáticamente
Su modo por defecto no llama a ninguna red	<code>provider='mock'</code> produce una lectura heurística local de brillo y RGB con Pillow
<code>decision/optional_ai.py</code> es un stub	<code>suggest_step_order</code> devuelve la lista sin tocarla y nadie lo importa . Ver 15

Conclusión: el sistema es determinista de extremo a extremo tal y como se distribuye. El adaptador de visión es un punto de extensión latente, sin uso, cuya activación exigiría escribir un flow nuevo que invoque `vision.inspect_screen_target` con un `vision_provider` distinto de `mock`. Documentado en detalle en 09 · APIs e integraciones.

8. Componentes más importantes

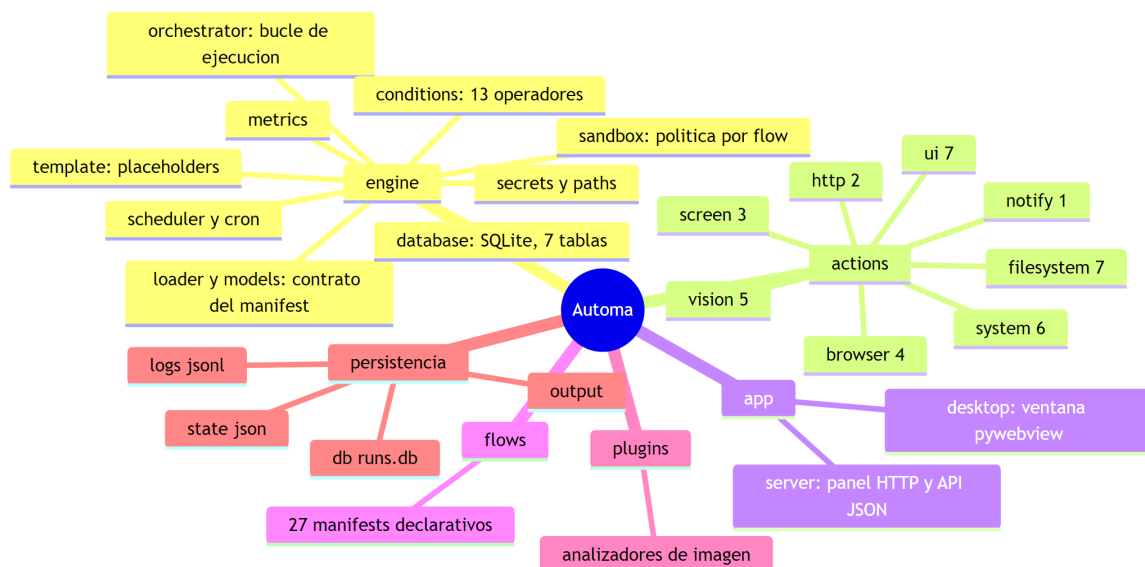


Diagrama 2 (Mermaid, renderizado)

Lo que el mapa muestra: las cinco piezas del sistema y el reparto de las 36 acciones por familia. **Lo que no muestra:** que `app/server.py` concentra 1 753 líneas —el 29 % del Python de producción— porque además del enrutado HTTP contiene el HTML, el CSS y el JavaScript del panel embebidos como cadenas Python. Es el archivo con más peso y el que más cuidado exige al modificar (ver 15).

9. Tecnologías y dependencias

Dependencias declaradas en `pyproject.toml`

Paquete	Cota	Para qué se usa
<code>Pillow</code>	<code>>=12.2.0, <13</code>	Análisis de imagen, recorte, captura de respaldo
<code>psutil</code>	<code>>=5.9.8, <8</code>	CPU, memoria, disco, procesos
<code>requests</code>	<code>>=2.32.2, <3</code>	<code>http.fetch_url</code> , <code>http.check_urls</code> , webhooks de salida, <code>robots.txt</code>
<code>mss</code>	<code>>=9.0.1, <11</code>	Captura de pantalla primaria
<code>pyautogui</code>	<code>>=0.9.54, <1</code>	Teclado y ratón
<code>pytesseract</code>	<code>>=0.3.10, <1</code>	Puente al binario OCR <code>tesseract</code>
<code>pyperclip</code>	<code>>=1.8.2, <2</code>	Lectura del portapapeles
<code>PyGetWindow</code>	<code>>=0.0.9, <1</code>	Rectángulo de la ventana activa
<code>pywebview</code>	<code>>=5.4, <7</code>	Ventana nativa del escritorio

Extras: `schema` (`jsonschema>=4`) y `dev` (`pytest`, `pytest-cov`, `ruff`, `jsonschema`, `pre-commit`).

Un comentario del propio `pyproject.toml` explica la política de cotas: «Cotas de seguridad: piso explícito = primera versión sin CVE conocida al corte de la auditoría 2026-06-01». Es una decisión documentada, no un rango arbitrario.

Dependencia crítica que NO está declarada

`playwright` **no aparece** en `pyproject.toml` ni en `requirements.txt`, pero nueve flows (02, 07, 21–27) no funcionan sin él. Las tres acciones que lo usan lo importan dentro de la función y levantan un `RuntimeError` con la instrucción de instalación:

```
# actions/browser_capture.py – patrón repetido en browser_form.py y browser_extract.py
raise RuntimeError(
    'playwright no está instalado. Ejecuta: '
    'pip install playwright && python -m playwright install chromium'
) from exc
```

Es una degradación explícita y con mensaje útil, no un crash opaco. Aun así, un tercio del catálogo depende de un paquete que `pip install -e .` no instala. Registrado en [15 · Riesgos y deuda técnica](#).

Dependencia externa opcional: `tesseract`

`vision.ocr_image` necesita el binario `tesseract` instalado en el sistema. `plugins/analyzers/ocr_image_analyzer.py::OCRImageAnalyzer` lo comprueba antes de usarlo y, si falta, devuelve `status: "unavailable"` con instrucciones por sistema operativo en vez de fallar. Un comentario del archivo explica la decisión: permite que un flow con rama de recuperación siga su camino alternativo.

10. Límites del sistema

Lo que Automa **no** hace, y conviene decirlo antes de que alguien lo suponga:

- **No aísla a nivel de sistema operativo.** El sandbox es declarativo: el motor comprueba nombres de acción y prefijos de ruta antes de llamar a la función. Una vez dentro, la acción corre con todos los permisos del usuario. [docs/SEGURIDAD.md](#) lo declara y el README lo marca como «[bajo] Sandbox declarativo, no proceso».
- **Ese sandbox es opcional y 14 de 27 flows no lo usan.** Solo los flows 08–20 declaran `allowed_actions`; los flows 01–07 y 21–27 corren con `SandboxPolicy` completamente permisiva. Detalle y conteo en [11 · Seguridad](#).
- **No hay autenticación de usuario.** Hay protección anti-CSRF y un token opcional (`AUTOMA_PANEL_TOKEN`), pero cualquiera con acceso a la sesión de Windows tiene acceso total al panel.
- **No hay retención ni rotación.** `db/runs.db`, `logs/*.jsonl` y `state/*.json` crecen sin límite. **NO IDENTIFICADO:** no existe ninguna rutina de purga en el repositorio.
- **No hay multiusuario ni permisos.**
- **Los atajos de teclado del panel cubren 12 flows de 27.** El propio README lo advierte: los casos por encima del 12 se ejecutan haciendo clic en su tarjeta.
- **Windows es la plataforma real.** La CI corre también en Ubuntu, pero eso valida la lógica pura y el motor; las acciones de UI, portapapeles, PowerShell y ventana activa no tienen sentido fuera de Windows.

11. Integraciones externas

Integración	Dirección	Autenticación	Módulo
Webhook entrante	Entra	<code>AUTOMA_WEBHOOK_TOKEN</code> en el header <code>X-Automa-Token</code>	<code>app/server.py::_check_webhook_token</code>
Notificación por webhook	Sale	<code>Bearer</code> opcional, con soporte <code>@secret:NOMBRE</code>	<code>actions/notify.py::send_notification</code>
Scraping / captura web	Sale	Ninguna	Playwright + Chromium
Verificación de enlaces	Sale	Ninguna	<code>actions/http_actions.py::check_urls</code>
<code>robots.txt</code>	Sale	Ninguna	<code>actions/browser_extract.py::RobotsCache</code>
Prometheus	Entra (lectura)	Ninguna	<code>GET /metrics</code>
Proveedor de visión	Sale	<code>Bearer</code> opcional	<code>VisionModelAnalyzer</code> , sin uso

Todas las salidas de red son **opt-in por manifest**: si el flow no declara un paso que las use, no hay tráfico. Los siete flows web (21–27) apuntan por defecto a HTML locales del propio repositorio (`data/web/`), de modo que la demo funciona sin internet.

12. Estado general observado

Medido en el commit analizado, ejecutando los comandos del propio repositorio:

Señal	Resultado	Comando
Suite de pruebas	150 pasan, 0 fallan en 16,62 s	<code>python -m pytest</code>
Cobertura	58,92 %, umbral configurado 54 %	incluida en <code>pytest</code> vía <code>--cov-fail-under=54</code>
Lint	Sin hallazgos	<code>python -m ruff check .</code>
Validación de manifests	27 flows, 36 acciones, 0 errores	<code>python scripts/validate_project.py</code>
Árbol de trabajo	Limpio al iniciar el análisis	<code>git status --porcelain</code>

La CI tiene cinco workflows (`ci`, `security`, `workflow-security`, `markdown-docs`, `dependency-hygiene`) más el de release. Todas las acciones de terceros están fijadas a SHA, hay un verificador propio (`pin-check`) que falla si alguien introduce un `uses:` sin SHA, y `security.yml` corre CodeQL `security-extended`, `detect-secrets` sobre el filesystem y los últimos 50 commits, y `pip-audit`.

Lectura honesta del estado: el sustrato es sólido y está bien probado en su lógica pura. Los huecos están donde el sistema toca el mundo real: las acciones de UI, captura y navegador tienen poca o ninguna cobertura automatizada, y el empaquetado tiene un hallazgo abierto (§10 del 15).

13. El sistema explicado para una persona no técnica

Imagine que tiene un ayudante muy obediente y muy literal sentado frente a su computador.

Usted le deja una **receta escrita**: «primero saca una foto de la pantalla, después guarda un informe con lo que veas, y si el informe dice que la memoria está por encima del 80 %, avísame». El ayudante hace exactamente eso, ni más ni menos, en ese orden. No improvisa, no interpreta, no consulta a nadie. Si un paso falla, mira la receta a ver si usted dejó escrito qué hacer en ese caso.

Eso es Automa. Las recetas son archivos de texto que se pueden leer y corregir; el ayudante es un programa que vive en su computador y no habla con internet salvo que la receta se lo pida.

Lo que gana con esto:

- **Queda registro de todo.** Cada vez que el ayudante ejecuta una receta anota en un cuaderno la hora, cada paso, cuánto tardó y qué resultado dio. Puede volver a mirarlo meses después.
- **Las recetas se pueden acotar.** Puede escribir en la receta «solo puedes escribir en esta carpeta» o «si tardas más de cinco segundos, para». El programa lo respeta.
- **Puede programarlo.** «Ejecuta esta receta cada quince minutos» o «todos los lunes a las nueve».
- **Todo se maneja desde una ventana.** Hay una pantalla con tarjetas: una por receta. Se hace clic y el ayudante empieza, mostrando en qué paso va.

Lo que NO hace, y es importante saberlo antes de confiar en él:

- **No piensa.** No hay inteligencia artificial decidiendo nada. Si la receta está mal escrita, hará lo que dice la receta, no lo que usted quería decir.
- **No es una caja fuerte.** Las restricciones de la receta las hace cumplir el propio programa, no el sistema operativo. Es como un cartel de «no pasar»: funciona si quien pasa respeta el cartel. Y además, catorce de las veintisiete recetas actuales no llevan cartel puesto.
- **Cualquiera que use su computador puede usarlo.** No hay contraseña de usuario. Si alguien se sienta en su sesión de Windows abierta, puede ejecutar cualquier receta.
- **No borra nada solo.** El cuaderno de registro y las capturas se acumulan indefinidamente. Con el tiempo ocupan espacio y hay que limpiarlos a mano.

- **Es para Windows.** Buena parte de las recetas (abrir el Explorador, bloquear la sesión, leer el portapapeles) solo tienen sentido ahí.
- **Algunas recetas necesitan piezas extra.** Las que abren páginas web necesitan que antes se instale un navegador especial; las que leen texto de una imagen necesitan un programa de reconocimiento de texto. Si falta la pieza, el programa lo dice con claridad en vez de fallar de forma confusa.

En una frase: **Automa convierte tareas repetitivas de Windows en recetas escritas que se ejecutan solas, dejando rastro de todo, sin enviar nada a internet y sin adivinar nada.**

Documentos relacionados: [02 · Instalación](#) · [03 · Arquitectura](#) · [11 · Seguridad](#) · [15 · Riesgos](#) · [17 · Resumen ejecutivo](#)

02 · Instalación y ejecución

Cómo pasar de un clon limpio a un sistema funcionando, en cualquiera de las tres vías que el repositorio soporta. Todos los comandos de este documento se transcriben del propio repositorio; los que se ejecutaron durante el análisis llevan su salida real.

1. Requisitos previos

Requisito	Versión	Obligatorio	Evidencia
Python	<code>>=3.10</code>	Sí, salvo si se usa el instalador <code>.exe</code>	<code>pyproject.toml</code> , <code>requires-python</code>
Sistema operativo	Windows 10 / 11	Para el catálogo completo	Las acciones de UI, portapapeles, PowerShell y ventana activa son de Windows
<code>pip</code> o <code>uv</code>	—	Sí	<code>Makefile</code> ofrece ambos caminos
Chromium de Playwright	—	Solo para los 9 flows de navegador	<code>python -m playwright install chromium</code>
Binario <code>tesseract</code>	—	Solo para los flows con OCR (12, 17)	<code>plugins/analyzers/ocr_image_analyzer.py</code> lo busca y degrada si falta
Escritorio gráfico activo	—	Para capturas y acciones de teclado/ratón	<code>actions/screen.py</code> levanta <code>RuntimeError</code> sin escritorio

La CI valida **Python 3.10, 3.11 y 3.12** sobre `ubuntu-latest` y `windows-latest` (matriz de `.github/workflows/ci.yml`, seis combinaciones).

Nota sobre `playwright`. No está declarado en `pyproject.toml` ni en `requirements.txt`, pero nueve flows lo necesitan. Hay que instalarlo aparte. Las tres acciones que lo usan lo importan dentro de la función y devuelven un `RuntimeError` con el comando exacto, así que el fallo es legible; pero no se resuelve solo. Registrado en [15 · Riesgos y deuda técnica](#).

2. Opción A · Instalador de Windows (usuarios finales)

1. Descargar `Automa-Setup-vX.Y.Z.exe` del último release.
2. Ejecutarlo. El asistente instala **por usuario** (`PrivilegesRequired=lowest` en `installer/Automa.iss`), sin pedir permisos de administrador.
3. Al terminar, «Launch Automa» abre una ventana nativa con el panel.

Detalles verificables del instalador, leídos de `installer/Automa.iss`:

Aspecto	Valor
Directorio por defecto	<code>{autopf}\Automa</code>
Idiomas	Español (por defecto) e inglés
Acceso directo en escritorio	Opcional, desmarcado por defecto (<code>Flags: unchecked</code>)
Arquitectura	<code>x64compatible</code>
Compresión	<code>lzma2/ultra</code> con <code>SolidCompression</code>

Efecto que sobrevive a la desinstalación. El binario empaquetado escribe sus datos fuera del directorio de instalación: `engine/paths.py::data_dir` devuelve `%LOCALAPPDATA%\Automa` cuando detecta que corre congelado. El desinstalador de Inno Setup borra `{app}`, no `%LOCALAPPDATA%`. **INFERENCIA:** la base de datos, los logs y las capturas **quedan en el disco tras desinstalar**. Si quiere borrarlos, hay que hacerlo a mano.

3. Opción B · Desde el código con `uv` (recomendada para desarrollo)

```
git clone https://github.com/vladimiracunadev-create/automa-pc.git
cd automa-pc
uv sync --extra dev --extra schema
python -m playwright install chromium # necesario para los flows 02, 07 y 21-27
uv run automa-desktop                 # ventana nativa pywebview
```

`uv sync --extra dev --extra schema` es exactamente lo que hace la CI antes de correr los tests (`.github/workflows/ci.yml`, paso «Install project (runtime + dev + schema)»).

4. Opción C · Desde el código con `pip`

```
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux / macOS
pip install -e ".[dev,schema]"
python -m playwright install chromium
automa-desktop
```

Equivalente vía `Makefile`:

```
make install-dev # pip install -e ".[dev,schema]"
make sync        # uv sync --extra dev --extra schema
```

`requirements.txt` **no basta**. Declara solo seis paquetes (`Pillow`, `psutil`, `requests`, `mss`, `pyautogui`, `pytesseract`) y le faltan `pyperclip`, `PyGetWindow` y `pywebview`, que sí están en `pyproject.toml`. `make install` usa ese archivo y deja el entorno incompleto: la ventana nativa y los flows 15 y 16 no arrancarían. Usar la vía `pyproject.toml` (opciones B o C). Registrado en [15 · Riesgos y deuda técnica](#).

5. Comandos disponibles tras instalar

Los cuatro *entry points* declarados en `pyproject.toml`, `[project.scripts]`:

Comando	Apunta a	Qué hace
<code>automa</code>	<code>engine.runner:main</code>	CLI con tres subcomandos: <code>list</code> , <code>run</code> , <code>scheduler</code>
<code>automa-panel</code>	<code>app.server:run_server</code>	Panel HTTP en <code>127.0.0.1:8787</code>
<code>automa-desktop</code>	<code>app.desktop:main</code>	Panel dentro de una ventana nativa <code>pywebview</code>
<code>automa-validate</code>	<code>scripts.validate_project:main</code>	Valida los 27 manifests contra el JSON Schema

Sintaxis del CLI

```
automa list # inventario de flows en JSON
automa run flows/05_system_healthcheck # ejecuta un flow
```

```
automa run flows/03_folder_inventory --context mi_contexto.json
automa scheduler --interval 2 # bucle del scheduler, chequeo cada 2 s
```

Equivalentes sin instalar el paquete:

```
python -m engine.runner list
python -m engine.runner run flows/05_system_healthcheck
python -m app.server # panel; equivale a make run-panel
```

[!] El CLI falla al imprimir en una consola Windows por defecto

Verificado durante este análisis. `engine/runner.py::main` termina con `print(json.dumps(..., ensure_ascii=False, indent=2))`. Los manifests y los estados contienen caracteres no representables en `cp1252` (por ejemplo `→`, presente en varias descripciones de flow), así que la impresión revienta:

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '\u2192' in position 6042:
character maps to <undefined>
```

Ocorre con `list` y con `run`. **Detalle crítico:** en `run` el flow se ejecuta completo y correctamente —la corrida queda persistida en SQLite y el reporte escrito— y el error salta *después*, solo al volcar el JSON a la consola. No se pierde trabajo, pero el código de salida es de error.

Solución inmediata:

```
# PowerShell
$env:PYTHONIOENCODING = "utf-8"; python -m engine.runner list

# Git Bash / cmd
set PYTHONIOENCODING=utf-8
python -m engine.runner list
```

Verificado: con `PYTHONIOENCODING=utf-8` el comando devuelve el JSON completo sin error. El panel HTTP no sufre este problema porque escribe bytes UTF-8 a un socket, no a la consola. Registrado como hallazgo en [15 · Riesgos y deuda técnica](#) y con guía en [14 · Solución de problemas](#).

6. Ejecución del panel

```
automa-panel # o: python -m app.server
```

Salida esperada: `Panel disponible en http://127.0.0.1:8787`.

El servidor es `ThreadingHTTPServer` de la biblioteca estándar (`app/server.py::run_server`), ligado a `127.0.0.1` por defecto. **Importar `app.server` tiene efectos secundarios:** al cargarse el módulo se arranca el scheduler en un hilo de fondo, se inicializa la base de datos y se sincroniza el catálogo de flows.

```
# app/server.py — se ejecuta al importar el módulo, no al llamar run_server()
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Consecuencia práctica: cualquier proceso que importe `app.server` —incluidos los tests de `tests/test_panel_endpoints.py`— levanta un scheduler. Es una decisión implícita, no documentada en el repositorio.

Parámetros de la ventana nativa

```
automa-desktop --host 127.0.0.1 --port 8787 --width 1200 --height 800 --fullscreen
```

`app/desktop.py::launch` arranca el servidor en un hilo, espera hasta 8 segundos a que el puerto responda (`_wait_for_server`) y solo entonces abre la ventana. Si el puerto no responde, imprime el error y devuelve código 1 sin abrir ventana. Si falta `pywebview`, devuelve código 2 con la instrucción de instalación.

7. Configuración inicial

No hace falta configurar nada para arrancar: los 27 flows traen su `context.example.json` y funcionan con los valores por defecto. Los flows web apuntan por defecto a HTML locales de `data/web/`, de modo que la demo corre sin internet.

Para personalizar un flow hay tres vías, resueltas en este orden por `engine/loader.py::FlowLoader.load_context`:

- 1. `configs/<carpeta>.json` — lo que escribe el panel desde `/flow/<folder>/config`
- 2. `flows/<carpeta>/context.user.json` — override local, no versionado
- 3. `flows/<carpeta>/context.example.json` — valor por defecto que viene con el flow

La primera que exista gana; las demás se ignoran por completo (no hay mezcla de claves). Detalle en [10 · Configuración](#).

Variables de entorno

Variable	Obligatoria	Efecto
AUTOMA_PANEL_TOKEN	No	Si se define, toda mutación del panel exige el header <code>X-Automa-Token</code> con ese valor
AUTOMA_WEBHOOK_TOKEN	No	Habilita <code>POST /api/hook/<folder></code> . Sin ella el webhook responde 401 siempre
AUTOMA_ROOT	No	Sobrescribe la raíz de solo lectura (donde viven <code>flows/</code> y <code>schemas/</code>)
AUTOMA_DATA_ROOT	No	Sobrescribe la raíz escribible (<code>db/</code> , <code>state/</code> , <code>logs/</code> , <code>configs/</code> , <code>secrets/</code>)
PYTHONIOENCODING	No	Recomendada en <code>utf-8</code> para usar el CLI en Windows (\$5)

Ninguna es obligatoria. Catálogo completo en [10 · Configuración](#).

8. Base de datos: no hay que crearla

`db/runs.db` se crea sola. `engine/database.py::init_db` ejecuta un `CREATE TABLE IF NOT EXISTS` de las siete tablas y se invoca desde `Orchestrator.__init__`, desde `SchedulerService.__init__`, desde el CLI y al importar `app.server`. No hay migraciones que aplicar ni semillas que cargar.

La única migración existente es una línea al final de `init_db`, que añade la columna `cron_expression` a bases anteriores a la v0.2.0:

```
_ensure_column(conn, 'schedules', 'cron_expression', 'cron_expression TEXT')
```

Detalle del esquema en [07 · Base de datos](#).

9. Ejecución de las pruebas y de los validadores

Los tres comandos que el README declara obligatorios antes de subir. **Salidas reales obtenidas en el commit analizado**, sobre Python 3.12.9 y Windows 11:

```
python -m pytest

150 passed in 16.62s
Required test coverage of 54% reached. Total coverage: 58.92%

python -m ruff check .
```

```
All checks passed!
```

```
python scripts/validate_project.py
```

```
{
  "ok": true,
  "flows_checked": 27,
  "registered_actions": 36,
  "errors": []
}
```

Y el smoke test de integración, que la CI corre en un job aparte:

```
python scripts/smoke_test.py
```

```
{
  "ok": true,
  "runs": 20,
  "db_path": "C:\\dev\\automa-pc\\db\\runs.db"
}
```

Efecto no obvio del smoke test. `scripts/smoke_test.py` llama a `set_flow_config('03_folder_inventory', {'path_override': 'data/inbox'})`, que **reescribe el archivo versionado** `configs/03_folder_inventory.json`. Tras ejecutarlo, `git status --porcelain` puede marcar ese archivo como modificado. Se restaura con `git checkout -- configs/03_folder_inventory.json`. Registrado en [15 · Riesgos y deuda técnica](#).

Atajos equivalentes del Makefile: `make test`, `make lint`, `make validate`, `make smoke`, `make flow-health`, `make list`, `make run-panel`, `make clean`.

10. Ejecución en producción

No hay «producción» en el sentido de servidor: el modo de uso previsto es el equipo del operador. Las dos formas soportadas son:

- **Instalador .exe** (opción A). Es el modo de distribución oficial.
- **automa-desktop desde el código** (opciones B y C).

Para dejar el scheduler corriendo sin ventana:

```
automa scheduler --interval 2
```

Ese proceso solo dispara las tareas programadas; no levanta panel. El panel arranca su propio scheduler al importarse, así que **no hay que ejecutar ambos**: dos schedulers sobre la misma base compiten, aunque la tabla `run_locks` impide que un mismo flow corra dos veces en paralelo (`engine/database.py::acquire_run_lock` usa la clave primaria `folder` y captura `sqlite3.IntegrityError`).

11. Compilación del binario

```
pwsh installer/build_local.ps1      # genera dist/Automa/
iscc installer/Automa.iss            # genera installer/output/Automa-Setup-vX.Y.Z.exe
```

Requiere PyInstaller e Inno Setup 6+. La CI lo hace en `windows-latest` ([.github/workflows/release.yml](#)), donde Inno Setup viene preinstalado como `iscc`. Detalle en [13 · Despliegue y operación](#).

REQUIERE VALIDACIÓN: el build no se ejecutó en este análisis. Hay un hallazgo abierto sobre `installer/automa.spec` (falta `actions.browser_extract` en `hiddenimports`) que solo se puede confirmar compilando.

12. Errores frecuentes al instalar y arrancar

Síntoma	Causa	Solución
---------	-------	----------

UnicodeEncodeError: 'charmap' codec... al usar el CLI	Consola Windows en cp1252 y ensure_ascii=False	set PYTHONIOENCODING=utf-8 antes del comando (\$5)
RuntimeError: playwright no está instalado	Flows 02, 07, 21–27 sin Playwright	pip install playwright && python -m playwright install chromium
ERROR: pywebview no esta instalado (código 2)	Falta pywebview	pip install pywebview, o usar automa-panel en el navegador
El flow OCR devuelve status: "unavailable"	Falta el binario tesseract	Windows: choco install tesseract. Linux: apt-get install tesseract-ocr. macOS: brew install tesseract
RuntimeError: No fue posible capturar la pantalla	Sin escritorio gráfico (sesión SSH, contenedor)	Ejecutar en una sesión Windows interactiva
PermissionError [WinError 5] al escribir en db/	Bundle instalado bajo Program Files con AUTOMA_DATA_ROOT mal apuntado	Corregido en v0.2.0 con data_dir(); ver CHANGELOG.md
El puerto 8787 está ocupado	Otra instancia corriendo	automa-desktop --port 8888, o cerrar la otra instancia
Los 12 primeros atajos Alt+N funcionan, los demás no	El panel solo mapea 12 atajos	Ejecutar el flow haciendo clic en su tarjeta

Guía completa con diagnóstico y riesgo de cada solución en [14 · Solución de problemas](#).

13. Verificación de que la instalación quedó bien

```
python scripts/validate_project.py      # debe devolver "ok": true, 27 flows
curl http://127.0.0.1:8787/healthz     # debe devolver {"status": "ok"}
python -m engine.runner run flows/05_system_healthcheck # con PYTHONIOENCODING=utf-8
```

Si el tercero deja un archivo nuevo en [output/reports/](#) y una fila nueva en la pestaña **Histórico** del panel, el sistema está operativo de extremo a extremo.

Documentos relacionados: [01 · Descripción general](#) · [10 · Configuración](#) · [13 · Despliegue y operación](#) · [14 · Solución de problemas](#) · [18 · Guía para nuevo desarrollador](#)

03 · Arquitectura

Estilo arquitectónico, capas, patrones, manejo de estado y errores, concurrencia y persistencia. Cada diagrama va acompañado del párrafo que explica **lo que muestra y lo que no**; ningún diagrama es imprescindible para entender el texto.

1. Estilo arquitectónico

Automa es una **arquitectura en capas con motor de flujo declarativo dirigido por datos** (*data-driven workflow engine*). Tres afirmaciones la caracterizan:

1. **El comportamiento vive en datos, no en código.** El manifest describe qué pasa; el motor solo sabe ejecutar pasos, evaluar condiciones y seguir transiciones. Añadir un caso de uso es añadir un JSON.
2. **La ejecución es una máquina de estados explícita.** Cada paso decide el siguiente mediante `transitions`, con `success`, `failure` o `any` como disparador y una condición opcional. Sin transiciones, el orden es el del array.
3. **La política de seguridad es parte del contrato, no del entorno.** Cada manifest puede declarar `allowed_actions`, `allowed_paths`, `required_secrets` y `max_runtime_seconds`, y el motor los aplica.

Es deliberadamente **monolítico y de un solo proceso**. No hay colas, ni brokers, ni servicios. La justificación está en el propio producto: es una herramienta local para un operador. `docs/ARQUITECTURA.md` del repositorio lo describe en la misma línea.

2. Capas y responsabilidades

Capa	Directorio	Responsabilidad	Depende de
Presentación	<code>app/</code>	Panel HTML+CSS+JS, API JSON, ventana nativa	<code>engine/</code>
Orquestación	<code>engine/orchestrator.py</code> , <code>scheduler.py</code> , <code>runner.py</code>	Bucle de ejecución, disparo por horario, CLI	Contrato, política, acciones, persistencia
Contrato	<code>engine/loader.py</code> , <code>models.py</code> , <code>manifest_schema.py</code> , <code>schemas/</code>	Leer y validar el manifest, convertirlo en dataclasses	—
Política	<code>engine/sandbox.py</code> , <code>secrets.py</code>	Restricciones por flow, resolución de secretos	<code>engine/paths.py</code>
Evaluación	<code>engine/conditions.py</code> , <code>template.py</code>	13 operadores, sustitución de placeholders	—
Acciones	<code>actions/</code>	Todo el trabajo efectivo sobre el sistema	Bibliotecas externas
Extensión	<code>plugins/analyzers/</code>	Analizadores de imagen intercambiables	Pillow, pyteseract, requests
Persistencia	<code>engine/database.py</code> , <code>logger.py</code> , <code>state_store.py</code>	SQLite, JSONL, snapshots JSON	<code>engine/paths.py</code>
Observabilidad	<code>engine/metrics.py</code> , <code>introspection.py</code>	Agregados SQL, detección de outputs	<code>engine/database.py</code>
Catálogo	<code>flows/</code> , <code>engine/catalog.py</code>	Los 27 casos y su lectura	<code>engine/database.py</code>

Regla de dependencia respetada: `actions/` no importa `engine/orchestrator`, y el orquestador no conoce ninguna acción concreta. La única excepción deliberada es `actions/notify.py`, que importa `engine.secrets.get_secret` para resolver la sintaxis `@secret:NOMBRE`.

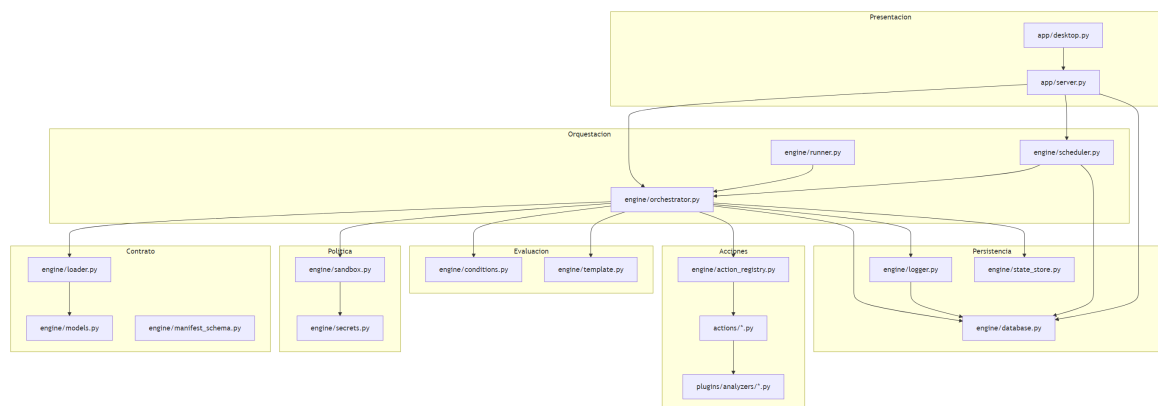


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: el orquestador es el único punto que toca todas las capas; las acciones no vuelven hacia arriba.

`JsonLogger` escribe a la vez en el archivo y en la tabla `events`, por eso apunta a `engine/database.py`.

Lo que no muestra: que `engine/manifest_schema.py` no se invoca durante la ejecución. Solo lo usa `scripts/validate_project.py`. En tiempo de ejecución, `FlowLoader.load_manifest` lee el JSON directamente sin validarlo contra el schema: un manifest inválido llega hasta el motor y falla ahí, no antes. Es una decisión implícita con consecuencias reales, registrada en 15.

3. El bucle de ejecución paso a paso

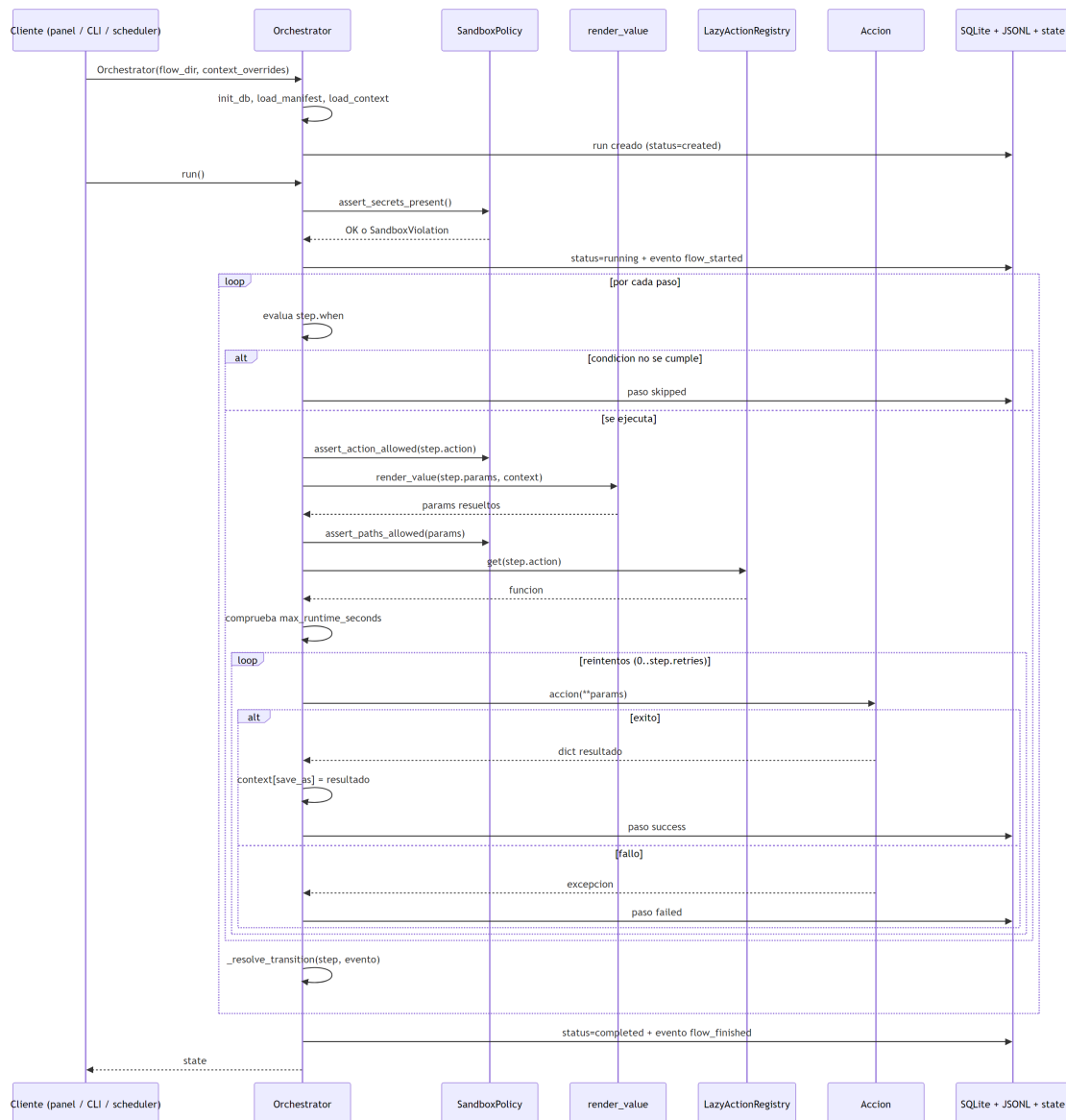


Diagrama 2 (Mermaid, renderizado)

Lo que la secuencia muestra: el orden exacto de los cuatro controles antes de cada acción —secretos (una vez, al inicio), acción permitida, rutas permitidas, tiempo máximo— y que el resultado de cada paso entra en el contexto antes de resolver la transición, de modo que la condición del siguiente paso puede leerlo.

Lo que no muestra, y es importante:

- `assert_paths_allowed` se aplica **sobre los parámetros ya renderizados**, no sobre el manifest crudo. Es lo correcto: un placeholder sin resolver no se puede validar.
- `max_runtime_seconds` se comprueba **entre pasos**. Una acción que se cuelga dentro (una llamada HTTP sin timeout, un `page.goto` bloqueado) no se interrumpe: el límite solo se nota cuando el paso termina y el motor va a por el siguiente.
- Los reintentos comparten los **mismos parámetros renderizados**. `render_value` se llama una vez por paso, fuera del bucle de reintentos, así que un `{now}` no cambia entre intentos.
- Al fallar el último intento, el motor busca una transición **on: "failure"**. Si esa transición apunta a un paso **distinto** del siguiente por defecto, el flow **se recupera** y continúa; si no, la corrida se marca **failed**. Esa comparación (`recovery_next != self._default_next(step.id)`) es el mecanismo de recuperación completo, y es fácil pasarlo por alto leyendo el manifest.

4. Patrones de diseño identificados

Patrón	Dónde	Por qué está
Registry con carga perezosa	<code>engine/action_registry.py::LazyActionRegistry</code>	Una acción solo importa su módulo cuando un flow la usa. Evita cargar Playwright, psutil o Pillow en cada arranque
Plugin por entry points	<code>LazyActionRegistry._maybe_load_entry_point</code>	Terceros publican acciones con el grupo <code>automa.actions</code> en su <code>pyproject.toml</code> sin tocar este repo
Strategy	<code>actions/vision.py::ANALYZERS</code>	<code>mock</code> , <code>metadata</code> y <code>ocr</code> implementan el mismo <code>AnalyzerProtocol</code> ; el manifest elige cuál
Máquina de estados	<code>Orchestrator._resolve_transition</code>	Grafo de pasos con eventos y guardas
Template method con datos	<code>engine/template.py::render_value</code>	<code>{clave}</code> y <code>{objeto.campo}</code> se resuelven contra el contexto aplanado
Repository	<code>engine/database.py</code>	Todo el SQL vive en un módulo; nadie más abre una conexión
Context manager	<code>engine/database.py::connect</code>	<code>commit</code> automático al salir sin excepción, <code>close</code> siempre
Guard clauses / fail fast	<code>engine/sandbox.py</code>	Cada <code>assert_*</code> lanza <code>SandboxViolation</code> y aborta la corrida
Degradación explícita	<code>OCRImageAnalyzer</code> , <code>system.read_clipboard</code> , <code>check_urls</code>	Devuelven <code>status/available/truncated</code> en vez de fallar o truncar en silencio
Separación puro / impuro	<code>actions/browser_extract.py</code>	La lógica (hash, links, BFS, CSV) es pura y testeable; solo <code>scrape_page</code> toca Playwright

El último es el patrón más valioso del repositorio y conviene protegerlo: los 31 tests que la v0.3.0 añadió para la familia web **corren sin navegador**, porque toda la lógica está en funciones puras y la interacción con Playwright se reduce a una capa de pegamento. Ver [12 · Pruebas y calidad](#).

5. Manejo del estado

Hay tres estados distintos y conviene no confundirlos:

Estado	Vive en	Alcance	Quién lo escribe
Contexto del flow	Memoria, <code>Orchestrator.context</code>	Una corrida	<code>save_as</code> de cada paso, más <code>_last_result</code> y <code>_last_error</code>
Estado de la corrida	<code>Orchestrator.state</code> , volcado a <code>state/*.json</code> y a la tabla <code>runs</code>	Una corrida, persistido	<code>Orchestrator._persist</code> , tras cada paso
Estado del sistema	<code>db/runs.db</code>	Todo el histórico	<code>engine/database.py</code>

El contexto es mutable y compartido

```
# engine/orchestrator.py, dentro del bucle de reintentos
result = action(**rendered_params)
self.context['_last_result'] = result
if step.save_as:
    self.context[step.save_as] = result
self.state['context'] = self.context
```

`self.state['context']` es la **misma referencia**, no una copia. Es intencional y hace que el snapshot persistido refleje siempre el contexto vivo, pero implica que el contexto crece con cada paso que declara `save_as`, y que **todo él acaba serializado en la**

columna `context_json` de la tabla `runs`. Un flow que guarde el texto completo de una página web grande (`browser.extract_content` admite hasta 200 000 caracteres por defecto) deja ese texto dentro de la fila de la corrida. Consecuencia de tamaño registrada en 15.

Persistencia tras cada paso

`_persist()` se llama después de cada paso, con éxito o sin él. Escribe simultáneamente el JSON de `state/` y hace `upsert` de la fila de `runs`. El costo es una escritura de disco y una transacción SQLite por paso; el beneficio es que una corrida interrumpida por un corte deja rastro de hasta dónde llegó. Para un sistema de escritorio con flows de pocos pasos, la decisión está bien calibrada.

6. Manejo de errores

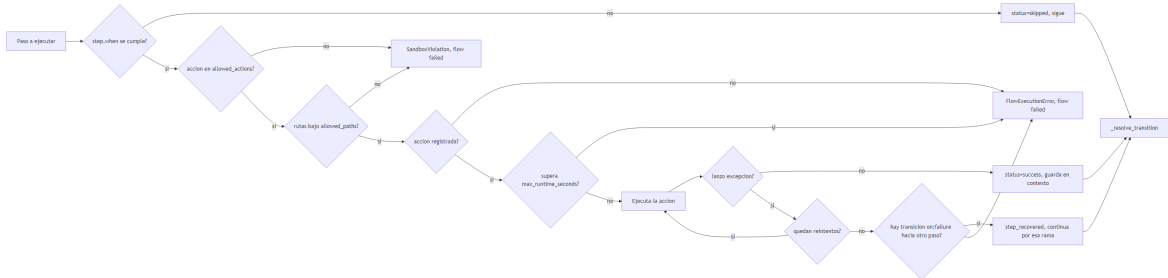


Diagrama 3 (Mermaid, renderizado)

Lo que el diagrama muestra: los siete puntos donde una corrida puede detenerse, y que la recuperación por transición es el último recurso, después de agotar los reintentos.

Lo que no muestra: que el manejo de excepciones dentro de la acción es deliberadamente amplio (`except Exception` con `noqa: BLE001`). Cualquier fallo —de red, de disco, de sintaxis en los parámetros— se convierte en el mismo `step_failed` con el texto de la excepción. Es una decisión que favorece la robustez del bucle frente a la precisión del diagnóstico. Tampoco muestra que el scheduler se traga las excepciones por completo:

```
# engine/scheduler.py::_run_job
try:
    Orchestrator(Path(flow['flow_path'])).run()
except Exception:
    # No interrumpe el loop. El error queda en la corrida.
    pass
```

El comentario es honesto: el error queda persistido en la corrida. Pero para el operador significa que una tarea programada que falla siempre no genera ninguna alerta; hay que mirar el histórico. Registrado en 15.

7. Concurrencia: qué es paralelo y qué no

Elemento	Modelo	Detalle
Pasos dentro de una corrida	Secuencial, un hilo	El bucle <code>while current_step_id</code> no tiene paralelismo
Corridas lanzadas desde el panel	Hilo por corrida	<code>threading.Thread(target=_runner, daemon=True)</code> en <code>do_POST</code>
Servidor HTTP	<code>ThreadingHTTPServer</code>	Un hilo por petición
Scheduler	Hilo de fondo + hilo por tarea	<code>start_in_background</code> , y <code>_run_job</code> en su propio hilo
Webhook <code>POST /api/hook</code>	Síncrono	A diferencia de <code>/api/run</code> , ejecuta el flow y espera. Una petición de webhook a un flow largo mantiene la conexión abierta

Protección contra ejecución doble	Tabla <code>run_locks</code>	Solo la aplica el scheduler . Ver aviso abajo
-----------------------------------	------------------------------	--

El lock protege menos de lo que parece. `acquire_run_lock` / `release_run_lock` se invocan **únicamente** desde `SchedulerService._run_job`. Ni `POST /api/run/<folder>`, ni `POST /run`, ni `POST /api/hook/<folder>`, ni el CLI lo usan. Verificado buscando las llamadas en todo el repositorio. Consecuencia: **el panel sí permite lanzar el mismo flow dos veces en paralelo**, aunque el scheduler no. Para flows con tracking persistente (07, 23, 26) eso significa dos procesos escribiendo el mismo archivo de estado. Registrado en 15.

SQLite y concurrencia

`engine/database.py::connect` abre una conexión nueva por operación y la cierra al salir. No hay pool ni conexión compartida entre hilos, lo que evita el clásico `ProgrammingError: SQLite objects created in a thread can only be used in that same thread`. El costo es abrir y cerrar el archivo en cada consulta. **NO IDENTIFICADO**: el repositorio no configura `PRAGMA journal_mode=WAL` ni `busy_timeout`, de modo que dos escrituras simultáneas pueden producir `database is locked`.

8. Autenticación y autorización

No hay usuarios. Lo que sí hay es un modelo de protección de las mutaciones, documentado en el propio `app/server.py` con un comentario de bloque de 20 líneas:

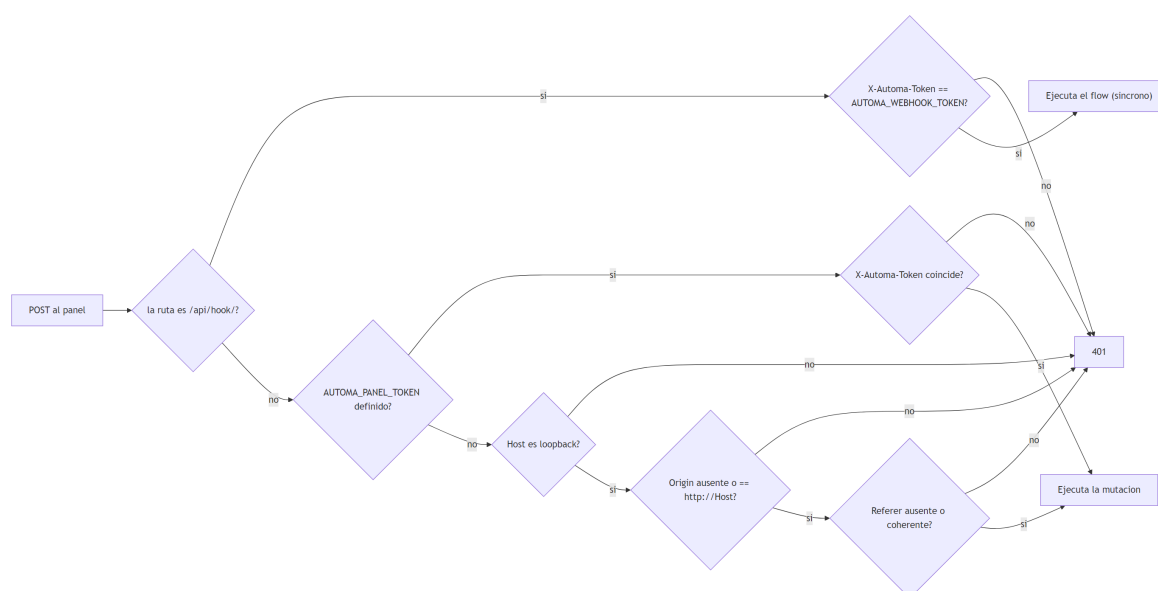


Diagrama 4 (Mermaid, renderizado)

Lo que el diagrama muestra: dos modos excluyentes. Con `AUTOMA_PANEL_TOKEN` definido, todo se decide por token comparado en tiempo constante (`hmac.compare_digest`, que cierra CWE-208). Sin token, la defensa es anti-CSRF y anti-DNS-rebinding: `Host` debe ser loopback y, **si vienen**, `Origin` y `Referer` deben ser coherentes.

Lo que no muestra: que un cliente que **no envía** `Origin` ni `Referer` —cualquier `curl`, cualquier script— pasa el control sin más, siempre que el `Host` sea loopback. La defensa está calibrada contra el ataque real (una web maliciosa que hace `fetch` a `127.0.0.1`, donde el navegador siempre añade `Origin`), no contra un proceso local hostil. Es una decisión razonable y está documentada, pero conviene tenerla explícita. Análisis completo en 11 · Seguridad.

Los GET no se protegen. `do_GET` no llama a `_authorize_mutation`. `/api/flows`, `/api/runs`, `/metrics`, `/file?path=...` y el HTML del panel son accesibles sin token, incluso con `AUTOMA_PANEL_TOKEN` definido. Es coherente con «lecturas inofensivas», pero `/api/runs` devuelve el `context_json` completo de cada corrida, que puede contener datos sensibles capturados por los flows.

9. Persistencia y caché

Persistencia

Tres almacenes simultáneos, sin caché intermedia:

Almacén	Formato	Contenido	Rotación
<code>db/runs.db</code>	SQLite, 7 tablas	Catálogo, corridas, pasos, eventos, config, locks, horarios	Ninguna
<code>logs/<flow_id>_<run_id>.jsonl</code>	JSON Lines	Un evento por línea, con marca de tiempo	Ninguna
<code>state/<flow_id>_<run_id>.json</code>	JSON	Snapshot completo del estado de la corrida	Ninguna

Los eventos se escriben **dos veces**: `JsonLogger.write` añade la línea al archivo `.jsonl` y llama a `insert_event` para la tabla `events`. Es redundancia deliberada —el archivo sobrevive a un borrado de la base, la tabla permite consultas— pero duplica el tamaño de la traza.

Caché

Hay tres cachés, todas en memoria y de vida corta:

- `LazyActionRegistry._cache` — función ya importada por nombre de acción.
- `RobotsCache._parsers` — un `robots.txt` por host durante un crawl.
- `data/seeds/.used_indices.json` y `data/web_watch/*.json` — no son caché sino **tracking persistente entre corridas**, y son el mecanismo que permite que el flow 07 no repita registros y que el 23 detecte cambios.

NO IDENTIFICADO: no hay caché de resultados de acciones ni de consultas SQL.

10. Procesos en segundo plano

Proceso	Arranca cuando	Frecuencia	Se detiene con
<code>SchedulerService.serve_forever</code>	Se importa <code>app.server</code> , o <code>automa.scheduler</code>	Cada <code>loop_sleep_seconds</code> (2 s por defecto)	<code>stop()</code> , o al morir el proceso (hilo daemon)
Hilo por corrida del panel	<code>POST /api/run/<folder></code>	Una vez por corrida	Al terminar la corrida (hilo daemon)
Hilo por tarea programada	El scheduler la ve vencida	Una vez por disparo	Al terminar (hilo daemon)
Servidor HTTP del panel	<code>app.desktop.launch</code>	Permanente	Al cerrar la ventana

Todos son hilos **daemon**: al cerrar la ventana o matar el proceso, mueren sin esperar. **INFERENCIA**: una corrida en curso al cerrar la ventana se interrumpe a mitad, dejando la fila de `runs` en `status='running'` para siempre. No hay ningún mecanismo de detección de corridas huérfanas ni de liberación automática de `run_locks` al arrancar. El `RUNBOOK.md` del repositorio contempla la liberación manual de locks, lo que confirma que el escenario es conocido.

11. Modelo de despliegue

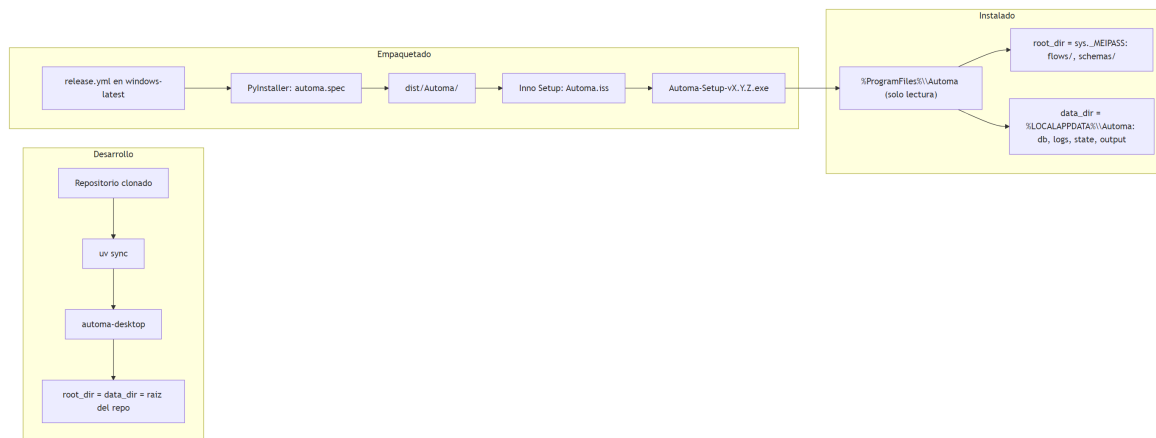


Diagrama 5 (Mermaid, renderizado)

Lo que el diagrama muestra: la separación entre raíz de solo lectura y raíz escribible, que es la decisión arquitectónica más importante del empaquetado. En desarrollo ambas coinciden con el repositorio; en el binario instalado se separan.

Lo que no muestra: el motivo. El `CHANGELOG.md` de la v0.2.1 lo documenta: instalado bajo `Program Files`, `init_db()` levantaba `PermissionError [WinError 5]` al intentar crear `_internal/db/`. La solución fue `engine/paths.py::data_dir`, que en modo `frozen` apunta a `%LOCALAPPDATA%\Automa`. `installer/automa_entry.py` además hace `os.chdir(data_dir())` para que las rutas relativas de los flows (`output/screenshots/...`) caigan en disco escribible. Es un ejemplo de bug de despliegue resuelto en la arquitectura, no con un parche.

12. Principios de ingeniería observados

Lo que está bien y conviene proteger:

- **Determinismo declarado y sostenido.** `crawl_pages` documenta en su docstring que el BFS toma los links en orden de aparición y sin aleatoriedad. `check_urls` verifica en orden de entrada. No hay `set()` iterado donde importe el orden.
- **Cotas explícitas por todas partes.** `max_links`, `max_text_chars`, `max_urls`, `max_pages`, `max_depth`, `max_steps_per_run`, `max_runtime_seconds`. Y cuando una cota se aplica, el resultado lo dice: `truncated: True`, nunca en silencio.
- **Degradación con motivo legible.** Falta `tesseract` → `status: "unavailable"` con instrucciones por sistema operativo. Falta backend de portapapeles → `available: False` con la razón. Falta Playwright → `RuntimeError` con el comando de instalación.
- **Comentarios que explican el porqué.** `engine/introspection.py` explica por qué limita los outputs a `output/`; `engine/template.py` explica por qué no usa `str.format_map`; `actions/ui.py` explica por qué `shell=True` está prohibido.
- **La seguridad del CI como parte de la arquitectura.** El repositorio trata el pipeline como frontera de confianza, con un razonamiento explícito en `security.yml`: «un commit malicioso fusionado a main se traduce directamente en RCE local cuando el operador hace pull».

Dónde la arquitectura tiene tensión:

- `app/server.py` con 1 753 líneas mezcla enrutado, HTML, CSS, JavaScript y lógica de presentación en un solo archivo.
- El sandbox es opcional y la mayoría de los flows no lo usan.
- El lock de ejecución solo cubre una de las cuatro vías de disparo.
- El JSON Schema existe pero el motor no lo aplica en runtime.

Cada uno de estos puntos está desarrollado, clasificado y priorizado en [15 · Riesgos y deuda técnica](#).

Documentos relacionados: [04 · Mapa del código](#) · [06 · Explicación profunda](#) · [07 · Base de datos](#) · [08 · Flujo de datos](#) · [11 · Seguridad](#)

04 · Mapa completo del código

Inventario jerárquico del repositorio en el commit **ff246ab**. Para cada elemento: ubicación, responsabilidad, dependencias, quién lo usa y **estado aparente** (activo · legado · duplicado · experimental · no determinado).

1. Árbol de primer nivel

automa-pc/		
■■■■ actions/	12 archivos	· las 36 acciones ejecutables
■■■■ app/	3 archivos	· panel HTTP y ventana nativa
■■■■ engine/	20 archivos	· motor, contrato, política, persistencia
■■■■ decision/	3 archivos	· stubs sin uso (ver §6)
■■■■ plugins/	6 archivos	· analizadores de imagen intercambiables
■■■■ flows/	27 carpetas	· el catálogo declarativo
■■■■ schemas/	1 archivo	· JSON Schema del manifest
■■■■ scripts/	2 archivos	versionados · validador y smoke test
■■■■ installer/	4 archivos	· PyInstaller, Inno Setup, entry point, build local
■■■■ tests/	17 archivos	· 150 pruebas
■■■■ data/		· HTML de demo, dataset semilla, carpeta de entrada
■■■■ configs/		· contexto persistido por flow
■■■■ db/ logs/ state/ output/ secrets/		· almacenes en tiempo de ejecución
■■■■ docs/		· documentación previa + esta carpeta
■■■■ .github/workflows/	6 workflows	de CI

Conteo verificado con `git ls-files`: **210 archivos versionados**, de los cuales **64 son .py** (47 de producción, 17 de pruebas).

2. engine/ — el motor

20 archivos, 1 841 líneas. Es el núcleo del sistema y ningún módulo suyo depende de `actions/` ni de `app/`.

Archivo	Líneas	Responsabilidad	Depende de	Lo usan	Estado
<code>orchestrator.py</code>	263	Bucle de ejecución, política, transiciones, persistencia	Casi todo <code>engine/</code>	<code>app/server.py</code> , <code>runner.py</code> , <code>scheduler.py</code> , <code>smoke_test.py</code>	Activo · crítico
<code>database.py</code>	378	Las 7 tablas SQLite y todo el SQL del sistema	<code>paths.py</code> , <code>cron.py</code> (import diferido)	<code>orchestrator</code> , <code>logger</code> , <code>catalog</code> , <code>metrics</code> , <code>scheduler</code> , <code>app/server</code>	Activo · crítico
<code>action_registry.py</code>	109	Registro perezoso de las 36 acciones + entry points	<code>importlib</code>	<code>orchestrator</code> , <code>validate_project</code>	Activo · crítico
<code>sandbox.py</code>	112	<code>SandboxPolicy</code> : acciones, rutas, secretos, tiempo	<code>os</code> , <code>pathlib</code>	<code>orchestrator</code>	Activo · crítico
<code>cron.py</code>	114	Cron de 5 campos, cálculo de próxima ejecución	<code>datetime</code>	<code>database.set_schedule</code> , <code>mark_schedule_run</code> , <code>app/server</code>	Activo
<code>metrics.py</code>	107	Agregados SQL y exposición Prometheus	<code>database.connect</code>	<code>app/server</code>	Activo
<code>catalog.py</code>	87	Lectura de <code>flows/</code> y unión con datos de la base	<code>database</code>	<code>app/server</code> , <code>runner</code> , <code>scheduler</code>	Activo

<code>scheduler.py</code>	86	Bucle de disparo por horario con lock	<code>catalog, database, orchestrator</code>	<code>app/server, runner</code>	Activo
<code>loader.py</code>	74	<code>manifest.json</code> → dataclasses; resolución del contexto	<code>models</code>	<code>orchestrator</code>	Activo · crítico
<code>introspection.py</code>	69	Detecta qué archivos de <code>output/</code> produjo la corrida	<code>pathlib</code>	<code>orchestrator._refresh</code>	Activo
<code>manifest_schema.py</code>	64	Validación contra JSON Schema con respaldo estructural	<code>jsonschema</code> (opcional), <code>paths</code>	<code>Solo scripts/validate_project</code>	Activo · no en runtime
<code>template.py</code>	63	<code>render_value:</code> {clave} y {obj.campo}	<code>re, datetime</code>	<code>orchestrator</code>	Activo · crítico
<code>paths.py</code>	61	<code>root_dir()</code> de solo lectura y <code>data_dir()</code> escribible	<code>os, sys</code>	<code>database, secrets, orchestrator, app/server, manifest_schema</code>	Activo · crítico
<code>conditions.py</code>	61	13 operadores + <code>all/any/not</code>	<code>re</code>	<code>orchestrator</code>	Activo · crítico
<code>secrets.py</code>	59	Entorno primero, archivo después	<code>paths</code>	<code>actions/notify, app/server</code>	Activo
<code>runner.py</code>	49	CLI <code>list / run / scheduler</code>	<code>catalog, database, orchestrator, scheduler</code>	Entry point <code>automa</code>	Activo · con defecto conocido
<code>models.py</code>	38	<code>FlowDefinition, StepDefinition, TransitionDefinition</code>	<code>dataclasses</code>	<code>loader, orchestrator</code>	Activo · crítico
<code>logger.py</code>	26	JSONL + inserción en la tabla <code>events</code>	<code>database</code>	<code>orchestrator</code>	Activo
<code>state_store.py</code>	21	Volcado del estado a <code>state/*.json</code>	<code>json, pathlib</code>	<code>orchestrator</code>	Activo
<code>__init__.py</code>	0	Marcador de paquete	—	—	Activo

Nota sobre `runner.py`: su `main()` termina imprimiendo JSON con `ensure_ascii=False`, lo que provoca `UnicodeEncodeError` en una consola Windows [cp1252](#). Verificado durante el análisis. Detalle en [02 §5](#) y [15](#).

Nota sobre `catalog.py`: define su propio `root_dir()` local (`Path(__file__).resolve().parent.parent`) en vez de importar `engine.paths.root_dir`. Ambas devuelven lo mismo en desarrollo, pero **la de `catalog.py` ignora `AUTOMA_ROOT` y `sys._MEIPASS`**. INFERENCIA: en el binario empaquetado, `catalog.flows_dir()` no apuntaría a los flows extraídos por PyInstaller. Duplicación con consecuencia, registrada en [15](#).

3. `actions/` — el trabajo efectivo

12 archivos, 1 832 líneas. Cada función pública es una acción invocable desde un manifest.

Archivo	Líneas	Acciones que expone	Dependencia externa	Estado
---------	--------	---------------------	---------------------	--------

<code>browser_extract.py</code>	567	<code>browser.extract_content</code> , <code>browser.crawl_site</code>	Playwright, requests	Activo
<code>browser_form.py</code>	204	<code>browser.fill_form</code>	Playwright	Activo
<code>vision.py</code>	197	<code>vision.analyze_image</code> , <code>ocr_image</code> , <code>find_text_in_image</code> , <code>select_image</code> , <code>inspect_screen_target</code>	Pillow (vía plugins)	Activo (parcial)
<code>system.py</code>	162	<code>system.wait_seconds</code> , <code>read_clipboard</code> , <code>run_powershell</code> , <code>snapshot_system</code> , <code>top_processes</code> , <code>watch_processes</code>	psutil, pyperclip, PowerShell	Activo
<code>screen.py</code>	158	<code>screen.capture_screenshot</code> , <code>capture_region</code> , <code>capture_active_window</code>	mss, Pillow, PyGetWindow	Activo
<code>filesystem.py</code>	121	<code>filesystem.ensure_directory</code> , <code>list_directory</code> , <code>write_json</code> , <code>read_text_file</code> , <code>classify_file_inventory</code> , <code>summarize_text_folder</code> , <code>move_file</code>	Ninguna	Activo (parcial)
<code>http_actions.py</code>	97	<code>http.fetch_url</code> , <code>http.check_urls</code>	requests	Activo (parcial)
<code>browser_capture.py</code>	93	<code>browser.capture_page</code>	Playwright	Activo
<code>ui.py</code>	91	<code>ui.open_url</code> , <code>open_file_in_browser</code> , <code>launch_process</code> , <code>hotkey</code> , <code>type_text</code> , <code>click</code> , <code>click_bbox</code>	pyautogui, webbrowser	Activo (parcial)
<code>notify.py</code>	78	<code>notify.send</code>	requests (import perezoso)	Activo
<code>rules.py</code>	64	<code>rules.evaluate</code>	Ninguna	Activo
<code>__init__.py</code>	0	Marcador de paquete	—	Activo

Las 10 acciones que ningún flow usa

Verificado leyendo los 27 manifests y comparando con `_BUILT_IN_ACTIONS`: de las **36 acciones registradas**, solo **26 aparecen en algún flow**. Las diez restantes existen, están registradas, funcionan y no tienen ningún caso que las ejerza:

Acción	Módulo	Estado
<code>filesystem.ensure_directory</code>	<code>actions/filesystem.py</code>	Disponible sin uso
<code>filesystem.read_text_file</code>	<code>actions/filesystem.py</code>	Disponible sin uso
<code>filesystem.move_file</code>	<code>actions/filesystem.py</code>	Disponible sin uso
<code>http.fetch_url</code>	<code>actions/http_actions.py</code>	Disponible sin uso
<code>ui.open_file_in_browser</code>	<code>actions/ui.py</code>	Disponible sin uso

<code>ui.click</code>	<code>actions/ui.py</code>	Disponible sin uso
<code>ui.click_bbox</code>	<code>actions/ui.py</code>	Disponible sin uso
<code>vision.select_image</code>	<code>actions/vision.py</code>	Disponible sin uso
<code>vision.find_text_in_image</code>	<code>actions/vision.py</code>	Disponible sin uso
<code>vision.inspect_screen_target</code>	<code>actions/vision.py</code>	Disponible sin uso · única vía al adaptador de visión multimodal

No son código muerto en sentido estricto —forman parte del contrato público del registro y del `pyproject.toml`— pero sí superficie sin cobertura de caso. La última es la más relevante: es el único camino hacia `VisionModelAnalyzer`, y al no usarla ningún flow, **el catálogo publicado no puede llamar a un proveedor de IA**. Ver 09 §5.

Discrepancia entre el registro y los entry points

`engine/action_registry.py::_BUILT_IN_ACTIONS` declara 36 acciones. `pyproject.toml`, sección `[project.entry-points."automa.actions"]`, declara 31. Las cinco que faltan en el `pyproject.toml`:

```
browser.capture_page
browser.crawl_site
browser.extract_content
browser.fill_form
http.check_urls
```

En la práctica no rompe nada, porque `LazyActionRegistry` consulta primero su diccionario interno y solo carga los entry points si no encuentra el nombre. Pero un paquete de terceros que inspeccione el grupo `automa.actions` verá un catálogo incompleto. Registrado en 15.

4. app/ — presentación

Archivo	Líneas	Responsabilidad	Estado
<code>server.py</code>	1 753	Panel HTML+CSS+JS embebido, API JSON, 10 rutas GET y 7 POST, autorización	Activo · crítico · voluminoso
<code>desktop.py</code>	114	Arranca el servidor en un hilo y abre la ventana <code>pywebview</code>	Activo
<code>__init__.py</code>	0	Marcador de paquete	Activo

`app/server.py` es el 29 % del Python de producción. Su estructura interna, por bloques:

Rango aproximado	Contenido
1–105	Imports, <code>_FOLDER_RE</code> , <code>_safe_folder</code> , <code>_is_preview</code> , <code>_resolve_under_root</code> , arranque del scheduler al importar
107–300	<code>_run_status_payload</code> : estado paso a paso de una corrida
300–730	Hoja de estilo del panel como cadena Python
734–1400	Funciones de renderizado: <code>html_page</code> , <code>badge</code> , tarjetas, historial, <code>_smart_summary</code> , detalle de corrida, dashboard de métricas
1401–1745	<code>AppHandler</code> : helpers de respuesta, autorización, <code>do_GET</code> , <code>do_POST</code>
1746–1753	<code>run_server</code>

Efecto secundario de importación (líneas 100–104): al importar el módulo se arranca el scheduler, se inicializa la base y se sincroniza el catálogo. Cualquier `import app.server` —incluido el de `tests/test_panel_endpoints.py`— levanta un hilo de scheduler.

5. flows/ — el catálogo declarativo

27 carpetas, cada una con `manifest.json`, `context.example.json` y `README.md`. Inventario completo verificado leyendo los 27 manifests:

Carpeta	id	Familia	Pasos	Acciones	allowed_actions	allowed_paths	max_runtime_seconds
01_screen_capture_analyze	screen_capture_analyze	pantalla	3	screen, vision, filesystem	—	—	—
02_screen_capture_browser	screen_capture_browser	navegador	1	browser.capture_page	—	—	—
03_folder_inventory	folder_inventory	filesystem	3	filesystem ×3	—	—	—
04_document_drop_pipeline	document_drop_pipeline	documentos	4	filesystem ×4	—	—	—
05_system_health_check	system_healthcheck	sistema	3	system, rules, filesystem	—	—	—
06_process_watchdog	process_watchdog	sistema	3	system ×2, filesystem	—	—	—
07_browser_form_filler	browser_form_filler	navegador	1	browser.fill_form	—	—	—
08_windows_lock_workstation	windows_lock_workstation	sistema	1	ui.hotkey	[OK]	—	5
09_show_desktop_capture	show_desktop_capture	pantalla	3	ui, system, screen	[OK]	[OK]	10
10_explorer_open_path	explorer_open_path	sistema	1	ui.launch_process	[OK]	—	5
11_settings_open_section	settings_open_section	sistema	1	ui.open_url	[OK]	—	5
12_desktop_ocr_inventory	desktop_ocr_inventory	pantalla	3	screen, vision, filesystem	[OK]	[OK]	30
13_notepad_quick_note	notepad_quick_note	sistema	3	ui, system, ui	[OK]	—	15
14_run_dialog_command	run_dialog_command	sistema	4	ui, system, ui ×2	[OK]	—	10
15_clipboard_capture	clipboard_capture	sistema	2	system, filesystem	[OK]	[OK]	5
16_active_window_screenshot	active_window_screenshot	pantalla	1	screen.capture_active_window	[OK]	[OK]	10
17_taskmgr_snapshots	taskmgr_snapshot	pantalla	5	ui, system, screen, vision, filesystem	[OK]	[OK]	30
18_powershell_audit	powershell_audit	sistema	2	system.run_powershell, filesystem	[OK]	[OK]	40
19_taskbar_capture	taskbar_capture	pantalla	1	screen.capture_region	[OK]	[OK]	10
20_volume_mute_toggle	volume_mute_toggle	sistema	1	ui.hotkey	[OK]	—	3
21_web_content_extractor	web_content_extractor	navegador	3	browser ×2, filesystem	—	—	—
22_web_site_map	web_site_map	navegador	2	browser.crawl_site, filesystem	—	—	—
23_web_change_detector	web_change_detector	navegador	4	browser, rules, notify, filesystem	—	—	—
24_web_link_audit	web_link_audit	navegador	4	browser, http, rules, filesystem	—	—	—

25_web_table_ext ract	web_table_extrac t	navegador	2	browser, filesystem	—	—	—
26_web_value_mon itor	web_value_monito r	navegador	4	browser, rules, notify, filesystem	—	—	—
27_web_page_arch ive	web_page_archive	navegador	3	browser ×2, filesyste m	—	—	—

Lectura del cuadro, y es el hallazgo más importante de este inventario:

- 13 de 27 flows declaran `allowed_actions` — exactamente los del bloque 08–20, la tanda añadida en la v0.2.0.
- 7 de 27 declaran `allowed_paths` — 09, 12, 15, 16, 17, 18, 19.
- 14 flows corren con `SandboxPolicy` completamente permisiva: los 01–07 y los 21–27. Entre ellos, `07_browser_form_filler` (lanza un navegador visible y escribe archivos) y los siete de la familia web (salen a internet si se les cambia la URL).
- Ningún flow declara `required_secrets`. El campo existe en el schema y el motor lo aplica, pero el catálogo no lo ejerce.
- Ningún flow está marcado `preview: true`. Los 27 son operativos.

Análisis de las consecuencias en [11 · Seguridad](#).

6. `decision/` — stubs sin uso

Archivo	Líneas	Símbolo	Estado
<code>rules.py</code>	9	<code>prioritize_steps(step_ids, context)</code> — devuelve la lista sin tocar	Muerto
<code>optional_ai.py</code>	9	<code>suggest_step_order(step_ids, context)</code> — devuelve la lista sin tocar	Muerto
<code>__init__.py</code>	0	Marcador de paquete	—

Verificado con búsqueda en todo el repositorio: **ningún módulo importa `decision`**, y ninguna de las dos funciones se invoca desde ningún sitio. Sin embargo, el paquete sí aparece en `[tool.hatch.build.targets.wheel].packages` y en la cobertura de `pytest` (`--cov=decision`), donde figura con **0 %**.

La docstring de `optional_ai.py` documenta la intención original: «Stub para futura integración IA. La IA solo debe sugerir orden o prioridad, nunca reemplazar la ejecución del motor». Es una decisión de diseño registrada, aunque el código nunca se llegó a escribir. Ver [15](#).

7. `plugins/analyzers/` — analizadores de imagen

Archivo	Líneas	Clase	Qué hace	Usado por	Estado
<code>vision_model_analyzer.py</code>	222	<code>VisionModelAnalyzer</code>	<code>mock</code> local, <code>openai_compatible</code> , <code>ollama</code>	Solo <code>vision.inspect_screenshot</code> , que ningún flow usa	Experimental · sin uso
<code>ocr_image_analyzer.py</code>	87	<code>OCRImageAnalyzer</code>	pytesseract con degradación explícita	<code>vision.ocr_image</code> , flows 12 y 17	Activo
<code>mock_image_analyzer.py</code>	40	<code>MockImageAnalyzer</code>	Brillo medio y estado visual con Pillow	<code>vision.analyze_image</code> por defecto, flow 01	Activo
<code>metadata_image_analyzer.py</code>	21	<code>MetadataImageAnalyzer</code>	Dimensiones, modo y SHA-256	Registrado en <code>ANALYZERS</code> , sin flow que lo pida	Disponible sin uso

base.py	9	AnalyzerProtocol	Contrato tipado	Los cuatro anteriores	Activo
---------	---	------------------	-----------------	-----------------------	--------

VisionModelAnalyzer se instancia al importar actions/vision.py (VISION_ANALYZER = VisionModelAnalyzer()), pero el constructor no hace nada y ningún método se llama sin pasar por inspect_screen_target. No hay tráfico de red implícito.

8. tests/ — 17 módulos, 150 pruebas

Archivo	Líneas	Qué cubre
test_browser_extract.py	358	Lógica pura de la familia web: links, hash, parseo numérico, tracking, markdown, CSV, BFS, robots
test_actions_basic.py	235	Acciones de filesystem, sistema, reglas, UI en modo dry_run
test_security_hardening.py	175	_safe_folder, path traversal en /file, allowlist de PowerShell, shell=True prohibido
test_orchestrator.py	139	Ejecución completa, transiciones, reintentos, condiciones
test_panel_endpoints.py	117	Rutas del panel y autorización
test_sandbox.py	83	SandboxPolicy en sus cuatro dimensiones
test_cron.py	57	Parseo y cálculo de próxima ejecución
test_secrets_and_notify.py	52	Resolución de secretos y backends de notificación
test_manifest_schema.py	52	Validación con y sin jsonschema
test_metrics.py	49	Agregados y formato Prometheus
test_loader.py	49	Manifest → dataclasses y precedencia del contexto
test_template.py	39	Placeholders exactos, compuestos y llaves literales
test_run_locks.py	36	Adquisición, liberación y liberación forzada
test_conditions.py	33	Los 13 operadores y los combinadores
test_action_registry.py	23	Registro estático y dinámico
conftest.py	28	Fixtures compartidas
__init__.py	0	Marcador de paquete

Cobertura medida por módulo y huecos priorizados en 12 · Pruebas y calidad.

9. scripts/ e installer/

Archivo	Responsabilidad	Estado
---------	-----------------	--------

<code>scripts/validate_project.py</code>	JSON Schema + acciones registradas + transiciones resueltas + <code>start_step</code> válido	Activo · gate de CI
<code>scripts/smoke_test.py</code>	Corre tres flows de punta a punta y comprueba la base	Activo · gate de CI
<code>scripts/build_docs_pdf.py</code>	Genera los PDF de esta documentación	Activo · añadido por este análisis
<code>installer/automa.spec</code>	Spec de PyInstaller: datafiles, <code>hiddenimports</code> , <code>COLLECT</code>	Activo · con hallazgo abierto
<code>installer/Automa.iss</code>	Script de Inno Setup: instalación por usuario, dos idiomas	Activo
<code>installer/automa_entry.py</code>	Entry point del bundle: fija <code>AUTOMA_ROOT</code> y hace <code>chdir(data_dir())</code>	Activo
<code>installer/build_local.ps1</code>	Ayuda de compilación local	Activo

Hallazgo en `installer/automa.spec`. Su lista `hiddenimports` enumera diez módulos de `actions/` pero **no incluye `actions.browser_extract`**, añadido en la v0.3.0. El registro carga las acciones con `import_module` dinámico, que PyInstaller no rastrea. **INFERENCIA:** los siete flows de la familia web (21–27) fallarían con `ModuleNotFoundError` dentro del `.exe`. **REQUIERE VALIDACIÓN:** confirmar compilando. Detalle en 15.

10. `.github/workflows/` — seis workflows

Workflow	Dispara	Qué hace	Estado
<code>ci.yml</code>	push y PR a <code>main</code>	Matriz 2 SO × 3 Python: ruff, <code>validate_project</code> , pytest. Job <code>smoke</code> aparte	Activo · gate
<code>security.yml</code>	push, PR, lunes 06:00 UTC, manual	CodeQL <code>security-extended</code> , <code>detect-secrets</code> sobre filesystem e historial de 50 commits, Trojan Source, ofuscación, <code>pip-audit</code>	Activo · gate
<code>workflow-security.yml</code>	push, PR, programado	<code>actionlint</code> con verificación de checksum, <code>zizmor</code> , <code>pin-check</code> con parser YAML real	Activo · gate
<code>markdown-docs.yml</code>	PR que toque <code>**/*.md</code>	Verifica que todos los enlaces relativos de todos los <code>.md</code> resuelvan	Activo · gate
<code>dependency-hygiene.yml</code>	PR, lunes 04:00 UTC, manual	<code>pip list --outdated</code> (informativo, <code>continue-on-error</code>)	Activo · informativo
<code>release.yml</code>	tag <code>v*.*.*</code> o manual	PyInstaller + Inno Setup en <code>windows-latest</code> , sube el <code>.exe</code> al release	Activo

`markdown-docs.yml` es el verificador que valida esta propia documentación: recorre `rglob('*.md')` y falla si algún enlace relativo apunta a un archivo inexistente.

11. Directorios de datos

Directorio	Versionado	Contenido	Ignorado por <code>.gitignore</code>
------------	------------	-----------	--------------------------------------

data/web/	[OK]	7 HTML de demo: <code>form_demo</code> , <code>control_page</code> , <code>demo_page</code> , <code>site_demo/</code> x4	—
data/seeds/	[OK] (<code>form_seeds.json</code>)	100 registros para el flow 07	<code>.used_indices.json</code> sí
data/inbox/, data/dropbox/inbox/	[OK]	Archivos de ejemplo para los flows 03 y 04	—
data/web_watch/	[NO]	Tracking de los flows 23 y 26	[OK]
configs/	[OK] (<code>03_folder_inventory.json</code>)	Contexto persistido desde el panel	—
db/	Solo <code>.gitkeep</code>	<code>runs.db</code>	<code>db/*.db</code>
logs/, state/	Solo <code>.gitkeep</code>	JSONL y snapshots	<code>logs/*.jsonl</code> , <code>state/*.json</code>
output/reports/, output/screenshots/	Solo <code>.gitkeep</code>	Salidas de los flows	<code>output/**/*</code> por extensión
secrets/	Solo <code>.gitkeep</code>	<code>secrets.json</code>	<code>secrets/*.json</code>

Carpeta que NO está y sorprende al clonar: `data/web_watch/`. Los flows 23 y 26 la necesitan para su tracking; la crean solas en la primera corrida (`apply_tracking` hace `path.parent.mkdir(parents=True, exist_ok=True)`), pero está en `.gitignore` y no existe en un clon limpio. No es un error, es diseño: la primera corrida establece la línea base y lo reporta con `first_run: true`.

12. Resumen del estado del código

Estado	Elementos
Activo y crítico	<code>engine/orchestrator.py</code> , <code>database.py</code> , <code>sandbox.py</code> , <code>loader.py</code> , <code>models.py</code> , <code>template.py</code> , <code>conditions.py</code> , <code>paths.py</code> , <code>action_registry.py</code> , <code>app/server.py</code>
Activo	El resto de <code>engine/</code> , las 26 acciones con flow, <code>app/desktop.py</code> , los tres analizadores en uso, los seis workflows
Disponible sin uso	10 acciones registradas sin flow, <code>MetadataImageAnalyzer</code>
Experimental sin uso	<code>VisionModelAnalyzer</code> y su único punto de entrada <code>vision.inspect_screen_target</code>
Muerto	<code>decision/rules.py::prioritize_steps</code> , <code>decision/optional_ai.py::suggest_step_order</code>
Duplicado	<code>engine/catalog.py::root_dir</code> frente a <code>engine/paths.py::root_dir</code>
No en runtime	<code>engine/manifest_schema.py</code> (solo lo usa el validador de CI)
No determinado	<code>installer/build_local.ps1</code> : no se ejecutó en este análisis

Documentos relacionados: 03 · Arquitectura · 05 · Referencia técnica · 06 · Explicación profunda · 15 · Riesgos · 19 · Matriz de trazabilidad

05 · Referencia técnica

Catálogo de consulta: las 36 acciones, las funciones relevantes del motor, los endpoints HTTP, los comandos, las variables de entorno, los operadores de condición y los errores del sistema. Para cada función relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién la llama, a quién llama y **riesgo al modificarla**.

1. Las 36 acciones registradas

Fuente: `engine/action_registry.py::_BUILT_IN_ACTIONS`. La columna **Flows** indica en cuántos de los 27 manifests aparece la acción.

1.1 `filesystem.*` — 7 acciones

Acción	Función	Firma	Retorno	Flows
<code>filesystem.ensure_directory</code>	<code>ensure_directory</code>	<code>(path: str)</code>	<code>{path, exists}</code>	0
<code>filesystem.list_directory</code>	<code>list_directory</code>	<code>(path: str, recursive: bool = False)</code>	<code>{path, files[], total_files}</code>	2
<code>filesystem.write_json</code>	<code>write_json</code>	<code>(path: str, data: Any)</code>	<code>{path, written}</code>	16
<code>filesystem.read_text_file</code>	<code>read_text_file</code>	<code>(path: str, max_chars: int = 4000)</code>	<code>{path, chars, preview}</code>	0
<code>filesystem.classify_file</code>	<code>classify_file_inventory</code>	<code>(files: list[dict])</code>	<code>{total_files, total_size_bytes, by_extension, largest_file}</code>	2
<code>filesystem.summarize_text</code>	<code>summarize_text_folder</code>	<code>(path: str, max_files: int = 10, max_chars_per_file: int = 800)</code>	<code>{path, processed_files, summaries[]}</code>	1
<code>filesystem.move_file</code>	<code>move_file</code>	<code>(source_path: str, destination_path: str, overwrite: bool = False)</code>	<code>{source, destination, moved}</code>	0

Excepciones: `list_directory`, `read_text_file`, `summarize_text_folder` y `move_file` levantan `FileNotFoundError` si la ruta no existe. `move_file` levanta `FileExistsError` si el destino existe y `overwrite=False`.

Efectos secundarios: `ensure_directory`, `write_json` y `move_file` crean el directorio padre con `mkdir(parents=True, exist_ok=True)`. `move_file` con `overwrite=True` hace `destination.unlink()` antes de mover: si el `shutil.move` falla después, el destino ya se perdió. `write_json` sobrescribe sin avisar.

Riesgo al modificar: `write_json` es la acción más usada del catálogo (16 de 27 flows). Cualquier cambio en su forma de retorno rompe el resumen del panel (`_smart_summary`) y la detección de outputs.

`summarize_text_folder` solo procesa extensiones `.txt`, `.md`, `.log`, `.csv`, `.json` y **corta en `max_files`** (10 por defecto). Un archivo fuera de esa lista se ignora en silencio: el resultado no dice cuántos se saltaron. `list_directory` solo devuelve **archivos**, nunca subcarpetas.

1.2 `screen.*` — 3 acciones

Acción	Función	Firma	Retorno	Flows
<code>screen.capture_screenshot</code>	<code>capture_screenshot</code>	<code>(output_path: str)</code>	<code>{image_path, width, height, method}</code>	4
<code>screen.capture_region</code>	<code>capture_region</code>	<code>(output_path: str, bbox: dict)</code>	<code>{image_path, width, height, bbox, method}</code>	1
<code>screen.capture_active_window</code>	<code>capture_active_window</code>	<code>(output_path: str)</code>	<code>igual + window_title</code>	1

Estrategia de respaldo: las tres intentan primero `mss` y, si falla, `Pillow (ImageGrab)`. El campo `method` del retorno dice cuál se usó. Si ambas fallan, levantan `RuntimeError` con el mensaje «No fue posible capturar la pantalla. Revisa si el entorno tiene escritorio gráfico disponible».

Formato del `bbox` (`_resolve_bbox`): acepta `{left, top, width, height}` o `{left, top, right, bottom}`. Valores negativos en `left/top` se interpretan **relativos al borde opuesto, estilo CSS** — así el flow 19 captura la barra de tareas con `{"left": 0, "top": -48, "width": 99999, "height": 48}`, donde el ancho desmedido se recorta solo al borde del monitor. Un `bbox` que produzca dimensiones ≤ 0 levanta `ValueError`.

`capture_active_window` resuelve el rectángulo con `pygetwindow.getActiveWindow()`. Si no hay ventana identificable (sesión sin foco) levanta `RuntimeError` con motivo legible. **Solo captura el monitor primario** (`sct.monitors[1]`): una ventana en un segundo monitor daría un recorte incorrecto. **REQUIERE VALIDACIÓN**.

Riesgo al modificar: cambiar el orden `mss` → `Pillow` alteraría el color de las capturas en algunos equipos; `mss` devuelve BGRA y `mss.tools.to_png` lo maneja.

1.3 `vision.*` — 5 acciones

Acción	Función	Firma resumida	Flows
<code>vision.analyze_image</code>	<code>analyze_image</code>	<code>(image_path: str, analyzer: str = 'mock')</code>	1
<code>vision.ocr_image</code>	<code>ocr_image</code>	<code>(image_path: str)</code>	2
<code>vision.find_text_in_image</code>	<code>find_text_in_image</code>	<code>(image_path: str, query: str, case_sensitive: bool = False)</code>	0
<code>vision.select_image</code>	<code>select_image</code>	<code>(image_path: str)</code>	0
<code>vision.inspect_screen_target</code>	<code>inspect_screen_target</code>	12 parámetros; ver §1.9	0

`analyze_image` despacha sobre el diccionario `ANALYZERS` (`mock`, `metadata`, `ocr`) y levanta `ValueError` si el nombre no existe. Las cinco levantan `FileNotFoundError` si la imagen no está.

`ocr_image` **no falla cuando falta OCR**: `OCRImageAnalyzer` comprueba `pytesseract` y el binario `tesseract` (incluidas las rutas típicas de Windows fuera del PATH) y devuelve `status: "unavailable"` con `reason` y una `summary` con instrucciones por sistema operativo. Es una degradación deliberada, documentada en el propio archivo: permite que un flow con rama de recuperación siga su camino alternativo.

Riesgo al modificar: `find_text_in_image` e `inspect_screen_target` dependen de la clave `matches` que produce `OCRImageAnalyzer`, con `left/top/width/height` por palabra. Cambiar esa forma rompe el encadenamiento OCR → `ui.click_bbox`.

1.4 `system.*` — 6 acciones

Acción	Función	Firma	Flows
<code>system.wait_seconds</code>	<code>wait_seconds</code>	<code>(seconds: float)</code>	4

<code>system.snapshot_system</code>	<code>snapshot_system</code>	<code>()</code>	1
<code>system.top_processes</code>	<code>top_processes</code>	<code>(limit: int = 10, sort_by: str = 'memory')</code>	1
<code>system.watch_processes</code>	<code>watch_processes</code>	<code>(processes: list[dict], memory_mb_threshold: float = 250.0, cpu_percent_threshold: float = 60.0)</code>	1
<code>system.read_clipboard</code>	<code>read_clipboard</code>	<code>(max_chars: int = 10000)</code>	1
<code>system.run_powershell</code>	<code>run_powershell</code>	<code>(command: str, allowlist: list[str] \ None = None, timeout_seconds: float = 30.0)</code>	1

`snapshot_system` mide CPU con `psutil.cpu_percent(interval=0.2)`: **bloquea 200 ms** en cada llamada. El disco se mide sobre `psutil.disk_usage("/")`, que en Windows resuelve a la unidad del directorio actual.

`top_processes` acepta `sort_by` con valor `"cpu"`; **cualquier otro valor ordena por memoria**, sin error ni aviso. Los procesos que lanzan `NoSuchProcess` o `AccessDenied` se saltan en silencio, así que `total_seen` puede ser menor que el número real de procesos del equipo.

`read_clipboard` **nunca lanza excepción**: si falta `pyperclip` o no hay backend de portapapeles devuelve `{available: False, reason: "...", text: "", length: 0}`. Cuando trunca lo declara con `truncated: True` y `length` con el tamaño real.

`system.run_powershell` — la acción de mayor superficie

```
def run_powershell(command: str,
                   allowlist: list[str] | None = None,
                   timeout_seconds: float = 30.0) -> dict[str, Any]
```

Dos controles en cascada, ambos **antes** de invocar PowerShell:

1. **Tokens prohibidos.** Si el comando contiene `;`, `|`, `&`, ```, `>`, `<`, `$`(o `$_` → `ValueError`. Cierra el encadenamiento y la redirección.
2. **Allowlist de verbos.** La **primera palabra** del comando debe coincidir exactamente (sensible a mayúsculas) con un elemento de la lista. Sin `allowlist` explícita se usa `_PS_DEFAULT_ALLOWLIST`, 13 verbos de solo lectura: `Get-Date`, `Get-Process`, `Get-Service`, `Get-ComputerInfo`, `Get-CimInstance`, `Get-WmiObject`, `Get-Disk`, `Get-Volume`, `Get-NetAdapter`, `Get-NetIPAddress`, `Get-EventLog`, `Get-Host`, `Get-Location`.

Se ejecuta con `subprocess.run(["powershell", "-NoProfile", "-NonInteractive", "-Command", trimmed], shell=False)`. Devuelve `stdout` recortado a 50 000 caracteres y `stderr` a 5 000, **declarando el recorte** con `stdout_truncated` / `stderr_truncated`.

Riesgo al modificar: alto. El parámetro `allowlist` es *overridable desde el manifest*. Un flow que declare `"allowlist": ["Remove-Item"]` obtiene exactamente eso. La seguridad de esta acción depende de que nadie escriba ese manifest, no de un control del motor. Ver 11 · Seguridad.

1.5 `ui.*` — 7 acciones

Acción	Función	Firma	<code>dry_run</code>	Flows
<code>ui.open_url</code>	<code>open_url</code>	<code>(url: str, new_tab: bool = True)</code>	[NO]	1
<code>ui.open_file_in_browser</code>	<code>open_file_in_browser</code>	<code>(path: str, new_tab: bool = True)</code>	[NO]	0

<code>ui.launch_process</code>	<code>launch_process</code>	<code>(command: str, wait_seconds: float = 0.0, shell: bool = False, dry_run: bool = False)</code>	[OK]	2
<code>ui.hotkey</code>	<code>hotkey</code>	<code>(keys: list[str], interval: float = 0.0, dry_run: bool = False)</code>	[OK]	6
<code>ui.type_text</code>	<code>type_text</code>	<code>(text: str, interval: float = 0.01, dry_run: bool = False)</code>	[OK]	2
<code>ui.click</code>	<code>click</code>	<code>(x, y, clicks=1, interval=0.0, button='left', dry_run=False)</code>	[OK]	0
<code>ui.click_bbox</code>	<code>click_bbox</code>	<code>(bbox: dict, clicks=1, interval=0.0, button='left', dry_run=False)</code>	[OK]	0

dry_run es el mecanismo de prueba de todo el bloque de UI. Con **dry_run=True** la función devuelve el payload que habría producido, con **sent/typed/clicked** en **False**, sin tocar teclado ni ratón. Es lo que permite que `tests/test_actions_basic.py` las ejecute sin efectos. Todos los flows de UI exponen **dry_run** en su `context.example.json`.

launch_process prohíbe **shell=True** explícitamente, con un comentario que explica la decisión:

```
# Se elimina deliberadamente la rama shell=True para cerrar el vector de
# command injection (CWE-78).
raise ValueError('launch_process: shell=True está deshabilitado por seguridad. ...')
```

El comando se tokeniza con `shlex.split` y se lanza con **shell=False**. **Nota para Windows:** `shlex` usa reglas POSIX, así que una ruta con contrabarras (`C:\Users\x`) puede tokenizarse de forma inesperada. Los flows 10 y 13 pasan rutas con barra normal.

`ui.hotkey`, `type_text`, `click` y `click_bbox` importan `pyautogui` dentro de la función (`_import_pyautogui`) y levantan `RuntimeError` con mensaje explicativo si no está disponible o el entorno no permite control de UI.

Riesgo al modificar: muy alto. Estas acciones mueven teclado y ratón sobre la sesión real del operador. Un cambio que ignore **dry_run** haría que la suite de tests empiece a teclear en la máquina de quien la ejecute.

1.6 http.* — 2 acciones

Acción	Función	Firma	Flows
<code>http.fetch_url</code>	<code>fetch_url</code>	<code>(url: str, output_path: str \ None = None, timeout: float = 15.0)</code>	0
<code>http.check_urls</code>	<code>check_urls</code>	<code>(urls: list[str], timeout: float = 10.0, max_urls: int = 100, delay_seconds: float = 0.0)</code>	1

`fetch_url` hace `raise_for_status()`: un 404 se convierte en excepción y el paso falla.

`check_urls` verifica **en orden de entrada** (determinista):

- `http/https` → **HEAD** con redirects; si el servidor devuelve 405 o 501, reintentará con **GET** en modo **stream** y lo cierra. `ok = response.ok`.

- `file://` → comprueba existencia del archivo local. Sirve para auditar los enlaces de las páginas de demo sin red.
- Otros esquemas (`mailto:`, `tel:`) → `skipped: True`, no cuentan como rotos.

Devuelve `truncated: True` si la lista superaba `max_urls`. **Nunca trunca en silencio.**

Riesgo al modificar: el flow 24 lee `broken_count` para decidir si alertar. Cambiar la semántica de `ok` (por ejemplo tratando un 301 como roto) cambiaría el comportamiento del flow sin tocar su manifest.

1.7 `notify.send` — 1 acción

```
def send_notification(message: str, backend: str = 'log', target: str | None = None,
                    token: str | None = None, extra: dict | None = None,
                    timeout: float = 10.0) -> dict[str, Any]
```

backend	Requiere <code>target</code>	Efecto
<code>log</code>	No	<code>print</code> a stdout
<code>file</code>	Sí (ruta)	Añade <code>timestamp\tmessage\n</code> al archivo, creando el directorio
<code>webhook</code>	Sí (URL)	POST JSON <code>{"text": message, "timestamp": ...}</code> más <code>extra</code>

Un `backend` desconocido levanta `ValueError`. Falta de `target` en `file` o `webhook` también.

Resolución de secretos: si `token` empieza por `@secret:`, el resto se resuelve con `engine.secrets.get_secret`. El token nunca se escribe en el resultado: `record` devuelve `backend`, `message`, `timestamp`, `sent`, `target` y `status_code`, **no el token**. Es un detalle deliberado y correcto, porque ese resultado va al contexto y de ahí a la columna `context_json` de SQLite.

Usada por los flows 23 y 26, ambos con `backend: "file"` por defecto.

1.8 `browser.*` — 4 acciones

Acción	Función	Módulo	Flows
<code>browser.capture_page</code>	<code>capture_page</code>	<code>actions/browser_capture.py</code>	3
<code>browser.fill_form</code>	<code>fill_form</code>	<code>actions/browser_form.py</code>	1
<code>browser.extract_content</code>	<code>extract_content</code>	<code>actions/browser_extract.py</code>	6
<code>browser.crawl_site</code>	<code>crawl_site</code>	<code>actions/browser_extract.py</code>	1

Las cuatro importan `playwright.sync_api.sync_playwright` dentro de la función y levantan `RuntimeError` con el comando de instalación si falta. Las cuatro aceptan una URL (`http`, `https`, `file`) o una **ruta local a un `.html`**, que `_to_url` convierte a `file://` tras comprobar que existe; si no existe y no es URL, `FileNotFoundError`.

`browser.capture_page`

```
capture_page(target, output_path, full_page=True, viewport_width=1280,
            viewport_height=800, wait_seconds=1.0, timeout_seconds=30.0)
```

Chromium **headless**. Devuelve `{image_path, url, title, width, height, full_page, size_bytes, method}`. `width/height` son los del `viewport solicitado`, no los del PNG resultante: con `full_page=True` la imagen es más alta. Detalle que conviene conocer al leer los reportes.

`browser.fill_form`

```
fill_form(target, seeds_path='data/seeds/form_seeds.json',
         used_path='data/seeds/used_indices.json', headless=False,
         slow_mo_ms=250, save_data_path=None, viewport_width=1280,
```

```
viewport_height=900, timeout_seconds=30.0)
```

headless=False por defecto: lanza una ventana de Chromium visible. Rellena nueve campos con `page.fill` más `#pais` con `select_option`, hace clic en `#btn-submit` y espera `#validation-result.show` durante 5 s.

Selectores acoplados al HTML de demo. Los nombres de campo están escritos en el código (`nombre`, `apellido`, `email`, `telefono`, `direccion`, `ciudad`, `fecha_nacimiento`, `profesion`, `comentario`, `pais`). Apuntar esta acción a otro formulario no funciona sin tocar Python. Es una limitación real del alcance actual.

Éxito determinado por texto: `is_success` se calcula como `validation_text.startswith('[OK]')` or `'válido' in validation_text.lower()`. Depende del texto que renderiza la página de demo.

Efecto secundario persistente: `_pick_record` escribe `data/seeds/.used_indices.json` en cada llamada, incluso si el llenado falla después. Cuando los 100 registros se agotan, el tracking se reinicia solo y lo declara con `cycle_resetted: True`.

Aleatoriedad: `random.choice` sin semilla. Es la única fuente de no-determinismo del sistema, y es intencional (el objetivo es no repetir registros).

`browser.extract_content`

```
extract_content(target, selector=None, parse_number=False, include_tables=False,
                max_links=200, max_text_chars=200_000, track_state_path=None,
                save_markdown_path=None, save_tables_dir=None,
                viewport_width=1280, viewport_height=800,
                wait_seconds=1.0, timeout_seconds=30.0)
```

La acción más versátil del repositorio: seis flows la usan con parámetros distintos. Devuelve siempre `url`, `title`, `text`, `text_chars`, `text_truncated`, `content_hash` (SHA-256 del texto normalizado), `links`, `link_urls`, `links_count`, `links_truncated`, `meta`, `method`. Con `selector` añade `selector_found`, `selector_value` y —si `parse_number`— `selector_number`. Con `include_tables` añade `tables` y `tables_count`. Con `track_state_path` añade `first_run`, `changed`, `previous_value`, `state_path`.

`browser.crawl_site`

```
crawl_site(start_url, max_pages=10, max_depth=2, same_domain_only=True,
            delay_seconds=0.5, respect_robots=True, viewport_width=1280,
            viewport_height=800, wait_seconds=0.5, timeout_seconds=30.0)
```

BFS determinista. Devuelve `pages[]` con `url`, `depth`, `title`, `links_count`, `text_chars`, `content_hash`, más `truncated`, `robots_blocked`, `errors` y `robots_checked_hosts`.

Riesgo al modificar: alto en `browser_extract.py`. Sus funciones puras (`normalize_text`, `content_hash`, `resolve_links`, `parse_number`, `apply_tracking`, `crawl_pages`, `build_result`, `render_markdown`, `save_tables_csv`) concentran 31 de los 150 tests. Cambiar `normalize_text` cambia todos los hashes ya guardados y haría que el flow 23 reporte un cambio falso en la primera corrida posterior.

1.9 `rules.evaluate` y la acción de visión avanzada

```
evaluate_rules(input_data: dict, rules: list[dict], default_status: str = 'no_match')
```

Evalúa las reglas **en orden y se detiene en la primera que coincide**. Cada regla lleva `path`, `operator` (`eq`, `ne`, `gt`, `lt`, `contains`, `in`), `value`, y opcionalmente `id`, `status`, `status_on_fail`, `message`. Devuelve `{status, matched_rule, evaluations[]}`. Un operador no soportado levanta `ValueError`.

Cuidado: dos motores de condición distintos. `actions/rules.py::_matches` soporta **6 operadores**; `engine/conditions.py::matches` soporta **13**. Los `when` de los pasos y transiciones usan el segundo; las reglas de `rules.evaluate` usan el primero. `gte`, `lte`, `exists`, `not_exists`, `truthy`, `falsy` y `regex` **no funcionan dentro de `rules.evaluate`**. Duplicación registrada en 15.

`vision.inspect_screen_target` combina OCR y visión multimodal con tres modos (`ocr`, `vision`, `hybrid`), un `prefer_source` y un `fallback_bbox`. Devuelve `target_found`, `target_bbox`, `selected_source`, `decision` (`click` o `recover`) y `diagnostics[]`. Los errores de cada fuente **no abortan**: se acumulan en `diagnostics`. Es el único camino hacia `VisionModelAnalyzer` y ningún flow lo usa. Ver 09 §5.

2. Funciones centrales del motor

Orchestrator.__init__(flow_dir, context_path=None, context_overrides=None)

- **Efectos secundarios:** llama a `init_db()` (crea `db/runs.db` y sus 7 tablas), lee el manifest y el contexto, construye la `SandboxPolicy`, genera el `run_id` y crea el `JsonLogger` y el `StateStore` —lo que **crea los directorios `logs/` y `state/`**.
- **run_id:** `datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')`, con microsegundos. Es la clave primaria de `runs`. **INFERENCIA:** una colisión exigiría dos corridas en el mismo microsegundo.
- **Llamada por:** `app/server.py::do_POST`, `engine/runner.py::main`, `engine/scheduler.py::_run_job`, `scripts/smoke_test.py`.
- **Riesgo al modificar:** **crítico.** Es el constructor de todo el sistema.

Orchestrator.run() -> dict

- **Retorno:** el `state` completo con `status` (`completed` o `failed`), `steps[]`, `route[]`, `outputs[]`, `duration_seconds`.
- **Excepciones:** `FlowExecutionError` en violación de sandbox, acción no registrada, paso inexistente, exceso de `max_steps_per_run`, exceso de `max_runtime_seconds`, o fallo del último reintento sin rama de recuperación.
- **Efectos secundarios:** escribe en SQLite (`runs`, `steps`, `events`), en `logs/*.jsonl` y en `state/*.json` **tras cada paso**.
- **Riesgo al modificar:** **crítico.** 139 líneas de bucle con siete puntos de salida.

SandboxPolicy.assert_paths_allowed(params)

```
def _path_strings(self, params: Any) -> Iterable[str]
```

Recorre los parámetros recursivamente y considera candidata a ruta **cualquier cadena cuya clave contenga** `path`, `destination`, `source`, `output` o `file`. Descarta las que contienen `{` (placeholders sin resolver). Resuelve con `Path.resolve()` y comprueba `relative_to` contra cada base de `allowed_paths`.

Consecuencia no obvia: el filtro es por **nombre de clave**. Un parámetro que sea una ruta pero se llame `target` (como `browser.extract_content(target=...)`) **no se comprueba**. Es coherente con el catálogo actual, pero un autor de flows debe saberlo. Registrado en 11 · Seguridad.

render_value(value, context)

Recursiva sobre dict, list y str. Para una cadena:

1. Convierte `{{ → { y }} → }`.
2. Aplana el contexto (`flatten_context` genera claves compuestas `a.b.c`).
3. Añade `now` con formato `%Y%m%d_%H%M%S`.
4. Si la cadena es **exactamente** un placeholder, devuelve el valor **con su tipo original** (int, bool, dict, `None`).
5. Si no, sustituye cada `{clave}` presente y **deja literal** la que no exista.

Los dos últimos puntos son la clave. Un manifest que escribe `"headless": "{ { headless } }"` recibe el booleano real, no la cadena `"True"`. Y una llave que no es placeholder (JSON embebido en un comando) queda intacta, a diferencia de `str.format_map`, que crashearía. El propio módulo lo documenta.

Riesgo al modificar: **crítico.** Los 27 manifests dependen de esta semántica.

evaluate_condition(condition, context) y matches(actual, operator, expected)

`evaluate_condition` soporta los combinadores `all`, `any`, `not` de forma recursiva, y en la hoja usa `path` + `operator` + `value`. **Una condición vacía o `None` devuelve `True`** — es lo que permite que un paso sin `when` se ejecute siempre.

Los 13 operadores de `matches`:

Operador	Semántica
----------	-----------

<code>eq / ne</code>	Igualdad / desigualdad
<code>gt / gte / lt / lte</code>	Comparación; <code>None</code> siempre da <code>False</code>
<code>contains</code>	<code>str(expected).lower() in str(actual).lower()</code> — sin distinguir mayúsculas
<code>in</code>	<code>actual in expected</code> , solo si <code>expected</code> es lista
<code>exists / not_exists</code>	<code>actual is not None / is None</code>
<code>truthy / falsy</code>	<code>bool(actual) / not bool(actual)</code>
<code>regex</code>	<code>re.search(str(expected), str(actual))</code>

Un operador desconocido levanta `ValueError`, que el orquestador convierte en fallo del paso.

`LazyActionRegistry.get(action_name) -> ActionFn | None`

Cachea por nombre. Si no está en `_action_paths`, carga los entry points del grupo `automa.actions` **una sola vez** y reintenta. Devuelve `None` si sigue sin aparecer, y el orquestador lo convierte en `FlowExecutionError('Acción no registrada: ...')`.

Riesgo al modificar: `register_callable` guarda un `dotted_path` falso (`<inline>:nombre`) y mete la función directo en caché. Es el mecanismo que usan los tests.

`engine.paths.root_dir()` y `data_dir()`

Función	Orden de resolución
<code>root_dir()</code>	<code>\$AUTOMA_ROOT</code> → <code>sys._MEIPASS</code> → carpeta padre del archivo
<code>data_dir()</code>	<code>\$AUTOMA_DATA_ROOT</code> → si no está congelado, <code>root_dir()</code> → <code>%LOCALAPPDATA%\Automa</code> (Windows) o <code>\$XDG_DATA_HOME/automa-pc</code>

`data_dir()` crea el directorio si no existe. Separar ambas raíces fue el arreglo de la v0.2.1 al `PermissionError` bajo `Program Files`.

3. Endpoints HTTP

Servidor: `ThreadingHTTPServer` en `127.0.0.1:8787`.

3.1 GET — sin autorización

Ruta	Respuesta	Notas
<code>/</code>	HTML	Panel de 3 pestañas
<code>/healthz</code>	<code>{"status": "ok"}</code>	Sonda de vida
<code>/metrics</code>	<code>text/plain; version=0.0.4</code>	Formato Prometheus
<code>/api/metrics</code>	<code>{"overview": ..., "by_flow": [...]}</code>	JSON completo
<code>/metrics/dashboard</code>	HTML	Dashboard visual
<code>/api/flows</code>	<code>[{folder, id, name, family, description, steps, flow_path}]</code>	Catálogo
<code>/api/runs?flow_id=&limit=50</code>	Lista de corridas	Incluye <code>context_json</code> completo

<code>/api/runs/<run_id>/status</code>	Estado paso a paso	Para el polling del panel
<code>/flow/<folder></code>	HTML	Ficha del flow
<code>/flow/<folder>/config</code>	HTML	Editor de contexto
<code>/flow/<folder>/history</code>	HTML	Últimas corridas
<code>/run/<flow_id>/<run_id></code>	HTML	Detalle de una corrida
<code>/file?path=<relativa></code>	Bytes del archivo	Con cuatro controles; ver §3.3

Ningún GET exige token, ni siquiera con `AUTOMA_PANEL_TOKEN` definido.

3.2 POST — con autorización

Ruta	Cuerpo	Respuesta	Autorización
<code>/api/run/<folder></code>	<code>{"context_overrides": {...}}</code> opcional	<code>{ok, run_id, status, flow_id}</code> — asíncrono	<code>_authorize_mutation</code>
<code>/api/hook/<folder></code>	—	<code>{ok, run_id, status, flow_id}</code> — síncrono	<code>AUTOMA_WEBHOOK_TOKEN</code> obligatorio
<code>/api/form/submit</code>	JSON libre	<code>{ok, saved_path}</code>	<code>_authorize_mutation</code>
<code>/run?flow=<folder></code>	Formulario	Redirección 303 al detalle	<code>_authorize_mutation</code>
<code>/flow/<folder>/config</code>	<code>config_json=...</code>	HTML	<code>_authorize_mutation</code>
<code>/flow/<folder>/schedule</code>	<code>enabled, interval_seconds, cron_expression</code>	Redirección 303	<code>_authorize_mutation</code>

Códigos de estado: 200 OK · 303 redirección · 400 folder o JSON inválido · 401 no autorizado · 404 flow o ruta inexistente · 409 flow en preview · 415 extensión no servida por `/file` · 500 fallo de ejecución.

3.3 Los cuatro controles de `/file`

En orden, tal como están en `do_GET`:

1. Rechaza cadenas con `\x00` o cualquier carácter de control (`ord(c) < 32`).
2. Rechaza rutas absolutas: el cliente pide siempre relativas a la raíz.
3. `os.path.normpath(os.path.join(base_path, rel))` debe empezar por `base_path + os.sep` — el `os.sep` cierra el bypass por prefijo hermano (`/repo-evil` frente a `/repo`).
4. Allowlist negativa de extensiones: `.html`, `.htm`, `.xhtml`, `.xml`, `.svg`, `.js`, `.mjs`, `.css` devuelven 415. Cierra XSS reflejado desde el mismo origen.

Además envía `X-Content-Type-Options: nosniff`. Cubierto por `tests/test_security_hardening.py`.

3.4 `_safe_folder` — la primera línea de defensa

```
_FOLDER_RE = re.compile(r'^[A-Za-z0-9_\-]{1,64}$')
```

Todo segmento `<folder>` de una URL pasa por aquí antes de tocar el catálogo o el filesystem. Rechaza `..`, separadores, NUL y no-ASCII. Se aplica en las siete rutas que reciben un folder.

4. Comandos

Comando	Qué hace	Código de salida
---------	----------	------------------

<code>automa list</code>	Catálogo en JSON	0, salvo <code>UnicodeEncodeError</code> en consola <code>cp1252</code>
<code>automa run <flow_dir> [--context RUTA]</code>	Ejecuta un flow e imprime el estado	0 / traza si falla
<code>automa scheduler [--interval N]</code>	Bucle del scheduler	No retorna
<code>automa-panel</code>	Panel HTTP	No retorna
<code>automa-desktop [--host --port --title --width --height --fullscreen]</code>	Ventana nativa	0 · 1 si el servidor no responde · 2 si falta <code>pywebview</code>
<code>automa-validate</code>	Valida los 27 manifests	0 si <code>ok</code> , 1 si hay errores
<code>python scripts/smoke_test.py</code>	Tres flows de punta a punta	0 / <code>AssertionError</code>
<code>python scripts/build_docs_pdf.py [--only N] [--check] [--out DIR] [--no-consolidado]</code>	Genera los PDF	0 / 1
<code>make test / lint / validate / smoke / list / run-panel / flow-health / clean</code>	Atajos	Según el comando

5. Variables de entorno

Variable	Leída por	Efecto si falta
<code>AUTOMA_PANEL_TOKEN</code>	<code>app/server.py::_authorize_mutation</code> vía <code>get_secret</code>	Se aplica el modo anti-CSRF por <code>Host/Origin/Referer</code>
<code>AUTOMA_WEBHOOK_TOKEN</code>	<code>app/server.py::_check_webhook_token</code>	<code>POST /api/hook/*</code> responde 401 siempre
<code>AUTOMA_ROOT</code>	<code>engine/paths.py::_root_dir</code>	Se usa la carpeta padre del paquete, o <code>sys._MEIPASS</code>
<code>AUTOMA_DATA_ROOT</code>	<code>engine/paths.py::_data_dir</code>	<code>root_dir()</code> en desarrollo, <code>%LOCALAPPDATA%\Automa</code> congelado
<code>OPENAI_API_KEY</code>	<code>VisionModelAnalyzer._analyze_openai_compatible</code>	Se envía la petición sin <code>Authorization</code> . Sin uso en el catálogo
<code>PYTHONIOENCODING</code>	Python	Recomendada en <code>utf-8</code> para el CLI en Windows

Ambos tokens se resuelven con `engine.secrets.get_secret`, así que también pueden vivir en `secrets/secrets.json` (ignorado por git) además de en el entorno.

`engine.secrets.list_secret_names()` expone los **nombres** conocidos —los del archivo, más las variables de entorno que empiecen por `AUTOMA_` o terminen en `_API_KEY` o `_TOKEN`— **sin exponer los valores**.

6. Campos del manifest

Contrato completo en `schemas/manifest.schema.json` (`additionalProperties: false` en la raíz y en cada paso: un campo desconocido es error de validación).

Campo	Tipo	Obligatorio	Por defecto
<code>id</code>	string, patrón <code>^[a-z0-9_]+\$</code>	[OK]	—

<code>name</code>	string no vacío	[OK]	—
<code>steps</code>	array con al menos 1 elemento	[OK]	—
<code>description</code>	string	—	""
<code>family</code>	string	—	"general"
<code>start_step</code>	string	—	Primer paso del array
<code>max_steps_per_run</code>	integer 1–10000	—	200
<code>allowed_actions</code>	array de strings únicos	—	null (sin restricción)
<code>required_secrets</code>	array de strings únicos	—	[]
<code>allowed_paths</code>	array de strings únicos	—	null (sin restricción)
<code>max_runtime_seconds</code>	number ≥ 0	—	null
<code>preview</code>	boolean	—	false

Campos de un paso: `id` [OK], `action` [OK], `params`, `save_as`, `retries` (0–100), `when`, `transitions`. **Campos de una transición:** `on` (`success`, `failure`, `any`), `next`, `end`, `when`.

7. Errores y excepciones

Excepción	Módulo	Cuándo
<code>FlowExecutionError</code>	<code>engine/orchestrator.py</code>	Violación de sandbox, acción no registrada, paso inexistente, exceso de pasos o de tiempo, fallo sin recuperación
<code>SandboxViolation</code>	<code>engine/sandbox.py</code>	Secreto ausente, acción bloqueada, ruta fuera de <code>allowed_paths</code>
<code>CronExpressionError</code>	<code>engine/cron.py</code>	Expresión con ≠ 5 campos, paso ≤ 0, rango fuera de límites, u horizonte de 4 años agotado
<code>FileNotFoundError</code>	Varias acciones	Ruta o imagen inexistente
<code>FileExistsError</code>	<code>filesystem.move_file</code>	Destino existente sin <code>overwrite</code>
<code>ValueError</code>	<code>rules</code> , <code>conditions</code> , <code>ui.launch_process</code> , <code>system.run_powershell</code> , <code>vision.analyze_image</code>	Operador o analizador desconocido, <code>shell=True</code> , token prohibido, verbo fuera de allowlist
<code>RuntimeError</code>	<code>screen</code> , <code>ui</code> , <code>browser.*</code> , <code>vision_model_analyzer</code>	Sin escritorio, sin <code>pyautogui</code> , sin Playwright, sin endpoint o modelo de visión
<code>UnicodeEncodeError</code>	<code>engine/runner.py::main</code>	Impresión de JSON no-ASCII en consola <code>cp1252</code> . Defecto conocido
<code>sqlite3.IntegrityError</code>	<code>engine/database.py::acquire_run_lock</code>	Capturada internamente: devuelve <code>False</code>

Kinds de error persistidos en `runs.error_json`: `sandbox_violation` (con `message` y, si aplica, `step_id`) o un objeto con `step_id` y `message` para los fallos de paso.

Eventos del log JSONL y de la tabla `events`: `flow_started`, `flow_finished`, `flow_blocked`, `step_started`, `step_finished`, `step_failed`, `step_skipped`, `step_blocked`, `step_recovered`.

Documentos relacionados: [04 · Mapa del código](#) · [06 · Explicación profunda](#) · [07 · Base de datos](#) · [09 · APIs e integraciones](#) · [10 · Configuración](#)

06 · Explicación profunda del código

Flujo interno módulo a módulo: qué entra, qué sale, qué decide, dónde falla y qué casos límite hay. Se explica por bloques lógicos donde la literalidad sería inútil, y línea a línea donde el detalle importa. Cada fragmento cita archivo y símbolo.

1. `engine/orchestrator.py` — el corazón

Objetivo. Convertir un `manifest.json` en una corrida trazable. **Entrada.** Carpeta del flow, ruta opcional de contexto, diccionario opcional de overrides. **Salida.** El diccionario `state` completo, y tres almacenes persistentes.

1.1 Construcción

```
def __init__(self, flow_dir, context_path=None, context_overrides=None):
    init_db()
    self.definition = FlowLoader.load_manifest(flow_dir)
    self.context = FlowLoader.load_context(flow_dir, context_path)
    if context_overrides:
        self.context = {**self.context, **context_overrides}
```

Cinco hechos que conviene tener presentes:

1. `init_db()` en el constructor. Crear un `Orchestrator` crea `db/runs.db` y sus siete tablas. No hay forma de instanciarlo sin tocar el disco. Es lo que permite que el CLI, el panel, el scheduler y el smoke test compartan inicialización sin coordinarse.
2. Los overrides son una fusión superficial. `{**base, **overrides}` reemplaza claves de primer nivel completas. Un override de `{"bbox": {"top": -48}}` borra el resto de claves de `bbox`. No hay fusión profunda.
3. La política se construye desde la definición, no desde el manifest crudo. El `FlowLoader` ya normalizó los campos, así que `SandboxPolicy.from_manifest` —que existe— no se usa aquí; el orquestador instancia `SandboxPolicy(...)` directamente con los campos de `FlowDefinition`.
4. El `run_id` se genera en el constructor, no en `run()`. Por eso el panel puede instanciar, persistir y devolver el `run_id` al cliente antes de que la corrida empiece.
5. `log_path` y `state_path` se construyen sobre `data_dir()`, no sobre `root_dir()`. Es lo que hace que el binario empaquetado escriba en `%LOCALAPPDATA%` en vez de en `Program Files`.

El `state` inicial trae 16 claves, incluidas `status: 'created'`, `steps: []`, `outputs: []`, `route: []` y `error: None`.

1.2 `run()` — bloque por bloque

Bloque 1 · secretos. Antes de nada, `self.policy.assert_secrets_present()`. Si falta alguno, el flow se marca `failed` con `kind: 'sandbox_violation'`, se persiste, se escribe el evento `flow_blocked` y se levanta `FlowExecutionError`. Es el único control que corre una vez por corrida y no por paso.

Bloque 2 · arranque. `status = 'running'`, `started_at`, `state['policy'] = policy.summary()`. Ese `summary()` incluye `permissive: bool`, así que el histórico registra si la corrida tenía sandbox o no. Es una decisión de auditabilidad excelente y poco visible: consultando `runs.context_json` se puede saber, corrida a corrida, cuáles corrieron sin restricción.

Bloque 3 · el bucle.

```
current_step_id = self.definition.start_step or (self.step_order[0] if self.step_order else None)
executed_count = 0
while current_step_id:
    if executed_count >= self.definition.max_steps_per_run:
        raise FlowExecutionError('Se alcanzó el máximo de pasos permitidos para esta corrida.')
    if current_step_id == '__END__':
        break
```

`__END__` es el centinela que devuelve `_resolve_transition` cuando una transición lleva `"end": true`. No es un paso real; nunca aparece en `steps_by_id`.

`max_steps_per_run` (200 por defecto) es la protección contra ciclos infinitos. Un flow con transiciones circulares no cuelga el proceso: se detiene con error tras 200 pasos. **Nota:** el contador cuenta *ejecuciones de paso*, no pasos distintos, así que un bucle

de dos pasos se corta a las 200 iteraciones.

Bloque 4 · condición del paso.

```
if step.when and not evaluate_condition(step.when, self.context):
    step_record = {..., 'status': 'skipped', 'result': {'reason': 'condition_not_met'}, ...}
```

Un paso saltado **sí se registra** en `state['steps']` y en la tabla `steps`, con `attempt: 0` y `duration_seconds: 0.0`. Es lo que permite al panel distinguir «no se ejecutó por condición» de «no llegó a ejecutarse». Después se resuelve la transición con evento `success` —un salto se considera éxito, no fallo— y `executed_count` **sí se incrementa**, contando contra `max_steps_per_run`.

Bloque 5 · los tres controles previos, en orden estricto.

```
self.policy.assert_action_allowed(step.action)          # 1
rendered_params = render_value(step.params, self.context)
self.policy.assert_paths_allowed(rendered_params)       # 2
action = ACTION_REGISTRY.get(step.action)
if action is None:
    raise FlowExecutionError(f'Acción no registrada: {step.action}')
if self.policy.max_runtime_seconds is not None:         # 3
    elapsed = (datetime.now(timezone.utc) - started_at).total_seconds()
    if elapsed > self.policy.max_runtime_seconds:
        raise FlowExecutionError(...)
```

El orden importa y está bien elegido: la acción se valida antes de renderizar (más barato), y las rutas se validan **después** de renderizar, porque un `{output_dir}` sin resolver no se puede comprobar. `assert_paths_allowed` además descarta explícitamente cualquier cadena que aún contenga `{`.

Caso límite del tiempo: `max_runtime_seconds` se compara con el tiempo transcurrido **antes** de lanzar el paso. Si un paso tarda diez minutos, el límite de 30 s no lo interrumpe: se detecta al ir a por el siguiente paso, y si era el último, la corrida termina `completed` habiendo excedido el límite. Es un control de *arranque de paso*, no un timeout real.

Bloque 6 · el bucle de reintentos.

```
attempts = 0
while attempts <= step.retries:
    attempts += 1
    try:
        result = action(**rendered_params)
        ...
        break
    except Exception as exc: # noqa: BLE001
        last_error = str(exc)
        self.context['_last_error'] = {'message': last_error, 'step_id': step.id}
        if attempts > step.retries:
            ...
```

Tres detalles no evidentes:

- **rendered_params** se calcula una sola vez, fuera del bucle. Un `{now}` en el nombre de archivo produce el mismo nombre en todos los reintentos.
- **No hay espera entre reintentos.** Tres intentos consecutivos contra un servidor caído ocurren en milisegundos. Para reintentos con espera hay que meter un paso `system.wait_seconds` en el manifest.
- **_last_error** queda en el contexto aunque el paso acabe teniendo éxito en un intento posterior. Una condición que consulte `_last_error` puede leer un error viejo.

Bloque 7 · la recuperación. Cuando se agotan los reintentos:

```
recovery_next = self._resolve_transition(step, event='failure')
if recovery_next and recovery_next != self._default_next(step.id):
    self.logger.write('step_recovered', {...})
    current_step_id = recovery_next
    self._persist()
    break
self.state['status'] = 'failed'
```

Esta comparación es todo el mecanismo de recuperación. `_resolve_transition` devuelve `_default_next` cuando ninguna transición coincide, así que la única forma de distinguir «hay rama de recuperación» de «no hay ninguna» es comprobar que el

destino sea distinto del siguiente por defecto.

Consecuencia sutil: un manifest que declare `{"on": "failure", "next": "<el paso siguiente>"}` —es decir, que quiera continuar por el camino normal tras un fallo— **no se recupera**, porque el destino coincide con el por defecto y la comparación falla. Hay que apuntar a un paso distinto. No está documentado en [docs/CREAR_FLUJOS.md](#).

Bloque 8 · cierre. El `except Exception` externo persiste el estado con `finished_at` y `duration_seconds` y escribe `flow_finished` antes de re-lanzar. Así, **incluso una corrida fallida deja duración medida y evento de cierre**. En el camino feliz, `status = 'completed'` y se devuelve el `state`.

1.3 `_resolve_transition`

```
for transition in step.transitions:
    if transition.on not in {event, 'any'}:
        continue
    if transition.when and not evaluate_condition(transition.when, self.context):
        continue
    if transition.end:
        return '__END__'
    if transition.next_step:
        return transition.next_step
return self._default_next(step.id)
```

Primera coincidencia gana. Una transición con `on` compatible y `when` cumplida pero **sin `next` ni `end`** no devuelve nada y el bucle sigue a la siguiente: es una transición inerte, y el validador no la detecta.

1.4 `_persist` y `_refresh_outputs`

```
def _persist(self):
    self._refresh_outputs()
    self.state_store.save(self.state)
    upsert_run(self.state, self.flow_dir.name, str(self.state_path), str(self.log_path))
```

Se llama tras **cada** paso. Cada llamada recorre todo el `state` buscando rutas de archivo (`extract_existing_paths`), reescribe el JSON completo de `state/` y hace un `INSERT ... ON CONFLICT DO UPDATE` en `runs`. Para un flow de tres pasos son tres recorridos completos y tres escrituras. **INFERENCIA:** en un flow de 200 pasos con un contexto grande, el costo de `_persist` domina el tiempo de ejecución.

2. `engine/template.py` — la sustitución de placeholders

Objetivo. Resolver `{clave}` y `{objeto.campos}` dentro de los `params` del manifest.

2.1 `flatten_context`

Convierte `{"a": {"b": 1}}` en `{"a": {"b": 1}, "a.b": 1}`. Conserva **ambas** claves, así que `{a}` devuelve el diccionario entero y `{a.b}` devuelve `1`.

2.2 La decisión clave: placeholder exacto frente a placeholder embebido

```
def render_value(value, context):
    if isinstance(value, str):
        prepared = value.replace("{", "{").replace("}", "}")
        flat = flatten_context(context)
        flat["now"] = datetime.now().strftime("%Y%m%d_%H%M%S")
        exact = _resolve_exact_placeholder(prepared, flat)
        if exact is not _MISSING:
            return exact
        return _substitute_placeholders(prepared, flat)
```

Si la cadena es **exactamente** un placeholder, se devuelve el valor **con su tipo original**. Por eso `"headless": "{ headless }"` en el manifest 07 llega a `fill_form` como el booleano `False`, no como la cadena `"False"`.

El centinela `_MISSING` distingue «la clave no existe» de «la clave existe y vale `None`». Sin él, un placeholder que resuelve a `None` caería al render de string y produciría `"None"` en vez de `null`. El módulo lo documenta.

2.3 Por qué no se usa `str.format_map`

```
def _replace(match):
    key = match.group(1).strip()
    if key in flat:
        return str(flat[key])
    return match.group(0) # queda literal
```

Dos razones, ambas en el docstring del módulo:

- 1. `str.format_map` interpreta el punto como **acceso a atributo**, así que `"{content.content_hash}"` lanzaría `AttributeError` sobre un dict. Con la sustitución manual sobre el contexto aplanado, funciona.
- 2. Una llave que no es placeholder —JSON embebido en un comando, por ejemplo— haría crashear a `format_map`. Aquí queda intacta.

El `CHANGELOG.md` de la v0.3.0 confirma que esto fue un cambio de comportamiento real: antes, un mensaje de `notify` con `{content.content_hash}` reventaba.

{now} es especial: se inyecta en cada llamada a `render_value`. Dentro de un mismo paso todos los `{now}` coinciden, pero **entre pasos pueden diferir** si el segundo cambia. Un flow que escriba una imagen en el paso 1 y la referencie en el paso 2 con otro `{now}` fallaría. Los flows del repositorio evitan el problema pasando la ruta por el contexto con `save_as`.

3. engine/sandbox.py — la política

3.1 Las cuatro dimensiones

Método	Cuándo	Qué comprueba
<code>assert_secrets_present</code>	Una vez, al arrancar	Que cada nombre de <code>required_secrets</code> esté en <code>os.environ</code> y no vacío
<code>assert_action_allowed</code>	Antes de cada paso	Que <code>step.action</code> esté en <code>allowed_actions</code> , si la lista existe
<code>assert_paths_allowed</code>	Antes de cada paso, sobre params renderizados	Que toda ruta candidata caiga bajo alguna base
<code>max_runtime_seconds</code>	Entre pasos, en el orquestador	Tiempo total transcurrido

`check_required_secrets` lee **solo** `os.environ`, no `engine.secrets.get_secret`. Un secreto que viva únicamente en `secrets/secrets.json` **no satisface** `required_secrets`. Es una inconsistencia entre dos mecanismos que se presentan como equivalentes. **NO DOCUMENTADO EN EL REPOSITORIO.**

3.2 `_path_strings` — la heurística de detección de rutas

```
if any(token in key.lower() for token in ('path', 'destination', 'source', 'output', 'file')):
    if isinstance(value, str):
        yield value
    yield from self._path_strings(value)
```

Detecta rutas **por el nombre de la clave**, no por la forma del valor. Consecuencias verificables:

- [OK] `output_path`, `save_data_path`, `seeds_path`, `destination_path`, `state_path`, `track_state_path` → se comprueban.
- [NO] `target` de `browser.extract_content` y `browser.capture_page` → **no se comprueba**, aunque puede ser una ruta local a un `.html`.
- [NO] `command` de `ui.launch_process` → **no se comprueba**, aunque contiene una ruta.

En el catálogo actual no genera un agujero, porque los siete flows con `allowed_paths` no usan esas acciones. Pero es una limitación que un autor de flows debe conocer. Registrada en [11 · Seguridad](#).

Además, `yield from self._path_strings(value)` recorre el valor **aunque la clave ya haya coincidido**, lo que significa que un dict anidado bajo una clave `path` se recorre dos veces. Sin consecuencia funcional, solo trabajo duplicado.

3.3 `is_permissive` y `summary`

`is_permissive()` devuelve `True` cuando las cuatro dimensiones están vacías. `summary()` lo incluye, y el orquestador lo guarda en `state['policy']`. Así queda registrado en el histórico si la corrida tenía o no restricciones. Para los 14 flows sin política, el histórico dirá `permissive: true`.

4. `engine/conditions.py` — la evaluación

4.1 `get_path` navega con tolerancia

```
for part in path.split('.'):
    if isinstance(current, dict):
        current = current.get(part, default)
    else:
        return default
```

Un camino que atraviesa una lista devuelve `default` (`None`). **No hay acceso por índice**: `steps.0.status` no funciona. Es una limitación real del lenguaje de condiciones.

4.2 Los operadores y sus trampas

- `gt`, `gte`, `lt`, `lte` comprueban `actual is not None` antes de comparar, así que nunca lanzan `TypeError` por comparar con `None`. Pero **sí lo lanzarían** al comparar una cadena con un número (`"80" > 70`). Un contexto que traiga el umbral como texto falla en tiempo de ejecución.
- `contains` normaliza a minúsculas **ambos lados**: `str(expected).lower() in str(actual).lower()`. Es una búsqueda insensible a mayúsculas por diseño.
- `in` devuelve `False` si `expected` no es una lista, sin avisar.
- `regex` usa `re.search`, no `re.fullmatch`: basta con que el patrón aparezca en algún punto.

4.3 Los combinadores

`all` sobre lista vacía devuelve `True`; `any` sobre lista vacía devuelve `False`. Es la semántica de Python y probablemente la esperada, pero un manifest con `{"all": []}` como guarda deja pasar siempre.

5. `engine/loader.py` — del JSON a las dataclasses

5.1 `load_manifest`

Lee el JSON **sin validarlo contra el schema**. Los campos obligatorios se acceden con corchetes (`raw['id']`, `raw['steps']`, `step['id']`, `step['action']`), así que un manifest incompleto produce un `KeyError` crudo, no un mensaje del validador.

`allowed_actions` y `allowed_paths` usan `if raw.get(...)` en vez de `is not None`, de modo que **una lista vacía se convierte en `None`**, es decir, en «sin restricción». Escribir `"allowed_actions": []` con la intención de bloquear todo produce el efecto contrario: política permisiva. Es una trampa real. Registrada en 15.

5.2 `load_context` — la precedencia

```
candidates = []
if explicit_context_path:
    candidates.append(explicit_context_path)
candidates.extend([
    Path('configs') / f'{flow_dir.name}.json',
    flow_dir / 'context.user.json',
    flow_dir / 'context.example.json',
])
for candidate in candidates:
    if candidate.exists():
```

```

        return json.load(fh)
    return {}

```

La primera que exista gana y las demás se ignoran por completo. No hay mezcla de claves. Editar la configuración desde el panel crea `configs/<folder>.json`, que a partir de ese momento **oculta** el `context.example.json`: si una versión posterior del flow añade una clave nueva al ejemplo, el flow configurado no la verá y el parámetro llegará sin resolver.

Trampa adicional: `Path('configs')` es una ruta **relativa al directorio de trabajo**, no a `root_dir()` ni a `data_dir()`. Ejecutar `automa run` desde otra carpeta hace que `configs/` no se encuentre. Contrasta con `set_flow_config`, que sí escribe en `data_dir() / 'configs'`. **Son dos rutas distintas** cuando el binario corre congelado: el panel escribiría en `%LOCALAPPDATA%\Automa\configs` y el loader leería `./configs`. Registrado en 15.

6. engine/database.py — la persistencia

6.1 El patrón de conexión

```

@contextmanager
def connect():
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()

```

Conexión nueva por operación, `commit` solo si no hubo excepción, `close` siempre. Evita el problema de compartir conexiones entre hilos. El costo es abrir el archivo en cada consulta.

NO IDENTIFICADO: no se configura `journal_mode=WAL` ni `busy_timeout`. Con el panel lanzando corridas en paralelo y el scheduler escribiendo a la vez, `database is locked` es posible. **REQUIERE VALIDACIÓN** bajo carga.

6.2 init_db y la migración

`executescript` con siete `CREATE TABLE IF NOT EXISTS`. Después, una única migración:

```

_ensure_column(conn, 'schedules', 'cron_expression', 'cron_expression TEXT')

```

`_ensure_column` consulta `PRAGMA table_info` y añade la columna solo si falta. Es toda la maquinaria de migración del proyecto: suficiente para el cambio que hubo, insuficiente para un cambio de tipo o un renombrado.

6.3 acquire_run_lock — el lock que casi nadie usa

```

try:
    conn.execute('INSERT INTO run_locks(folder, run_id, acquired_at) VALUES(?,?,?)', ...)
    return True
except sqlite3.IntegrityError:
    return False

```

La exclusión mutua se apoya en la **clave primaria folder** de `run_locks`: el segundo insert viola la restricción y se traduce en `False`. Es correcto y elegante.

Pero solo lo usa SchedulerService._run_job. Verificado buscando en todo el repositorio: `POST /api/run`, `POST /run`, `POST /api/hook` y el CLI **no adquieren el lock**. El panel sí permite lanzar el mismo flow dos veces en paralelo.

Y no hay liberación al arrancar. Si el proceso muere con un lock tomado, la fila queda para siempre y el scheduler no volverá a disparar ese flow. El `RUNBOOK.md` documenta la liberación manual con `force_release_lock`, lo que confirma que el escenario ocurre.

6.4 set_flow_config escribe en dos sitios


```
config_path = data_dir() / 'configs' / f'{folder}.json'
config_path.write_text(json.dumps(config, ensure_ascii=False, indent=2), encoding='utf-8')
conn.execute('INSERT INTO flow_configs(...) ON CONFLICT(folder) DO UPDATE ...')
```

Archivo y tabla. El archivo es lo que lee `FlowLoader.load_context`; la tabla es lo que lee `get_flow_by_folder` para mostrarlo en el panel. **No hay transacción que abarque ambos**: si el `write_text` tiene éxito y el `INSERT` falla, quedan desincronizados.

`write_text` sin `newline='\n'` produce CRLF en Windows. Como `configs/03_folder_inventory.json` está versionado y el smoke test lo reescribe, ejecutar el smoke test puede dejar el árbol de trabajo marcado como modificado.

7. actions/browser_extract.py — el módulo mejor diseñado

567 líneas divididas en dos mitades con una frontera explícita, documentada en el propio docstring: «la interacción con Playwright vive en `scrape_page/extract_content/crawl_site` (glue mínimo), y toda la lógica son funciones puras testeables sin navegador».

7.1 normalize_text — por qué el hash es estable

```
lines = [line.strip() for line in (text or '').splitlines()]
collapsed = []
for line in lines:
    if line or (collapsed and collapsed[-1]):
        collapsed.append(line)
while collapsed and not collapsed[-1]:
    collapsed.pop()
return '\n'.join(collapsed)
```

Recorta cada línea, colapsa líneas vacías consecutivas en una sola y quita las finales. Sin esto, un cambio de espaciado en el HTML cambiaría el SHA-256 y el flow 23 reportaría un cambio falso.

Riesgo de tocarla: cualquier modificación invalida **todos** los hashes ya guardados en `data/web_watch/*.json`. La primera corrida posterior reportaría `changed: true` en todas las páginas vigiladas.

7.2 parse_number — la heurística documentada

```
if ',' in cleaned and '.' in cleaned:
    dec = max(cleaned.rfind(','), cleaned.rfind('.')) # el separador más a la derecha
elif ',' in cleaned or '.' in cleaned:
    if len(parts) == 2 and (len(parts[1]) != 3 or parts[0] in ('', '0')):
        value = float(...) # es decimal
    else:
        value = float(''.join(parts)) # es separador de miles
```

Convierte `$1,499.90` y `1.499,90 €` al mismo `1499.9`. La regla con un solo separador: si detrás hay **exactamente tres dígitos**, se asume separador de miles, **salvo** que la parte entera sea vacía o 0.

Casos límite reales: `"1.234"` → `1234.0` (interpretado como miles). Si el valor vigilado fuera realmente `1.234` como decimal, el flow 26 leería `1234.0` y cruzaría cualquier umbral. La heurística está documentada en el docstring; es una decisión, no un descuido, pero un usuario debe conocerla.

7.3 resolve_links — orden y deduplicación

Preserva el orden de aparición en el DOM, deduplica por URL absoluta, descarta anclas (`#`), `javascript:` y `data:`, y **conserva** `mailto:` y `tel:` sin resolver contra la base. Corta en `max_links` y devuelve `truncated`. `urldefrag` elimina el fragmento, de modo que `pagina#a` y `pagina#b` cuentan como el mismo enlace.

7.4 apply_tracking — el detector de cambios

```
first_run = previous is None
previous_value = (previous or {}).get('watch_value')
changed = (not first_run) and previous_value != watch_value
```

La primera corrida nunca reporta cambio, aunque el archivo de estado no exista. Establece la línea base y devuelve `first_run: True`. Es lo que evita una notificación espuria al estrenar el flow.

Un archivo de estado corrupto se trata como inexistente (`except (json.JSONDecodeError, OSError): previous = None`), lo que reinicia la línea base **en silencio**: el flow diría `first_run: true` otra vez sin explicar por qué. Es el único punto del módulo donde una degradación no se declara.

7.5 `crawl_pages` — BFS acotado

```
queue = deque([(start_url, 0)])
seen = {start_url}
while queue and len(pages) < max_pages:
    url, depth = queue.popleft()
    if robots_check is not None and not robots_check(url):
        robots_blocked.append(url); continue
    try:
        raw = fetch_page(url)
    except Exception:
        errors.append({'url': url, 'error': str(exc)}); continue
```

Cuatro decisiones visibles:

1. **Un enlace caído no aborta el crawl**. Se registra en `errors` y se sigue.
2. **`seen` se marca al encolar, no al visitar**, evitando duplicados en la cola.
3. **El filtro de dominio compara `netloc` exacto**. `www.example.com` y `example.com` son dominios distintos.
4. **`truncated` es `bool(queue)`**: dice si quedaron URLs sin visitar al alcanzar el tope. Honesto.

`robots_check` se aplica **antes** de la petición, incluida la URL inicial: un `robots.txt` que prohíba la raíz produce un crawl de cero páginas con `robots_blocked` de uno.

7.6 `RobotsCache` — el fallo permisivo declarado

```
except requests.RequestException:
    self.checked_hosts[host] = False
    return None          # parser None -> allows() devuelve True
```

Si `robots.txt` no se puede leer, **se permite el acceso**, pero se registra `checked=False` en `robots_checked_hosts`, que va al reporte del flow 22. El docstring lo declara: «se permite el acceso pero se registra `checked=False` para reportarlo con honestidad». Un 4xx sí cuenta como comprobado (no hay reglas) y también permite.

7.7 `crawl_site` y la recuperación de página rota

```
def fetch_page(page_url):
    nonlocal page
    try:
        page.goto(page_url, wait_until='load')
    except Exception:
        try: page.close()
        except Exception: pass
        page = new_page()
        raise
```

Un `goto` fallido deja la pestaña de Playwright en estado de error que rompería la navegación siguiente. La solución: cerrar la pestaña, abrir una limpia y **propagar** el error para que `crawl_pages` lo registre. El comentario del código lo explica. Es el tipo de detalle que solo aparece tras ejecutar el crawl contra un sitio real.

8. `actions/browser_form.py` — la operación más compleja

8.1 `_pick_record` — tracking sin repetición

```
used = _load_used_ids(used_path)
available_ids = [r['id'] for r in records if r['id'] not in used]
if not available_ids:
    used = set(); available_ids = [r['id'] for r in records]; cycle_resetted = True
chosen_id = random.choice(available_ids)
used.add(chosen_id)
```

```
_save_used_ids(used_path, used, len(records))
```

El registro se marca como usado antes de intentar el llenado. Si el navegador falla después, ese registro se pierde para el ciclo. Es una decisión conservadora —prefiere saltarse un registro a repetirlo— pero no está documentada en el README del flow.

El reinicio automático al agotar los 100 se declara con `cycle_resetted: True` en el resultado.

8.2 El llenado

```
for field in ('nombre', 'apellido', 'email', 'telefono', 'direccion',
             'ciudad', 'fecha_nacimiento', 'profesion', 'comentario'):
    page.fill(f'#{field}', form_data[field])
page.select_option('#pais', form_data['pais'])
```

Nueve `fill` más un `select_option`. Los identificadores están **escritos en Python**, no en el manifest: la acción solo sirve para el formulario de demo o para uno con exactamente los mismos `id`. Es la limitación de alcance más importante del flow más vistoso del repositorio.

Un `KeyError` si el registro semilla no trae alguna de las diez claves; el orquestador lo convierte en fallo de paso.

8.3 La validación

```
page.wait_for_selector('#validation-result.show', timeout=5000)
...
except Exception:
    validation_text = '(timeout esperando #validation-result.show)'
```

El timeout **no falla el paso**: deja constancia textual y sigue. Después:

```
is_success = validation_text.startswith('[OK]') or 'válido' in validation_text.lower()
```

El éxito se decide comparando emoji y texto en español. Depende por completo del HTML de demo. Si la página cambia su copy, `is_success` pasa a `False` sin que nada más falle.

9. `engine/cron.py` — el planificador propio

Objetivo. Calcular la próxima ejecución de una expresión de 5 campos sin dependencias.

9.1 Lo que soporta y lo que no

Soporta `*`, números, listas (`1,3,5`), rangos (`9-17`) y pasos (`*/5`, `0-30/10`). **No soporta** nombres simbólicos (`MON`, `JAN`) ni `L/W/#`. El docstring lo declara y explica la decisión: «Es deliberadamente pequeño: cubre los casos comunes [...] sin meter una dependencia externa».

9.2 El día de la semana no es el estándar

```
(0, 6),      # day of week (0=lunes, 6=domingo, estilo ISO)
...
def _iso_weekday(dt): return dt.weekday()
```

En cron estándar, **0 es domingo**. Aquí **0 es lunes**. Una expresión copiada de `crontab(5)` se ejecutará el día equivocado. Está documentado en el comentario del código, pero **no** en `docs/OPERACION.md` ni en la ayuda del panel. Registrado en [15](#).

9.3 El avance por saltos

```
for _ in range(60 * 24 * 366 * 4):    # ≈ 4 años en minutos
    if candidate.month not in months.values:
        candidate = datetime(year, month, 1, 0, 0, tzinfo=timezone.utc); continue
    if candidate.day not in days.values:
        candidate = (candidate + timedelta(days=1)).replace(hour=0, minute=0); continue
    ...
```

En vez de avanzar minuto a minuto, salta al inicio del mes, del día o de la hora siguientes cuando el campo no coincide. Reduce drásticamente las iteraciones. El límite de cuatro años convierte una expresión imposible (`0 0 30 2 *`, 30 de febrero) en `CronExpressionError` en vez de un cuelgue.

Combinación día-del-mes / día-de-la-semana: el código exige que **ambos** coincidan (AND). Cron estándar usa OR cuando ninguno es `*`. Otra divergencia con `crontab(5)`.

9.4 Todo en UTC

`next_after` trabaja siempre en UTC. Una expresión `0 9 * * 1` se dispara a las 09:00 UTC, no a las 09:00 locales. Para un operador en UTC-3 eso son las 06:00. **NO DOCUMENTADO EN EL REPOSITORIO.**

10. engine/scheduler.py — el bucle de disparo

```
def run_pending_once(self):
    for schedule in list_schedules():
        if self._should_run(schedule):
            thread = threading.Thread(target=self._run_job, args=(...), daemon=True)
            thread.start()
```

Cada tarea vencida arranca su propio hilo daemon. `_should_run` compara `next_run_at` con `now` y exige `enabled`. Una tarea sin `next_run_at` nunca se dispara.

`_run_job` adquiere el lock, ejecuta y **libera el lock en `finally`**, de modo que una excepción no deja el lock tomado *dentro del proceso vivo*. Lo que sí lo deja es matar el proceso a mitad.

El error se traga por completo:

```
try:
    Orchestrator(Path(flow['flow_path'])).run()
except Exception:
    pass # "No interrumpe el loop. El error queda en la corrida."
```

Y `mark_schedule_run` se llama **después del `except`**, es decir, **también cuando el flow falla**. Consecuencia: una tarea programada que falla siempre reprograma su siguiente ejecución con normalidad y no genera ninguna señal. Solo se descubre mirando el histórico.

11. app/server.py — el panel

11.1 Los efectos de importar el módulo

```
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Cuatro efectos a nivel de módulo: hilo de scheduler vivo, base creada, catálogo sincronizado. Cualquier `import app.server` los dispara, incluido el de los tests.

11.2 `_is_preview` — dos mecanismos

```
if (flow_path / '.disabled').exists():
    return True
manifest = json.loads(...)
return bool(manifest.get('preview'))
```

El archivo marcador `.disabled` permite desactivar un flow **sin tocar el manifest ni commitear**. Es un detalle operativo útil y poco visible. Un manifest ilegible devuelve `False` (flow operativo), decisión discutible: un manifest corrupto se considera ejecutable.

Ningún flow del catálogo está en preview.

11.3 `_run_status_payload` — pending frente a not_taken

```
fallback_status = 'pending' if is_running else 'not_taken'
```

Un paso del manifest sin registro en **steps** significa cosas distintas según el estado de la corrida: si sigue viva, está **pendiente**; si terminó, es una **rama no tomada**. El comentario del código lo explica. Es el detalle que hace que el panel no muestre pasos «pendientes» eternos en corridas ya cerradas.

11.4 `/api/run/<folder>` — la ejecución asíncrona

```
orch = Orchestrator(Path(flow['flow_path']), context_overrides=overrides or None)
orch.state['status'] = 'running'
orch._persist() # sincrónico, antes del hilo
...
threading.Thread(target=_runner, daemon=True).start()
return self._send_json({'ok': True, 'run_id': run_id, ...})
```

El comentario explica el porqué: «Persistimos sincrónicamente antes de lanzar el thread para garantizar que el polling vea el run desde el primer tick». Sin eso, el navegador podría pedir `/api/runs/<run_id>/status` antes de que la fila existiera.

Nótese que se llama a `orch._persist()`, un método privado, desde fuera de la clase. Es acoplamiento con el interior del orquestador.

11.5 `_authorize_mutation` — el modelo de dos modos

Explicado en 03 §8 y analizado en 11 · Seguridad. El punto de código que importa aquí:

```
if hmac.compare_digest(provided, expected)
```

Comparación en tiempo constante, que cierra CWE-208 (fuga por temporización). Está bien hecho y el comentario lo cita explícitamente.

12. `engine/introspection.py` — qué cuenta como salida

```
if len(value) > 260 or '\n' in value:
    return
try:
    candidate = Path(value)
    if not candidate.exists() or not candidate.is_file(): return
    resolved = candidate.resolve()
    resolved.relative_to(output_root)
except (ValueError, OSError):
    return
```

Tres filtros con motivo explicado en comentarios del propio archivo:

1. **Longitud > 260 o con salto de línea** → no es una ruta. El comentario razona: los paths reales rara vez pasan de 260 caracteres en Windows, y llamar a `Path.exists()` sobre una cadena larguísima produce `OSError ENAMETOOLONG` en Linux.
2. **Debe existir y ser archivo.**
3. **Debe estar bajo `output/`.** El comentario explica que antes se aceptaba cualquier ruta existente y eso contaminaba la lista con archivos que el flow solo había leído.

`_output_root()` usa `Path('output').resolve()`, **relativo al directorio de trabajo**. Por eso `installer/automa_entry.py` hace `os.chdir(data_dir())` en el bundle: sin ese `chdir`, los outputs no se detectarían.

13. `plugins/analyzers/ocr_image_analyzer.py` — degradación bien hecha

```
@staticmethod
def _tesseract_binary_available() -> bool:
    if shutil.which('tesseract'): return True
    for candidate in ('C:/Program Files/Tesseract-OCR/tesseract.exe',
                     'C:/Program Files (x86)/Tesseract-OCR/tesseract.exe'):
        if Path(candidate).exists(): return True
    return False
```

Busca en el PATH y, además, en las dos rutas donde el instalador de Windows deja el binario sin añadirlo al PATH. Es un detalle que ahorra un caso de soporte muy frecuente.

Cuando falta, devuelve `status: 'unavailable'` con `reason` (`pytesseract_missing` o `tesseract_binary_missing`) y una `summary` con el comando de instalación por sistema operativo. El flow continúa; los pasos siguientes recibirán `matches: []`.

La extracción de coordenadas usa `pytesseract.image_to_data` con `output_type=Output.DICT` y descarta las entradas con texto vacío, quedándose con `text`, `conf`, `left`, `top`, `width`, `height` por palabra. Ese formato es el que consume `ui.click_bbox`.

14. `scripts/validate_project.py` — el gate

Cinco comprobaciones por manifest, en orden:

1. **JSON Schema** (o el respaldo estructural si falta `jsonschema`). Si falla, se **detiene ahí** para ese flow: no tiene sentido comprobar acciones sobre un manifest inválido.
2. **IDs de paso duplicados**.
3. **Acción registrada** en `ACTION_REGISTRY.keys()`.
4. **Acción dentro de `allowed_actions`**, si el manifest lo declara. Este control es valioso: impide publicar un flow cuya política se contradiga con sus pasos.
5. **`transitions.next` y `start_step` apuntan a pasos existentes**.

Devuelve JSON con `ok`, `flows_checked`, `registered_actions` y `errors[]`, y sale con código 1 si hay errores. Es el gate que corre la CI en las seis combinaciones de la matriz.

Lo que NO comprueba: que los `params` de cada paso coincidan con la firma de la acción. Un `screen.capture_screenshot` sin `output_path` pasa la validación y falla en ejecución con `TypeError`. Registrado en 12.

Documentos relacionados: 03 · [Arquitectura](#) · 04 · [Mapa del código](#) · 05 · [Referencia técnica](#) · 08 · [Flujo de datos](#) · 15 · [Riesgos](#)

07 · Base de datos y persistencia

Motor, conexión, esquema completo, diccionario de datos, ERD, consultas reales y relación tabla ↔ módulo. Además, los **otros tres mecanismos de persistencia** que conviven con SQLite y que no son base de datos.

1. Motor y ubicación

Aspecto	Valor
Motor	SQLite 3, módulo <code>sqlite3</code> de la biblioteca estándar
ORM	Ninguno. SQL escrito a mano en <code>engine/database.py</code>
Archivo	<code>db/runs.db</code>
Ruta resuelta	<code>engine/database.py: DB_PATH = data_dir() / 'db' / 'runs.db'</code>
En desarrollo	<code><raíz del repo>/db/runs.db</code>
En el binario instalado	<code>%LOCALAPPDATA%\Automa\db\runs.db</code>
Override	<code>AUTOMA_DATA_ROOT</code>
Creación	Automática: <code>init_db()</code> en el constructor del orquestador, del scheduler, del CLI y al importar <code>app.server</code>
Versionado en git	No: <code>.gitignore</code> excluye <code>db/*.db</code> y <code>db/*.db-*</code> . Solo se versiona <code>db/.gitkeep</code>

Por qué SQLite y no otro motor: el producto es una herramienta local para un operador. SQLite viene con Python, no necesita servicio, no necesita configuración y el archivo se puede copiar como respaldo. **NO DOCUMENTADO EN EL REPOSITORIO:** la decisión no está escrita en ningún ADR, pero es coherente con el resto del diseño local-first.

Patrón de conexión

```
@contextmanager
def connect() -> Iterable[sqlite3.Connection]:
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

Conexión nueva por operación, `commit` implícito al salir sin excepción, `close` garantizado. `row_factory = sqlite3.Row` permite acceder a las columnas por nombre y convertirlas a `dict`.

Configuración ausente: **NO IDENTIFICADO** — el repositorio no establece `PRAGMA journal_mode=WAL`, `PRAGMA busy_timeout` ni `PRAGMA foreign_keys=ON`. La última es relevante: **no hay ninguna clave foránea declarada** en el esquema (ver §4).

2. Esquema completo

Siete tablas, todas creadas por `init_db()` con `CREATE TABLE IF NOT EXISTS`.

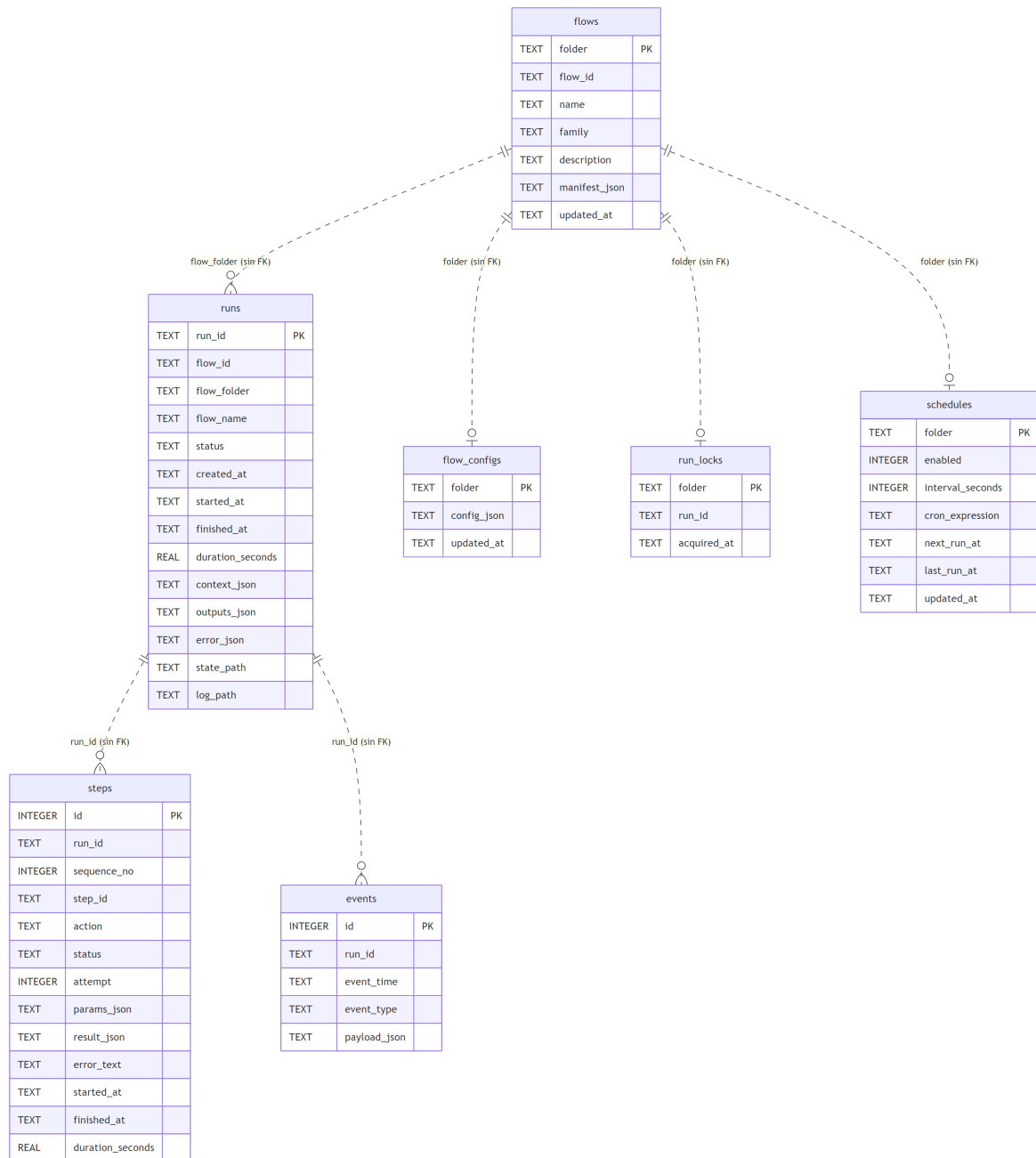


Diagrama 1 (Mermaid, renderizado)

Lo que el ERD muestra: el modelo real es una estrella alrededor de **flows** y **runs**. Un flow tiene muchas corridas; una corrida tiene muchos pasos y muchos eventos; un flow tiene como mucho una configuración, un horario y un lock.

Lo que el ERD NO muestra, y es la característica más importante del esquema: ninguna de esas relaciones existe como **clave foránea en el DDL**. Están dibujadas con línea discontinua a propósito. Son relaciones lógicas que el código mantiene, no restricciones que la base haga cumplir. Consecuencias reales en §4.

3. Diccionario de datos

3.1 flows — catálogo sincronizado

Espejo en base de datos de las carpetas de **flows/**. Se reescribe con **sync_flows()** al arrancar el panel o el CLI.

Columna	Tipo	Nulo	Descripción
---------	------	------	-------------

folder	TEXT	PK	Nombre de la carpeta: 05_system_healthcheck
flow_id	TEXT	No	id del manifest: system_healthcheck
name	TEXT	No	Nombre legible
family	TEXT	Sí	sistema, navegador, pantalla, filesystem, documentos
description	TEXT	Sí	Descripción del manifest
manifest_json	TEXT	No	El manifest completo serializado, incluidos los pasos
updated_at	TEXT	No	ISO-8601 UTC de la última sincronización

sync_flows hace INSERT ... ON CONFLICT(folder) DO UPDATE. **No borra filas:** un flow eliminado del disco deja su fila huérfana en flows para siempre. El panel no lo mostrará (lee de list_flows(), que recorre el disco), pero la fila persiste.

3.2 runs — la tabla central

Una fila por corrida, sea del panel, del CLI, del scheduler o del webhook.

Columna	Tipo	Nulo	Descripción
run_id	TEXT	PK	20260827T143052123456Z, o sched_..., si lo genera el scheduler
flow_id	TEXT	No	id del manifest
flow_folder	TEXT	No	Carpeta del flow
flow_name	TEXT	No	Nombre legible, copiado en el momento de la corrida
status	TEXT	No	created → running → completed failed
created_at	TEXT	No	ISO-8601 UTC, momento de la instanciación
started_at	TEXT	Sí	Momento de entrar en run()
finished_at	TEXT	Sí	Momento de terminar, con éxito o con error
duration_seconds	REAL	Sí	Redondeado a 4 decimales
context_json	TEXT	Sí	Contexto completo al final, con todos los save_as
outputs_json	TEXT	Sí	Lista de archivos detectados bajo output/
error_json	TEXT	Sí	null, o {step_id?, message, kind?}
state_path	TEXT	Sí	Ruta del snapshot JSON

log_path	TEXT	Sí	Ruta del JSONL
----------	------	----	----------------

context_json es la columna de mayor riesgo de tamaño y de privacidad. Contiene el contexto entero al terminar, incluidos los resultados de cada paso con `save_as`. Un flow que use `browser.extract_content` guarda ahí el texto completo de la página (hasta 200 000 caracteres por defecto). Un flow con `system.read_clipboard` guarda el contenido del portapapeles. Y `GET /api/runs` la devuelve **sin autenticación**. Ver [11 · Seguridad](#).

Valores de **status** observados:

Estado	Significado
created	Instanciado, aún no arrancado. Rara vez persiste
running	En ejecución. Puede quedarse aquí para siempre si el proceso muere
completed	Todos los pasos ejecutados sin fallo irrecuperable
failed	Violación de sandbox, o fallo sin rama de recuperación

NO IDENTIFICADO: no hay ningún mecanismo que detecte y corrija corridas huérfanas en `running`.

3.3 steps — un registro por intento

Columna	Tipo	Nulo	Descripción
id	INTEGER	PK AUTOINCREMENT	Clave técnica
run_id	TEXT	No	Corrida a la que pertenece
sequence_no	INTEGER	No	Posición: <code>len(state['steps'])</code> tras añadir
step_id	TEXT	No	<code>id</code> del paso en el manifest
action	TEXT	No	Nombre de la acción
status	TEXT	No	<code>success</code> , <code>failed</code> , <code>skipped</code>
attempt	INTEGER	No	Número de intento. 0 si el paso se saltó
params_json	TEXT	Sí	Parámetros ya renderizados
result_json	TEXT	Sí	Resultado completo de la acción
error_text	TEXT	Sí	<code>str(excepción)</code>
started_at / finished_at	TEXT	Sí	ISO-8601 UTC
duration_seconds	REAL	Sí	4 decimales; 0.0 en pasos saltados

params_json guarda los parámetros resueltos. Si un flow recibiera un secreto por `context_overrides` y lo pasara como parámetro, ese secreto quedaría en claro en esta columna. En el catálogo actual no ocurre —ningún flow declara `required_secrets`— pero es una propiedad del diseño que un autor de flows debe conocer.

Detalle no obvio: solo se inserta **un** registro por paso, incluso con reintentos. El **INSERT** ocurre al final: en éxito con el número del intento que funcionó, en fallo con el último. Los intentos intermedios fallidos **solo aparecen en events** como `step_failed`.

3.4 events — la traza fina

Columna	Tipo	Nulo	Descripción
id	INTEGER	PK AUTOINCREMENT	Clave técnica
run_id	TEXT	No	Corrida
event_time	TEXT	No	ISO-8601 UTC, generado por <code>JsonlLogger</code>
event_type	TEXT	No	Ver tabla siguiente
payload_json	TEXT	Sí	Carga del evento

event_type	Cuándo	Carga
flow_started	Al arrancar	flow_id, run_id, policy
flow_finished	Al terminar	flow_id, run_id, status, error si lo hubo
flow_blocked	Secretos ausentes	reason
step_started	Cada intento	step_id, action, attempt, params
step_finished	Intento con éxito	step_id, status, result
step_failed	Cada intento fallido	step_id, attempt, error
step_skipped	Condición no cumplida	step_id, reason
step_blocked	Acción o ruta prohibida	step_id, reason
step_recovered	Rama de recuperación tomada	step_id, next, error

Redundancia deliberada: cada evento se escribe **dos veces** —línea en `logs/*.jsonl` y fila en `events`— porque `JsonlLogger.write` hace ambas cosas. El archivo sobrevive a un borrado de la base; la tabla permite consultar.

3.5 flow_configs, run_locks, schedules

T Columna	Tipo	Descripción
flfolderfigs	TEXT PK	Carpeta del flow
config_json	TEXT	Contexto guardado desde el panel
updated_at	TEXT	ISO-8601 UTC
rufolders	TEXT PK	La PK es el mecanismo de exclusión mutua
run_id	TEXT	Corrida que lo tomó
acquired_at	TEXT	ISO-8601 UTC
scfolders	TEXT PK	Carpeta del flow
enabled	INTEGER	0 o 1. Por defecto 0
interval_seconds	INTEGER	Modo intervalo

<code>cron_expression</code>	TEXT	Modo cron, 5 campos. Añadida por migración
<code>next_run_at</code>	TEXT	Próxima ejecución en UTC
<code>last_run_at</code>	TEXT	Última ejecución
<code>updated_at</code>	TEXT	ISO-8601 UTC

`interval_seconds` y `cron_expression` son **excluyentes en la práctica**: `set_schedule` prioriza `cron_expression` si viene, y el panel envía `interval_seconds` solo si no hay cron. Pero el esquema permite ambas a la vez, y `mark_schedule_run` también prioriza el cron. **INFERENCIA**: escribir ambas por API dejaría el intervalo inerte.

4. Integridad referencial: la ausencia declarada

No hay ninguna **FOREIGN KEY** en el esquema. Verificado leyendo el DDL completo de `init_db()`.

Consecuencia	Detalle
Se puede insertar un <code>step</code> con <code>run_id</code> inexistente	Nada lo impide
Borrar una corrida no borra sus pasos ni sus eventos	No hay <code>ON DELETE CASCADE</code>
Un flow borrado del disco deja fila en <code>flows</code> , <code>schedules</code> , <code>flow_configs</code> y <code>run_locks</code>	Ninguna limpieza automática
Un lock huérfano bloquea el scheduler para siempre	Se libera a mano con <code>force_release_lock</code>

Para un sistema de un solo escritor coordinado por código, la decisión es defendible: la integridad la mantiene `engine/database.py`, que es el único módulo que escribe. Pero significa que **cualquier limpieza manual de la base debe hacerse tabla por tabla**.

Índices

Los únicos índices son los implícitos de las claves primarias (`flows.folder`, `runs.run_id`, `steps.id`, `events.id`, `flow_configs.folder`, `run_locks.folder`, `schedules.folder`). No hay ningún `CREATE INDEX` explícito.

Consultas que hacen escaneo completo:

Consulta	Filtra por	Índice
<code>list_runs(flow_id=...)</code>	<code>runs.flow_id</code>	[NO]
<code>list_runs()</code> sin filtro	<code>ORDER BY created_at DESC</code>	[NO]
<code>get_steps(run_id)</code>	<code>steps.run_id</code>	[NO]
<code>get_events(run_id)</code>	<code>events.run_id</code>	[NO]
<code>metrics.overview()</code>	<code>GROUP BY status, GROUP BY action</code>	[NO]

INFERENCIA: con unos cientos de corridas es irrelevante. Con decenas de miles —y sin retención, es cuestión de tiempo— el panel y el dashboard de métricas se degradarán. Registrado en 15.

5. Consultas reales del sistema

Todas viven en `engine/database.py` y `engine/metrics.py`.

Escritura del histórico

```
-- upsert_run: se ejecuta tras CADA paso de CADA corrida
INSERT INTO runs(run_id, flow_id, flow_folder, flow_name, status, created_at, started_at,
                finished_at, duration_seconds, context_json, outputs_json, error_json,
                state_path, log_path)
VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)
ON CONFLICT(run_id) DO UPDATE SET
    status=excluded.status, started_at=excluded.started_at, finished_at=excluded.finished_at,
    duration_seconds=excluded.duration_seconds, context_json=excluded.context_json,
    outputs_json=excluded.outputs_json, error_json=excluded.error_json,
    state_path=excluded.state_path, log_path=excluded.log_path;
```

Lectura del histórico

```
-- list_runs
SELECT * FROM runs WHERE flow_id = ? ORDER BY created_at DESC LIMIT ?;
-- get_steps
SELECT * FROM steps WHERE run_id = ? ORDER BY sequence_no ASC, id ASC;
```

El desempate por `id` en `get_steps` importa porque `sequence_no` puede repetirse si dos inserciones ocurrieran con el mismo `len(state['steps'])`.

El lock

```
INSERT INTO run_locks(folder, run_id, acquired_at) VALUES(?,?,?); -- IntegrityError = ocupado
DELETE FROM run_locks WHERE folder = ? AND run_id = ?; -- release
DELETE FROM run_locks WHERE folder = ?; -- force_release
```

Métricas

```
-- Duración media por acción, top 10 más lentas
SELECT action, AVG(duration_seconds) AS avg_d, COUNT(*) AS c
FROM steps WHERE duration_seconds IS NOT NULL
GROUP BY action ORDER BY avg_d DESC LIMIT 10;

-- Reintentos acumulados por acción
SELECT action, SUM(attempt - 1) AS retry_count
FROM steps WHERE attempt > 1 GROUP BY action ORDER BY retry_count DESC LIMIT 10;

-- Resumen por flow
SELECT flow_id, COUNT(*) AS runs_total,
    SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END) AS runs_completed,
    SUM(CASE WHEN status='failed' THEN 1 ELSE 0 END) AS runs_failed,
    AVG(duration_seconds) AS avg_duration_seconds,
    MAX(created_at) AS last_run_at
FROM runs GROUP BY flow_id ORDER BY runs_total DESC LIMIT ?;
```

Sesgo conocido en `retry_count`: cuenta `attempt - 1` sobre la fila **final** de cada paso. Como solo se guarda un registro por paso, un paso que falló dos veces y tuvo éxito al tercero aporta **2**; uno que falló tres veces y se rindió aporta **2** también (`attempt=3`). Es coherente, pero no equivale al número de fallos.

6. Transacciones

No hay transacciones explícitas. Cada llamada a `connect()` abre su propia transacción implícita y hace `commit` al salir. Consecuencias:

- `insert_step` y el `upsert_run` que le sigue son **dos transacciones separadas**. Un corte entre ambas deja el paso guardado y la corrida con el estado anterior.
- `set_flow_config` escribe **primero el archivo y luego la fila**. No hay atomicidad entre ambos medios.
- `JsonLogger.write` escribe **primero el archivo y luego la fila** de `events`. Mismo caso.

INFERENCIA: para un sistema de escritorio con corridas cortas, el riesgo es bajo y el diseño simple lo compensa. En caso de inconsistencia, el archivo JSONL y el snapshot de `state/` son la fuente de verdad más completa.

7. Relación tabla ↔ módulo

Tabla	Escriben	Leen
flows	<code>sync_flows ← app/server.py</code> (al importar), <code>engine/runner.py</code>	<code>engine/catalog.py</code> (indirectamente)
runs	<code>upsert_run ← Orchestrator._persist</code>	<code>catalog.list_runs</code> , <code>find_run</code> , <code>metrics.*</code> , <code>app/server.py</code>
steps	<code>insert_step ← Orchestrator.run</code>	<code>catalog.load_run_steps</code> , <code>metrics.overview</code> , <code>_run_status_payload</code>
events	<code>insert_event ← JsonlLogger.write</code>	<code>catalog.load_run_events</code> , detalle de corrida
flow_configs	<code>set_flow_config ← POST</code> <code>/flow/<f>/config</code> , <code>smoke_test.py</code>	<code>get_flow_config ←</code> <code>catalog.get_flow_by_folder</code>
run_locks	<code>acquire_run_lock/release_run_lock ←</code> <code>solo scheduler._run_job</code>	<code>list_run_locks ← RUNBOOK.md</code>
schedules	<code>set_schedule ← POST</code> <code>/flow/<f>/schedule</code> ; <code>mark_schedule_run</code> <code>← scheduler</code>	<code>list_schedules ←</code> <code>scheduler.run_pending_once</code> ; <code>get_schedule ← panel</code>

8. Los otros tres mecanismos de persistencia

SQLite no es el único almacén. Hay tres más, y dos de ellos son funcionalmente imprescindibles.

8.1 `state/<flow_id>_<run_id>.json` — snapshot completo

Escrito por `StateStore.save` tras cada paso, con `indent=2` y `ensure_ascii=False`. Es el `state` completo: definición, contexto, todos los pasos con sus resultados, la ruta recorrida y los outputs. Es un superconjunto de lo que hay en `runs`, y sirve para reconstruir una corrida si la base se pierde.

Se reescribe entero en cada paso. Un flow de 20 pasos con contexto grande escribe el JSON completo 20 veces.

8.2 `logs/<flow_id>_<run_id>.jsonl` — traza append-only

Escrito por `JsonlLogger.write` con modo `'a'`. Una línea JSON por evento, con `timestamp`, `event` y la carga. Es el único almacén **append-only** del sistema: nada lo reescribe.

8.3 Tracking persistente entre corridas — el más importante funcionalmente

Tres archivos JSON fuera de SQLite que dan memoria a los flows:

Archivo	Escrito por	Para qué	Versionado
<code>data/seeds/.used_indices.json</code>	<code>browser_form._save_used_ids</code>	Que el flow 07 no repita registro	[NO] <code>.gitignore</code>
<code>data/web_watch/demo_page.json</code>	<code>browser_extract.apply_tracking</code>	Línea base del flow 23	[NO] <code>.gitignore</code>
<code>data/web_watch/precio_demo.json</code>	<code>browser_extract.apply_tracking</code>	Línea base del flow 26	[NO] <code>.gitignore</code>

Sin ellos, los flows 07, 23 y 26 pierden su función. Un `git clean` o un borrado de `data/web_watch/` hace que la próxima corrida del 23 reporte `first_run: true` y no detecte el cambio que estaba vigilando. La carpeta `data/web_watch/` no existe en un clon limpio; se crea sola en la primera corrida.

Formato de `.used_indices.json`:

```
{
  "total_in_dataset": 100,
  "used_count": 7,
  "remaining": 93,
  "used_ids": [3, 12, 41, 55, 68, 77, 91]
}
```

Formato de un archivo de `data/web_watch/`:

```
{
  "url": "file:///C:/dev/automa-pc/data/web/demo_page.html",
  "watch_value": "a3f5...(SHA-256 del texto normalizado)",
  "checked_at": "2026-08-27T14:30:52.123456+00:00"
}
```

8.4 secrets/secrets.json y configs/<folder>.json

- `secrets/secrets.json` — escrito por `engine.secrets.set_secret`, **texto plano sin cifrar**. El control de acceso son los permisos del sistema de archivos. El docstring del módulo lo declara: «permisos del FS son el control de acceso». Ignorado por git.
- `configs/<folder>.json` — escrito por `set_flow_config`. **Versionado** en el caso de `configs/03_folder_inventory.json`.

9. Respaldo y recuperación

NO IDENTIFICADO: el repositorio **no incluye ninguna rutina de respaldo, purga, compactación ni retención**. Verificado buscando `VACUUM`, `DELETE FROM runs`, `backup` y `retention` en todo el código.

Procedimiento manual, **INFERENCIA** a partir de la estructura:

```
# Respaldo en caliente (SQLite lo soporta con el proceso corriendo)
sqlite3 db/runs.db ".backup 'db/runs-backup-$(date +%Y%m%d).db'"

# Respaldo completo: incluir los almacenes que NO están en la base
tar -czf automa-backup.tar.gz db/runs.db logs/ state/ configs/ data/seeds/.used_indices.json data/web_watch/
```

Restauración total: parar el panel, sustituir `db/runs.db`, arrancar. `init_db()` no destruye nada.

Reinicio limpio: borrar `db/runs.db`. Se recrea vacía en el siguiente arranque. El `RUNBOOK.md` del repositorio documenta este procedimiento junto con la liberación de locks.

Crecimiento sin control: las tres carpetas crecen indefinidamente. Un flow programado cada 15 minutos genera 96 corridas al día, cada una con su fila en `runs`, sus filas en `steps` y `events`, su `.jsonl` y su `.json` de estado. **INFERENCIA:** en un año son unas 35 000 corridas y 70 000 archivos sueltos en `logs/` y `state/`. Registrado como riesgo en 15.

10. Datos sensibles almacenados

Dato	Dónde acaba	Riesgo
Contenido del portapapeles (flow 15)	<code>runs.context_json</code> , <code>steps.result_json</code> , <code>state/*.json</code> , <code>output/reports/*.json</code>	Alto: puede contener contraseñas copiadas
Capturas del escritorio (flows 01, 09, 12, 16, 17, 19)	<code>output/screenshots/*.png</code>	Alto: cualquier cosa visible en pantalla
Texto OCR del escritorio (flows 12, 17)	<code>runs.context_json</code> , reportes JSON	Alto: texto de ventanas abiertas
Salida de PowerShell (flow 18)	<code>runs.context_json</code> , reportes	Medio: inventario del equipo
Rutas y nombres de archivo (flows 03, 04)	<code>runs.context_json</code>	Bajo-medio
Datos del formulario (flow 07)	<code>output/reports/form_submission_*.json</code>	Bajo: son datos sintéticos del seed

Contenido de páginas web (flows 21–27)	<code>runs.context_json</code> , reportes	Depende de la URL configurada
Tokens (<code>AUTOMA_*</code>)	<code>secrets/secrets.json</code> sin cifrar	Medio

El punto crítico: todo eso es legible desde `GET /api/runs`, que **no exige autenticación**, y desde `GET /file?path=...`, que sirve cualquier archivo bajo la raíz del proyecto salvo las extensiones bloqueadas —los PNG **no** están bloqueados. Análisis completo en [11 · Seguridad](#).

11. Cómo inspeccionar la base a mano

```
sqlite3 db/runs.db ".tables"
sqlite3 db/runs.db ".schema runs"

# Últimas 10 corridas
sqlite3 db/runs.db "SELECT run_id, flow_id, status, duration_seconds FROM runs
                  ORDER BY created_at DESC LIMIT 10;"

# Corridas fallidas con su motivo
sqlite3 db/runs.db "SELECT run_id, flow_id, error_json FROM runs WHERE status='failed';"

# Corridas colgadas en running
sqlite3 db/runs.db "SELECT run_id, flow_id, started_at FROM runs WHERE status='running';"

# Locks activos
sqlite3 db/runs.db "SELECT * FROM run_locks;"

# Horarios habilitados
sqlite3 db/runs.db "SELECT folder, enabled, interval_seconds, cron_expression, next_run_at
                  FROM schedules WHERE enabled=1;"

# Tamaño de los contextos guardados (la columna que más crece)
sqlite3 db/runs.db "SELECT run_id, LENGTH(context_json) AS bytes FROM runs
                  ORDER BY bytes DESC LIMIT 10;"
```

El `RUNBOOK.md` del repositorio incluye consultas equivalentes para la operación diaria.

Documentos relacionados: [03 · Arquitectura](#) · [06 · Explicación profunda](#) · [08 · Flujo de datos](#) · [11 · Seguridad](#) · [13 · Despliegue y operación](#)

08 · Flujo de datos

De dónde vienen los datos, cómo se validan, cómo se transforman, dónde se guardan, quién los consume, dónde se pueden perder y qué datos personales o sensibles atraviesan el sistema.

1. Panorama: las cinco fuentes y los cuatro destinos

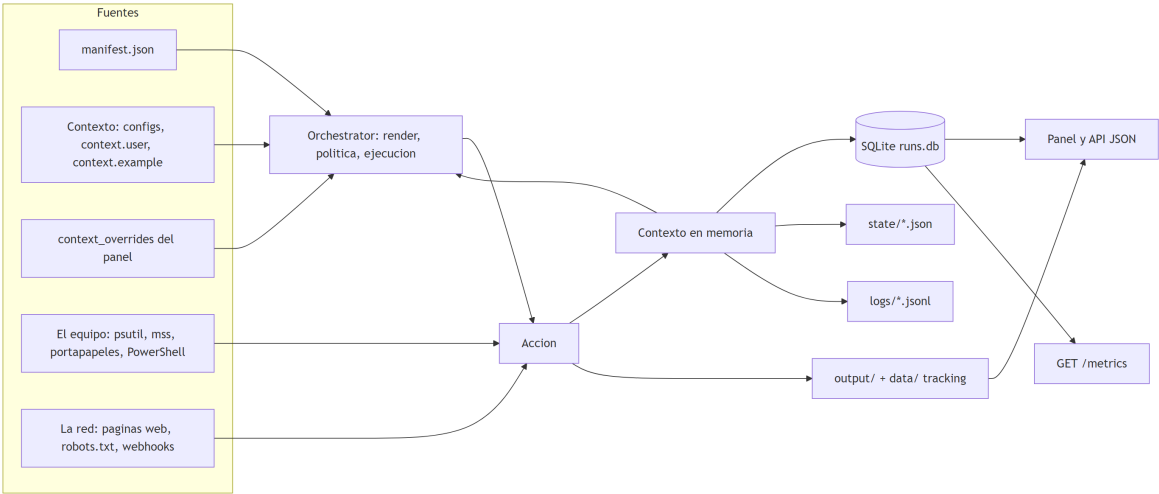


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: las tres fuentes de configuración entran por el motor y las dos fuentes de mundo real entran por la acción. Todo converge en el contexto, y el contexto se persiste por triplicado.

Lo que no muestra: que el contexto es **acumulativo y una sola referencia**. No hay copia por paso: `self.state['context'] = self.context` apunta al mismo diccionario. Lo que un paso guarda con `save_as` queda visible para todos los siguientes y acaba entero en la columna `context_json` de la tabla `runs`.

2. Origen de los datos

2.1 El manifest — datos de estructura

`flows/<carpeta>/manifest.json`, leído por `FlowLoader.load_manifest`. Define pasos, acciones, parámetros con placeholders, condiciones y transiciones. **No se valida contra el JSON Schema en tiempo de ejecución:** el schema solo lo aplica `scripts/validate_project.py` en la CI. Un manifest inválido llega hasta el motor y falla con `KeyError`.

2.2 El contexto — datos de configuración

`FlowLoader.load_context` resuelve la **primera fuente que exista** y descarta las demás:

Prioridad	Fuente	Quién la escribe
1	<code>--context RUTA</code> del CLI	El operador
2	<code>configs/<carpeta>.json</code>	El panel, en <code>POST /flow/<f>/config</code>
3	<code>flows/<carpeta>/context.user.json</code>	El operador, a mano. No versionado
4	<code>flows/<carpeta>/context.example.json</code>	El autor del flow. Versionado

No hay mezcla de claves. Si existe `configs/03_folder_inventory.json`, el `context.example.json` del flow 03 se ignora por completo, aunque una versión posterior del flow añada claves nuevas. Es la trampa de configuración más probable del sistema.

2.3 Los overrides — datos de invocación

```
POST /api/run/03_folder_inventory
{"context_overrides": {"path_override": "C:/Users/ejemplo/Documentos"}}
```

Se fusionan con `{**contexto, **overrides}`: **reemplazo superficial de claves de primer nivel**. Un override de un objeto anidado borra el resto de sus claves.

2.4 El equipo — datos del mundo real

Dato	Acción	Biblioteca
CPU, RAM, disco	<code>system.snapshot_system</code>	<code>psutil</code>
Procesos	<code>system.top_processes</code>	<code>psutil</code>
Píxeles del escritorio	<code>screen.capture_*</code>	<code>mss</code> , respaldo <code>Pillow</code>
Ventana en foco	<code>screen.capture_active_window</code>	<code>PyGetWindow</code>
Portapapeles	<code>system.read_clipboard</code>	<code>pyperclip</code>
Inventario del sistema	<code>system.run_powershell</code>	<code>subprocess</code> + PowerShell
Texto en imágenes	<code>vision.ocr_image</code>	<code>pytesseract</code> + binario <code>tesseract</code>
Archivos y carpetas	<code>filesystem.*</code>	Biblioteca estándar

2.5 La red — datos externos

Dato	Acción	Salida
DOM renderizado	<code>browser.extract_content</code> , <code>crawl_site</code> , <code>capture_page</code>	Chromium
Estado de enlaces	<code>http.check_urls</code>	<code>requests</code> HEAD/GET
Contenido de una URL	<code>http.fetch_url</code>	<code>requests</code> GET
<code>robots.txt</code>	<code>RobotsCache._fetch</code>	<code>requests</code> GET
Notificación	<code>notify.send</code> con backend: <code>webhook</code>	<code>requests</code> POST

Todos los flows web apuntan por defecto a archivos locales del repositorio (`data/web/demo_page.html`, `data/web/site_demo/index.html`, `data/web/control_page.html`). Verificado leyendo los ocho `context.example.json` correspondientes. Sin cambiar la configuración, **el sistema no genera tráfico de red**.

3. Validación: qué se comprueba y qué no

Punto	Qué valida	Cuándo	Módulo
JSON Schema	Estructura completa del manifest, <code>additionalProperties: false</code>	Solo en CI	<code>scripts/validate_project.py</code>
Acciones registradas	Que cada <code>action</code> exista	Solo en CI	<code>validate_project.py</code>
Coherencia con <code>allowed_actions</code>	Que ningún paso use una acción que su propia política prohíbe	Solo en CI	<code>validate_project.py</code>

Transiciones y <code>start_step</code>	Que apunten a pasos existentes	Solo en CI	<code>validate_project.py</code>
<code>_safe_folder</code>	Slug de la URL: <code>^[A-Za-z0-9_\-\]{1,64}\$</code>	En cada petición	<code>app/server.py</code>
Ruta de <code>/file</code>	NUL, controles, absolutas, prefijo, extensión	En cada petición	<code>app/server.py::do_GET</code>
<code>_authorize_mutation</code>	Token, o <code>Host/Origin/Referer</code>	En cada POST	<code>app/server.py</code>
<code>assert_secrets_present</code>	Variables de entorno presentes y no vacías	Una vez por corrida	<code>engine/sandbox.py</code>
<code>assert_action_allowed</code>	Acción en la lista blanca	Antes de cada paso	<code>engine/sandbox.py</code>
<code>assert_paths_allowed</code>	Rutas renderizadas bajo <code>allowed_paths</code>	Antes de cada paso	<code>engine/sandbox.py</code>
Tokens de PowerShell	<code>;</code> , <code>\</code> , <code>&</code> , backtick, <code>></code> , <code><</code> , <code>\$</code> (, <code>\$_</code>	En la acción	<code>actions/system.py</code>
Allowlist de verbos	Primera palabra del comando	En la acción	<code>actions/system.py</code>
<code>shell=True</code> prohibido	Cierra CWE-78	En la acción	<code>actions/ui.py</code>

Lo que NO se valida en ningún punto:

- Que los `params` de un paso coincidan con la **firma** de la acción. Un `screen.capture_screenshot` sin `output_path` pasa la CI y falla en ejecución con `TypeError`.
- Que el **tipo** de un valor del contexto sea el esperado. Un umbral escrito como `"80"` en vez de `80` produce `TypeError` al comparar en `conditions.matches`.
- El **contenido** de un `context_overrides`: se acepta cualquier diccionario.
- La **URL** de `browser.*`: `_to_url` solo comprueba que, si no es URL, el archivo exista.

4. Transformación: el ciclo de un dato dentro de una corrida

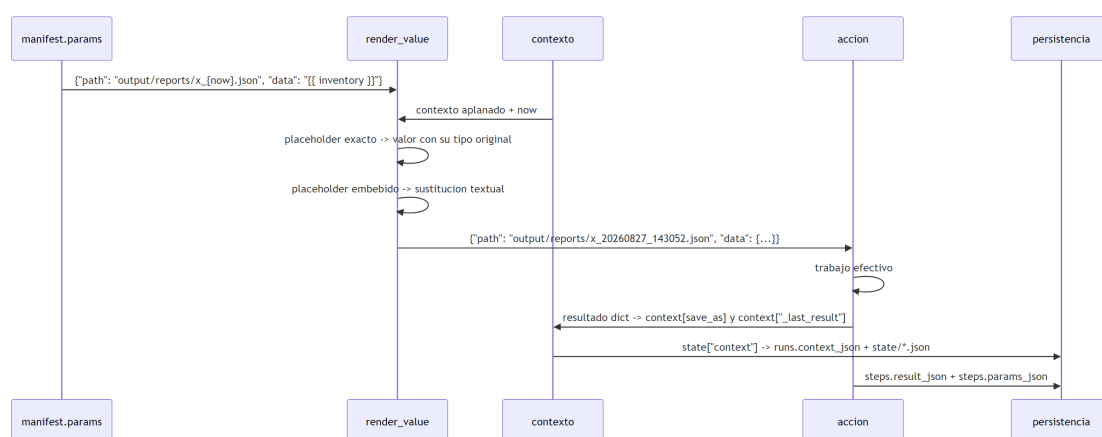


Diagrama 2 (Mermaid, renderizado)

Lo que la secuencia muestra: los dos modos de `render_value`. Un `"{{ inventory }}"` solo (placeholder exacto) devuelve el **diccionario entero**; un `"x_{now}.json"` (embebido) hace sustitución textual. Es lo que permite que el flow 04 pase el resultado completo de un paso como `data` del siguiente.

Lo que no muestra: que `{now}` se recalcula en **cada** llamada a `render_value`, es decir, una vez por paso. Dos pasos del mismo flow pueden obtener timestamps distintos si cruzan el cambio de segundo. Los flows del repositorio lo evitan pasando la ruta por el contexto con `save_as` en lugar de recomputar `{now}`.

Ejemplo real y completo: 04_document_drop_pipeline

Cuatro pasos, encadenados por contexto. Rastro del dato paso a paso:

Paso	Acción	Entrada	save_as	Qué añade al contexto
scan_dropbox	filesystem.list_directory{{ dropbox_path }}		inventory	{path, files[], total_files}
classify_dropbox	filesystem.classify_file{{ inventory.files }}		stats	{total_files, total_size_bytes, by_extension, largest_file}
summarize_texts	filesystem.summarize_text{{ dropbox_path }}, max_files: 10, max_chars_per_file: 500		summary	{path, processed_files, summaries[]}
write_pipeline_report	filesystem.write_json	{{ inventory }}, {{ stats }}, {{ summary }}	report	{path, written}

Nótese {{ inventory.files }} en el paso 2: usa el **contexto aplanado** para bajar un nivel dentro del resultado del paso 1. Es el mecanismo que hace innecesario cualquier código de pagemento.

Pérdida de información silenciosa en este flow: summarize_text_folder corta en max_files: 10 y solo acepta cinco extensiones. Una carpeta con 50 logs produce un informe de 10, y el JSON **no dice cuántos quedaron fuera** — solo processed_files: 10. Es la única cota del sistema que no se declara explícitamente. Registrado en 15.

5. Almacenamiento: qué acaba dónde

Dato	runs.context_json	steps.result_json	state/*.json	logs/*.jsonl	output/
Parámetros renderizados	—	[OK] (params_json)	[OK]	[OK] (step_started)	—
Resultado de cada paso	[OK]	[OK]	[OK]	[OK] (step_finished)	—
Texto de una página web	[OK]	[OK]	[OK]	[OK]	[OK] si hay markdown
Portapapeles	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Texto OCR del escritorio	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Captura PNG	Solo la ruta	Solo la ruta	Solo la ruta	Solo la ruta	[OK] binario
Salida de PowerShell	[OK]	[OK]	[OK]	[OK]	[OK] JSON
Mensaje de error	[OK] (error_json)	[OK] (error_text)	[OK]	[OK] (step_failed)	—

Cuatro copias del mismo dato es la norma, no la excepción. El resultado de un paso se escribe en steps.result_json, en runs.context_json, en el snapshot de state/ y en la línea step_finished del JSONL. Es redundancia deliberada por trazabilidad, y explica por qué el volumen crece rápido.

Tracking persistente entre corridas

Tres archivos JSON dan **memoria** a los flows más allá de una corrida:

Archivo	Flow	Qué recuerda	Qué pasa si se borra
<code>data/seeds/.used_indices.json</code>	07	IDs de registros ya usados	El ciclo empieza de cero; se repiten registros
<code>data/web_watch/demo_page.json</code>	23	SHA-256 del texto de la última corrida	Próxima corrida: <code>first_run: true</code> , no detecta el cambio
<code>data/web_watch/precio_demo.json</code>	26	Último valor leído del selector CSS	Igual

Ninguno está versionado. `data/web_watch/` ni siquiera existe en un clon limpio: se crea sola en la primera corrida. Es diseño, no defecto — la primera corrida establece la línea base y lo declara con `first_run: true`.

6. Consumo: quién lee qué

Consumidor	Fuente	Ruta
Pestaña Ejecutar	<code>flows</code> en disco + última corrida de <code>runs</code>	<code>render_home</code>
Pestaña Programadas	<code>schedules</code>	<code>render_home</code>
Pestaña Histórico	<code>runs</code>	<code>render_home</code> , <code>render_flow_history</code>
Detalle de corrida	<code>runs + steps + events</code>	<code>render_run_detail</code>
Polling durante la ejecución	<code>runs + steps</code> + manifest	<code>GET /api/runs/<run_id>/status</code>
Dashboard de métricas	Agregados de <code>runs + steps</code>	<code>render_metrics_dashboard</code>
Prometheus	Agregados	<code>GET /metrics</code>
Vistas previas e imágenes	<code>output/</code> vía <code>outputs.json</code>	<code>GET /file?path=...</code>
Integrador externo	<code>runs</code>	<code>GET /api/runs</code> , <code>GET /api/flows</code>

El resumen legible del panel

`app/server.py::_smart_summary` traduce el contexto crudo en texto para humanos. **Pero solo cubre 6 de los 27 flows.** Verificado leyendo la cadena de `elif` completa: hay rama para `screen_capture_analyze`, `screen_capture_browser`, `folder_inventory`, `document_drop_pipeline`, `system_healthcheck` y `process_watchdog`. Para los otros 21 —incluidos `browser_form_filler` y los siete de la familia web— la función devuelve cadena vacía y `render_run_detail` omite el bloque entero; el operador ve solo los pasos colapsables con su JSON.

Es una divergencia con el `README.md` del repositorio, que en su demo de cinco minutos afirma que el detalle del flow 07 muestra «los 10 datos enviados como lista legible». Registrado en 15 · Riesgos y deuda técnica.

Además, cada rama depende de las **claves concretas** que devuelve cada acción (`capture.width`, `stats.by_extension`, `snapshot.cpu_percent`...): cambiar el nombre de un campo de retorno degrada el resumen sin romper nada. Acoplamiento implícito entre presentación y acciones.

7. Dónde se pierden o corrompen datos

Punto	Qué se pierde	Se declara	Gravedad
<code>summarize_text_folder</code> con más de <code>max_files</code>	Los archivos sobrantes	[NO]	Media
<code>summarize_text_folder</code> con extensiones no soportadas	Los archivos ignorados	[NO]	Media

<code>top_processes</code> con <code>AccessDenied</code>	Procesos privilegiados	Parcial (<code>total_seen</code>)	Baja
<code>apply_tracking</code> con archivo de estado corrupto	La línea base	[NO] Se reinicia en silencio	Media
<code>resolve_links</code> con más de <code>max_links</code>	Enlaces sobrantes	[OK] <code>links_truncated</code>	Baja
<code>extract_content</code> con más de <code>max_text_chars</code>	Texto sobrante	[OK] <code>text_truncated</code>	Baja
<code>check_urls</code> con más de <code>max_urls</code>	URLs sobrantes	[OK] <code>truncated</code>	Baja
<code>run_powershell</code> con salida larga	Más allá de 50 000 / 5 000 chars	[OK] <code>*_truncated</code>	Baja
<code>read_clipboard</code> con más de <code>max_chars</code>	Texto sobrante	[OK] <code>truncated</code> + <code>length</code> real	Baja
<code>crawl_pages</code> con <code>max_pages</code> alcanzado	Páginas pendientes	[OK] <code>truncated</code>	Baja
<code>_pick_record</code> cuando el llenado falla después	Un registro del seed, marcado como usado sin llegar a enviarse	[NO]	Baja
Proceso muerto a mitad de corrida	Nada persistido; la fila queda en <code>running</code> para siempre	[NO]	Media
<code>set_flow_config</code> si falla el <code>INSERT</code> tras escribir el archivo	Desincronización archivo/tabla	[NO]	Baja
Dos corridas del mismo flow en paralelo desde el panel	Escritura concurrente del archivo de tracking	[NO]	Media

Balance honesto: el sistema declara nueve de sus trece cotas. Las cuatro que no lo hacen están concentradas en `summarize_text_folder` y en `apply_tracking`.

8. Datos personales y sensibles

Categoría	Flows	Dónde acaba	Retención
Contenido del portapapeles	15	<code>context_json</code> , <code>result_json</code> , <code>state/</code> , <code>output/reports/</code>	Indefinida
Capturas de pantalla	01, 09, 12, 16, 17, 19	<code>output/screenshots/*.png</code>	Indefinida
Texto OCR de ventanas abiertas	12, 17	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Inventario del equipo	05, 06, 18	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Rutas y nombres de archivo personales	03, 04, 10	<code>context_json</code> , <code>output/reports/</code>	Indefinida
Contenido de páginas web	21–27	<code>context_json</code> , <code>output/reports/</code> , <code>output/*.md</code>	Indefinida
Datos de formulario	07	<code>output/reports/form_submission_*</code>	Indefinida. Sintéticos: vienen del seed

Los tres puntos de mayor exposición, en orden:

- 1. `system.read_clipboard` — el portapapeles puede contener contraseñas recién copiadas de un gestor. El flow 15 las escribiría en claro en cuatro sitios distintos.
- 2. **Las capturas de escritorio** — cualquier cosa visible: correo abierto, chat, un documento confidencial. Y `GET /file?path=...` sirve PNG sin autenticación: la allowlist de extensiones bloquea `.html`, `.js`, `.svg` y `.css`, pero no `.png`.
- 3. `GET /api/runs` — devuelve `context_json` de todas las corridas, sin exigir token, incluso con `AUTOMA_PANEL_TOKEN` definido (`do_GET` no llama a `_authorize_mutation`).

Ningún control de privacidad está implementado: no hay anonimización, no hay redacción de campos, no hay cifrado en reposo, no hay retención. Análisis completo en 11 · Seguridad.

9. Qué sale del equipo

Con la configuración por defecto: **nada**. Verificado leyendo los 27 `context.example.json` y los 27 manifests.

Salida potencial	Se activa cuando	Por defecto
Petición HTTP a una página	El operador cambia <code>target_url</code> a una URL remota	Apunta a <code>data/web/*.html</code>
<code>robots.txt</code>	Igual, en el flow 22	Local, sin red
Verificación de enlaces	Flow 24 sobre una página con enlaces externos	Página local con enlaces locales
Webhook de notificación	<code>notify_backend: "webhook"</code> en los flows 23 o 26	<code>"file"</code>
Proveedor de IA	Un flow nuevo que llame a <code>vision.inspect_screen_target</code> con proveedor $\neq$ <code>mock</code>	Ningún flow lo usa
Telemetría del producto	—	No existe

El único servidor que Automa levanta escucha en `127.0.0.1`. No hay cliente que se conecte a ningún servicio del proyecto.

10. Tres flujos completos, de punta a punta

10.1 15_clipboard_capture — el más corto y el más sensible

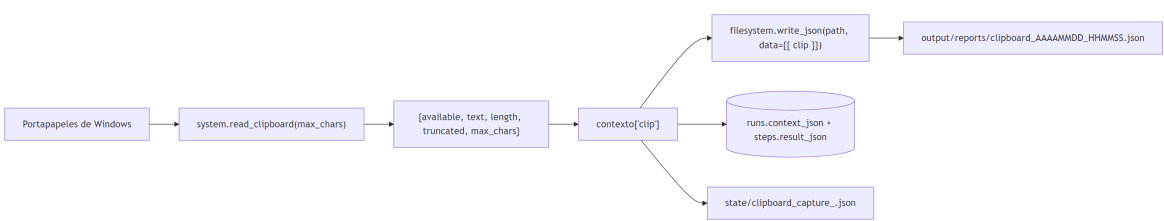


Diagrama 3 (Mermaid, renderizado)

Dos pasos, una lectura del sistema, **cuatro copias del texto copiado**. El flow declara `allowed_actions` y `allowed_paths: ["output/reports"]`, así que `assert_paths_allowed` comprueba la clave `path` del segundo paso. Pero la política **no impide** que el texto acabe en `context_json`: el sandbox controla dónde escriben las acciones, no qué guarda el motor.

10.2 23_web_change_detector — el que usa memoria entre corridas

Paso	Acción	Decisión
------	--------	----------

<code>extract_and_track</code>	<code>browser.extract_content</code> con <code>track_state_path</code> y <code>retries: 1</code>	Compara el SHA-256 del texto normalizado con el guardado
<code>evaluate_change</code>	<code>rules.evaluate</code>	<code>content.changed == true</code> → alerta; <code>content.first_run == true</code> → baseline; si no, <code>ok</code>
<code>notify_change</code>	<code>notify.send</code>	Solo si <code>decision.status == "alerta"</code>
<code>write_report</code>	<code>filesystem.write_json</code>	Siempre

La condición del tercer paso vive en el manifest, no en el código:

```
"when": {"path": "decision.status", "operator": "eq", "value": "alerta"}
```

Cuando no se cumple, el paso se registra como `skipped` con `reason: condition_not_met` — queda constancia de que se evaluó y no se disparó.

El dato que cruza corridas es `data/web_watch/demo_page.json`, escrito por `apply_tracking` antes de devolver el resultado. Si el flow falla en un paso posterior, la línea base ya se actualizó: la próxima corrida comparará contra el valor nuevo y no detectará el cambio que se perdió.

10.3 07_browser_form_filler — el que más datos mueve

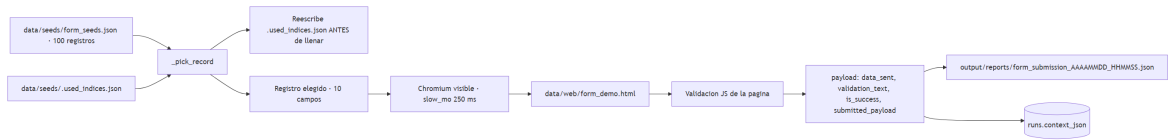


Diagrama 4 (Mermaid, renderizado)

Tres detalles del flujo que no se ven en el manifest:

- 1. El tracking se escribe **antes** del llenado. Un fallo del navegador consume el registro igualmente.
- 2. `is_success` se decide comparando texto: `validation_text.startswith('[OK]')` or `'válido' in validation_text.lower()`. Depende del copy de la página de demo.
- 3. Los nombres de los diez campos están **en el código Python**, no en el manifest. Apuntar el flow a otro formulario exige tocar `actions/browser_form.py`.

11. Integridad y determinismo

Lo determinista:

- El BFS de `crawl_pages` recorre los enlaces en orden de aparición en el DOM, sin aleatoriedad. El docstring lo declara.
- `check_urls` verifica en orden de entrada.
- `resolve_links` preserva el orden y deduplica por URL absoluta.
- `rules.evaluate` se detiene en la primera regla que coincide, siempre la misma.
- `normalize_text` produce el mismo texto para el mismo HTML, lo que hace estable el SHA-256.

Lo no determinista, y es una sola cosa:

- `random.choice` en `browser_form._pick_record`, **sin semilla**. Es intencional: el objetivo es no repetir registros entre corridas.

El docstring de `browser_extract.py` distingue con precisión las dos cosas: «El contenido de una web puede cambiar entre corridas — lo determinista es el comportamiento y la estructura de salida, no el contenido remoto».

09 · APIs e integraciones

Endpoints internos, contratos de datos, integraciones salientes, dependencias de plataforma y el adaptador de visión multimodal que existe pero ningún flow usa. Todos los ejemplos usan valores ficticios y dominios reservados.

1. La API HTTP del panel

Aspecto	Valor
Servidor	<code>ThreadingHTTPServer</code> de la biblioteca estándar
Bind por defecto	<code>127.0.0.1:8787</code> (<code>app/server.py::run_server</code>)
Protocolo	HTTP/1.1 sin TLS
Formatos	<code>application/json; charset=utf-8, text/html; charset=utf-8, text/plain; version=0.0.4</code>
Versionado	Ninguno. No hay <code>/v1/</code> , ni cabecera de versión
CORS	No se emite ninguna cabecera CORS. El navegador bloquea cualquier lectura cross-origin
Documentación de API	NO DOCUMENTADO EN EL REPOSITORIO: no hay OpenAPI ni Swagger

Sin TLS y sin CORS, la superficie efectiva es la máquina local. La única cabecera de seguridad que se emite es `X-Content-Type-Options: nosniff`, y solo en `/file`.

1.1 Endpoints GET

Método	Ruta	Devuelve	Auth
GET	<code>/</code>	HTML del panel de 3 pestañas	[NO]
GET	<code>/healthz</code>	<code>{"status": "ok"}</code>	[NO]
GET	<code>/metrics</code>	Texto Prometheus	[NO]
GET	<code>/api/metrics</code>	<code>{"overview": {...}, "by_flow": [{...}]}</code>	[NO]
GET	<code>/metrics/dashboard</code>	HTML del dashboard	[NO]
GET	<code>/api/flows</code>	Array de flows	[NO]
GET	<code>/api/runs?flow_id=&limit=50</code>	Array de corridas	[NO]
GET	<code>/api/runs/<run_id>/status</code>	Estado paso a paso	[NO]
GET	<code>/flow/<folder></code>	HTML de la ficha	[NO]
GET	<code>/flow/<folder>/config</code>	HTML del editor de contexto	[NO]
GET	<code>/flow/<folder>/history</code>	HTML del histórico del flow	[NO]
GET	<code>/run/<flow_id>/<run_id></code>	HTML del detalle	[NO]
GET	<code>/file?path=<relativa></code>	Bytes del archivo	[NO]

Ninguna lectura exige autenticación, ni siquiera con AUTOMA_PANEL_TOKEN definido: do_GET no llama a _authorize_mutation. Ver 11 · Seguridad.

1.2 Endpoints POST

Método	Ruta	Cuerpo	Devuelve	Auth
POST	/api/run/<folder>	{"context_overrides": {...}} opcional	{ok, run_id, status, flow_id}	_authorize_mutation
POST	/api/hook/<folder>	Ignorado	{ok, run_id, status, flow_id}	AUTOMA_WEBHOOK_TOKEN
POST	/api/form/submit	JSON libre	{ok, saved_path}	_authorize_mutation
POST	/run?flow=<folder>	Formulario	303 → /run/<flow_id>/<run_id>	_authorize_mutation
POST	/flow/<folder>/config	config_json=<JSON>	HTML	_authorize_mutation
POST	/flow/<folder>/schedule	enabled, interval_seconds, cron_expression	303 → /#schedule	_authorize_mutation

1.3 Códigos de estado

Código	Cuándo
200	Éxito
303	Redirección tras un POST de formulario
400	folder que no pasa _safe_folder, JSON malformado, cuerpo vacío en /api/form/submit, error al guardar config o schedule
401	No autorizado: token inválido, Host no loopback, Origin/Referer incoherentes, o AUTOMA_WEBHOOK_TOKEN sin definir
404	Flow inexistente, corrida inexistente, ruta no servida, archivo no encontrado en /file
409	Flow marcado preview: true o con archivo .disabled
415	/file con extensión de la lista bloqueada
500	FlowExecutionError en la ejecución síncrona (/run, /api/hook) o al instanciar el orquestador

2. Ejemplos de uso

Disparar un flow y hacer polling

```
# 1. Disparo asíncrono: devuelve de inmediato
curl -s -X POST http://127.0.0.1:8787/api/run/05_system_healthcheck \
  -H 'Content-Type: application/json' \
  -d '{}'
```

```
{
  "ok": true,
  "run_id": "20260827T143052123456Z",
  "status": "running",
```

```
"flow_id": "system_healthcheck"
}
```

```
# 2. Polling del estado paso a paso
curl -s http://127.0.0.1:8787/api/runs/20260827T143052123456Z/status
```

```
{
  "ok": true,
  "run_id": "20260827T143052123456Z",
  "flow_id": "system_healthcheck",
  "flow_folder": "05_system_healthcheck",
  "flow_name": "Healthcheck del sistema",
  "status": "completed",
  "steps": [
    {"step_id": "take_snapshot", "action": "system.snapshot_system", "status": "success", "attempt": 1, "duration_": 0.001},
    {"step_id": "evaluate_snapshot", "action": "rules.evaluate", "status": "success", "attempt": 1, "duration_": 0.001},
    {"step_id": "write_snapshot", "action": "filesystem.write_json", "status": "success", "attempt": 1, "duration_": 0.001}
  ],
  "duration_seconds": 0.4312,
  "started_at": "2026-08-27T14:30:52.200000+00:00",
  "finished_at": "2026-08-27T14:30:52.631200+00:00",
  "error": null
}
```

Las 11 claves de primer nivel son exactamente las que devuelve `app/server.py:_run_status_payload`, que combina la fila de `runs`, los registros de `steps` y la lista de pasos del manifest. Un paso sin registro aparece como `running` si es el primero pendiente de una corrida viva, `pending` para los siguientes de esa misma corrida, y `not_taken` si la corrida ya terminó —porque entonces significa «rama no tomada», no «pendiente».

Disparar con override de contexto

```
curl -s -X POST http://127.0.0.1:8787/api/run/03_folder_inventory \
  -H 'Content-Type: application/json' \
  -d '{"context_overrides": {"path_override": "C:/Users/ejemplo/Documentos"}}'
```

El override reemplaza claves de **primer nivel**; no hay fusión profunda.

Webhook entrante

```
# Requiere AUTOMA_WEBHOOK_TOKEN definido en el entorno del panel
curl -s -X POST http://127.0.0.1:8787/api/hook/05_system_healthcheck \
  -H 'X-Automa-Token: TOKEN-DE-EJEMPLO-NO-REAL'
```

Diferencia crítica con `/api/run`: el webhook es **síncrono**. Ejecuta el flow completo y responde al terminar. Una petición contra un flow largo mantiene la conexión abierta hasta el final, y un timeout del cliente no cancela la corrida.

```
{"ok": true, "run_id": "20260827T143100987654Z", "status": "completed", "flow_id": "system_healthcheck"}
```

Sin token configurado:

```
{"ok": false, "error": "token inválido o AUTOMA_WEBHOOK_TOKEN no configurado"}
```

Panel con token

```
export AUTOMA_PANEL_TOKEN='TOKEN-DE-EJEMPLO-NO-REAL' # en el proceso del panel
curl -s -X POST http://127.0.0.1:8787/api/run/05_system_healthcheck \
  -H 'X-Automa-Token: TOKEN-DE-EJEMPLO-NO-REAL' -d '{}'
```

La comparación se hace con `hmac.compare_digest`, en tiempo constante (cierra CWE-208).

Programar un flow

```
# Cada 15 minutos, en punto (recordar: los campos van en UTC)
curl -s -X POST 'http://127.0.0.1:8787/flow/05_system_healthcheck/schedule' \
  -d 'enabled=on' --data-urlencode 'cron_expression=*/15 * * * *'

# 0 por intervalo simple, en segundos
curl -s -X POST 'http://127.0.0.1:8787/flow/05_system_healthcheck/schedule' \
  -d 'enabled=on' -d 'interval_seconds=900'
```

`set_schedule` prioriza `cron_expression` si viene; el panel envía `interval_seconds` solo cuando no hay cron.

Servir un archivo de salida

```
curl -s -o captura.png 'http://127.0.0.1:8787/file?path=output/screenshots/desktop_clean_20260827_143052.png'
```

`/file` acepta **solo rutas relativas** a la raíz del proyecto y rechaza `.html`, `.htm`, `.xhtml`, `.xml`, `.svg`, `.js`, `.mjs` y `.css` con 415. Los `.png`, `.json`, `.csv`, `.md`, `.log` y `.jsonl` **sí se sirven**.

3. Contrato de datos: `/metrics` en formato Prometheus

`engine/metrics.py::prometheus_text` emite un puñado de series. Ejemplo real de la forma:

```
# HELP flujo_runs_total Total de corridas por estado.
# TYPE flujo_runs_total counter
flujo_runs_total{status="completed"} 148
flujo_runs_total{status="failed"} 3
# HELP flujo_run_duration_seconds_avg Duración promedio histórica.
# TYPE flujo_run_duration_seconds_avg gauge
flujo_run_duration_seconds_avg 0.8123456
flujo_runs_window_completed 96
flujo_runs_window_failed 2
```

Observaciones sobre el contrato:

- El prefijo es `flujo_`, no `automa_`. Es un resto del nombre anterior del proyecto y quedaría feo cambiarlo sin romper dashboards existentes. Referencia histórica, no error.
- `flujo_runs_window_completed` y `flujo_runs_window_failed` se emiten **sin # HELP ni # TYPE**. Prometheus lo tolera, pero es una inconsistencia del formato.
- La ventana es de 200 corridas, fija (`overview(window_runs=200)`).
- El endpoint **no exige autenticación**. Un scraper en la red local podría leerlo si el panel se expusiera fuera de loopback.

`/api/metrics` devuelve el mismo material en JSON, con más detalle: `slowest_actions`, `retries_top_actions`, `failed_top_actions` y el desglose `by_flow`.

4. Integraciones salientes

4.1 Chromium vía Playwright

Aspecto	Detalle
Acciones	<code>browser.capture_page</code> , <code>browser.fill_form</code> , <code>browser.extract_content</code> , <code>browser.crawl_site</code>
Instalación	<code>pip install playwright && python -m playwright install chromium</code>
No declarada	<code>playwright</code> no está en <code>pyproject.toml</code> ni en <code>requirements.txt</code>
Modo	Headless salvo <code>browser.fill_form</code> , que por defecto abre ventana visible

User-Agent	<code>automa-pc</code> en <code>extract_content</code> y <code>crawl_site</code> ; el de Chromium por defecto en los otros dos
Timeout	<code>timeout_seconds=30.0</code> por defecto, aplicado con <code>page.set_default_timeout</code>
Degradación	<code>RuntimeError</code> con el comando de instalación exacto

`_to_url` acepta `http://`, `https://`, `file://` o una ruta local a un `.html`. Una ruta inexistente que no sea URL levanta `FileNotFoundError`.

4.2 robots.txt

`actions/browser_extract.py::RobotsCache` consulta `<esquema>://<host>/robots.txt` una vez por host, con `User-Agent: automa-pc` y timeout de 5 s.

Situación	Comportamiento
<code>robots.txt</code> legible	Se aplica <code>RobotFileParser.can_fetch</code>
Estado ≥ 400	Se permite; <code>checked=True</code>
Error de red	Se permite ; <code>checked=False</code> , reportado en <code>robots_checked_hosts</code>
Esquema no http(s) (por ejemplo <code>file://</code>)	Se permite sin consultar

El fallo es permisivo pero **declarado**: el docstring lo dice y el reporte del flow 22 lo expone. Es la decisión honesta.

4.3 Verificación de enlaces

`http.check_urls` hace `HEAD` con redirects y, ante `405` o `501`, reintentra con `GET` en modo `stream` cerrando la respuesta sin descargar el cuerpo. `file://` se verifica por existencia en disco. `mailto:` y `tel:` se marcan `skipped`.

Sin control de reintentos ni de concurrencia: las peticiones son secuenciales, con `delay_seconds` opcional entre ellas. Un `max_urls` de 100 contra un servidor lento con timeout de 10 s puede tardar hasta 1 000 segundos. **INFERENCIA**.

4.4 Notificaciones salientes

`actions/notify.py::send_notification` con `backend: "webhook"`:

```
POST <target>
Content-Type: application/json
Authorization: Bearer <token resuelto>    ← solo si se pasó token

{
  "text": "[!] Cambio detectado en https://ejemplo.example/pagina · hash a3f5...",
  "timestamp": "2026-08-27T14:30:52.123456+00:00"
}
```

Compatible con Slack y Discord si el endpoint acepta `{"text": "..."}`. El campo `extra` del manifest se fusiona en el cuerpo.

Resolución de secretos: un `token` que empiece por `@secret:` se resuelve con `engine.secrets.get_secret`. Ejemplo de manifest:

```
{"action": "notify.send",
 "params": {"message": "...", "backend": "webhook",
            "target": "https://hooks.ejemplo.example/servicios/XXXX",
            "token": "@secret:NOTIFY_TOKEN"}}
```

El token nunca se devuelve en el resultado. `record` contiene `backend`, `message`, `timestamp`, `sent`, `target` y `status_code` — no el token. Es correcto y deliberado, porque ese resultado acaba en `runs.context_json`.

Sin reintentos ni cola: un webhook caído produce una excepción de `requests` que el orquestador convierte en fallo del paso. La única política de reintento disponible es el campo `retries` del paso en el manifest.

Ningún flow usa **backend**: **"webhook"** por defecto. Los flows 23 y 26 traen `notify_backend: "file"` en su `context.example.json`.

4.5 Tesseract OCR

Integración con un **binario externo**, no con un servicio. `pytesseract` invoca `tesseract` en el sistema. `OCRImageAnalyzer._tesseract_binary_available` lo busca en el PATH y, además, en `C:/Program Files/Tesseract-OCR/tesseract.exe` y su equivalente (**x86**), que es donde el instalador de Windows lo deja sin añadirlo al PATH.

Si falta, devuelve un payload válido con `status: "unavailable"`, `reason` (`pytesseract_missing` o `tesseract_binary_missing`) y una `summary` con el comando de instalación por sistema operativo. **El flow no falla.**

5. Adaptador opcional de visión multimodal (no usado por ningún flow)

Este es el único punto del repositorio donde existe código capaz de llamar a un modelo de IA. Conviene documentarlo con precisión, porque el producto se presenta —correctamente— como determinista y sin LLM.

5.1 Qué existe

`plugins/analyzers/vision_model_analyzer.py::VisionModelAnalyzer.analyze` soporta tres proveedores:

provider	Endpoint	Autenticación	Red
mock (por defecto)	—	—	[NO]
openai_compatible	<endpoint>/chat/completions	Bearer desde <code>api_key</code> , <code>api_key_env</code> o <code>OPENAI_API_KEY</code>	[OK]
ollama	<endpoint o <code>http://127.0.0.1:11434</code> >/api/chat	Ninguna	[OK] (local por defecto)

`mock` no llama a nada: abre la imagen con Pillow, calcula brillo medio y RGB, y devuelve `target_found: False`, `confidence: 0.0`. Su propia `summary` lo declara: «Visión mock completada sin OCR ni proveedor externo [...] no identifica texto real».

Los dos proveedores reales codifican la imagen en base64 (data URL en el caso OpenAI-compatible), piden una respuesta **solo JSON** con `summary`, `target_found`, `confidence`, `target_bbox` y `visible_text`, y usan `_extract_json_object` para recuperar el objeto aunque venga envuelto en texto. `temperature: 0` en el caso OpenAI-compatible. Sin endpoint o sin modelo, levantan `RuntimeError`.

5.2 Por qué es inalcanzable desde el catálogo

Cadena de invocación completa, verificada con búsqueda en todo el repositorio:

```
VisionModelAnalyzer.analyze
  ← llamado SOLO por actions/vision.py::inspect_screen_target
    ← registrado como acción 'vision.inspect_screen_target'
      ← usada por 0 de los 27 manifests
```

Comprobación reproducible:

```
grep -l "inspect_screen_target" flows/*/manifest.json      # sin resultados
```

Conclusión verificable: con el catálogo tal y como se distribuye, **ningún flow puede provocar una llamada a un proveedor de IA**. La instancia `VISION_ANALYZER = VisionModelAnalyzer()` se crea al importar `actions/vision.py`, pero su constructor no hace nada y ningún método se ejecuta sin pasar por `inspect_screen_target`.

Activarlo exigiría **escribir un flow nuevo** que declarara esa acción con `vision_provider` distinto de `mock` y un `vision_endpoint`. Es una decisión explícita del operador, no un comportamiento por defecto.

5.3 Riesgos si alguien lo activara

Riesgo	Detalle
Exfiltración de pantalla	La imagen enviada es una captura del escritorio, codificada íntegra en base64
Clave en el manifest	<code>vision_api_key</code> es un parámetro de la acción: escribirla ahí la dejaría en <code>steps.params_json</code> en claro
Sin validación del endpoint	Cualquier URL es aceptada
Determinismo perdido	La decisión <code>click</code> / <code>recover</code> pasaría a depender de un modelo

La forma segura sería `vision_api_key_env` con el nombre de una variable, nunca `vision_api_key` con el valor. **NO DOCUMENTADO EN EL REPOSITORIO**: no hay guía sobre esto.

5.4 Los stubs de `decision/`

`decision/optional_ai.py::suggest_step_order` y `decision/rules.py::prioritize_steps` devuelven la lista de pasos sin tocarla y **nadie los importa**. La docstring del primero documenta la intención de diseño, que sigue siendo válida aunque el código no exista:

«Stub para futura integración IA. La IA solo debe sugerir orden o prioridad, nunca reemplazar la ejecución del motor.»

Es una decisión arquitectónica registrada. El código muerto asociado está en [15 · Riesgos y deuda técnica](#).

6. Interfaces de plataforma (Windows)

No son APIs de red, pero son las integraciones que definen el producto.

Interfaz	Vía	Acciones	Qué falla sin ella
Captura de framebuffer	<code>mss</code> , respaldo <code>Pillow.ImageGrab</code>	<code>screen.*</code>	<code>RuntimeError</code> explícito
Teclado y ratón	<code>pyautogui</code>	<code>ui.hotkey</code> , <code>type_text</code> , <code>click</code> , <code>click_bbox</code>	<code>RuntimeError</code> con instrucción
Ventana en foco	<code>pygetwindow</code>	<code>screen.capture_active_window</code>	<code>RuntimeError</code> con motivo
Portapapeles	<code>pyperclip</code>	<code>system.read_clipboard</code>	<code>available: False</code> con razón, sin excepción
PowerShell	<code>subprocess</code>	<code>system.run_powershell</code>	Depende del SO
Lanzador de procesos	<code>subprocess.Popen</code> + <code>shlex</code>	<code>ui.launch_process</code>	—
Navegador del sistema	<code>webbrowser</code>	<code>ui.open_url</code> , <code>open_file_in_browser</code>	—
Ventana nativa	<code>pywebview</code> (<code>edgechromium</code> en Windows)	<code>automa-desktop</code>	Código de salida 2
CPU, RAM, disco, procesos	<code>psutil</code>	<code>system.*</code>	—

URIs `ms-settings:` — el flow 11 usa `ui.open_url` con URIs del tipo `ms-settings:network`. `webbrowser.open` delega en el manejador de protocolo de Windows. Es una integración con el shell del sistema operativo, no con la web.

7. Extensión por terceros: entry points

`LazyActionRegistry._maybe_load_entry_points` descubre el grupo `automa.actions`:

```
# pyproject.toml de un paquete de terceros
[project.entry-points."automa.actions"]
"miempresa.exportar_sap" = "mi_paquete.acciones:exportar_sap"
```

Reglas del mecanismo, verificadas en el código:

- Los entry points se cargan **una sola vez**, de forma perezosa, cuando se pide un nombre desconocido o se llama a `keys()`.
- Se usa `setdefault`: **una acción interna nunca puede ser sobrescrita** por un paquete externo. Es una decisión de seguridad relevante.
- La función debe aceptar los parámetros como argumentos por nombre y devolver un `dict` serializable a JSON.

Contrato completo en `docs/EXTENSION.md`.

Discrepancia verificada: `_BUILT_IN_ACTIONS` declara 36 acciones y `[project.entry-points."automa.actions"]` del `pyproject.toml` solo 31. Faltan `browser.capture_page`, `browser.crawl_site`, `browser.extract_content`, `browser.fill_form` y `http.check_urls`. No rompe el runtime —el diccionario interno se consulta primero— pero un paquete externo que inspeccione el grupo verá un catálogo incompleto. Registrado en 15.

8. Lo que NO existe

Verificado con búsqueda en todo el repositorio:

No existe	Comprobación
Cliente de base de datos remota	Solo <code>sqlite3</code>
Cola de mensajes, broker, <code>celery</code>	Sin dependencias de ese tipo
Autenticación OAuth / OIDC / SSO	Sin bibliotecas de auth
Telemetría o <i>analytics</i> del producto	Sin cliente saliente propio
API pública versionada	Sin <code>/v1/</code> , sin OpenAPI
Servidor gRPC o WebSocket	Solo HTTP/1.1
Cliente de correo	Sin <code>smtplib</code> en el código de producción
Integración con servicios en la nube	Sin SDK de ningún proveedor
Actualización automática	El instalador no incluye <i>updater</i>

```
# Comando que respalda la afirmación de ausencia de red no declarada
grep -rn "requests\|urllib.request\|http.client\|socket\." --include="*.py" \
  actions/ engine/ app/ plugins/ decision/ | grep -v "^Binary"
```

Devuelve únicamente: `actions/http_actions.py`, `actions/notify.py`, `actions/browser_extract.py` (`RobotsCache`), `plugins/analyzers/vision_model_analyzer.py` y `app/desktop.py` (`socket.create_connection` contra `127.0.0.1`, para esperar al servidor local). No hay ninguna otra salida de red en el sistema.

Documentos relacionados: 05 · Referencia técnica · 08 · Flujo de datos · 10 · Configuración · 11 · Seguridad · 13 · Despliegue y operación

10 · Configuración

Todos los puntos de configuración del sistema: archivos, variables de entorno, valores por defecto, precedencias, diferencias entre entornos y **qué pasa exactamente cuando algo está mal configurado**. Ningún valor de este documento es real: los ejemplos usan marcadores explícitos y dominios reservados.

1. Mapa de la configuración

Automa no tiene un archivo de configuración global. La configuración vive repartida en seis sitios, cada uno con un alcance distinto:

Nivel	Dónde	Alcance	Versionado
1 · Manifest	<code>flows/<carpeta>/manifest.json</code>	Estructura y política de un flow	[OK]
2 · Contexto por defecto	<code>flows/<carpeta>/context.example.com</code>	Valores del flow, provistos por su autor	[OK]
3 · Contexto local	<code>flows/<carpeta>/context.user.json</code>	Override local del operador	[NO]
4 · Contexto del panel	<code>configs/<carpeta>.json</code>	Lo que guarda POST /flow/<f>/config	[OK] (solo existe uno)
5 · Overrides de invocación	Body de POST /api/run/<folder>	Una sola corrida	—
6 · Entorno y secretos	Variables de entorno, <code>secrets/secrets.json</code>	Todo el proceso	[NO]

No hay archivo `.env`, ni `settings.py`, ni `config.toml`. Verificado con `git ls-files`. El único manifiesto del proyecto es `pyproject.toml`, y no contiene configuración de runtime.

2. El manifest: contrato de un flow

`schemas/manifest.schema.json` es el contrato completo. `additionalProperties: false` en la raíz y en cada paso: **un campo desconocido es un error de validación**, no una clave ignorada.

2.1 Campos de la raíz

Campo	Tipo	Obligatorio	Por defecto	Qué pasa si falta o está mal
<code>id</code>	string <code>^[a-z0-9_]+\$</code>	[OK]	—	<code>KeyError</code> en <code>load_manifest</code> ; la CI lo detecta antes
<code>name</code>	string no vacío	[OK]	—	Igual
<code>steps</code>	array ≥ 1 elemento	[OK]	—	Igual
<code>description</code>	string	—	<code>""</code>	Se muestra vacía en el panel
<code>family</code>	string	—	<code>"general"</code>	El panel agrupa por familia

<code>start_step</code>	string	—	Primer paso del array	Si apunta a un paso inexistente: <code>FlowExecutionError</code> en la primera iteración
<code>max_steps_per_run</code>	integer 1–10000	—	200	Un flow con transiciones circulares se corta al alcanzarlo
<code>allowed_actions</code>	array de strings únicos	—	<code>null</code> = sin restricción	Ver §2.2
<code>required_secrets</code>	array de strings únicos	—	<code>[]</code>	Si el secreto no está en <code>os.environ</code> , el flow no arranca
<code>allowed_paths</code>	array de strings únicos	—	<code>null</code> = sin restricción	Ver §2.2
<code>max_runtime_seconds</code>	number ≥ 0	—	<code>null</code> = sin límite	Se comprueba entre pasos , no interrumpe uno en curso
<code>preview</code>	boolean	—	<code>false</code>	<code>true</code> → el panel muestra badge y los endpoints de ejecución responden 409

2.2 La trampa de la lista vacía

```
# engine/loader.py::load_manifest
allowed_actions=list(raw['allowed_actions']) if raw.get('allowed_actions') else None
```

`raw.get('allowed_actions')` con una **lista vacía es falsy**, así que `"allowed_actions": []` se convierte en `None`, es decir, en **política permisiva**.

Escribir una lista vacía con la intención de bloquear todas las acciones produce el efecto exactamente contrario. Lo mismo vale para `allowed_paths`. Es un caso límite real, verificado en el código y no cubierto por ninguna prueba. Registrado en [15 · Riesgos y deuda técnica](#).

2.3 Configuración de sandbox en el catálogo real

Configuración	Flows que la declaran	Cuáles
<code>allowed_actions</code>	13 de 27	08–20
<code>allowed_paths</code>	7 de 27	09, 12, 15, 16, 17, 18, 19
<code>max_runtime_seconds</code>	13 de 27	08–20, entre 3 y 40 segundos
<code>required_secrets</code>	0 de 27	Ninguno
<code>preview</code>	0 de 27	Ninguno

Los 14 flows restantes (01–07 y 21–27) corren con `SandboxPolicy` permisiva y el histórico lo registra:

`state['policy']['permissive'] == True`. Análisis en [11 · Seguridad](#).

2.4 Campos de un paso y de una transición

Campo del paso	Tipo	Por defecto	Notas
<code>id</code>	string	—	[OK] obligatorio, único dentro del flow

<code>action</code>	string	—	[OK] obligatorio, debe estar registrado
<code>params</code>	object	<code>{}</code>	Se pasan como argumentos por nombre a la acción
<code>save_as</code>	string	—	Clave del contexto donde guardar el resultado
<code>retries</code>	integer 0–100	<code>0</code>	Sin espera entre intentos
<code>when</code>	condition	—	Si no se cumple: <code>status: skipped</code>
<code>transitions</code>	array	<code>[]</code>	Sin transiciones, el orden es el del array

Campo de transición	Valores	Por defecto
<code>on</code>	<code>success, failure, any</code>	<code>success</code>
<code>next</code>	id de un paso existente	—
<code>end</code>	boolean	<code>false</code>
<code>when</code>	condition	—

Detalle no obvio de las transiciones de fallo. Para que un flow se recupere de un error, la transición `on: "failure"` debe apuntar a un paso **distinto del siguiente por defecto**. Si apunta al que ya venía a continuación, el motor no lo distingue de «no hay rama» y marca la corrida como `failed`. Ver [06 §1.2](#).

3. El contexto de un flow

3.1 Precedencia: la primera que exista gana

```
# engine/loader.py::load_context – orden de búsqueda
[--context RUTA] → configs/<carpeta>.json → flows/<c>/context.user.json → flows/<c>/context.example.json
```

No hay mezcla de claves. La primera fuente que exista se devuelve entera y las demás se ignoran por completo.

La consecuencia práctica más importante: guardar la configuración desde el panel crea `configs/<carpeta>.json`, que a partir de ese momento **oculta el `context.example.json`**. Si una versión posterior del flow añade una clave nueva al ejemplo, el flow configurado no la verá y el parámetro llegará a la acción como el placeholder literal sin resolver (`"{{ nueva_clave }}"`), que probablemente provoque un error de tipo.

Segunda trampa, más sutil: `Path('configs')` es **relativa al directorio de trabajo**, no a `root_dir()` ni a `data_dir()`. Pero `set_flow_config` escribe en `data_dir() / 'configs'`. En desarrollo coinciden; en el binario empaquetado **no**: el panel escribiría en `%LOCALAPPDATA%\Automa\configs\` y el loader leería `./configs` relativo al cwd. `installer/automa_entry.py` hace `os.chdir(data_dir())` precisamente para que coincidan, pero cualquier ejecución del CLI desde otra carpeta rompe la resolución. Registrado en [15](#).

3.2 Los contextos reales del catálogo

Los 27 `context.example.json`, agrupados por lo que configuran:

Flow	Claves	Valores por defecto
<code>01_screen_capture_analyze</code>	<code>analyzer_override</code>	—

02_screen_capture_browser	target_url, full_page, viewport_width, viewport_height, wait_seconds	data/web/control_page.html, true, 1280, 800, 1
03_folder_inventory	path_override	—
04_document_drop_pipeline	dropbox_path, notes	—
05_system_healthcheck	(ninguna)	—
06_process_watchdog	(ninguna)	—
07_browser_form_filler	target_url, seeds_path, used_path, headless, slow_mo_ms, viewport_width, viewport_height	HTML local, seed de 100, false, 250
08 10 13 14 20	dry_run + parámetro propio (folder_path, note, command)	dry_run presente en todos
09_show_desktop_capture	dry_run	—
11_settings_open_section	section	URI ms-settings:
12 16 17 19	(ninguna o dry_run)	—
15_clipboard_capture	max_chars	—
18_powershell_audit	command	Comando de la allowlist
21_web_content_extract	target_url, include_tables, max_links, wait_seconds, take_screenshot	data/web/demo_page.html, true, 100, 0.5, true
22_web_site_map	start_url, max_pages, max_depth, same_domain_only, delay_seconds, respect_robots, wait_seconds	data/web/site_demo/index.html, 10, 2, true, 0.2, true, 0.2
23_web_change_detector	target_url, state_path, wait_seconds, notify_backend, notify_target	data/web/demo_page.html, data/web_watch/demo_page.json, 0.5, file, output/notifications/web_changes.log
24_web_link_audit	target_url, max_links, timeout, wait_seconds	data/web/site_demo/index.html, 100, 10, 0.2
25_web_table_extract	target_url, wait_seconds	data/web/demo_page.html, 0.5
26_web_value_monitor	target_url, selector, threshold, state_path, wait_seconds, notify_backend, notify_target	data/web/demo_page.html, #precio, 1000, data/web_watch/precio_demo.json, 0.5, file, output/notifications/value_monitor.log
27_web_page_archive	target_url, archive_slug, wait_seconds	data/web/demo_page.html, demo_page, 0.5

Tres propiedades del catálogo por defecto que conviene proteger:

1. Todos los flows web apuntan a HTML locales del repositorio. La demo funciona sin internet y es determinista.
2. Los dos flows con notificación usan backend: "file". Nada sale del equipo hasta que el operador lo cambie a webhook.
3. Los flows que mueven teclado o ratón exponen dry_run. Un operador puede probarlos sin que ocurra nada.

3.3 dry_run: el interruptor de seguridad

Presente en los flows 08, 09, 10, 13, 14, 17 y 20. Con `dry_run: true`, las acciones de UI devuelven el payload que habrían producido con `sent/typed/clicked/launched` en `false`, **sin tocar el sistema**.

```
{"dry_run": true}
```

Es la forma recomendada de probar un flow de UI por primera vez. Registro: es también lo que permite que `tests/test_actions_basic.py` ejecute esas acciones en la CI sin efectos.

4. Variables de entorno

Variable	Obligatoria	Por defecto	Efecto	Leída por
AUTOMA_PANEL_TOKEN	[NO]	Sin definir	Si está: toda mutación exige X-Automa-Token idéntico. Si no: modo anti-CSRF por <code>Host/Origin/Referer</code>	<code>app/server.py::_authorize_mutation</code> vía <code>get_secret</code>
AUTOMA_WEBHOOK_TOKEN	[NO]	Sin definir	Habilita <code>POST /api/hook/<folder></code> . Sin ella, el webhook responde 401 siempre	<code>app/server.py::_check_webhook_token</code>
AUTOMA_ROOT	[NO]	Carpeta padre de <code>engine/</code>	Sobrescribe la raíz de solo lectura (<code>flows/</code> , <code>schemas/</code>)	<code>engine/paths.py::root_dir</code>
AUTOMA_DATA_ROOT	[NO]	<code>root_dir()</code> en desarrollo	Sobrescribe la raíz escribible (<code>db/</code> , <code>state/</code> , <code>logs/</code> , <code>configs/</code> , <code>secrets/</code>)	<code>engine/paths.py::data_dir</code>
OPENAI_API_KEY	[NO]	Sin definir	Respaldo de credencial del adaptador de visión. Ningún flow lo usa	<code>VisionModelAnalyzer._analyze_opena</code>
LOCALAPPDATA	Windows	Del sistema	Base de <code>data_dir()</code> en modo empaquetado	<code>engine/paths.py::data_dir</code>
XDG_DATA_HOME	Linux/macOS	<code>~/local/share</code>	Base de <code>data_dir()</code> en modo empaquetado	<code>engine/paths.py::data_dir</code>
PYTHONIOENCODING	[NO]	Del sistema	Recomendada en utf-8 para usar el CLI en Windows. Ver §7	Python

Ninguna es obligatoria para arrancar. El sistema funciona sin definir ni una sola.

Ambos tokens también pueden vivir en archivo

`_authorize_mutation` y `_check_webhook_token` usan `engine.secrets.get_secret`, que busca primero en `os.environ` y después en `secrets/secrets.json`:

```
{
  "AUTOMA_PANEL_TOKEN": "PON-AQUI-UN-TOKEN-LARGO-Y-ALEATORIO",
  "AUTOMA_WEBHOOK_TOKEN": "PON-AQUI-OTRO-TOKEN-DISTINTO"
}
```

Esos valores son marcadores, no credenciales. Genere los suyos con `python -c "import secrets; print(secrets.token_urlsafe(32))"`.

Inconsistencia verificada entre los dos mecanismos. `SandboxPolicy.check_required_secrets` lee **solo** `os.environ`, no `get_secret`. Un secreto declarado en `required_secrets` que viva únicamente en `secrets/secrets.json` **no satisface**

la comprobación y el flow no arranca. Los tokens del panel sí funcionan desde el archivo. Registrado en 15.

El archivo de secretos no está cifrado

`secrets/secrets.json` se escribe en texto plano con `json.dumps`. El docstring de `engine/secrets.py` declara la decisión sin adornos: «permisos del FS son el control de acceso». Está en `.gitignore` (`secrets/*.json`, con excepción de `.gitkeep`).

`engine.secrets.list_secret_names()` devuelve los **nombres** conocidos —los del archivo, más las variables de entorno que empiecen por `AUTOMA` o terminen en `_API_KEY` o `_TOKEN`— **sin exponer ningún valor**. Es el diseño correcto para una pantalla de diagnóstico.

5. Diferencias entre entornos

Aspecto	Desarrollo	Binario instalado	CI
<code>root_dir()</code>	Raíz del repositorio	<code>sys._MEIPASS</code> (bundle extraído)	Raíz del repositorio
<code>data_dir()</code>	Igual que <code>root_dir()</code>	<code>%LOCALAPPDATA%\Automa</code>	Raíz del repositorio
Directorio de trabajo	Donde se lance el comando	<code>data_dir()</code> (por <code>os.chdir</code>)	Raíz del repositorio
Base de datos	<code>db/runs.db</code>	<code>%LOCALAPPDATA%\Automa\db\runs.db</code> , efímera	<code>db/runs.db</code> , efímera
Salidas	<code>output/</code> del repo	<code>%LOCALAPPDATA%\Automa\output\</code>	<code>output/</code> del repo
Playwright	Instalado a mano	Chromium no viene en el bundle	Sin instalar
Tesseract	Opcional	No viene en el bundle	Sin instalar
Panel	<code>automa-panel</code> o <code>automa-desktop</code>	Ventana nativa	No se levanta como servicio
Tokens	Sin definir	Sin definir	Sin definir

No hay archivos de configuración por entorno. No existe `config.dev.json` ni `config.prod.json`. La única diferencia entre entornos la resuelve `engine/paths.py` en función de si el proceso está congelado.

NO IDENTIFICADO: no existe ningún sistema de *feature flags*. El único conmutador de disponibilidad es `preview` en el manifest y el archivo marcador `.disabled` en la carpeta del flow —y ningún flow del catálogo usa ninguno de los dos.

El archivo marcador `.disabled`

```
touch flows/07_browser_form_filler/.disabled
```

`app/server.py::_is_preview` lo comprueba antes que el campo `preview` del manifest. Permite **desactivar un flow localmente sin tocar el manifest ni commitear**. Los endpoints de ejecución responderán 409. Es un detalle operativo útil y poco visible; **NO DOCUMENTADO EN EL REPOSITORIO** fuera del propio docstring.

Con el flow desactivado, el **CLI sigue funcionando**: el comentario del schema lo declara como intencional, para poder probar un flow antes de quitarle la marca.

6. Consecuencias de una configuración incorrecta

Error de configuración	Síntoma	Gravedad
<code>"allowed_actions": []</code>	Se convierte en política permisiva , no restrictiva	Alta : falsa sensación de seguridad

<code>configs/<f>.json</code> obsoleto tras actualizar el flow	Placeholder sin resolver llega a la acción → <code>TypeError</code> o ruta literal <code>{{ clave }}</code>	Media
Umbral como texto (<code>"threshold": "1000"</code>)	<code>TypeError</code> al comparar en <code>conditions.matches</code>	Media
<code>cron_expression</code> con día de semana estilo crontab	Se ejecuta el día equivocado : aquí <code>0</code> es lunes, no domingo	Alta : silenciosa
<code>cron_expression</code> pensada en hora local	Se ejecuta en UTC	Media: silenciosa
<code>interval_seconds</code> y <code>cron_expression</code> a la vez	El cron gana, el intervalo queda inerte	Baja
<code>allowed_paths</code> con ruta relativa mal escrita	<code>SandboxViolation</code> y flow bloqueado	Baja: falla ruidosamente
<code>required_secrets</code> con secreto solo en <code>secrets.json</code>	El flow no arranca : <code>assert_secrets_present</code> solo mira el entorno	Media
<code>notify_backend: "webhook"</code> sin <code>notify_target</code>	<code>ValueError</code> y paso fallido	Baja
<code>AUTOMA_WEBHOOK_TOKEN</code> sin definir	Todo <code>POST /api/hook/*</code> responde <code>401</code>	Baja: es el diseño
<code>AUTOMA_PANEL_TOKEN</code> definido y el cliente no lo envía	<code>401</code> en toda mutación, incluido el propio panel web	Media
<code>target_url</code> apuntando a un archivo inexistente	<code>FileNotFoundError</code> en <code>_to_url</code>	Baja
<code>AUTOMA_DATA_ROOT</code> a una ruta sin permisos	<code>PermissionError</code> al crear el directorio	Media
<code>system.run_powershell</code> con <code>allowlist</code> ampliada en el manifest	La allowlist por defecto queda anulada	Alta

El caso del cron, con detalle

```
# engine/cron.py
(0, 6), # day of week (0=lunes, 6=domingo, estilo ISO)
```

En crontab(5) estándar, `0` es **domingo**. Aquí es **lunes**. Una expresión copiada de internet como `0 9 * * 1` («lunes a las 9» en cron estándar) se dispararía aquí en **martes**. Está documentado en un comentario del código, pero no en `docs/OPERACION.md` ni en la ayuda del panel.

Además, cuando ni `day of month` ni `day of week` son `*`, este cron exige que **ambos** coincidan (AND); crontab(5) usa OR. Y `next_after` trabaja siempre en **UTC**.

Tres divergencias con el estándar, todas silenciosas. Registradas en [15](#).

7. Configuración del entorno de ejecución en Windows

```
# Necesario para usar el CLI: sin esto, `automa list` y `automa run` revientan
# al imprimir JSON con caracteres no representables en cp1252.
$env:PYTHONIOENCODING = "utf-8"
```

Verificado durante el análisis: sin la variable, `python -m engine.runner list` termina en `UnicodeEncodeError: 'charmap' codec can't encode character '→'`. Con ella, devuelve el JSON completo. **El flow sí se ejecuta**; lo que falla es la impresión posterior.

8. Configuración de las herramientas de desarrollo

Toda en `pyproject.toml`:

Sección	Configuración
<code>[tool.pytest.ini_options]</code>	<code>-q --strict-markers --cov=engine --cov=actions --cov=app --cov=decision --cov-report=term-missing --cov-fail-under=54; testpaths = ["tests"]</code> ; marcadores <code>integration</code> y <code>slow</code>
<code>[tool.ruff]</code>	<code>line-length = 120, target-version = "py310"</code>
<code>[tool.ruff.lint]</code>	<code>select = ["E", "F", "W", "I", "B", "UP"], ignore = ["E501"]</code>
<code>[tool.ruff.lint.per-file-ignores]</code>	<code>"tests/**" = ["B011"]</code> (permite <code>assert False</code>)
<code>[tool.hatch.build.targets.wheel]</code>	<code>packages = ["engine", "actions", "app", "decision", "plugins", "scripts"]</code>

`--cov-fail-under=54` es un gate real: la suite falla si la cobertura baja de 54 %. Medición actual: **58,92 %**. El margen es de menos de 5 puntos.

`ignore = ["E501"]` con `line-length = 120` significa que el límite de línea se declara pero no se aplica; `app/server.py` tiene líneas mucho más largas por el HTML embebido.

Y `.pre-commit-config.yaml` añade hooks locales: `trailing-whitespace`, `end-of-file-fixer`, `check-yaml`, `check-json` (excluyendo los manifests, validados aparte), `check-toml`, `check-merge-conflict`, `check-added-large-files` (500 KB), `mixed-line-ending --fix=lf`, `ruff --fix`, `ruff-format` y `markdownlint-cli2`.

Nota sobre `ruff-format` en `pre-commit`: el hook está declarado pero la CI **no** ejecuta `ruff format --check`. Un colaborador sin `pre-commit install` puede subir código con formato distinto sin que nada falle.

9. Cómo configurar sin romper nada: receta

1. **Para un cambio puntual y reversible**, use `context_overrides` en la llamada a `POST /api/run/<folder>`. No toca ningún archivo.
2. **Para un cambio local persistente**, cree `flows/<carpeta>/context.user.json`. No está versionado y no oculta futuras claves del ejemplo... salvo que exista un `configs/<carpeta>.json`, que tiene más prioridad.
3. **Evite guardar desde el panel** si el flow va a actualizarse: el `configs/<f>.json` resultante congelará el conjunto de claves.
4. **Antes de programar con cron**, recuerde: 0 es lunes y la hora es UTC. Verifique con `python -c "from engine.cron import next_after; from datetime import datetime, timezone; print(next_after('* /15 * * * *', datetime.now(timezone.utc)))"`.
5. **Antes de ejecutar un flow de UI por primera vez**, póngalo en `dry_run: true`.
6. **Nunca escriba una credencial en `params`**. Un valor en `params` acaba en claro en `steps.params.json`. Use `@secret:NOMBRE` en `notify.send`, o la variable de entorno.

10. Lo que no se puede configurar

Valores fijos en el código, sin punto de extensión:

Constante	Valor	Dónde
Puerto del panel	8787	<code>app/server.py:run_server</code> , <code>app/desktop.py:DEFAULT_PORT</code> — configurable solo por argumento de <code>automa-desktop</code>
Bind del panel	127.0.0.1	Igual

Intervalo del scheduler del panel	2.0 s	app/server.py, línea de módulo
Ventana de métricas	200 corridas	engine/metrics.py::overview
Límite de resultados de métricas	10	Todas las consultas LIMIT 10
Espera del arranque de la ventana	8.0 s	app/desktop.py::_wait_for_server
Tamaño de ventana	1200×800	app/desktop.py::DEFAULT_SIZE
Allowlist de PowerShell por defecto	13 verbos	actions/system.py::_PS_DEFAULT_ALLOWLIST
Extensiones bloqueadas en /file	8	app/server.py::do_GET
Patrón de folder	^[A-Za-z0-9_\-]{1,64}\$	app/server.py::_FOLDER_RE
Campos del formulario del flow 07	10 nombres	actions/browser_form.py::fill_form
Extensiones de summarize_text_folder	.txt .md .log .csv .json	actions/filesystem.py
Prefijo de las métricas Prometheus	flujo_	engine/metrics.py
Retención de datos	No existe	—

Documentos relacionados: [02 · Instalación](#) · [05 · Referencia técnica](#) · [08 · Flujo de datos](#) · [11 · Seguridad](#) · [14 · Solución de problemas](#)

11 · Seguridad

Superficie de ataque, controles **presentes** y controles **ausentes**, medidos sobre el código del commit analizado. No se ejecutó ninguna prueba destructiva ni ningún ataque contra sistemas externos: todo lo que sigue es análisis estático más ejecución de la suite de pruebas del propio repositorio. Los hallazgos se describen **sin publicar cadenas explotables**.

1. Modelo de amenaza declarado por el proyecto

El repositorio no esconde su situación. `.github/workflows/security.yml` abre con un comentario que es, de hecho, el modelo de amenaza:

«este repo ejecuta acciones de teclado/mouse/captura de pantalla en el equipo del operador. Un commit malicioso fusionado a main se traduce directamente en RCE local cuando el operador hace pull.»

Esa frase ordena bien las prioridades: **la cadena de suministro es el vector principal**, porque el producto, por diseño, ya tiene permiso para hacer casi cualquier cosa en la máquina. Y el repositorio invierte en consecuencia (\$2).

Activos a proteger

Activo	Dónde vive	Por qué importa
La sesión de Windows del operador	El equipo	El sistema puede teclear, hacer clic y lanzar procesos en ella
Capturas del escritorio	<code>output/screenshots/*.png</code>	Pueden contener cualquier cosa visible en pantalla
Contenido del portapapeles	<code>output/reports/clipboard_*.json</code> , SQLite	Puede contener credenciales recién copiadas
Histórico completo	<code>db/runs.db</code> , <code>state/</code> , <code>logs/</code>	Contiene todo lo anterior, sin cifrar
Tokens del panel	<code>secrets/secrets.json</code> o entorno	Sin cifrar
El propio catálogo de flows	<code>flows/*/manifest.json</code>	Un manifest es código ejecutable en la práctica

2. Lo que está bien y hay que proteger

Un informe que solo enumerase problemas daría una imagen falsa. Estos controles están implementados, verificados y algunos son mejores que la media del sector.

2.1 Hardening de la cadena de suministro — 12 capas

`SECURITY.md` documenta la política completa. Verificado en los workflows:

#	Capa	Evidencia en el repositorio
1	SHA pin en toda acción de terceros	<code>actions/checkout@9c091bb...</code> , <code>astral-sh/setup-uv@fac544c...</code> , <code>github/codeql-action/*@8aad20d...</code>
2	<code>pin-check</code> con parser YAML real	<code>.github/workflows/workflow-security.yml</code> , <code>job pin-check</code>
3	Allowlist de excepciones vacía	Sin excepciones silenciosas
4	<code>persist-credentials: false</code>	En todos los <code>checkout</code>

5	Permisos mínimos por job	<code>permissions: contents: read;</code> <code>release.yml</code> arranca con <code>permissions: {}</code>
6	<code>concurrency: cancel-in-progress</code>	En <code>ci</code> , <code>security</code> , <code>workflow-security</code> , <code>markdown-docs</code> , <code>dependency-hygiene</code>
7	Sin <code>pull_request_target</code>	Verificado: no se usa como disparador en ningún workflow. La única aparición es un comentario de <code>workflow-security.yml</code> que lo lista como patrón prohibido
8	CodeQL <code>security-extended</code>	<code>security.yml</code> , con <code>security-and-quality</code> añadido
9	<code>actionlint</code> (con verificación de checksum) + <code>zizmor</code>	<code>workflow-security.yml</code>
10	<code>detect-secrets</code> sobre filesystem y 50 commits de historial	<code>security.yml</code> , con <code>fetch-depth: 0</code>
11	Detección de Trojan Source, ofuscación y URLs de exfiltración	<code>security.yml</code>
12	<code>pip-audit</code>	<code>security.yml</code>

La capa 10 es la más infrecuente: escanear el árbol completo en cada uno de los últimos 50 commits detecta un secreto **aunque se haya borrado después**.

La capa 9 verifica el **checksum del propio `actionlint`** antes de ejecutarlo, cerrando el caso de una herramienta de seguridad comprometida.

2.2 Sanitización de entrada en el panel

Control	Implementación	Cierra
Slug de <code>folder</code>	<code>_FOLDER_RE = ^[A-Za-z0-9_\-]{1,64}\$</code> aplicado en las 7 rutas que reciben un folder	Path traversal, NUL, no-ASCII
Path traversal en <code>/file</code>	Cuatro controles en cascada; ver §2.3	CWE-22
XSS reflejado desde <code>/file</code>	Allowlist negativa de 8 extensiones + <code>X-Content-Type-Options: nosniff</code>	CWE-79
Escapado del HTML del panel	<code>html.escape</code> en las rutas de renderizado	CWE-79
Comparación de tokens	<code>hmac.compare_digest</code>	CWE-208 (fuga por temporización)
Anti-CSRF sin token	<code>Host</code> loopback + <code>Origin/Referer</code> coherentes	CSRF, DNS rebinding
Command injection	<code>shell=True</code> prohibido en <code>ui.launch_process</code> ; <code>shlex.split</code> + <code>shell=False</code>	CWE-78
Inyección en PowerShell	Tokens prohibidos + allowlist de verbos	CWE-78
Inyección SQL	Todas las consultas son parametrizadas con <code>?</code>	CWE-89
Sanitización del tag en release	Regex <code>^v\d+\.\d+\.\d+([-.]?[\w]+)?\$,</code> con <code>throw</code> si no coincide	Inyección en el pipeline

Los comentarios del código citan explícitamente los CWE que cierran. Es una práctica poco habitual y muy útil para auditar.

2.3 Los cuatro controles de `/file`, en orden

1. Rechaza `\x00` y cualquier carácter de control (`ord(c) < 32`)
2. Rechaza rutas absolutas
3. `normpath(join(base, rel))` debe empezar por `base + os.sep`
4. Extensión no puede estar en `{.html .htm .xhtml .xml .svg .js .mjs .css}` → 415

El `os.sep` del punto 3 es el detalle fino: sin él, un directorio hermano llamado `repo-evil` pasaría el `startswith` de `repo`. El comentario del código lo señala.

Cubierto por `tests/test_security_hardening.py`.

2.4 Higiene de secretos en el resultado

`notify.send` con `backend: "webhook"` resuelve el token y lo usa en la cabecera `Authorization`, pero el **record** que devuelve no lo incluye. Es correcto y deliberado: ese resultado va al contexto y de ahí a `runs.context_json`.

`engine.secrets.list_secret_names()` devuelve **nombres**, nunca valores.

2.5 Salidas de red opt-in

Con la configuración por defecto, **el sistema no genera tráfico de red**. Verificado leyendo los 27 `context.example.json`: los nueve flows de navegador apuntan a HTML locales del repositorio, y los dos con notificación usan `backend: "file"`.

3. Autenticación y autorización

3.1 No hay usuarios

No existe autenticación de usuario, ni sesión, ni roles, ni permisos. El README del repositorio lo declara: «Multiusuario / RBAC — [alto] No — un operador local».

El modelo de confianza es: *quien tiene la sesión de Windows tiene el sistema*. Es coherente con un producto de escritorio, pero conviene enunciarlo porque cambia la lectura de todo lo demás.

3.2 Protección de las mutaciones: dos modos

```
# app/server.py::_authorize_mutation
panel_token = get_secret('AUTOMA_PANEL_TOKEN')
if panel_token:
    return (True, '') if self._check_token('AUTOMA_PANEL_TOKEN') else (False, 'token inválido ...')

host_only = (self.headers.get('Host') or '').split(':', 1)[0].lower()
if host_only not in {'127.0.0.1', 'localhost', ':::1', ':::1'}:
    return False, 'Host no loopback: cliente remoto debe usar AUTOMA_PANEL_TOKEN'
origin = self.headers.get('Origin')
if origin and origin != expected_origin: return False, ...
referer = self.headers.get('Referer')
if referer and not referer.startswith(expected_origin + '/') and referer != expected_origin:
    return False, ...
return True, ''
```

Modo 1 · con token. Toda mutación exige `X-Automa-Token` idéntico, comparado en tiempo constante. Robusto.

Modo 2 · sin token. Tres comprobaciones que cierran el ataque real: una web maliciosa que el operador visita y que intenta `fetch('http://127.0.0.1:8787/api/run/...')`. El navegador **siempre** envía `Origin` en un `fetch` cross-site, así que se bloquea.

Lo que el modo 2 no cubre, y hay que decirlo: las comprobaciones de `Origin` y `Referer` son condicionales (`if origin and ...`). Un cliente que **no envíe** esas cabeceras —`curl`, un script Python, cualquier proceso local— pasa el control con solo poner `Host: 127.0.0.1`. Es una decisión calibrada contra el navegador, no contra un proceso local hostil. El comentario del código es honesto sobre cuál es el caso que cubre.

3.3 Las lecturas no están protegidas

`do_GET` no llama a `_authorize_mutation` en ningún caso. Verificado leyendo el método completo. Consecuencia: incluso con `AUTOMA_PANEL_TOKEN` definido, siguen abiertos:

Ruta	Qué expone
GET /api/runs	context_json completo de cada corrida: portapapeles, texto OCR, contenido web, inventarios
GET /api/flows	Catálogo con todos los pasos y parámetros
GET /api/metrics,/metrics	Volumen, duraciones, tasas de fallo
GET /file?path=...	Cualquier archivo bajo la raíz salvo 8 extensiones. Los .png de las capturas SÍ se sirven
GET /run/<flow_id>/<run_id>	Detalle completo en HTML

Este es el hallazgo de mayor impacto del documento. Con el panel escuchando en loopback y un solo usuario, el riesgo real es bajo. Deja de serlo si alguien cambia el bind, monta un reverse proxy, o si hay más de una cuenta en el equipo.

3.4 El webhook

POST /api/hook/<folder> exige AUTOMA_WEBHOOK_TOKEN **siempre**, independientemente del modo. Sin la variable definida responde 401: está **deshabilitado por defecto**. Es la decisión correcta para la única superficie pensada para llamadas no locales.

REQUIERE VALIDACIÓN: el webhook es **síncrono**. Una petición contra un flow largo mantiene la conexión abierta y no hay límite de peticiones concurrentes. **INFERENCIA**: un atacante con el token podría agotar recursos disparando el mismo flow muchas veces —el **run_locks** no se aplica en esta ruta.

4. El sandbox: qué protege de verdad

4.1 Es declarativo, no de sistema operativo

docs/SEGURIDAD.md lo declara y el README lo marca como «[bajo] Sandbox declarativo, no proceso». El motor comprueba nombres de acción y prefijos de ruta **antes** de llamar a la función. Una vez dentro, la acción corre con **todos los permisos del usuario**: no hay contenedor, ni jaula, ni token restringido, ni AppContainer.

4.2 La mayoría del catálogo no lo usa

Conteo verificado leyendo los 27 manifests:

Control	Flows que lo declaran	Flows sin él
allowed_actions	13 (08–20)	14 (01–07, 21–27)
allowed_paths	7 (09, 12, 15–19)	20
max_runtime_seconds	13 (08–20)	14
required_secrets	0	27

Entre los 14 sin política están:

- **07_browser_form_filler** — lanza un navegador visible y escribe archivos.
- **Los siete flows web (21–27)** — salen a internet si se cambia la URL.
- **01, 03, 04, 05, 06** — leen el sistema de archivos y el estado del equipo.

INFERENCIA: el bloque 08–20 se añadió en la v0.2.0 con la política ya en mente, y la familia web de la v0.3.0 no la incorporó. No hay ninguna regla en la CI que exija declarar **allowed_actions**, así que el hueco puede crecer con cada caso nuevo.

4.3 La detección de rutas es por nombre de clave

```
if any(token in key.lower() for token in ('path', 'destination', 'source', 'output', 'file')):
```

`assert_paths_allowed` considera candidata a ruta **cualquier cadena cuya clave contenga** uno de esos cinco fragmentos. Consecuencias verificadas:

Parámetro	¿Se comprueba?
<code>output_path</code> , <code>save_data_path</code> , <code>seeds_path</code> , <code>state_path</code> , <code>track_state_path</code>	[OK]
<code>target</code> de <code>browser.extract_content / capture_page</code>	[NO] (puede ser una ruta local)
<code>command</code> de <code>ui.launch_process</code>	[NO] (contiene una ruta)
<code>path</code> de <code>filesystem.*</code>	[OK]

En el catálogo actual no abre un agujero, porque ninguno de los siete flows con `allowed_paths` usa esas acciones. Pero un autor de flows que confíe en `allowed_paths` para acotar un flow de navegador tendría una protección que no protege.

4.4 `max_runtime_seconds` no es un timeout

Se comprueba **entre pasos**. Una acción que se cuelga dentro no se interrumpe. Un `browser.crawl_site` contra un servidor lento, o un `http.check_urls` de 100 URLs con timeout de 10 s, pueden superar largamente el límite sin que nada los corte. Es un control de *arranque de paso*.

4.5 `required_secrets` mira solo el entorno

`SandboxPolicy.check_required_secrets` usa `os.environ.get`, no `engine.secrets.get_secret`. Un secreto que viva solo en `secrets/secrets.json` **no satisface** el requisito y el flow no arranca. Incoherencia entre dos mecanismos presentados como equivalentes.

5. La superficie de mayor riesgo: `system.run_powershell`

Dos controles bien contruidos y una puerta abierta por diseño.

Los controles:

```
forbidden_chars = (";", "|", "&", "`", ">", "<", "$(", "$_")
for token in forbidden_chars:
    if token in trimmed:
        raise ValueError(...)
verbs = tuple(allowlist) if allowlist else _PS_DEFAULT_ALLOWLIST
head = trimmed.split()[0]
if head not in verbs:
    raise ValueError(...)
```

Se ejecuta con `shell=False` y argumentos como lista, `-NoProfile -NonInteractive`. Los 13 verbos por defecto son todos `Get-*` de solo lectura.

La puerta abierta: el parámetro `allowlist` es **sobrescribible desde el manifest**. Un flow que declare `"allowlist": ["Remove-Item"]` obtiene exactamente eso, y los tokens prohibidos no impiden un comando destructivo de una sola palabra con argumentos.

Lectura correcta: no es una vulnerabilidad explotable desde fuera —requiere escribir un manifest, es decir, tener ya acceso de escritura al repositorio o al directorio de flows—. Pero significa que **la seguridad de esta acción depende de la revisión de código de los manifests, no de un control del motor**. Es exactamente el modelo de amenaza que `security.yml` describe, y por eso el hardening del CI es la defensa correcta.

NO IDENTIFICADO: ninguna regla de `scripts/validate_project.py` comprueba los `params` de un paso. Un manifest con una `allowlist` ampliada pasa la validación sin comentarios.

6. Almacenamiento de datos sensibles

Dato	Almacén	Cifrado	Control de acceso
------	---------	---------	-------------------

Tokens del panel	<code>secrets/secrets.json</code>	[NO] Texto plano	Permisos del sistema de archivos
Histórico completo	<code>db/runs.db</code>	[NO]	Permisos del sistema de archivos
Portapapeles capturado	SQLite + <code>output/reports/*.json</code>	[NO]	Igual
Capturas de escritorio	<code>output/screenshots/*.png</code>	[NO]	Igual, más GET /file sin auth
Texto OCR de ventanas	SQLite + reportes	[NO]	Igual
Salida de PowerShell	SQLite + reportes	[NO]	Igual

No hay cifrado en reposo en ningún punto. El docstring de `engine/secrets.py` lo declara sin adornos: «permisos del FS son el control de acceso». Para un producto local de un solo usuario es defendible; en un equipo compartido o con respaldo automático a la nube, no.

No hay retención ni purga. Verificado buscando `VACUUM`, `DELETE FROM runs`, `retention` y `backup` en todo el código: no existe ninguna rutina. Un flow programado cada 15 minutos genera 96 corridas diarias, cada una con su fila en SQLite, su `.jsonl` y su `.json` de estado, indefinidamente.

7. Dependencias

Política declarada

`pyproject.toml` fija un piso explícito por paquete, con el motivo escrito:

«Cotas de seguridad: piso explícito = primera versión sin CVE conocida al corte de la auditoría 2026-06-01 (ver SECURITY.md §"Auditoría 2026-06").»

Es una política razonada, no un rango arbitrario. Todas las dependencias llevan además cota superior de *major* (<13, <8, <3...), lo que evita saltos de versión mayor inesperados.

Lo que no está pinneado

No hay lockfile versionado. El comentario de `requirements.txt` lo reconoce: «Para reproducibilidad estricta usar `uv export` (lockfile en CI) — este archivo solo fija un piso seguro». Verificado: no existe `uv.lock` ni `requirements.lock` en `git ls-files`.

INFERENCIA: dos instalaciones en fechas distintas pueden resolver versiones distintas dentro del mismo rango. Un escáner de vulnerabilidades sobre las declaraciones no puede pronunciarse con precisión sobre versiones no fijadas.

`requirements.txt` incompleto

Declara seis paquetes y le faltan `pyperclip`, `PyGetWindow` y `pywebview`, que sí están en `pyproject.toml`. `make install` usa ese archivo. No es un riesgo de seguridad, pero deja el entorno incompleto.

Análisis de vulnerabilidades

`pip-audit` corre en `security.yml` (soft en PR, hard en `main`). **No se ejecutó en este análisis:** **REQUIERE VALIDACIÓN.** La CI del repositorio es la fuente para ese dato.

Dependencias con superficie especial

Paquete	Superficie
<code>pyautogui</code>	Control de teclado y ratón: si se compromete, controla la sesión
<code>playwright</code> (no declarado)	Descarga y ejecuta un Chromium completo
<code>pywebview</code>	Motor de navegador embebido (<code>edgechromium</code> en Windows)

pytesseract	Invoca un binario externo del sistema
-------------	---------------------------------------

8. Riesgos de inyección: revisión punto por punto

Vector	Estado	Evidencia
SQL injection	[OK] Cerrado	Todas las consultas usan ?. Sin concatenación de strings en SQL
Command injection (shell)	[OK] Cerrado	shell=True prohibido con ValueError; shlex.split + shell=False
PowerShell injection	[!] Mitigado	Tokens prohibidos + allowlist; pero la allowlist es sobrescribible desde el manifest
Path traversal	[OK] Cerrado en el panel	_safe_folder + los 4 controles de /file
Path traversal en acciones	[!] Parcial	allowed_paths es opcional y solo lo declaran 7 flows
XSS reflejado	[OK] Mitigado	Allowlist de extensiones en /file + nosniff + html.escape
XSS almacenado	[!] REQUIERE VALIDACIÓN	El panel renderiza contenido de las corridas; el escapado se aplica en las rutas revisadas, pero el HTML se construye por concatenación de f-strings en ~660 líneas. Una ruta sin html.escape sería difícil de detectar por lectura
CSRF	[OK] Mitigado	Modo 2 de _authorize_mutation
DNS rebinding	[OK] Mitigado	Comprobación de Host loopback
SSRF	[!] Abierto por diseño	http.fetch_url, check_urls y browser.* aceptan cualquier URL, incluidas IPs internas. Es la funcionalidad del producto
Deserialización insegura	[OK] Sin riesgo	Solo json.loads. Sin pickle, yaml.load ni eval
Carga de archivos	[OK] Sin superficie	No hay endpoint de subida. POST /api/form/submit escribe JSON con nombre generado por el servidor

Verificación de la penúltima fila:

```
grep -rn "pickle\|eval(\|exec(\|yaml.load\|os.system" --include="*.py" \
  engine/ actions/ app/ plugins/ decision/
```

Sin resultados en el código de producción.

9. Controles ausentes

Lista honesta de lo que **no** existe, verificado por búsqueda:

Control ausente	Impacto	Prioridad sugerida
Autenticación en los GET	Todo el histórico es legible sin token	Alta
Cifrado en reposo de secrets.json	Tokens en texto plano	Media

Cifrado del histórico	Capturas, OCR y portapapeles sin cifrar	Media
Retención y purga de datos	Crecimiento sin límite	Media
<code>allowed_actions</code> obligatorio en la CI	14 flows sin política	Alta
Validación de <code>params</code> contra la firma de la acción	Un manifest puede ampliar la allowlist de PowerShell sin aviso	Alta
Timeout real por acción	<code>max_runtime_seconds</code> no interrumpe	Media
Lock en las rutas del panel	El mismo flow puede correr dos veces en paralelo	Media
Lockfile de dependencias	Superficie no auditable con precisión	Media
Cabeceras de seguridad (CSP, X-Frame-Options, Referrer-Policy)	Solo se emite <code>nosniff</code> , y solo en <code>/file</code>	Baja
Límite de tasa en el webhook	Agotamiento de recursos con token válido	Baja
Registro de auditoría de accesos al panel	<code>log_message</code> está silenciado a propósito	Baja
<code>ruff format --check</code> en la CI	El hook existe en pre-commit pero la CI no lo aplica	Baja
Firma de código del <code>.exe</code>	<code>codesign_identity=None</code> en <code>automa.spec</code> ; SmartScreen avisará	Baja
Rotación de tokens	Sin mecanismo	Baja

Sobre el registro silenciado

```
def log_message(self, format: str, *args: Any) -> None: # silencia logs ruidosos
    return
```

`AppHandler.log_message` está sobrescrito para no imprimir nada. Es razonable para no llenar la consola de la ventana nativa, pero significa que **no queda ningún registro de qué peticiones recibió el panel**. Las corridas sí quedan en SQLite; los accesos de lectura, no.

10. Comprobación de secretos en el repositorio

Realizada durante este análisis sobre el árbol versionado:

```
grep -rni "(sk-|ghp_|gho_|AKIA|BEGIN [A-Z ]*PRIVATE KEY|password\s*=\s*[\"'])[^\"]{6,}|api[_-]?key\s*[:=])" \
$(git ls-files '*.py' '*.json' '*.toml' '*.md' '*.yaml' '*.ps1' '*.iss')
```

Sin coincidencias de credenciales reales. Las seis coincidencias del patrón son todas benignas y se listan aquí para que la comprobación sea reproducible:

Archivo	Qué es
<code>actions/vision.py</code> (x2)	Nombres de parámetro: <code>vision_api_key</code> , <code>api_key=</code>
<code>plugins/analyzers/vision_model_analyzer.py</code> (x3)	Nombres de parámetro: <code>api_key</code>
<code>docs/OPERACION.md</code> línea 130	Ejemplo de documentación: <code>set_secret("MY_API_KEY", "sk-...")</code> . El valor es literalmente <code>sk-...</code> , un marcador, no una clave

`secrets/` contiene únicamente `.gitkeep`. `.gitignore` excluye `secrets/*.json`, `db/*.db`, `logs/*.jsonl`, `state/*.json` y las salidas de `output/`.

La CI hace lo mismo con `detect-secrets==1.5.0` y además escanea los últimos 50 commits.

11. Protección de datos personales

NO IDENTIFICADO: no existe ninguna funcionalidad de privacidad. Sin anonimización, sin redacción de campos, sin consentimiento, sin política de retención, sin exportación ni borrado a petición.

Los datos que el sistema captura pueden ser personales o sensibles según lo que haya en pantalla o en el portapapeles del operador. Como todo permanece en el equipo y no sale a la red, la responsabilidad recae íntegramente en quien lo opera.

Recomendaciones para el operador, INFERENCIA a partir de la arquitectura:

- 1. No ejecute `15_clipboard_capture` con credenciales en el portapapeles.
- 2. Revise `output/screenshots/` antes de compartir el equipo o hacer respaldo.
- 3. Si el equipo tiene sincronización a la nube, excluya `output/`, `db/`, `logs/` y `state/`.
- 4. Defina `AUTOMA_PANEL_TOKEN` si hay más de una cuenta en el equipo — aunque recuerde que **no protege las lecturas**.
- 5. Purgue periódicamente a mano: no hay retención automática.

12. Recomendaciones priorizadas

Ninguna se aplicó: este documento es informativo, igual que 15 · Riesgos y deuda técnica.

#	Recomendación	Esfuerzo	Impacto
1	Exigir token también en <code>GET /api/runs, /api/flows, /api/metrics</code> y <code>/file</code> cuando <code>AUTOMA_PANEL_TOKEN</code> esté definido	Bajo (una llamada en <code>do_GET</code>)	Alto
2	Añadir a <code>validate_project.py</code> una regla que exija <code>allowed_actions</code> en todo flow nuevo	Bajo	Alto
3	Validar los <code>params</code> de cada paso contra la firma de la acción (<code>inspect.signature</code>)	Medio	Alto
4	Bloquear el override de <code>allowlist</code> en <code>system.run_powershell</code> , o exigir una lista blanca de allowlists	Bajo	Alto
5	Comprobar también <code>value</code> (no solo la clave) en <code>assert_paths_allowed</code> para detectar rutas en <code>target</code> y <code>command</code>	Medio	Medio
6	Generar y versionar un lockfile (<code>uv export</code>)	Bajo	Medio
7	Timeout real por acción con <code>concurrent.futures</code> o señal	Medio	Medio
8	Rutina de retención configurable	Medio	Medio
9	Aplicar <code>run_locks</code> también en las rutas del panel y del webhook	Bajo	Medio

10	Cifrar <code>secrets.json</code> con DPAPI en Windows	Medio	Medio
11	Auditar el escapado HTML de las ~660 líneas de renderizado del panel	Medio	Medio
12	Añadir <code>ruff format --check</code> a la CI	Bajo	Bajo

13. Lo que este análisis NO comprobó

Aspecto	Motivo
Vulnerabilidades conocidas de las dependencias	No se ejecutó <code>pip-audit</code> ni consulta a bases de CVE. La CI sí lo hace
Pruebas de penetración del panel	Fuera del alcance: sin pruebas destructivas
XSS almacenado real	Requeriría inyectar contenido en una corrida y observar el render
Comportamiento del binario empaquetado	No se compiló
Efectividad de <code>zizmor</code> , <code>actionlint</code> y CodeQL en este commit	Requiere ver las ejecuciones en GitHub Actions
Permisos efectivos de los archivos en Windows	No se auditaron las ACL
Resistencia a carga o denegación de servicio	Sin pruebas de estrés

Documentos relacionados: [03 · Arquitectura](#) · [07 · Base de datos](#) · [08 · Flujo de datos](#) · [10 · Configuración](#) · [15 · Riesgos y deuda técnica](#)

12 · Pruebas y calidad

Qué se prueba, qué no, con qué cobertura medida, qué herramientas de calidad hay y qué pruebas faltan, priorizadas. Todas las cifras de este documento se obtuvieron ejecutando los comandos del propio repositorio en el commit analizado.

1. Resultado medido

Comando	Resultado real	Duración
<code>python -m pytest</code>	150 passed, 0 failed	16,62 s
<code>python -m ruff check .</code>	All checks passed!	< 1 s
<code>python scripts/validate_project.py</code>	<code>{"ok": true, "flows_checked": 27, "registered_actions": 36, "errors": []}</code>	< 1 s
<code>python scripts/smoke_test.py</code>	<code>{"ok": true, "runs": 20, "db_path": ".../db/runs.db"}</code>	~2 s

Los cuatro gates del repositorio están **en verde** en el commit `ff246ab`.

2. Tipos de prueba

`pytest` con 17 archivos y **150 pruebas**. Dos marcadores declarados en `pyproject.toml` —`integration` y `slow`— con `--strict-markers` activo.

Tipo	Dónde	Qué ejercita
Unitarias puras	<code>test_template</code> , <code>test_conditions</code> , <code>test_cron</code> , <code>test_browser_extract</code> (mayoría)	Funciones sin efectos: placeholders, operadores, cron, hash, links, BFS
Unitarias con filesystem	<code>test_loader</code> , <code>test_actions_basic</code> , <code>test_secrets_and_notify</code>	Escriben en <code>tmp_path</code>
Integración con SQLite	<code>test_orchestrator</code> , <code>test_run_locks</code> , <code>test_metrics</code> , <code>test_panel_endpoints</code>	Base de datos real, redirigida por fixture
Contrato	<code>test_manifest_schema</code> , <code>test_action_registry</code>	JSON Schema y registro de acciones
Seguridad	<code>test_security_hardening</code>	<code>_safe_folder</code> , path traversal, allowlist de PowerShell, <code>shell=True</code>
Humo de extremo a extremo	<code>scripts/smoke_test.py</code> (fuera de pytest)	Tres flows completos + scheduler + base

El patrón que hace posible probar la familia web sin navegador

`actions/browser_extract.py` separa deliberadamente lo puro de lo impuro. Su docstring lo declara: la interacción con Playwright vive en tres funciones de pegamento y **toda la lógica** —hash, normalización, links, tracking, markdown, CSV, BFS— son funciones puras.

`tests/test_browser_extract.py` (358 líneas, el archivo de pruebas más grande) explota eso con una `FakePage` que imita la interfaz de Playwright y un `fetch_page` falso para el crawl. Resultado medible: **`actions/browser_extract.py` alcanza 91 % de cobertura sin lanzar un solo Chromium**, mientras que `browser_capture.py` y `browser_form.py` —que no tienen esa separación— se quedan en 18 % y 0 %.

Es la propiedad de diseño más valiosa del repositorio en materia de pruebas, y la que hay que exigir a cualquier acción nueva.

3. Cobertura medida

Gate configurado: `--cov-fail-under=54` en `pyproject.toml`. Ámbitos: `--cov=engine` `--cov=actions` `--cov=app` `--cov=decision`.

Medición del análisis: 2 527 sentencias, cobertura total entre 58,9 % y 60,0 %.

La cobertura no es reproducible entre ejecuciones. Dos corridas consecutivas de `pytest` en el mismo commit dieron 58,92 % y 60,0 %. La causa **INFERENCIA**: importar `app.server` arranca un hilo de `scheduler` (`SCHEDULER.start_in_background()` a nivel de módulo), y cuántas líneas de `engine/scheduler.py` se ejecutan depende del temporizado del hilo. En la primera corrida `scheduler.py` marcó 51 %, en la segunda 76 %. El margen sobre el gate de 54 % es de apenas 5 puntos, así que una corrida desafortunada podría hacer fallar la CI sin ningún cambio de código. Registrado en 15 · Riesgos y deuda técnica.

Cobertura por módulo

Módulo	Sentencias	Cobertura	Lectura
engine/loader.py	27	100 %	Contrato del manifest, completo
engine/logger.py	17	100 %	
engine/metrics.py	36	100 %	
engine/models.py	31	100 %	Dataclasses
engine/template.py	40	100 %	Los placeholders, completos
engine/sandbox.py	57	95 %	La política, casi completa
engine/conditions.py	52	90 %	La lógica pura de la familia web
engine/introspection.py	41	90 %	
actions/browser_extract.py	245	91 %	
engine/orchestrator.py	175	87 %	El bucle central
engine/cron.py	70	87 %	
engine/paths.py	29	86 %	
engine/action_registry.py	44	84 %	

engine/scheduler.py	55	51–76 %	Variable entre corridas
engine/database.py	129	73 %	
engine/secrets.py	33	73 %	
engine/catalog.py	43	72 %	
engine/state_store.py	16	75 %	
actions/rules.py	36	75 %	
actions/system.py	71	65 %	
actions/filesystem.py	71	59 %	
actions/notify.py	45	58 %	
actions/ui.py	64	56 %	Solo la rama <code>dry_run</code>
engine/manifest_schema.py	45	53 %	El respaldo sin <code>jsonschema</code> no se ejercita
actions/http_actions.py	45	53 %	El archivo más grande, el menos cubierto
app/desktop.py	60	48 %	
app/server.py	593	38 %	
actions/screen.py	86	30 %	Requiere escritorio gráfico
actions/browser_capture.py	34	18 %	Requiere Chromium
actions/browser_form.py	90	0 %	El flow más complejo, sin ninguna prueba
actions/vision.py	104	0 %	Ninguna prueba

<code>engine/runner.py</code>	37	0 %	El CLI, sin ninguna prueba
<code>decision/optional_ai.py</code>	3	0 %	Código muerto
<code>decision/rules.py</code>	3	0 %	Código muerto

`plugins/analyzers/` **no está en el ámbito de cobertura**: `--cov` cubre `engine`, `actions`, `app` y `decision`, pero no `plugins`. Sus 379 líneas —incluido `VisionModelAnalyzer` con 222— no se miden. Como `actions/vision.py` está al 0 %, **INFERENCIA**: los cuatro analizadores solo se ejecutan de forma indirecta, si acaso, y su comportamiento de degradación no está verificado por ninguna prueba.

Los cinco módulos al 0 %

Módulo	Líneas	Por qué no se prueba	¿Es aceptable?
<code>actions/vision.py</code>	104	Depende de Pillow y del binario <code>tesseract</code>	No . <code>find_text_in_image</code> e <code>inspect_screen_target</code> son lógica pura sobre el resultado del OCR: se pueden probar con un <code>matches</code> falso
<code>actions/browser_form.py</code>	90	Depende de Chromium	Parcialmente . <code>_pick_record</code> , <code>_load_used_ids</code> y <code>_save_used_ids</code> son puras y críticas
<code>engine/runner.py</code>	37	El CLI	No . <code>build_parser</code> es trivial de probar, y ahí vive un defecto real (§7)
<code>decision/rules.py</code>	3	Código muerto	Irrelevante
<code>decision/optional_ai.py</code>	3	Código muerto	Irrelevante

4. Fixtures y datos de prueba

`tests/conftest.py`, 28 líneas y solo dos fixtures:

Fixture	Qué hace
<code>tmp_runtime</code>	Crea <code>db/</code> , <code>logs/</code> , <code>state/</code> , <code>output/</code> , <code>configs/</code> y <code>flows/</code> dentro de un <code>tmp_path</code> y hace <code>monkeypatch.chdir(tmp_path)</code>
<code>project_root</code>	Devuelve la raíz del repositorio

El aislamiento se consigue **cambiando el directorio de trabajo**, no parcheando módulos. Su propio docstring explica el porqué: «El motor escribe rutas relativas a la `cwd`. Para tests deterministas cambiamos `cwd` al `tmp` y dejamos que cree sus propias carpetas». Es simple y funciona, y de paso documenta una propiedad real del sistema: en modo desarrollo, **todo se resuelve contra el `cwd`**.

`tests/test_actions_basic.py` sí ejercita la otra vía —`monkeypatch.setenv("AUTOMA_DATA_ROOT", ...)`— en las tres pruebas que cubren `engine/paths.py::data_dir`, añadidas en la v0.2.1 según el `CHANGELOG.md`. Eso explica el 86 % de `paths.py`.

Recurso	Ubicación	Uso
---------	-----------	-----

<code>tests/assets/sample_ui.png</code>	Versionado	Imagen para las pruebas de análisis
<code>data/seeds/form_seeds.json</code>	Versionado, 100 registros	Dataset del flow 07
<code>data/web/*.html</code>	Versionado, 7 archivos	Páginas de demo para los flows web
<code>data/inbox/example.txt</code> , <code>data/dropbox/inbox/*</code>	Versionado	Entrada de los flows 03 y 04
<code>tmp_path</code> de pytest	Efímero	Escrituras de las pruebas de filesystem

No hay factories ni datos generados. Los fixtures son archivos reales versionados, lo que hace las pruebas deterministas y legibles a costa de flexibilidad.

5. Herramientas de calidad

Análisis estático y formato

Herramienta	Configuración	Dónde corre
<code>ruff check</code>	<code>select = ["E", "F", "W", "I", "B", "UP"], ignore = ["E501"], line-length = 120</code>	CI + pre-commit
<code>ruff format</code>	Por defecto	Solo pre-commit
CodeQL	<code>security-extended</code> , <code>security-and-quality</code> sobre Python	<code>security.yml</code>
<code>actionlint</code>	Con verificación de checksum del binario	<code>workflow-security.yml</code>
<code>zizmor</code>	<code>==1.5.2</code> , pinneado	<code>workflow-security.yml</code>
<code>detect-secrets</code>	<code>==1.5.0</code> , filesystem + 50 commits	<code>security.yml</code>
<code>pip-audit</code>	Soft en PR, hard en <code>main</code>	<code>security.yml</code>
<code>markdownlint-cli2</code>	<code>v0.15.0</code>	Solo pre-commit
Verificador de enlaces Markdown	Script Python propio	<code>markdown-docs.yml</code>

Dos huecos entre pre-commit y CI. `ruff format --check` y `markdownlint-cli2` están declarados en `.pre-commit-config.yaml` pero **no se ejecutan en ningún workflow**. Verificado: `grep -rn "ruff format" .github/workflows/` no devuelve nada. Un colaborador que no ejecute `pre-commit install` puede subir código con formato distinto y Markdown con problemas de lint sin que ningún gate lo detecte.

Gates de la CI

Workflow	Qué falla la construcción
<code>ci.yml · job test</code>	<code>ruff check</code> , <code>validate_project.py</code> o <code>pytest</code> (incluido el umbral de cobertura), en 6 combinaciones: 2 SO × 3 versiones de Python
<code>ci.yml · job smoke</code>	<code>smoke_test.py</code> en Ubuntu con Python 3.12
<code>security.yml</code>	<code>detect-secrets</code> con hallazgo, <code>pip-audit</code> en <code>main</code> , Trojan Source, ofuscación
<code>workflow-security.yml</code>	<code>actionlint</code> , <code>zizmor</code> , <code>pin-check</code>

markdown-docs.yml

Cualquier enlace relativo roto en cualquier .md del repositorio

dependency-hygiene.yml es informativo: usa `continue-on-error: true`.

Nota sobre markdown-docs.yml: solo se dispara en `pull_request` con `paths: '**/*.md'`. Un push directo a `main` con un enlace roto **no lo detecta**.

6. El validador de manifests

`scripts/validate_project.py` es el gate específico del dominio. Cinco comprobaciones por manifest:

1. JSON Schema completo (o respaldo estructural si falta `jsonschema`).
2. IDs de paso no duplicados.
3. Toda `action` está registrada en `ACTION_REGISTRY`.
4. Toda `action` está dentro del `allowed_actions` del propio manifest, si lo declara.
5. `transitions.next` y `start_step` apuntan a pasos existentes.

La comprobación 4 es especialmente valiosa: impide publicar un flow cuya política se contradiga con sus propios pasos.

Lo que no comprueba, y son huecos reales:

Hueco	Consecuencia
Que los <code>params</code> coincidan con la firma de la acción	Un paso sin un parámetro obligatorio pasa la CI y falla con <code>TypeError</code> en ejecución
Que el flow declare <code>allowed_actions</code>	14 de 27 flows no lo hacen y nadie avisa
Que una transición tenga <code>next</code> o <code>end</code>	Una transición inerte pasa desapercibida
Que <code>allowed_actions: []</code> no sea una lista vacía	Se convierte silenciosamente en política permisiva
Que los <code>params</code> de <code>system.run_powershell</code> no amplíen la allowlist	Ver 11 · Seguridad

7. Módulos y comportamientos sin cobertura

7.1 El defecto del CLI que ninguna prueba habría detectado

`engine/runner.py` tiene **0 % de cobertura** y contiene un defecto reproducible:

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '→' in position 6042
```

`print(json.dumps(..., ensure_ascii=False))` sobre una consola Windows `cp1252`. Afecta a `list` y a `run`. **Una sola prueba que capturase `stdout` con codificación forzada lo habría detectado**. Es el ejemplo más claro del costo de dejar un módulo sin pruebas.

7.2 Comportamientos críticos sin prueba

Comportamiento	Módulo	Riesgo de no probarlo
<code>_pick_record</code> sin repetir y con reinicio de ciclo	<code>browser_form.py</code>	El flow 07 podría repetir registros sin que nadie lo note
<code>is_success</code> por comparación de texto	<code>browser_form.py</code>	Un cambio de copy en la página lo rompe en silencio
<code>find_text_in_image</code> sobre <code>matches</code> del OCR	<code>vision.py</code>	Lógica pura, trivialmente testeable

<code>inspect_screen_target</code> en sus tres modos	<code>vision.py</code>	100 líneas de lógica de decisión sin cubrir
Degradación de <code>OCRImageAnalyzer</code> sin <code>tesseract</code>	<code>plugins/</code>	Fuera del ámbito de <code>--cov</code>
<code>_smart_summary</code> para los 6 flows que cubre	<code>app/server.py</code>	Sin prueba de renderizado
<code>_is_preview</code> con <code>.disabled</code> y con <code>preview: true</code>	<code>app/server.py</code>	Mecanismo de desactivación sin cubrir
<code>render_*</code> del panel (≈660 líneas)	<code>app/server.py</code>	Escapado HTML sin verificar
<code>allowed_actions: []</code> → política permisiva	<code>engine/loader.py</code>	Caso límite con impacto de seguridad
<code>data_dir()</code> en modo congelado	<code>engine/paths.py</code>	Cubierto en parte (<code>CHANGELOG</code> v0.2.1 menciona 3 casos)
Recuperación por transición <code>on: failure</code>	<code>engine/orchestrator.py</code>	Cubierto en <code>test_orchestrator.py</code>

7.3 Lo que no se puede probar en CI y hay que asumir

Área	Motivo
<code>screen.capture_*</code> real	Requiere escritorio gráfico
<code>ui.hotkey</code> , <code>type_text</code> , <code>click</code> reales	Moverían teclado y ratón de la máquina de CI
<code>system.read_clipboard</code> con contenido	Requiere backend de portapapeles
<code>system.run_powershell</code> real	Requiere Windows con PowerShell
<code>capture_active_window</code>	Requiere una ventana en foco
Los flows 02, 07, 21–27 completos	Requieren Chromium instalado
Los flows 12 y 17 completos	Requieren el binario <code>tesseract</code>
El binario empaquetado	Requiere PyInstaller + Inno Setup

Para las acciones de UI, el repositorio resolvió bien el problema: `dry_run` permite ejercitar la firma y el payload sin efectos, y `tests/test_actions_basic.py` lo usa. Es la mitigación correcta dentro de lo posible.

8. Criterios de aceptación observados

Deducidos de los gates, no declarados en un documento:

Criterio	Umbral	Dónde se aplica
Todas las pruebas pasan	150/150	<code>ci.yml</code> , 6 combinaciones
Cobertura mínima	54 %	<code>pyproject.toml</code>
Lint sin hallazgos	0	<code>ci.yml</code>
Manifests válidos	0 errores	<code>ci.yml</code>
Smoke test completo	<code>ok: true</code>	<code>ci.yml</code> , job <code>smoke</code>
Sin secretos en el árbol ni en 50 commits	0 hallazgos	<code>security.yml</code>

Toda acción de terceros con SHA pin	100 %	<code>workflow-security.yml</code>
Enlaces Markdown válidos	0 rotos	<code>markdown-docs.yml</code> (solo en PR)

`docs/VALIDACION.md` del repositorio documenta esta batería desde la perspectiva del autor de flows.

9. Casos límite relevantes: cubiertos y no cubiertos

Cubiertos

Caso	Prueba
Placeholder exacto que resuelve a <code>None</code>	<code>test_template.py</code>
Llaves que no son placeholders (JSON en un comando)	<code>test_template.py</code>
Los 13 operadores de condición	<code>test_conditions.py</code>
Cron con listas, rangos y pasos	<code>test_cron.py</code>
Lock ya tomado por otra corrida	<code>test_run_locks.py</code>
Liberación forzada de lock	<code>test_run_locks.py</code>
Path traversal por <code>..</code> y por prefijo hermano	<code>test_security_hardening.py</code>
<code>shell=True</code> rechazado	<code>test_security_hardening.py</code>
PowerShell fuera de la allowlist	<code>test_security_hardening.py</code>
Reintentos y recuperación por transición	<code>test_orchestrator.py</code>
Precedencia del contexto	<code>test_loader.py</code>
<code>parse_number</code> con formatos mixtos	<code>test_browser_extract.py</code>
BFS con cotas, errores y robots	<code>test_browser_extract.py</code>
Primera corrida del tracking (<code>first_run</code>)	<code>test_browser_extract.py</code>
Validación con y sin <code>jsonschema</code>	<code>test_manifest_schema.py</code>

No cubiertos

Caso	Impacto
<code>allowed_actions: []</code> → permisivo	Seguridad
Salida del CLI en consola no-UTF-8	Defecto real y reproducible
Archivo de tracking corrupto → línea base reiniciada en silencio	Datos
<code>max_runtime_seconds</code> con un paso que se cuelga	Nunca interrumpe
Dos corridas del mismo flow en paralelo desde el panel	Escritura concurrente del tracking
Corrida huérfana en <code>running</code> tras matar el proceso	Nunca se limpia
<code>set_flow_config</code> con fallo entre archivo y tabla	Desincronización

Día de la semana del cron (0 = lunes, no domingo)	Silencioso
<code>configs/</code> relativo al cwd frente a <code>data_dir()</code>	Empaquetado

10. Propuesta priorizada de pruebas faltantes

Ninguna se implementó: este documento es informativo.

Prioridad alta

#	Prueba	Esfuerzo	Por qué
1	Salida del CLI con codificación forzada no-UTF-8	Bajo	Detecta un defecto real y actual
2	<code>allowed_actions: []</code> produce política permisiva	Bajo	Documenta un caso límite de seguridad
3	<code>_pick_record</code> : no repite, reinicia al agotar, marca antes de llenar	Bajo	90 líneas al 0 % en el flow más complejo
4	<code>find_text_in_image</code> con <code>matches</code> falso	Bajo	Lógica pura sin cubrir
5	<code>inspect_screen_target</code> en <code>ocr</code> , <code>vision</code> e <code>hybrid</code> con analizadores falsos	Medio	100 líneas de decisión sin cubrir
6	Regla en <code>validate_project.py</code> : todo flow debe declarar <code>allowed_actions</code>	Bajo	Cierra el hueco de los 14 flows

Prioridad media

#	Prueba	Esfuerzo
7	Añadir <code>plugins</code> al ámbito de <code>--cov</code> y cubrir la degradación de <code>OCRImageAnalyzer</code>	Bajo
8	<code>_is_preview</code> con <code>.disabled</code> y con <code>preview: true</code>	Bajo
9	<code>_smart_summary</code> para los 6 flows que cubre	Medio
10	Escapado HTML en las rutas de renderizado con contenido controlado	Medio
11	<code>apply_tracking</code> con archivo corrupto	Bajo
12	Semántica del día de la semana del cron, con caso explícito	Bajo
13	Estabilizar la cobertura: no arrancar el scheduler al importar en modo test	Medio

Prioridad baja

#	Prueba	Esfuerzo
---	--------	----------

14	<code>engine/runner.py::build_parser</code> con los tres subcomandos	Bajo
15	<code>app/desktop.py::_wait_for_server</code> con timeout	Bajo
16	<code>metrics.prometheus_text</code> con base vacía	Bajo
17	<code>data_dir()</code> en las tres ramas de resolución	Bajo

11. Lo que está bien y conviene proteger

- **Los cuatro gates están en verde y son rápidos** (≈17 s la suite completa). Un ciclo corto es lo que hace que la gente los ejecute.
- **La separación puro/impuro de `browser_extract.py`** permite probar la familia web sin navegador. Debería ser el patrón obligatorio de toda acción nueva.
- **`dry_run` en las acciones de UI** hace testeable lo que por naturaleza no lo es.
- **Los fixtures aíslan cambiando el directorio de trabajo**, no parcheando módulos. Las pruebas ejercitan el mismo camino de resolución de rutas que producción, y las tres de `data_dir()` usan la variable de entorno real `AUTOMA_DATA_ROOT`.
- **El validador de manifests comprueba la coherencia interna de la política**, no solo la sintaxis.
- **La matriz de CI cubre 6 combinaciones** de sistema operativo y versión de Python.
- **`--strict-markers`** impide que un marcador mal escrito pase inadvertido.

12. Lo que no se pudo verificar

Aspecto	Motivo
Ejecución real de los 27 flows	Requiere escritorio Windows interactivo, Chromium y <code>tesseract</code>
Resultado de CodeQL, <code>zizmor</code> , <code>detect-secrets</code> y <code>pip-audit</code> en este commit	Requiere ver las ejecuciones en GitHub Actions
<code>markdownlint-cli2</code> sobre los <code>.md</code>	No se ejecutó: no forma parte de ningún gate de CI
Cobertura de <code>plugins/</code>	Fuera del ámbito de <code>--cov</code>
Comportamiento del binario empaquetado	No se compiló
Estabilidad de la suite bajo carga o en paralelo	Sin pruebas de estrés

Documentos relacionados: 04 · Mapa del código · 06 · Explicación profunda · 13 · Despliegue y operación · 15 · Riesgos y deuda técnica · 19 · Matriz de trazabilidad

13 · Despliegue y operación

Entornos, construcción, empaquetado, CI/CD, publicación, logs, métricas, respaldo, recuperación, rollback y mantenimiento. Incluye cómo regenerar esta documentación en PDF.

1. Entornos

Automa no tiene entornos en el sentido de un servicio. Tiene **tres modos de ejecución**:

Modo	Cómo se lanza	Raíz de lectura	Raíz de escritura	Destinatario
Desarrollo	<code>uv run automa-desktop</code> o <code>automa-panel</code>	Repositorio	La misma	Desarrollador
Empaquetado	<code>Automa.exe</code> del instalador	<code>sys._MEIPASS</code>	<code>%LOCALAPPDATA%\Automa</code>	Operador final
CI	<code>uv run pytest</code> en GitHub Actions	Repositorio	La misma, efímera	Automático

La separación entre raíz de lectura y raíz de escritura es la decisión de despliegue más importante del proyecto, y nació de un fallo real documentado en el `CHANGELOG.md` de la v0.2.1: instalado bajo `Program Files`, `init_db()` levantaba `PermissionError [WinError 5]`.

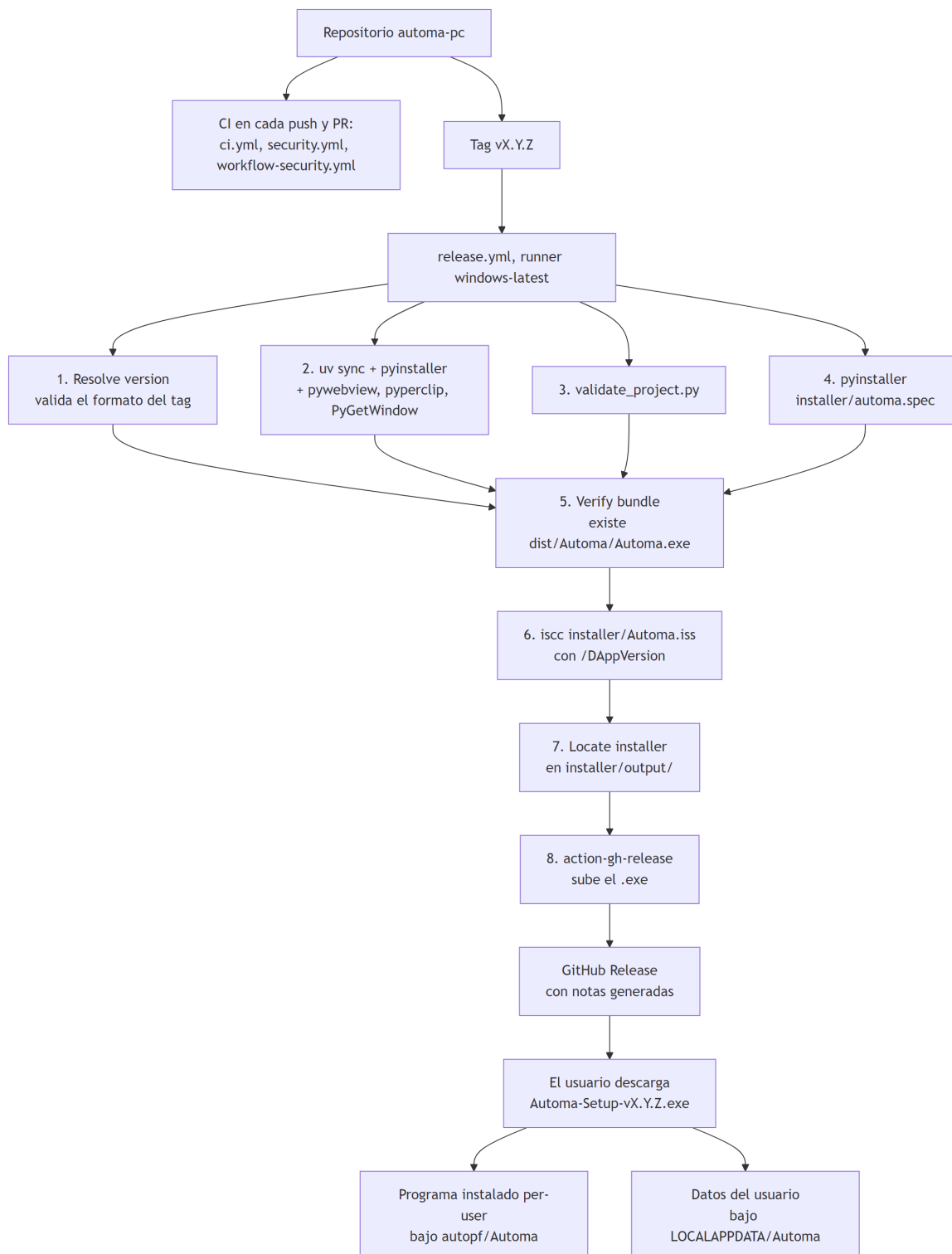


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: los ocho pasos del pipeline de release y dónde acaban el programa y sus datos.

Lo que no muestra: que `release.yml` no ejecuta la suite de pruebas. Corre `validate_project.py` pero no `pytest`. Un tag sobre un commit con pruebas rojas produciría un instalador igualmente. Es defendible —el tag suele ponerse sobre un commit ya validado por `ci.yml`— pero no hay nada que lo garantice. **INFERENCIA.**

2. Construcción del binario

PyInstaller

installer/automa.spec produce un bundle *one-folder* en dist/Automa/:

Elemento	Contenido
Entry point	installer/automa_entry.py
datas	flows/ y schemas/ del repositorio, más los datafiles de webview
hiddenimports	10 módulos de actions/, todos los submódulos de webview, mss, mss.tools, pyautogui, pygetwindow, pyperclip, PIL.Image, PIL.ImageGrab, pytesseract
excludes	tkinter, test, unittest
console	False — aplicación GUI, sin consola flotante
upx	False
icon	None
codesign_identity	None

[!] Hallazgo abierto: falta actions.browser_extract

La lista hiddenimports enumera actions.filesystem, screen, vision, system, rules, ui, http_actions, notify, browser_capture y browser_form. **No incluye actions.browser_extract**, añadido en la v0.3.0. Verificado: `grep -c "browser_extract" installer/automa.spec` devuelve 0.

LazyActionRegistry.get importa con `import_module(module_name)`, un import **dinámico** que el analizador estático de PyInstaller no rastrea. **INFERENCIA**: el módulo no se incluiría en el bundle y los siete flows de la familia web (21–27) fallarían con `ModuleNotFoundError: No module named 'actions.browser_extract'` dentro del .exe.

REQUIERE VALIDACIÓN: no se compiló durante este análisis. La comprobación es directa: compilar, instalar y ejecutar el flow 21.

Registrado en 15 · Riesgos y deuda técnica.

Segundo punto de atención en el mismo archivo: playwright no aparece ni en datas ni en hiddenimports, y el navegador Chromium tampoco se empaqueta. Los nueve flows de navegador no funcionarían en el binario aunque se arreglara lo anterior. **INFERENCIA** coherente con el tamaño del instalador: incluir Chromium añadiría cientos de megabytes.

Inno Setup

installer/Automa.iss empaqueta dist/Automa/ en un .exe instalable:

Directiva	Valor	Consecuencia
PrivilegesRequired	lowest	Instala por usuario, sin pedir administrado
PrivilegesRequiredOverridesAllowed	dialog	El usuario puede elegir instalación para todos

DefaultDirName	{autopf}\Automa	%LOCALAPPDA en modo per-user
ArchitecturesAllowed	x64compatible	Solo 64 bits
Compression	Lzma2/ultra + SolidCompression	Instalador más pequeño, compilación más lenta
LicenseFile	..\LICENSE	MIT mostrada en el asistente
[Languages]	Español (por defecto) e inglés	
[Tasks] desktopicon	Flags: unchecked	Acceso directo en escritorio desmarcado por defecto
[Run]	nowait postinstall skipifsilent	«Launch Automa» al terminar
AppVersion	/DAppVersion=X.Y.Z desde CI; 0.3.0 por defecto	El valor por defecto es para builds locales

Compilación local

```
pwsh installer/build_local.ps1      # limpia build/ y dist/, corre PyInstaller, verifica el .exe
iscc installer/Automa.iss            # requiere Inno Setup 6+ en PATH
```

build_local.ps1 usa \$ErrorActionPreference = "Stop", borra build/ y dist/, ejecuta python -m PyInstaller installer/automa.spec --noconfirm --clean y verifica que dist/Automa/Automa.exe exista, informando su tamaño. Si no está, hace Write-Error y sale con código 1. Es una verificación real, no un echo optimista.

REQUIERE VALIDACIÓN: no se ejecutó en este análisis.

3. CI/CD: los seis workflows

Workflow	Disparo	Jobs	¿Bloquea?
----------	---------	------	-----------

ci.yml	push y PR a main	test (matriz 2x3), smoke (needs: test)	[OK]
security.yml	push, PR, lunes 06:00 UTC, manual	codeql, detect-secrets, más los de Trojan Source / ofuscación / pip-audit	[OK]
workflow-security.yml	push, PR, programado	actionlint, zizmor, pin-check	[OK]
markdown-docs.yml	PR con paths: '**/*.md'	link-check	[OK] solo en PR
dependency-hygiene.yml	PR, lunes 04:00 UTC, manual	outdated	[NO] continue-on-error
release.yml	tag v*.*.* o manual	build-windows	—

ci.yml en detalle

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
    python-version: ["3.10", "3.11", "3.12"]
```

Seis combinaciones. Pasos: checkout (SHA pin, persist-credentials: false) → setup-uv con caché sobre pyproject.toml → uv python install → uv sync --extra dev --extra schema → ruff check → validate_project.py → pytest.

fail-fast: false hace que un fallo en una combinación no cancele las demás: se ve el cuadro completo en una sola ejecución.

El job smoke depende de test y corre scripts/smoke_test.py en Ubuntu con Python 3.12.

release.yml en detalle

Tres controles de seguridad dignos de mención:

- Validación del tag antes de usarlo. if (\$tag -notmatch '^v\d+\.\d+\.\d+([-.]?[\w]+)?\$') { throw ... } — cierra la inyección por nombre de tag.
- Validación de APP_VERSION con el mismo patrón antes de pasarlo a iscc.
- permissions: {} a nivel de workflow, elevado a contents: write solo en el job que lo necesita.

Además, concurrency: cancel-in-progress: false — a diferencia del resto de workflows, una construcción de release no se cancela si llega otra.

Verificación de la cadena de suministro

workflow-security.yml tiene tres jobs, todos gates:

- actionlint — descarga el binario verificando su checksum antes de ejecutarlo.
- zizmor — pinneado a 1.5.2, análisis estático de los propios workflows.
- pin-check — parser YAML real con PyYAML pinneado; falla si aparece un uses: sin SHA.

4. Publicación

Canal	Estado	Evidencia
GitHub Releases	Automático por tag	release.yml con softprops/action-gh-release
Notas de versión	Generadas automáticamente	generate_release_notes: true

GitHub Pages	<code>docs/index.html + docs/.nojekyll</code>	REQUIERE VALIDACIÓN: la activación es una opción del repositorio en GitHub, no un archivo
PyPI	No	Sin workflow de publicación
Winget / Chocolatey / Scoop	No	Sin manifests
Firma de código	No	<code>codesign_identity=None</code> . SmartScreen mostrará aviso

NO IDENTIFICADO: no hay mecanismo de actualización automática. El usuario debe descargar e instalar la versión nueva a mano.

Procedimiento de release

INFERENCIA a partir de los workflows y del `CHANGELOG.md`; el repositorio no tiene un documento de proceso de release:

1. Actualizar la versión en `pyproject.toml`, `[project].version`.
2. Actualizar `CHANGELOG.md` con la sección de la versión.
3. Actualizar las insignias del `README.md` que citan la versión.
4. Verificar en verde: `pytest`, `ruff check`, `validate_project.py`.
5. Confirmar y subir a `main`; esperar a `ci.yml`, `security.yml` y `workflow-security.yml`.
6. Crear y subir el tag `vX.Y.Z`.
7. `release.yml` construye y publica.
8. Verificar el `.exe` publicado descargándolo e instalándolo.

Propuesta, no aplicada: añadir a ese procedimiento la regeneración de los PDF de esta documentación con `python scripts/build_docs_pdf.py`. No se ha modificado ninguna lista de comprobación existente del usuario.

5. Logs y observabilidad

Dónde mirar

Fuente	Contenido	Rotación
<code>logs/<flow_id>_<run_id>.jsonl</code>	Un evento por línea, append-only	Ninguna
<code>state/<flow_id>_<run_id>.json</code>	Snapshot completo de la corrida	Ninguna
Tabla <code>events</code>	Los mismos eventos, consultables	Ninguna
Tabla <code>runs</code>	Estado, duración, error, contexto	Ninguna
Consola del panel	Solo <code>Panel</code> disponible en <code>http://...</code> y las salidas de <code>notify.send</code> con <code>backend: log</code>	—

El servidor HTTP no registra peticiones. `AppHandler.log_message` está sobrescrito para no imprimir nada («silencia logs ruidosos»). Consecuencia operativa: no hay forma de saber qué peticiones recibió el panel.

El scheduler tampoco registra sus fallos. `_run_job` captura toda excepción con `pass`. El error queda en la fila de la corrida, pero **no hay alerta ni traza en consola**. Una tarea programada que falla siempre reprograma con normalidad su siguiente ejecución.

Métricas

```
curl http://127.0.0.1:8787/metrics # texto Prometheus
curl http://127.0.0.1:8787/api/metrics # JSON detallado
```

El dashboard visual está en <http://127.0.0.1:8787/metrics/dashboard>. Series expuestas: `flujo_runs_total{status=...}`, `flujo_run_duration_seconds_avg`, `flujo_runs_window_completed`, `flujo_runs_window_failed`.

[docs/METRICAS.md](#) del repositorio documenta el formato en detalle.

Alertas

NO IDENTIFICADO: no existe ningún sistema de alertas. `notify.send` es una acción que un flow puede invocar, no un mecanismo de monitoreo del sistema. Nada avisa de:

- Una corrida fallida.
- Una tarea programada que lleva días fallando.
- Un lock huérfano que bloquea un flow.
- Una corrida colgada en `running`.
- Disco lleno por acumulación de logs.

Todo se descubre mirando el panel o consultando SQLite.

6. Mantenimiento

Comprobaciones periódicas recomendadas

INFERENCIA a partir de la arquitectura; el [RUNBOOK.md](#) cubre parte de esto.

```
# Corridas colgadas en running (proceso muerto a mitad)
sqlite3 db/runs.db "SELECT run_id, flow_id, started_at FROM runs WHERE status='running';"

# Locks huérfanos que bloquean el scheduler
sqlite3 db/runs.db "SELECT * FROM run_locks;"

# Tareas programadas que fallan de forma sistemática
sqlite3 db/runs.db "SELECT flow_id, COUNT(*) FROM runs WHERE status='failed'
                  AND created_at > date('now','-7 day') GROUP BY flow_id;"

# Tamaño de los almacenes
du -sh db/ logs/ state/ output/

# Las filas de contexto más grandes
sqlite3 db/runs.db "SELECT run_id, LENGTH(context_json) FROM runs ORDER BY 2 DESC LIMIT 10;"
```

Liberar un lock atascado

Procedimiento documentado en [RUNBOOK.md](#):

```
from engine.database import force_release_lock
force_release_lock("05_system_healthcheck")
```

Necesario cuando el proceso murió con un lock tomado: **no hay liberación automática al arrancar**.

Purga manual

No hay ninguna rutina de retención. Verificado buscando `VACUUM`, `DELETE FROM runs` y `retention` en todo el código: no existe.

```
# Reinicio limpio del histórico: se recrea vacío en el siguiente arranque
rm db/runs.db

# Purga de logs y snapshots antiguos (más de 30 días)
find logs/ -name "*.jsonl" -mtime +30 -delete
find state/ -name "*.json" -mtime +30 -delete
```

Aviso importante antes de purgar `data/`: los archivos de tracking (`data/seeds/.used_indices.json`, `data/web_watch/*.json`) **no son residuo**. Borrarlos reinicia la memoria de los flows 07, 23 y 26: el 07 volverá a repetir

registros y el 23 reportará `first_run: true` sin detectar el cambio que estaba vigilando.

Estimación de crecimiento

INFERENCIA: un flow programado cada 15 minutos genera 96 corridas diarias. Cada una produce una fila en `runs`, N filas en `steps`, M filas en `events`, un `.jsonl` y un `.json`. En un año son ~35 000 corridas y ~70 000 archivos sueltos entre `logs/` y `state/`. Sin índices en `runs.flow_id` ni en `steps.run_id`, el panel y el dashboard se degradarán antes que el disco.

7. Respaldo, recuperación y rollback

Respaldo

```
# Base de datos en caliente (SQLite lo soporta con el proceso corriendo)
sqlite3 db/runs.db ".backup 'db/runs-backup-20260827.db'"

# Respaldo completo: incluye lo que NO está en la base
tar -czf automa-backup-20260827.tar.gz \
  db/runs.db logs/ state/ configs/ secrets/ \
  data/seeds/.used_indices.json data/web_watch/
```

Los dos últimos elementos son los que se olvidan y los que más duele perder.

Recuperación

1. Cerrar el panel.
2. Restaurar `db/runs.db` y las carpetas del respaldo.
3. Arrancar. `init_db()` no destruye nada: solo crea lo que falte.

Rollback de versión

Situación	Procedimiento
Binario instalado	Desinstalar desde Configuración → Aplicaciones e instalar el <code>.exe</code> de la versión anterior desde Releases. Los datos de <code>%LOCALAPPDATA%\Automa</code> sobreviven
Desde el código	<code>git checkout vX.Y.Z && uv sync --extra dev --extra schema</code>
Compatibilidad de la base	El esquema solo ha crecido (una columna añadida por migración). INFERENCIA: una base nueva funciona con una versión antigua, salvo que use la columna nueva

Efecto que sobrevive a la desinstalación. El desinstalador de Inno Setup borra `{app}`, no `%LOCALAPPDATA%\Automa`. **INFERENCIA:** la base de datos, los logs y todas las capturas —que pueden contener información sensible— **quedan en el disco tras desinstalar**. Hay que borrarlas a mano.

8. Generación de la documentación en PDF

Los Markdown de `docs/system-documentation/` son la **fuentes única**. Los PDF se generan a partir de ellos con `scripts/build_docs_pdf.py`, añadido por este análisis.

Uso

```
python scripts/build_docs_pdf.py          # los 20 documentos + el consolidado
python scripts/build_docs_pdf.py --only 03 # solo el 03, para iterar rápido
python scripts/build_docs_pdf.py --check   # comprueba dependencias sin generar
python scripts/build_docs_pdf.py --no-consolidado # omite el PDF unificado
python scripts/build_docs_pdf.py --out OTRA/RUTA # directorio de salida alternativo
```

Requisitos

<code>pip install markdown xhtml2pdf</code>	<code># obligatorio</code>
<code>npm i -g @mermaid-js/mermaid-cli</code>	<code># opcional: rasteriza los diagramas</code>

Qué hace y qué no

Aspecto	Comportamiento
Portada	Nombre del sistema, título, versión, commit y fecha
Versión	Leída de <code>pyproject.toml</code> , <code>[project].version</code>
Commit	Leído de <code>.git/HEAD</code> sin invocar <code>git</code> (soporta <code>packed-refs</code>)
Índice	Solo si el documento tiene 4 o más encabezados de nivel 2
Pie	<code>Sistema · Documento · versión · fecha · pág. N de M</code>
Diagramas Mermaid	Rasterizados a PNG con <code>mmdc</code> , cacheados por SHA-256 del código fuente en <code>.tmp/mermaid-cache/</code>
Si falta <code>mmdc</code>	El diagrama degrada al código fuente en bloque monoespaciado con aviso visible, y el resumen final dice cuántos degradaron
Salida	Un PDF por documento más <code>00-documentacion-completa.pdf</code>
Verificación	Comprueba que cada PDF tenga tamaño > 0 antes de contarlo como generado

Nunca degrada en silencio. Si `mmdc` no está o falla, el script lo dice en `stderr` al terminar.

Detalle de Windows: `mmdc` se instala como `.cmd` y Node ≥ 20.12 se niega a lanzarlo sin shell (endurecimiento por CVE-2024-27980). Por eso la invocación pasa por `shell=True` con los argumentos entrecomillados. Está comentado en el propio script.

Caché de diagramas

`.tmp/mermaid-cache/` guarda un PNG por diagrama, nombrado por los 16 primeros caracteres del SHA-256 de su código fuente. La segunda ejecución solo rehace lo que cambió. La carpeta `.tmp/` ya está en `.gitignore`.

9. Procedimiento de despliegue de un flow nuevo

No hay despliegue: un flow nuevo es una carpeta más.

1. Crear `flows/NN_nombre/` con `manifest.json`, `context.example.json` y `README.md`.
2. Declarar `allowed_actions`, `allowed_paths` y `max_runtime_seconds` — aunque 14 flows existentes no lo hagan, es lo correcto.
3. `python scripts/validate_project.py` debe devolver `ok: true`.
4. `python -m engine.runner run flows/NN_nombre` con `PYTHONIOENCODING=utf-8`.
5. Reiniciar el panel: `sync_flows` solo corre al importar `app.server`.

Paso 5, no obvio: el catálogo del panel se sincroniza **al arrancar**. Un flow añadido con el panel corriendo aparece en `list_flows()` —que lee el disco— pero su fila en la tabla `flows` no se actualiza hasta el siguiente arranque.

Contrato completo en `docs/CREAR_FLUJOS.md`.

10. Lo que no se pudo verificar

Aspecto	Motivo
Compilación con PyInstaller	No se ejecutó. Requiere PyInstaller instalado
Compilación con Inno Setup	No se ejecutó. Requiere Inno Setup 6+
Si los flows 21–27 funcionan en el <code>.exe</code>	Depende del hallazgo de <code>automa.spec</code> (\$2)
Instalación y desinstalación reales	No se probaron
Estado de GitHub Pages	Depende de la configuración del repositorio en GitHub
Ejecuciones reales de los workflows	Requiere ver GitHub Actions
Comportamiento del scheduler en ejecución prolongada	Requiere días de operación
Degradación con volumen alto de corridas	Sin pruebas de carga

Documentos relacionados: [02 · Instalación](#) · [07 · Base de datos](#) · [10 · Configuración](#) · [12 · Pruebas y calidad](#) · [14 · Solución de problemas](#)

14 · Solución de problemas

Guía práctica. Cada entrada sigue el mismo formato: **síntoma** → **causa** → **diagnóstico** → **solución** → **archivos** → **riesgo de aplicar la solución**. Los casos marcados **VERIFICADO** se reprodujeron durante este análisis.

Índice de síntomas

Área	Casos
Instalación y arranque	1.1 – 1.6
CLI	2.1 – 2.2
Panel	3.1 – 3.5
Ejecución de flows	4.1 – 4.8
Navegador y web	5.1 – 5.5
Pantalla, OCR y UI	6.1 – 6.5
Scheduler	7.1 – 7.4
Base de datos y almacenamiento	8.1 – 8.4
Pruebas y CI	9.1 – 9.4
Empaquetado	10.1 – 10.3

1. Instalación y arranque

1.1 · ERROR: pywebview no esta instalado, código de salida 2

- **Causa:** automa-desktop necesita pywebview y no está en el entorno. Suele ocurrir tras `make install`, que usa `requirements.txt` — donde `pywebview` no está declarado.
- **Diagnóstico:** `python -c "import webview; print(webview.__version__)"`
- **Solución:** `pip install -e ".[dev,schema]"` (instala desde `pyproject.toml`), o como mínimo `pip install pywebview`. Alternativa sin ventana: `automa-panel` y abrir `http://127.0.0.1:8787` en el navegador.
- **Archivos:** `app/desktop.py`, `pyproject.toml`, `requirements.txt`
- **Riesgo:** Ninguno.

1.2 · ERROR: el servidor HTTP no respondió en 127.0.0.1:8787 tras 8s, código 1

- **Causa:** El puerto está ocupado por otra instancia, o el servidor falló al arrancar en su hilo. `app/desktop.py::start_server_in_thread` **captura y descarta** cualquier excepción del servidor, así que el motivo real no se ve.
- **Diagnóstico:** `powershell netstat -ano | findstr :8787 python -m app.server # arranca en primer plano y muestra la traza real`
- **Solución:** cerrar la otra instancia, o `automa-desktop --port 8888`.
- **Archivos:** `app/desktop.py::wait_for_server`, `app/server.py::run_server`
- **Riesgo:** Ninguno. Cambiar el puerto no afecta a los datos.

1.3 · `PermissionError [WinError 5]` al escribir en `db/`

- **Causa:** El binario instalado bajo `Program Files` intenta escribir en su propio directorio. Era un fallo real de la v0.2.0, corregido en la v0.2.1.
- **Diagnóstico:** `python from engine.paths import root_dir, data_dir print(root_dir(), data_dir())` # deben ser distintos si está congelado
- **Solución:** actualizar a $\geq 0.2.1$. Si aparece en desarrollo, definir `AUTOMA_DATA_ROOT` a una ruta escribible.
- **Archivos:** `engine/paths.py::data_dir`, `installer/automa_entry.py`, `CHANGELOG.md`
- **Riesgo:** cambiar `AUTOMA_DATA_ROOT` mueve la base de datos: el histórico anterior deja de verse. Copie `db/runs.db` a la ruta nueva si quiere conservarlo.

1.4 · `ModuleNotFoundError` al ejecutar `python -m app.server` desde otra carpeta

- **Causa:** El paquete no está instalado y no se ejecuta desde la raíz del repositorio.
- **Diagnóstico:** `python -c "import engine; print(engine.__file__)"`
- **Solución:** `pip install -e .`, o ejecutar desde la raíz.
- **Archivos:** `pyproject.toml`, `[tool.hatch.build.targets.wheel]`
- **Riesgo:** Ninguno.

1.5 · `make install` deja el entorno incompleto

- **Causa:** `requirements.txt` declara seis paquetes y le faltan `pyperclip`, `PyGetWindow` y `pywebview`. **VERIFICADO** comparando ambos manifiestos.
- **Diagnóstico:** `bash python -c "import pyperclip, pygetwindow, webview" # ImportError si falta alguno`
- **Solución:** usar `make install-dev` o `make sync`, que instalan desde `pyproject.toml`.
- **Archivos:** `requirements.txt`, `pyproject.toml`, `Makefile`
- **Riesgo:** Ninguno.

1.6 · SmartScreen bloquea el instalador

- **Causa:** El `.exe` no está firmado: `codesign_identity=None` en `automa.spec`.
- **Diagnóstico:** el aviso de Windows lo dice.
- **Solución:** «Más información» → «Ejecutar de todas formas», tras verificar que el archivo procede de la página oficial de releases del repositorio.
- **Archivos:** `installer/automa.spec`, `installer/Automa.iss`
- **Riesgo:** saltarse SmartScreen es un riesgo real si el origen no es el oficial. Verifique la URL antes.

2. CLI

2.1 · `UnicodeEncodeError: 'charmap' codec can't encode character '\u2192'` **VERIFICADO**

- **Síntoma:** `text $ python -m engine.runner list UnicodeEncodeError: 'charmap' codec can't encode character '\u2192' in position 6042`
- **Causa:** `engine/runner.py::main` hace `print(json.dumps(..., ensure_ascii=False, indent=2))`. Varios manifests contienen → y otros caracteres no representables en la página de códigos `cp1252` de la consola de Windows. Afecta a `list` y a `run`.
- **Detalle importante:** con `run`, el flow se ejecuta completo y correctamente —la corrida queda persistida y el reporte escrito—. El error salta después, solo al volcar el JSON. No se pierde trabajo, pero el código de salida es de error.
- **Diagnóstico:** `python -c "import sys; print(sys.stdout.encoding)"` → si dice `cp1252`, es este caso.

- **Solución:** powershell \$env:PYTHONIOENCODING = "utf-8"; python -m engine.runner list # PowerShell bash PYTHONIOENCODING=utf-8 python -m engine.runner list # Git Bash **VERIFICADO:** con la variable, el comando devuelve el JSON completo sin error.
- **Archivos:** engine/runner.py: :main, líneas 39 y 41
- **Riesgo:** Ninguno. PYTHONIOENCODING solo afecta a la codificación de la salida.
- **Nota:** el panel HTTP **no** sufre este problema: escribe bytes UTF-8 a un socket.

2.2 · automa: command not found

- **Causa:** El paquete no está instalado, o el Scripts/ del entorno virtual no está en el PATH.
- **Diagnóstico:** pip show automa-pc
- **Solución:** activar el entorno virtual, o usar la forma sin instalar: python -m engine.runner ..., python -m app.server.
- **Archivos:** pyproject.toml, [project.scripts]
- **Riesgo:** Ninguno.

3. Panel y API

3.1 · 401 no autorizado: Host no loopback

- **Causa:** Se accede al panel con un Host distinto de 127.0.0.1, localhost, ::1 o [::1], y no hay AUTOMA_PANEL_TOKEN definido. Ocurre típicamente al poner un reverse proxy delante, o al acceder por el nombre de la máquina.
- **Diagnóstico:** revise la cabecera Host de la petición.
- **Solución:** definir AUTOMA_PANEL_TOKEN y enviar X-Automa-Token en cada mutación.
- **Archivos:** app/server.py: :_authorize_mutation
- **Riesgo: Medio.** Exponer el panel fuera de loopback abre todas las lecturas, que **no** exigen token en ningún caso: GET /api/runs devuelve el contexto completo de cada corrida. Ver 11 · Seguridad.

3.2 · 401 Origin '...' no coincide con 'http://127.0.0.1:8787'

- **Causa:** Una petición cross-origin. Es exactamente el ataque que la defensa cierra: una web maliciosa haciendo fetch contra el panel local.
- **Diagnóstico:** si la petición la hizo usted desde otra página, es un falso positivo; si no, alguien intentó algo.
- **Solución:** usar el panel desde su propia URL, o el CLI, o curl (que no envía Origin).
- **Archivos:** app/server.py: :_authorize_mutation
- **Riesgo:** Ninguno al usar la vía correcta. **No desactive esta comprobación.**

3.3 · 401 token inválido o AUTOMA_WEBHOOK_TOKEN no configurado

- **Causa:** El webhook está **deshabilitado por defecto**. Sin la variable, responde 401 siempre.
- **Diagnóstico:** python -c "import os; print(bool(os.environ.get('AUTOMA_WEBHOOK_TOKEN')))"
- **Solución:** definir la variable en el entorno del panel —o la clave en secrets/secrets.json— y **reiniciar el panel**. Genere el valor con python -c "import secrets; print(secrets.token_urlsafe(32))".
- **Archivos:** app/server.py: :_check_webhook_token, engine/secrets.py
- **Riesgo: Medio.** Habilitar el webhook abre una vía de ejecución de flows. Si lo expone más allá de localhost, póngalo detrás de un reverse proxy con TLS, como advierte el README.md.

3.4 · 400 folder inválido

- **Causa:** El segmento de la URL no pasa `_FOLDER_RE = ^[A-Za-z0-9_\-]{1,64}$`. Contiene un punto, una barra, un espacio o un carácter no ASCII.
- **Diagnóstico:** compare el nombre con la carpeta real en `flows/`.
- **Solución:** usar el nombre exacto de la carpeta.
- **Archivos:** `app/server.py::_safe_folder`
- **Riesgo:** Ninguno. **No relaje esta expresión regular:** es la primera línea de defensa contra path traversal.

3.5 · 415 Extensión no servida en /file

- **Causa:** `/file` bloquea `.html`, `.htm`, `.xhtml`, `.xml`, `.svg`, `.js`, `.mjs` y `.css` para cerrar XSS reflejado desde el mismo origen.
- **Diagnóstico:** mire la extensión del archivo pedido.
- **Solución:** abra el archivo directamente desde el disco. Los `.png`, `.json`, `.csv`, `.md`, `.log` y `.jsonl` sí se sirven.
- **Archivos:** `app/server.py::do_GET`
- **Riesgo:** **Alto si se quita el control.** Servir HTML propio desde el mismo origen del panel permitiría ejecutar JavaScript con acceso a la API.

4. Ejecución de flows

4.1 · FlowExecutionError: Acción no registrada: <nombre>

- **Causa:** El manifest usa una acción que no está en `_BUILT_IN_ACTIONS` ni en los entry points instalados. Casi siempre, un error tipográfico.
- **Diagnóstico:** `bash python scripts/validate_project.py # lo detecta antes de ejecutar python -c "from engine.action_registry import ACTION_REGISTRY; print(sorted(ACTION_REGISTRY.keys()))"`
- **Solución:** corregir el nombre, o registrar la acción.
- **Archivos:** `engine/action_registry.py`, el `manifest.json` del flow
- **Riesgo:** Ninguno.

4.2 · SandboxViolation: Acción 'X' bloqueada por allowed_actions del manifest

- **Causa:** El paso usa una acción que la política del propio flow no permite.
- **Diagnóstico:** `validate_project.py` también lo detecta: su comprobación 4 verifica la coherencia entre pasos y `allowed_actions`.
- **Solución:** añadir la acción a `allowed_actions`, **entendiendo lo que se amplía.**
- **Archivos:** `engine/sandbox.py::assert_action_allowed`, el manifest
- **Riesgo:** **Medio.** Ampliar la lista amplía lo que el flow puede hacer sobre el equipo.

4.3 · SandboxViolation: Ruta fuera de allowed_paths: ...

- **Causa:** Un parámetro cuya clave contiene `path`, `destination`, `source`, `output` o `file` resuelve a una ruta fuera de las bases declaradas.
- **Diagnóstico:** el mensaje incluye la ruta y las bases. Compruebe que la ruta esté resuelta y sea relativa al directorio de trabajo esperado.
- **Solución:** corregir la ruta del manifest, o añadir la base a `allowed_paths`.
- **Archivos:** `engine/sandbox.py::assert_paths_allowed`
- **Riesgo:** **Medio.** Ampliar `allowed_paths` amplía dónde puede escribir el flow.
- **Aviso:** la detección es **por nombre de clave**. Un parámetro llamado `target` o `command` que contenga una ruta **no se comprueba**. No confíe en `allowed_paths` para acotar un flow de navegador.

4.4 · SandboxViolation: Faltan variables de entorno requeridas por el flow: X

- **Causa:** `required_secrets` declara una variable que no está en `os.environ`, o está vacía. El flow **no arranca**.
- **Diagnóstico:** `python -c "import os; print(os.environ.get('X'))"`
- **Solución:** definir la variable en el entorno, no solo en `secrets/secrets.json`.
- **Archivos:** `engine/sandbox.py::check_required_secrets`
- **Riesgo:** Ninguno.
- **Aviso importante:** `check_required_secrets` lee solo `os.environ`, no `engine.secrets.get_secret`. Un secreto que viva únicamente en `secrets/secrets.json` **no satisface** el requisito. Es una inconsistencia conocida entre dos mecanismos que se presentan como equivalentes.

4.5 · TypeError: X() got an unexpected keyword argument 'Y' o falta un argumento

- **Causa:** Los `params` del manifest no coinciden con la firma de la acción. **Ningún validador comprueba esto:** el manifest pasa la CI y falla en ejecución.
- **Diagnóstico:** `python import inspect from engine.action_registry import ACTION_REGISTRY print(inspect.signature(ACTION_REGISTRY.get('screen.capture_screenshot')))`
- **Solución:** corregir los `params`. Consulte [05 · Referencia técnica](#).
- **Archivos:** el manifest, `actions/*.py`
- **Riesgo:** Ninguno.

4.6 · Un {{ placeholder }} llega literal a la acción

- **Síntoma:** un archivo llamado `reporte_{{ nombre }}.json`, o un `TypeError` por recibir la cadena "`{{ threshold }}`" donde se esperaba un número.
- **Causa:** La clave no existe en el contexto resuelto. `render_value` **deja literal** lo que no puede resolver, a propósito.
- **Causa más frecuente:** existe un `configs/<carpeta>.json` guardado desde el panel que **oculta por completo** el `context.example.json`. Si el flow se actualizó y añadió claves, el archivo de config no las tiene.
- **Diagnóstico:** `python from pathlib import Path from engine.loader import FlowLoader print(FlowLoader.load_context(Path('flows/23_web_change_detector')))`
- **Solución:** añadir la clave al contexto en uso, o **borrar** `configs/<carpeta>.json` para que vuelva a leerse el ejemplo del flow.
- **Archivos:** `engine/loader.py::load_context`, `engine/template.py::render_value`
- **Riesgo:** Bajo-medio. Borrar el archivo de config **pierde la configuración personalizada** del flow. Cópiala antes.

4.7 · Un paso aparece como skipped y esperaba que se ejecutara

- **Causa:** Su `when` no se cumplió. Es comportamiento normal, no un error: el registro incluye `result: {"reason": "condition_not_met"}`.
- **Diagnóstico:** mire el valor real de la ruta consultada en el contexto de la corrida (pestaña Histórico → detalle → `<details>` del contexto).
- **Solución:** revisar la condición. Recuerde que `contains` **no distingue mayúsculas**, que `in` exige que `value` sea una lista, y que `get_path` **no soporta índices de lista**.
- **Archivos:** `engine/conditions.py`, el manifest
- **Riesgo:** Ninguno.

4.8 · Un flow con transición on: "failure" termina en failed igualmente

- **Causa:** La transición apunta al **mismo paso** que ya venía a continuación por orden. El motor compara `recovery_next != self._default_next(step.id)` y, si coinciden, no lo considera una rama de recuperación.

- **Diagnóstico:** compare el `next` de la transición con el paso siguiente del array.
- **Solución:** apuntar la transición de fallo a un paso **distinto** del siguiente por defecto.
- **Archivos:** `engine/orchestrator.py::run`, bloque de recuperación
- **Riesgo:** Ninguno.

5. Navegador y familia web

5.1 · `RuntimeError: playwright no está instalado`

- **Causa:** `playwright` no está declarado en `pyproject.toml` ni en `requirements.txt`, pero nueve flows (02, 07, 21–27) lo necesitan.
- **Diagnóstico:** `python -c "import playwright; print(playwright.__file__)"`
- **Solución:** `bash pip install playwright python -m playwright install chromium`
- **Archivos:** `actions/browser_capture.py`, `browser_form.py`, `browser_extract.py`
- **Riesgo:** Ninguno. Descarga ~150 MB de navegador.

5.2 · `Executable doesn't exist at ...ms-playwright...`

- **Causa:** El paquete está pero falta el navegador descargado.
- **Diagnóstico:** el mensaje de Playwright lo indica.
- **Solución:** `python -m playwright install chromium`
- **Archivos:** —
- **Riesgo:** Ninguno.

5.3 · El flow 23 no detecta un cambio que sí ocurrió

- **Causas posibles, en orden de probabilidad:** 1. Es la primera corrida. `apply_tracking` devuelve `first_run: true` y nunca reporta cambio: establece la línea base. Es intencional, evita una alerta espuria. 2. El archivo de tracking se borró. `data/web_watch/` está en `.gitignore`. Un `git clean -xdf` lo elimina y la próxima corrida vuelve a ser `first_run`. 3. El archivo estaba corrupto. `apply_tracking` trata un JSON ilegible como inexistente y reinicia la línea base en silencio, sin declararlo. 4. El cambio no afecta al texto normalizado. `normalize_text` colapsa espacios y líneas vacías: un cambio de espaciado o de un atributo HTML no altera el SHA-256. 5. Una corrida anterior falló después del tracking. El tracking se escribe antes de devolver el resultado: si un paso posterior falla, la línea base ya se actualizó.
- **Diagnóstico:** `bash cat data/web_watch/demo_page.json # ver watch_value y checked_at`
- **Solución:** correr dos veces (la primera es la base). Para forzar una base nueva, borre el archivo de tracking.
- **Archivos:** `actions/browser_extract.py::apply_tracking, normalize_text`
- **Riesgo: Medio.** Borrar el archivo de tracking **pierde la referencia anterior**: la próxima corrida no detectará el cambio que estaba vigilando.

5.4 · El flow 07 repite un registro del seed

- **Causa:** `data/seeds/.used_indices.json` se borró, o el ciclo se agotó y se reinició solo (el resultado lo declara con `cycle_resetted: true`).
- **Diagnóstico:** `bash python -c "import json;d=json.load(open('data/seeds/.used_indices.json'));print(d['used_count'],'/',d['total_in_dataset'])"`
- **Solución:** ninguna necesaria si `cycle_resetted` es `true`. Si el archivo se borró, la memoria se perdió.
- **Archivos:** `actions/browser_form.py::_pick_record`
- **Riesgo:** Ninguno.
- **Nota:** el registro se marca como usado **antes** de intentar el llenado. Si el navegador falla, ese registro se pierde para el ciclo.

5.5 · El flow 26 lee un valor numérico incorrecto

- **Causa:** La heurística de `parse_number`. Con un solo separador y **exactamente tres dígitos detrás**, se asume separador de miles: `"1.234"` se interpreta como `1234.0`, no como `1.234`.
- **Diagnóstico:** `python from actions.browser_extract import parse_number print(parse_number("1.234")) # 1234.0`
- **Solución:** ninguna en el código actual. Ajuste el `threshold` sabiendo cómo se parsea, o elija un selector cuyo formato no sea ambiguo.
- **Archivos:** `actions/browser_extract.py::parse_number`
- **Riesgo:** Ninguno al ajustar el umbral. Cambiar la heurística afectaría a todos los valores ya vigilados.

6. Pantalla, OCR y acciones de UI

6.1 · `RuntimeError: No fue posible capturar la pantalla`

- **Causa:** Fallaron **las dos** estrategias, `mss` y `Pillow`. Casi siempre por no haber escritorio gráfico: sesión SSH, contenedor, servicio sin sesión interactiva.
- **Diagnóstico:** `python -c "import mss; print(mss.mss().monitors)"`
- **Solución:** ejecutar en una sesión Windows interactiva.
- **Archivos:** `actions/screen.py::capture_screenshot`
- **Riesgo:** Ninguno.

6.2 · El OCR devuelve `status: "unavailable"` y `matches: []`

- **Causa:** Falta `pytesseract` (`reason: pytesseract_missing`) o el binario `tesseract` (`reason: tesseract_binary_missing`). **El flow no falla:** es una degradación deliberada.
- **Diagnóstico:** `bash tesseract --version python -c "from plugins.analyzers.ocr_image_analyzer import OCRImageAnalyzer as O; print(O._tesseract_binary_available())"`
- **Solución:** Windows `choco install tesseract` o el instalador de UB Mannheim; Linux `apt-get install tesseract-ocr`; macOS `brew install tesseract`. El analizador busca además en `C:/Program Files/Tesseract-OCR/tesseract.exe` y su variante (x86), así que no hace falta tocar el PATH.
- **Archivos:** `plugins/analyzers/ocr_image_analyzer.py`
- **Riesgo:** Ninguno.

6.3 · `RuntimeError: No hay ventana activa identificable`

- **Causa:** `pygetwindow.getActiveWindow()` devolvió `None`. Sesión sin foco, o el flow se disparó desde el scheduler sin ventana en primer plano.
- **Diagnóstico:** `python -c "import pygetwindow as gw; print(gw.getActiveWindow())"`
- **Solución:** ejecutar el flow con una ventana en foco. Para uso programado, prefiera `screen.capture_screenshot` (escritorio completo).
- **Archivos:** `actions/screen.py::capture_active_window`
- **Riesgo:** Ninguno.
- **Nota:** `capture_region` solo trabaja sobre el **monitor primario** (`sct.monitors[1]`). Una ventana en un segundo monitor daría un recorte incorrecto. **REQUIERE VALIDACIÓN.**

6.4 · `RuntimeError: pyautogui no está disponible o el entorno no permite control de UI`

- **Causa:** Falta `pyautogui`, o el entorno no permite control de teclado y ratón.

- **Diagnóstico:** `python -c "import pyautogui; print(pyautogui.size())"`
- **Solución:** `pip install pyautogui` y ejecutar en un escritorio real. Para probar sin efectos, ponga `"dry_run": true` en el contexto del flow.
- **Archivos:** `actions/ui.py::_import_pyautogui`
- **Riesgo:** Alto al quitar `dry_run`. Estas acciones mueven teclado y ratón de verdad sobre su sesión. Pruebe siempre primero en `dry_run`.

6.5 · `ValueError: launch_process: shell=True está deshabilitado por seguridad`

- **Causa:** El manifest pasa `"shell": true`. Está prohibido a propósito para cerrar CWE-78.
- **Diagnóstico:** busque `shell` en los `params` del paso.
- **Solución:** quitar el parámetro y pasar el comando como cadena tokenizable por `shlex`.
- **Archivos:** `actions/ui.py::launch_process`
- **Riesgo:** Alto si se reactiva. No lo reactive: el control existe por una razón.
- **Nota Windows:** `shlex` usa reglas POSIX. Una ruta con contrabarras (`C:\Users\ejemplo`) puede tokenizarse mal. Use barras normales.

7. Scheduler y tareas programadas

7.1 · Una tarea programada nunca se ejecuta

- **Causas posibles:** 1. `enabled` está a 0. 2. `next_run_at` es NULL. 3. Hay un lock huérfano en `run_locks` para esa carpeta. 4. No hay ningún proceso con scheduler vivo (el panel cerrado y sin `automa scheduler`). 5. El cron apunta a un momento distinto del esperado: **0 es lunes, no domingo**, y la hora es **UTC**.
- **Diagnóstico:** `bash sqlite3 db/runs.db "SELECT folder, enabled, interval_seconds, cron_expression, next_run_at, last_run_at FROM schedules;"` `sqlite3 db/runs.db "SELECT * FROM run_locks;"` `python from engine.cron import next_after from datetime import datetime, timezone print(next_after('0 9 * * 1', datetime.now(timezone.utc)))` # ¿es el día que espera?
- **Solución:** habilitar desde el panel; liberar el lock; arrancar el panel o `automa scheduler --interval 2`; corregir la expresión cron.
- **Archivos:** `engine/scheduler.py`, `engine/cron.py`, `engine/database.py`
- **Riesgo:** liberar un lock **mientras la corrida sigue viva** permitiría dos ejecuciones en paralelo. Compruebe antes que no haya una corrida en `running`.

7.2 · Lock huérfano tras matar el proceso

- **Causa:** `_run_job` libera el lock en un `finally`, pero matar el proceso salta ese bloque. **No hay liberación automática al arrancar.**
- **Diagnóstico:** `bash sqlite3 db/runs.db "SELECT * FROM run_locks;"` `sqlite3 db/runs.db "SELECT run_id, status FROM runs WHERE status='running';"`
- **Solución:** `python from engine.database import force_release_lock force_release_lock("05_system_healthcheck")`
- **Archivos:** `engine/database.py::force_release_lock`, `RUNBOOK.md`
- **Riesgo:** Medio. Si la corrida seguía viva, se abre la puerta a una ejecución paralela. Verifique primero.

7.3 · Una tarea programada falla siempre y nadie avisa

- **Causa:** `SchedulerService._run_job` captura toda excepción con `pass`, y llama a `mark_schedule_run` **después del except**. La tarea reprograma su siguiente ejecución con normalidad aunque haya fallado.

- **Diagnóstico:** `bash sqlite3 db/runs.db "SELECT flow_id, status, COUNT(*) FROM runs WHERE created_at > date('now','-7 day') GROUP BY flow_id, status;"`
- **Solución:** no hay ninguna en el sistema. Revise el histórico periódicamente. Como paliativo, añada un paso `notify.send` con transición `on: "failure"` al final del flow.
- **Archivos:** `engine/scheduler.py::_run_job`
- **Riesgo:** Ninguno al añadir la notificación.

7.4 · Dos schedulers corriendo a la vez

- **Causa:** El panel arranca un scheduler al importar `app.server`, y además se lanzó `automa.scheduler`.
- **Diagnóstico:** cuente los procesos Python vivos.
- **Solución:** ejecute uno de los dos, no ambos.
- **Archivos:** `app/server.py` (líneas de módulo), `engine/runner.py`
- **Riesgo:** Bajo. La tabla `run_locks` impide que el mismo flow corra dos veces desde el scheduler. Pero el lock no cubre las rutas del panel: `POST /api/run` sí permite duplicados.

8. Base de datos y almacenamiento

8.1 · `sqlite3.OperationalError: database is locked`

- **Causa:** Dos escrituras simultáneas. El repositorio no configura `PRAGMA journal_mode=WAL` ni `busy_timeout`.
- **Diagnóstico:** ocurre con corridas en paralelo desde el panel más el scheduler.
- **Solución:** reintentar. No lance el mismo flow varias veces en paralelo.
- **Archivos:** `engine/database.py::connect`
- **Riesgo:** Ninguno al reintentar.

8.2 · Una corrida queda para siempre en `running`

- **Causa:** El proceso murió a mitad. Todos los hilos son *daemon*: cerrar la ventana los mata sin esperar. No hay detección de corridas huérfanas.
- **Diagnóstico:** `bash sqlite3 db/runs.db "SELECT run_id, flow_id, started_at FROM runs WHERE status='running';"`
- **Solución:** corregir a mano si molesta en el panel: `sql UPDATE runs SET status='failed', error_json='{ "message": "proceso interrumpido" }' WHERE status='running' AND started_at < datetime('now', '-1 hour');`
- **Archivos:** `engine/orchestrator.py`
- **Riesgo:** Medio. Ese `UPDATE` marcaría como fallida una corrida que siguiera viva. Compruebe que no haya ningún proceso ejecutándola antes.

8.3 · `db/`, `logs/`, `state/` y `output/` crecen sin control

- **Causa:** No existe ninguna rutina de retención. Verificado buscando `VACUUM`, `DELETE FROM runs` y `retention` en todo el código.
- **Diagnóstico:** `bash du -sh db/ logs/ state/ output/ sqlite3 db/runs.db "SELECT COUNT(*) FROM runs;"`
- **Solución:** purga manual. `bash find logs/ -name "*.jsonl" -mtime +30 -delete find state/ -name "*.json" -mtime +30 -delete sqlite3 db/runs.db "DELETE FROM runs WHERE created_at < date('now', '-90 day');" sqlite3 db/runs.db "VACUUM;"`
- **Archivos:** `engine/database.py`, `engine/logger.py`, `engine/state_store.py`

- **Riesgo: Alto y en dos frentes.** 1. **No hay claves foráneas ni ON DELETE CASCADE.** Borrar de `runs` deja huérfanas las filas de `steps` y `events`. Bórralas también, o el dashboard de métricas seguirá contándolas. 2. **Nunca borre `data/seeds/.used_indices.json` ni `data/web_watch/*.json`.** No son residuo: son la memoria de los flows 07, 23 y 26.

8.4 · El histórico desapareció tras actualizar o mover la instalación

- **Causa:** `data_dir()` cambió. En desarrollo es la raíz del repositorio; en el binario es `%LOCALAPPDATA%\Automa;` con `AUTOMA_DATA_ROOT` es lo que diga esa variable.
- **Diagnóstico:** `python from engine.paths import data_dir print(data_dir())`
- **Solución:** copiar `runs.db` de la ubicación antigua a la nueva.
- **Archivos:** `engine/paths.py::data_dir`
- **Riesgo: Medio.** Sobrescribir un `runs.db` existente pierde su histórico. Haga copia primero.

9. Pruebas y CI

9.1 · FAIL Required test coverage of 54% not reached

- **Causa:** La cobertura bajó del umbral configurado en `pyproject.toml`.
- **Situación real medida:** la cobertura oscila entre **58,9 % y 60,0 %** entre corridas del mismo commit, **INFERENCIA** por el hilo de scheduler que se arranca al importar `app.server`. El margen sobre el gate es de menos de 5 puntos.
- **Diagnóstico:** `python -m pytest` y mire la columna `Missing` del módulo que bajó.
- **Solución:** añadir pruebas. **No baje el umbral.**
- **Archivos:** `pyproject.toml`, `[tool.pytest.ini_options]`
- **Riesgo:** bajar el umbral degrada permanentemente el gate.

9.2 · --collect-only falla con el error de cobertura

- **Causa:** `addopts` incluye `--cov-fail-under=54` y se aplica también al recuento.
- **Diagnóstico:** el error aparece justo tras la tabla de cobertura.
- **Solución:** `python -m pytest --collect-only -q --no-cov`
- **Archivos:** `pyproject.toml`
- **Riesgo:** Ninguno.

9.3 · Tras `python scripts/smoke_test.py`, `git status` marca un archivo modificado

- **Causa:** El smoke test llama a `set_flow_config('03_folder_inventory', {'path_override': 'data/inbox'})`, que reescribe el archivo versionado `configs/03_folder_inventory.json`. **VERIFICADO** durante este análisis.
- **Diagnóstico:** `git status --porcelain` y `git diff configs/`
- **Solución:** `git checkout -- configs/03_folder_inventory.json`
- **Archivos:** `scripts/smoke_test.py`, `engine/database.py::set_flow_config`
- **Riesgo:** Ninguno. **Pero revise siempre `git status` antes de confirmar** tras ejecutar el smoke test.

9.4 · La CI falla en `markdown-docs` por un enlace roto

- **Causa:** El verificador recorre **todos** los `.md` del repositorio (excluyendo `node_modules`, `.git`, `output`, `state`, `logs`) y falla si un enlace relativo no resuelve.
- **Diagnóstico:** el log del job lista hasta 50 enlaces rotos con archivo y posición.
- **Solución:** corregir la ruta relativa.
- **Archivos:** `.github/workflows/markdown-docs.yml`

- **Riesgo:** Ninguno.
 - **Aviso:** el workflow solo se dispara en `pull_request` con `paths: '**/*.md'`. Un push directo a `main` con un enlace roto no lo detecta.
-

10. Empaquetado y binario

10.1 · `ModuleNotFoundError: No module named 'actions.browser_extract'` en el `.exe`

- **Causa:** `installer/automa.spec` no lista `actions.browser_extract` en `hiddenimports`, y el registro lo carga con `import_module` dinámico, que PyInstaller no rastrea. **VERIFICADO** en el archivo; **REQUIERE VALIDACIÓN** el efecto, porque no se compiló.
- **Afectaría a:** los siete flows de la familia web (21–27).
- **Diagnóstico:** ejecutar el flow 21 desde el binario instalado.
- **Solución propuesta** —no aplicada, este documento es informativo—: añadir `"actions.browser_extract"` a la lista de `hiddenimports`.
- **Archivos:** `installer/automa.spec`
- **Riesgo:** Ninguno. Añadir un `hiddenimport` es aditivo.

10.2 · Los flows de navegador no funcionan en el binario aunque el módulo esté

- **Causa:** `playwright` y el Chromium descargado **no se empaquetan**. No aparecen ni en `datas` ni en `hiddenimports` de `automa.spec`. **INFERENCIA** coherente con el tamaño del instalador.
- **Diagnóstico:** ejecutar el flow 02 desde el binario.
- **Solución:** ninguna sin rediseñar el empaquetado.
- **Archivos:** `installer/automa.spec`
- **Riesgo:** —

10.3 · Tras desinstalar, quedan datos en el disco

- **Causa:** El desinstalador de Inno Setup borra `{app}`, pero los datos viven en `%LOCALAPPDATA%\Automa` por diseño (`engine/paths.py::data_dir` en modo congelado). **INFERENCIA**.
 - **Diagnóstico:** revise `%LOCALAPPDATA%\Automa` tras desinstalar.
 - **Solución:** borrar la carpeta a mano.
 - **Archivos:** `engine/paths.py::data_dir`, `installer/Automa.iss`
 - **Riesgo:** **Alto**. Esa carpeta contiene el histórico completo y **todas las capturas de pantalla**, que pueden ser sensibles. Bórrela de forma segura si el equipo cambia de manos; pero recuerde que el borrado es irreversible.
-

11. Diagnóstico general: por dónde empezar

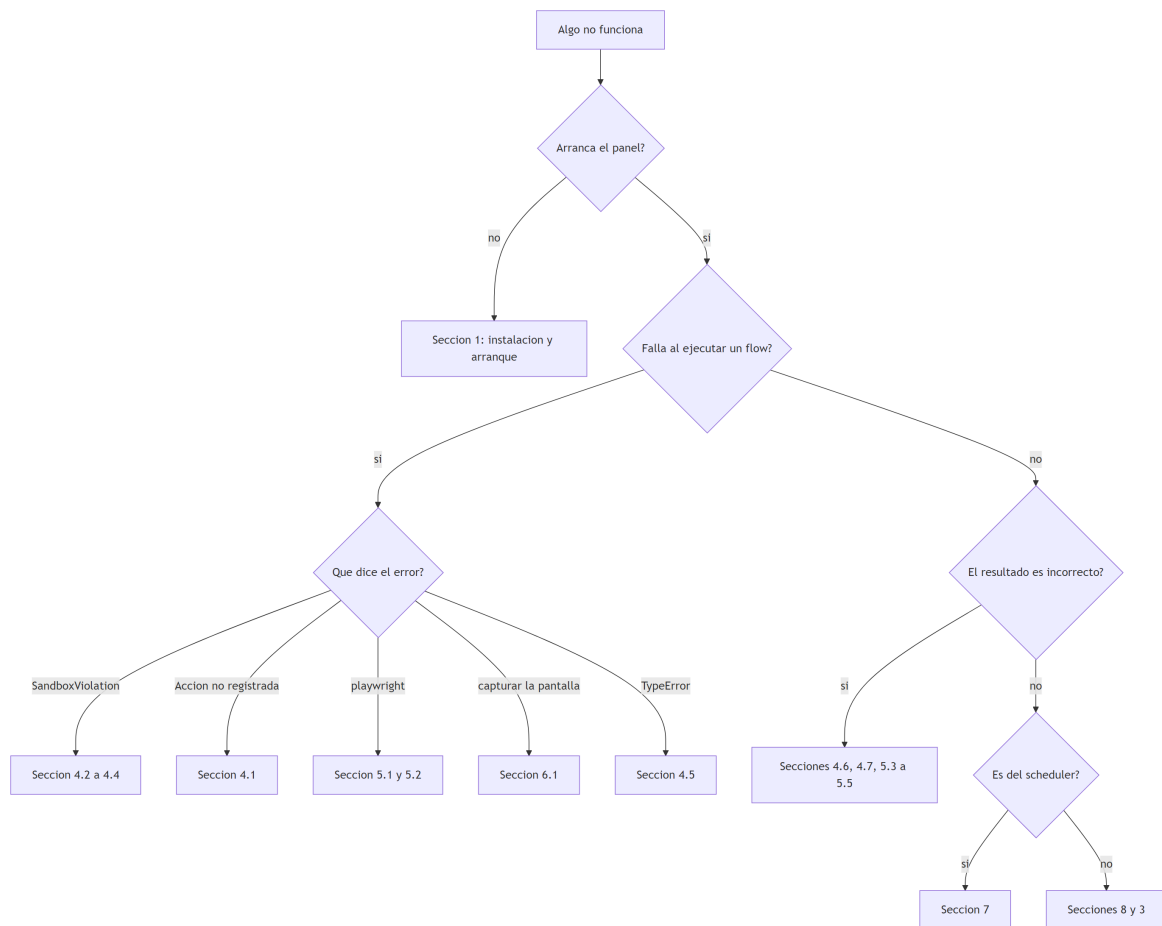


Diagrama 1 (Mermaid, renderizado)

Lo que el árbol muestra: el orden de descarte más rápido, del arranque al resultado. **Lo que no muestra:** los tres comandos que resuelven la mayoría de los diagnósticos y conviene ejecutar antes que nada:

```

python scripts/validate_project.py           # ¿el catálogo está sano?
curl http://127.0.0.1:8787/healthz          # ¿el panel responde?
sqlite3 db/runs.db "SELECT run_id, flow_id, status, error_json FROM runs
                                ORDER BY created_at DESC LIMIT 5;"    # ¿qué pasó en las últimas corridas?
  
```

Documentos relacionados: [02 · Instalación](#) · [10 · Configuración](#) · [11 · Seguridad](#) · [13 · Despliegue y operación](#) · [15 · Riesgos y deuda técnica](#)

15 · Riesgos y deuda técnica

Documento informativo. No se corrigió ninguno de estos hallazgos. Cada uno lleva severidad, impacto, probabilidad, evidencia verificable, ubicación, recomendación y prioridad. La sección final recoge lo que está bien y conviene proteger, porque un informe que solo enumera problemas da una imagen falsa.

1. Resumen

Severidad	Cantidad	Hallazgos
[alto] Alta	6	R-01 ... R-06
[medio] Media	12	R-07 ... R-18
[bajo] Baja	9	R-19 ... R-27
Total	27	

Categoría	Cantidad
Seguridad	7
Corrección funcional	6
Empaquetado y despliegue	3
Consistencia y duplicación	5
Operación y escalabilidad	4
Documentación	2

Todos los hallazgos se verificaron leyendo el código o ejecutando un comando. Los marcados **REPRODUCIDO** se ejecutaron durante el análisis; los marcados **INFERENCIA** dependen de una condición que no se pudo montar.

2. Severidad alta

R-01 · Los flows 21–27 probablemente no funcionan en el binario empaquetado

Campo	Valor
Severidad	[alto] Alta
Categoría	Empaquetado
Impacto	Siete de los 27 flows —la familia web completa, la novedad de la v0.3.0— fallarían en el producto distribuido
Probabilidad	Alta INFERENCIA

Ubicación	installer/automa.spec, lista hiddenimports
-----------	--

Evidencia. `grep -c "browser_extract" installer/automa.spec` devuelve 0. La lista enumera `actions.filesystem`, `screen`, `vision`, `system`, `rules`, `ui`, `http_actions`, `notify`, `browser_capture` y `browser_form` — diez módulos. Falta `actions.browser_extract`, añadido en la v0.3.0.

`LazyActionRegistry.get` resuelve con `import_module(module_name)`, un import **dinámico** que el analizador estático de PyInstaller no rastrea.

Recomendación. Añadir `"actions.browser_extract"` a `hiddenimports`. Mejor aún: derivar la lista de `_BUILT_IN_ACTIONS` para que nunca vuelva a desincronizarse.

REQUIERE VALIDACIÓN: confirmar compilando y ejecutando el flow 21 desde el `.exe`.

Prioridad: 1.

R-02 · GET /api/runs expone todo el histórico sin autenticación

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad · privacidad
Impacto	Portapapeles capturado, texto OCR de ventanas abiertas, contenido web e inventarios del equipo, legibles sin token
Probabilidad	Baja en loopback con un solo usuario; alta si se expone el panel
Ubicación	app/server.py::do_GET

Evidencia. `do_GET` no llama a `_authorize_mutation` en ningún caso. Verificado leyendo el método completo. Con `AUTOMA_PANEL_TOKEN` definido, siguen abiertos `/api/runs`, `/api/flows`, `/api/metrics`, `/metrics`, `/file` y todas las vistas HTML.

`GET /api/runs` devuelve la columna `context_json` completa de cada corrida. Y `/file` sirve los `.png` de las capturas: la `allowlist` de extensiones bloquea `.html`, `.js`, `.svg` y `.css`, **pero no** `.png`.

Recomendación. Cuando `AUTOMA_PANEL_TOKEN` esté definido, exigirlo también en los GET que devuelven datos (`/api/runs`, `/api/flows`, `/api/metrics`, `/file`). Es una llamada adicional en `do_GET`.

Prioridad: 2.

R-03 · 14 de 27 flows corren sin ninguna política de sandbox

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad
Impacto	La mitad del catálogo puede ejecutar cualquier acción y escribir en cualquier ruta
Probabilidad	Cierta: es el estado actual
Ubicación	flows/01-07/manifest.json, flows/21-27/manifest.json

Evidencia. Conteo leyendo los 27 manifests:

Control	Lo declaran	No lo declaran
<code>allowed_actions</code>	13 (08–20)	14
<code>allowed_paths</code>	7	20
<code>max_runtime_seconds</code>	13	14
<code>required_secrets</code>	0	27

Entre los 14 sin política están `07_browser_form_filler` (lanza un navegador visible y escribe archivos) y los siete de la familia web.

INFERENCIA: el bloque 08–20 se añadió en la v0.2.0 con la política ya en mente y la familia web de la v0.3.0 no la incorporó. Nada en la CI lo exige, así que el hueco puede crecer con cada caso nuevo.

Recomendación. Añadir a `scripts/validate_project.py` una regla que exija `allowed_actions` en todo flow. Retroadaptar los 14 existentes.

Prioridad: 3.

R-04 · `system.run_powershell` permite ampliar su allowlist desde el manifest

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad
Impacto	Un manifest puede habilitar verbos destructivos de PowerShell
Probabilidad	Baja: requiere escribir un manifest, es decir, acceso de escritura
Ubicación	<code>actions/system.py::run_powershell</code> , parámetro <code>allowlist</code>

Evidencia.

```
verbs = tuple(allowlist) if allowlist else _PS_DEFAULT_ALLOWLIST
```

`allowlist` es un parámetro de la acción y, por tanto, sobrescribible desde `params`. Un flow con `"allowlist": ["Remove-Item"]` obtiene exactamente eso. Los tokens prohibidos (`;`, `|`, `&...`) no impiden un comando destructivo de una sola palabra con argumentos.

Contexto que matiza la gravedad. No es explotable desde fuera: quien puede escribir un manifest ya tiene acceso al sistema de archivos. Es precisamente el modelo de amenaza que `security.yml` describe, y por eso el hardening del CI es la defensa principal. Pero significa que la seguridad de esta acción depende de la revisión humana de los manifests, no de un control del motor.

Recomendación. Eliminar el parámetro `allowlist` de la firma pública, o exigir que el conjunto ampliado sea subconjunto de una lista maestra.

Prioridad: 4.

R-05 · `allowed_actions: []` produce política permisiva, no restrictiva

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad · corrección

Impacto	Un autor que escriba una lista vacía creyendo bloquearlo todo obtiene lo contrario
Probabilidad	Media
Ubicación	<code>engine/loader.py::load_manifest</code>

Evidencia.

```
allowed_actions=list(raw['allowed_actions']) if raw.get('allowed_actions') else None
allowed_paths=list(raw['allowed_paths']) if raw.get('allowed_paths') else None
```

Una lista vacía es *falsy* en Python, así que `[]` → `None`, y `None` significa «sin restricción» en `SandboxPolicy`. Es un fallo silencioso con impacto de seguridad, sin ninguna prueba que lo cubra.

Recomendación. Cambiar la condición a `is not None` en ambos campos, y añadir una prueba de regresión.

Prioridad: 5.

R-06 · El CLI falla al imprimir en una consola Windows por defecto **REPRODUCIDO**

Campo	Valor
Severidad	[alto] Alta
Categoría	Corrección funcional
Impacto	<code>automa list</code> y <code>automa run</code> terminan con excepción y código de error en Windows
Probabilidad	Cierta: reproducido en este análisis
Ubicación	<code>engine/runner.py::main</code> , líneas 39 y 41

Evidencia reproducida.

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '→' in position 6042:
character maps to <undefined>
```

Ocorre con `list` y con `run`. La causa es `print(json.dumps(..., ensure_ascii=False, indent=2))` sobre una consola `cp1252`; varias descripciones de flow contienen →.

Matiz importante: con `run`, el flow se ejecuta completo y correctamente —la corrida queda persistida y el reporte escrito— y el error salta después, solo al volcar el JSON.

Verificado que la variable de entorno lo resuelve: con `PYTHONIOENCODING=utf-8`, el comando devuelve el JSON completo.

Recomendación. Escribir a `sys.stdout.buffer` con codificación UTF-8 explícita, o reconfigurar `sys.stdout` al inicio de `main()`. Añadir una prueba que capture la salida con codificación forzada — `engine/runner.py` está al **0 % de cobertura**, y una sola prueba habría detectado esto.

Prioridad: 6.

3. Severidad media

R-07 · El lock de ejecución solo cubre una de las cuatro vías de disparo

Campo	Valor
-------	-------

Severidad	[medio] Media
Impacto	El mismo flow puede correr dos veces en paralelo desde el panel, con escritura concurrente de los archivos de tracking (flows 07, 23, 26)
Probabilidad	Media
Ubicación	<code>engine/database.py::acquire_run_lock,</code> <code>app/server.py::do_POST</code>

Evidencia. Búsqueda en todo el repositorio: `acquire_run_lock` se invoca **solo** desde `engine/scheduler.py::_run_job`. Ni `POST /api/run/<folder>`, ni `POST /run`, ni `POST /api/hook/<folder>`, ni el CLI lo usan.

Recomendación. Aplicar el lock también en las rutas del panel y del webhook, con una respuesta **409** cuando esté tomado.

Prioridad: 7.

R-08 · No hay retención: los almacenes crecen sin límite

Campo	Valor
Severidad	[medio] Media
Impacto	Degradación del panel y de las métricas; consumo de disco
Probabilidad	Cierta a medio plazo
Ubicación	<code>engine/database.py</code> , <code>engine/logger.py</code> , <code>engine/state_store.py</code>

Evidencia. Búsqueda de `VACUUM`, `DELETE FROM runs`, `retention` y `backup` en todo el código: **no existe ninguna rutina**. Y no hay ningún `CREATE INDEX`: solo los índices implícitos de las claves primarias. `list_runs(flow_id=...)`, `get_steps(run_id)` y `get_events(run_id)` hacen escaneo completo.

INFERENCIA: un flow programado cada 15 minutos genera 96 corridas diarias, ~35 000 al año, con ~70 000 archivos sueltos entre `logs/` y `state/`.

Recomendación. Índices sobre `runs.flow_id`, `runs.created_at`, `steps.run_id` y `events.run_id`, más una rutina de purga configurable.

Prioridad: 8.

R-09 · `max_runtime_seconds` no interrumpe una acción en curso

Campo	Valor
Severidad	[medio] Media
Impacto	Un flow con límite de 30 s puede correr indefinidamente si un solo paso se cuelga
Probabilidad	Media
Ubicación	<code>engine/orchestrator.py::run</code>

Evidencia. El límite se comprueba **antes** de lanzar cada paso, comparando el tiempo transcurrido. Una acción que se bloquea dentro no se interrumpe. Afecta a los 13 flows que declaran el campo, incluidos `18_powershell_audit` (40 s) y `12_desktop_ocr_inventory` (30 s).

Recomendación. Ejecutar la acción en un `concurrent.futures.ThreadPoolExecutor` con `timeout`, o documentar explícitamente que es un control de arranque de paso.

Prioridad: 9.

R-10 · El scheduler se traga los errores sin ninguna señal

Campo	Valor
Severidad	[medio] Media
Impacto	Una tarea programada que falla siempre reprograma con normalidad y nadie se entera
Probabilidad	Media
Ubicación	engine/scheduler.py::_run_job

Evidencia.

```
try:
    Orchestrator(Path(flow['flow_path'])).run()
except Exception:
    pass # "No interrumpe el loop. El error queda en la corrida."
mark_schedule_run(folder, interval_seconds, cron_expression)
```

mark_schedule_run se llama **después** del **except**, es decir, también cuando el flow falla. No hay traza en consola —log_message está silenciado— ni alerta.

Recomendación. Registrar el fallo en el log del proceso y, opcionalmente, contar fallos consecutivos para deshabilitar la tarea tras N.

Prioridad: 10.

R-11 · engine/catalog.py duplica root_dir() e ignora el modo empaquetado

Campo	Valor
Severidad	[medio] Media
Impacto	En el binario, catalog.flows_dir() no apuntaría a los flows extraídos por PyInstaller
Probabilidad	Media INFERENCIA
Ubicación	engine/catalog.py, líneas 10–16

Evidencia.

```
# engine/catalog.py – definición local
def root_dir() -> Path:
    return Path(__file__).resolve().parent.parent
```

engine/paths.py::root_dir sí contempla \$AUTOMA_ROOT y sys._MEIPASS. La de catalog.py no. En desarrollo coinciden; congelado, no.

INFERENCIA matizada: installer/automa_entry.py define AUTOMA_ROOT y el bundle extrae los flows bajo _MEIPASS, que es también el directorio de engine/, así que podrían coincidir por accidente. **REQUIERE VALIDACIÓN** compilando.

Recomendación. Importar root_dir de engine.paths y borrar la definición local.

Prioridad: 11.

R-12 · configs/ se lee relativo al cwd pero se escribe en data_dir()

Campo	Valor
Severidad	[medio] Media
Impacto	La configuración guardada desde el panel puede no leerse al ejecutar el flow
Probabilidad	Media
Ubicación	<code>engine/loader.py::load_context</code> frente a <code>engine/database.py::set_flow_config</code>

Evidencia.

```
# loader.py – relativo al directorio de trabajo
Path('configs') / f'{flow_dir.name}.json'
# database.py – absoluto sobre la raíz escribible
config_path = data_dir() / 'configs' / f'{folder}.json'
```

En desarrollo coinciden si se ejecuta desde la raíz. `installer/automa_entry.py` hace `os.chdir(data_dir())` para que coincidan en el bundle. Pero cualquier ejecución del CLI desde otra carpeta rompe la resolución.

Recomendación. Usar `data_dir() / 'configs'` también en `load_context`.

Prioridad: 12.

R-13 · `required_secrets` solo mira el entorno, no la bóveda

Campo	Valor
Severidad	[medio] Media
Impacto	Un secreto guardado con <code>set_secret</code> no satisface <code>required_secrets</code> ; el flow no arranca
Probabilidad	Media si alguien empieza a usar el campo
Ubicación	<code>engine/sandbox.py::check_required_secrets</code>

Evidencia. Usa `os.environ.get(name)`, no `engine.secrets.get_secret`. Los tokens del panel sí se resuelven desde el archivo; `required_secrets` no. Inconsistencia entre dos mecanismos presentados como equivalentes.

Atenuante: ningún flow del catálogo declara `required_secrets`, así que hoy no afecta a nadie.

Recomendación. Usar `get_secret` en `check_required_secrets`.

Prioridad: 13.

R-14 · Dos motores de condiciones con capacidades distintas

Campo	Valor
Severidad	[medio] Media
Categoría	Duplicación
Impacto	Un operador que funciona en un <code>when</code> falla dentro de <code>rules.evaluate</code>
Probabilidad	Media

Ubicación	<code>engine/conditions.py::matches</code> frente a <code>actions/rules.py::_matches</code>
-----------	---

Evidencia.

Motor	Operadores
<code>engine/conditions.py</code> (when de pasos y transiciones)	13: <code>eq, ne, gt, gte, lt, lte, contains, in, exists, not_exists, truthy, falsy, regex</code>
<code>actions/rules.py</code> (<code>rules.evaluate</code>)	6: <code>eq, ne, gt, lt, contains, in</code>

Además, `contains` normaliza a minúsculas en el primero (`str(expected).lower() in str(actual).lower()`) y **no** en el segundo (`str(expected) in str(value)`). El mismo nombre de operador se comporta distinto según dónde se use.

Recomendación. Hacer que `actions/rules.py` delegue en `engine.conditions.matches`.

Prioridad: 14.

R-15 · `_smart_summary` solo cubre 6 de los 27 flows, y el README afirma otra cosa

Campo	Valor
Severidad	[medio] Media
Categoría	Documentación · experiencia de uso
Impacto	El detalle de 21 flows muestra solo JSON crudo; el README promete lo contrario
Probabilidad	Cierta
Ubicación	<code>app/server.py::_smart_summary</code> , <code>README.md</code>

Evidencia. La cadena de `elif` tiene rama para `screen_capture_analyze`, `screen_capture_browser`, `folder_inventory`, `document_drop_pipeline`, `system_healthcheck` y `process_watchdog`. Para el resto devuelve cadena vacía y `render_run_detail` omite el bloque entero.

El `README.md` afirma, en su demo de cinco minutos: «click el último run del flow 07 → verás los 10 datos enviados como **lista legible** (no JSON crudo)». **No hay rama para `browser_form_filler`.**

Recomendación. Añadir ramas para los flows más usados, o corregir el README.

Prioridad: 15.

R-16 · La cobertura no es reproducible entre ejecuciones **REPRODUCIDO**

Campo	Valor
Severidad	[medio] Media
Impacto	La CI podría fallar sin ningún cambio de código
Probabilidad	Baja pero real: el margen sobre el gate es < 5 puntos
Ubicación	<code>app/server.py</code> (líneas de módulo), <code>pyproject.toml</code>

Evidencia reproducida. Dos ejecuciones consecutivas de `python -m pytest` en el mismo commit dieron **58,92 %** y **60,0 %**. `engine/scheduler.py` marcó **51 %** en una y **76 %** en la otra.

INFERENCIA de la causa: importar `app.server` arranca un hilo de scheduler (`SCHEDULER.start_in_background()` a nivel de módulo). Cuántas líneas de `scheduler.py` se ejecutan depende del temporizado del hilo. El gate está en 54 %.

Recomendación. No arrancar el scheduler al importar: moverlo a `run_server()`, o condicionarlo a una variable de entorno que los tests no definan.

Prioridad: 16.

R-17 · Importar `app.server` tiene cuatro efectos secundarios

Campo	Valor
Severidad	[medio] Media
Impacto	Cualquier <code>import app.server</code> levanta un hilo, crea la base y sincroniza el catálogo
Probabilidad	Cierta
Ubicación	<code>app/server.py</code> , líneas 100–104

Evidencia.

```
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Se ejecutan al **importar** el módulo, no al llamar a `run_server()`. Afecta a `tests/test_panel_endpoints.py` y es la causa de R-16.

Recomendación. Mover los cuatro a `run_server()`.

Prioridad: 17.

R-18 · No hay lockfile de dependencias versionado

Campo	Valor
Severidad	[medio] Media
Categoría	Seguridad · reproducibilidad
Impacto	Dos instalaciones pueden resolver versiones distintas; un escáner de CVE no puede pronunciarse con precisión
Probabilidad	Cierta
Ubicación	Raíz del repositorio

Evidencia. `git ls-files` no incluye `uv.lock`, `requirements.lock`, `poetry.lock` ni `Pipfile.lock`. El comentario de `requirements.txt` lo reconoce: «Para reproducibilidad estricta usar `uv export` (lockfile en CI) — este archivo solo fija un piso seguro».

Atenuante fuerte: `pyproject.toml` fija un piso explícito por paquete con motivo documentado («primera versión sin CVE conocida al corte de la auditoría 2026-06-01») y una cota superior de *major*. Es mejor que un rango abierto.

Recomendación. Generar y versionar `uv.lock`.

Prioridad: 18.

4. Severidad baja

R-19 · requirements.txt está incompleto

Severidad [bajo] · Ubicación requirements.txt frente a pyproject.toml

Declara Pillow, psutil, requests, mss, pyautogui y pytesseract. Le faltan pyperclip, PyGetWindow y pywebview. make install usa ese archivo y deja el entorno sin la ventana nativa ni los flows 15 y 16. Además, el archivo **no termina en salto de línea**.

Recomendación. Sincronizarlo o eliminarlo y apuntar solo a pyproject.toml. **Prioridad: 19.**

R-20 · pyproject.toml declara 31 entry points frente a 36 acciones registradas

Severidad [bajo] · Ubicación pyproject.toml, [project.entry-points."automa.actions"]

Faltan browser.capture_page, browser.crawl_site, browser.extract_content, browser.fill_form y http.check_urls. Verificado programáticamente. No rompe el runtime —LazyActionRegistry consulta primero su diccionario interno— pero un paquete externo que inspeccione el grupo verá un catálogo incompleto.

Recomendación. Sincronizar, o generar la sección desde _BUILT_IN_ACTIONS. **Prioridad: 20.**

R-21 · Código muerto en decision/

Severidad [bajo] · Ubicación decision/rules.py, decision/optional_ai.py

prioritize_steps y suggest_step_order devuelven la lista sin tocarla y **nadie los importa**. Verificado con búsqueda en todo el repositorio. El paquete sí aparece en [tool.hatch.build.targets.wheel].packages y en --cov=decision, donde figura al 0 %, arrastrando la cobertura global hacia abajo.

Matiz: la docstring de optional_ai.py documenta una decisión arquitectónica que sigue siendo válida —«La IA solo debe sugerir orden o prioridad, nunca reemplazar la ejecución del motor»— aunque el código no exista.

Recomendación. Eliminar el paquete y conservar la decisión en un ADR, o marcarlo explícitamente como punto de extensión y sacarlo de --cov. **Prioridad: 21.**

R-22 · Tres divergencias silenciosas del cron con crontab(5)

Severidad [bajo] · Ubicación engine/cron.py

Divergencia	Aquí	crontab(5)
Día de la semana	0 = lunes	0 = domingo
Combinación día-mes / día-semana	AND	OR cuando ninguno es *
Zona horaria	Siempre UTC	Hora local del sistema

La primera está documentada en un comentario del código; las otras dos no. Ninguna aparece en docs/OPERACION.md ni en la ayuda del panel. Una expresión copiada de internet se ejecutará el día o la hora equivocados **sin ningún error**.

Recomendación. Documentarlas en la ayuda del panel y en docs/OPERACION.md. Considerar alinear el día de la semana con el estándar. **Prioridad: 22.**

R-23 · Diez acciones registradas sin ningún flow que las use

Severidad [bajo] · Ubicación actions/

De las 36 registradas, **26 aparecen en algún manifest**. Las otras diez son filesystem.ensure_directory, read_text_file, move_file, http.fetch_url, ui.open_file_in_browser, ui.click, ui.click_bbox, vision.select_image,

`find_text_in_image` e `inspect_screen_target`.

No son código muerto —forman parte del contrato público del registro— pero son superficie sin cobertura de caso.

`vision.inspect_screen_target` es la más relevante: es el **único camino** hacia `VisionModelAnalyzer`, y al no usarla ningún flow, el catálogo publicado no puede llamar a un proveedor de IA.

Recomendación. Escribir un caso que las ejercite, o marcarlas como experimentales. **Prioridad: 23.**

R-24 · Cotas no declaradas en `summarize_text_folder` y `apply_tracking`

Severidad [bajo] · **Ubicación** `actions/filesystem.py`, `actions/browser_extract.py`

El sistema declara nueve de sus trece cotas (`truncated`, `links_truncated`, `text_truncated`, `stdout_truncated`...). Dos no lo hacen:

1. `summarize_text_folder` corta en `max_files` (10 por defecto) y solo acepta cinco extensiones. El resultado no dice cuántos archivos quedaron fuera.
2. `apply_tracking` trata un JSON de estado corrupto como inexistente y **reinicia la línea base en silencio**, reportando `first_run: true` sin explicar por qué.

Recomendación. Añadir `skipped_count/truncated` al primero y un campo `state_reset_reason` al segundo. **Prioridad: 24.**

R-25 · `app/server.py` concentra 1 753 líneas con cuatro responsabilidades

Severidad [bajo] · **Ubicación** `app/server.py`

El 29 % del Python de producción, con enrutado HTTP, ~430 líneas de CSS, ~660 de renderizado HTML por concatenación de f-strings, y la lógica de autorización. Cobertura: **38 %**, la más baja de los módulos grandes.

El riesgo concreto: el escapado HTML se aplica con `html.escape` en las rutas revisadas, pero **una ruta sin escapar sería difícil de detectar por lectura** en 660 líneas de f-strings.

Recomendación. Separar al menos la hoja de estilo y las plantillas en archivos estáticos servidos desde disco. **Prioridad: 25.**

R-26 · `scripts/smoke_test.py` modifica un archivo versionado **REPRODUCIDO**

Severidad [bajo] · **Ubicación** `scripts/smoke_test.py`, `engine/database.py::set_flow_config`

Llama a `set_flow_config('03_folder_inventory', ...)`, que reescribe `configs/03_folder_inventory.json`, un archivo versionado. **Reproducido:** tras ejecutar el smoke test, `git status --porcelain` marcó ese archivo como modificado. Se restaura con `git checkout --`.

Contribuye `engine/database.py::set_flow_config`, que usa `Path.write_text` sin `newline='\n'`, produciendo CRLF en Windows.

Recomendación. Que el smoke test use un flow de prueba desechable o un `AUTOMA_DATA_ROOT` temporal. Añadir `newline='\n'` a las escrituras de `set_flow_config` y `set_secret`. **Prioridad: 26.**

R-27 · Dos verificadores de pre-commit no tienen equivalente en CI

Severidad [bajo] · **Ubicación** `.pre-commit-config.yaml`, `.github/workflows/`

`ruff-format` y `markdownlint-cli2` están declarados como hooks locales pero **no se ejecutan en ningún workflow**. Verificado: `grep -rn "ruff format" .github/workflows/` no devuelve nada. Un colaborador sin `pre-commit install` puede subir código con formato distinto sin que nada falle.

Además, `markdown-docs.yml` solo se dispara en `pull_request` con `paths: '**/*.md'`: un push directo a `main` con un enlace roto no lo detecta.

Recomendación. Añadir `ruff format --check` a `ci.yml` y disparar `markdown-docs.yml` también en `push` a `main`. **Prioridad: 27.**

5. Decisiones que requieren validación humana

No son defectos: son puntos donde el código toma una decisión que alguien debería confirmar conscientemente.

#	Decisión	Dónde	Por qué requiere confirmación
D-1	Mantener <code>plugins/analyzers/vision_model_analyzer.py</code> en el repositorio	<code>plugins/</code>	222 líneas capaces de enviar capturas de pantalla a un endpoint externo, sin ningún flow que las use. Contradice aparentemente el posicionamiento «sin IA», aunque en la práctica es inalcanzable
D-2	<code>browser.fill_form</code> con <code>headless=False</code> por defecto	<code>actions/browser_form.py</code>	Abre una ventana visible que interfiere con el trabajo del operador. Es intencional (la demo es el producto) pero sorprende en uso programado
D-3	<code>_pick_record</code> marca el registro como usado antes de llenar	<code>actions/browser_form.py</code>	Prefiere perder un registro a repetirlo. No está documentado en el README del flow
D-4	El webhook es síncrono	<code>app/server.py::do_POST</code>	Una petición contra un flow largo bloquea la conexión. <code>/api/run</code> sí es asíncrono
D-5	Los eventos se escriben dos veces (JSONL + tabla <code>events</code>)	<code>engine/logger.py</code>	Duplica el volumen de la traza. Es redundancia deliberada por resiliencia
D-6	<code>_persist()</code> tras cada paso	<code>engine/orchestrator.py</code>	Una escritura de disco y una transacción por paso. Correcto para flows cortos, caro para flows largos
D-7	El JSON Schema no se aplica en runtime	<code>engine/manifest_schema.py</code>	Solo lo usa la CI. Un manifest inválido llega hasta el motor y falla con <code>KeyError</code>
D-8	<code>release.yml</code> no ejecuta <code>pytest</code>	<code>.github/workflows/release.yml</code>	Corre <code>validate_project.py</code> pero no la suite. Un tag sobre un commit rojo produciría instalador
D-9	Los datos sobreviven a la desinstalación	<code>engine/paths.py::data_dir</code>	<code>%LOCALAPPDATA%\Automa</code> no lo borra el desinstalador. Incluye todas las capturas de pantalla

6. Lo que está bien y hay que proteger

Estas propiedades son las que hacen el sistema sólido. Alguien las romperá si nadie dice que importan.

Propiedad	Dónde	Por qué protegerla
Separación puro/impuro	<code>actions/browser_extract.py</code>	Permite 91 % de cobertura sin navegador. Debería ser obligatorio en toda acción nueva
<code>dry_run</code> en las acciones de UI	<code>actions/ui.py</code>	Hace testeable lo que por naturaleza no lo es, y permite probar sin efectos

Cotas explícitas y declaradas	Casi todas las acciones	<code>truncated, links_truncated, stdout_truncated...</code> Nunca truncado silencioso
Degradación con motivo legible	OCR, portapapeles, Playwright	<code>status: "unavailable"</code> con instrucciones por sistema operativo, en vez de un crash opaco
Determinismo declarado y sostenido	<code>crawl_pages, check_urls, resolve_links</code>	Orden de aparición, sin aleatoriedad. La única fuente de no-determinismo es <code>random.choice</code> en el flow 07, y es intencional
<code>policy.summary()</code> guardado en el histórico	<code>engine/orchestrator.py</code>	Se puede auditar, corrida a corrida, cuáles corrieron sin restricción
Comentarios que explican el porqué	<code>paths.py, introspection.py, template.py, ui.py, server.py</code>	Documentan decisiones y modelos de amenaza, no líneas de código
CWE citados en el código	<code>app/server.py, actions/ui.py, actions/system.py</code>	Cada control dice qué cierra. Práctica poco habitual y muy útil para auditar
12 capas de hardening del CI	<code>.github/workflows/</code>	SHA pin verificado por parser YAML propio, <code>detect-secrets</code> sobre 50 commits de historial, checksum del propio <code>actionlint</code>
Separación raíz lectura / raíz escritura	<code>engine/paths.py</code>	Resolvió un fallo real de despliegue en la arquitectura, no con un parche
SQL 100 % parametrizado	<code>engine/database.py</code>	Sin una sola concatenación
<code>_safe_folder</code> en las 7 rutas	<code>app/server.py</code>	Una única función, aplicada consistentemente
Los 4 controles de <code>/file</code>	<code>app/server.py</code>	Incluido el <code>+ os.sep</code> que cierra el bypass por prefijo hermano
El validador comprueba coherencia de política	<code>scripts/validate_project.py</code>	No solo sintaxis: verifica que ningún paso use una acción que su propio manifest prohíbe
Los flows web apuntan a HTML locales por defecto	<code>27 context.example.json</code>	La demo es determinista y sin internet. Cero tráfico de red por defecto
Cotas de dependencias con motivo escrito	<code>pyproject.toml</code>	«Primera versión sin CVE conocida al corte de la auditoría 2026-06-01»
Fixtures que aíslan sin parchear	<code>tests/conftest.py</code>	Cambian el cwd; las pruebas ejercitan el mismo camino que producción
El sistema crece por casos, no por refactor	<code>flows/</code>	27 casos construidos con las 36 acciones existentes. El motor no se toca

7. Plan sugerido, por esfuerzo e impacto

Ninguna acción se ejecutó.

Rápidas y de alto impacto (< 1 hora cada una)

#	Acción	Hallazgo
1	Añadir <code>actions.browser_extract</code> a <code>hiddenimports</code>	R-01

2	Cambiar <code>if raw.get(...)</code> por <code>is not None</code> en <code>load_manifest</code>	R-05
3	Exigir token en los GET cuando <code>AUTOMA_PANEL_TOKEN</code> esté definido	R-02
4	Reconfigurar <code>sys.stdout</code> a UTF-8 en <code>runner.main()</code>	R-06
5	Regla en <code>validate_project.py</code> : exigir <code>allowed_actions</code>	R-03
6	Importar <code>root_dir</code> de <code>engine.paths</code> en <code>catalog.py</code>	R-11
7	Sincronizar <code>requirements.txt</code> y los entry points	R-19, R-20
8	Añadir <code>ruff format --check</code> a <code>ci.yml</code>	R-27

Medias (medio día cada una)

#	Acción	Hallazgo
9	Mover el arranque del scheduler a <code>run_server()</code>	R-16, R-17
10	Aplicar <code>run_locks</code> en las rutas del panel y del webhook	R-07
11	Índices sobre <code>runs.flow_id</code> , <code>steps.run_id</code> , <code>events.run_id</code>	R-08
12	Unificar los dos motores de condiciones	R-14
13	Declarar las divergencias del cron en la ayuda del panel	R-22
14	Pruebas de <code>_pick_record</code> , <code>find_text_in_image</code> y del CLI	R-06, R-23

Mayores (varios días)

#	Acción	Hallazgo
15	Timeout real por acción	R-09
16	Rutina de retención configurable	R-08
17	Extraer CSS y plantillas de <code>app/server.py</code>	R-25
18	Generar y versionar el lockfile, e integrarlo en la CI	R-18

Documentos relacionados: [11 · Seguridad](#) · [12 · Pruebas y calidad](#) · [13 · Despliegue y operación](#) · [14 · Solución de problemas](#) · [17 · Resumen ejecutivo](#)

16 · Glosario

Términos técnicos, siglas y palabras del dominio, explicados para que los entienda alguien sin formación técnica. Los términos marcados **[propio]** propio de Automa significan algo específico en este sistema y no coinciden necesariamente con su uso general.

A

Acción · [propio] propio de Automa Una operación concreta que el sistema sabe hacer: sacar una foto de la pantalla, escribir un archivo, abrir el Explorador, leer una página web. Hay **36**. Cada una tiene un nombre con punto (`screen.capture_screenshot`) y un flow la invoca escribiendo ese nombre en su receta. *Analogía*: son los verbos del idioma en que se escriben las recetas.

allowed_actions · [propio] propio de Automa Lista escrita dentro de una receta que dice qué acciones puede usar. Si una receta la declara, el motor rechaza cualquier acción que no esté en la lista. **Es opcional, y 14 de las 27 recetas actuales no la tienen.**

[!] Escribir una lista **vacía** no bloquea nada: el sistema la interpreta como «sin restricción». Ver **15 · Riesgos**, hallazgo R-05.

allowed_paths · [propio] propio de Automa Lista de carpetas donde una receta puede leer o escribir. El motor comprueba las rutas antes de cada paso. *Detalle importante*: la comprobación se hace mirando **el nombre del parámetro**, no su contenido; un parámetro llamado `target` que contenga una ruta no se revisa.

Analizador · [propio] propio de Automa Componente intercambiable que examina una imagen. Hay cuatro: `mock` (mide el brillo medio), `metadata` (dimensiones y huella digital), `ocr` (lee el texto de la imagen) y uno de visión con inteligencia artificial que **ningún caso del sistema utiliza**.

API · *Application Programming Interface* La forma en que otro programa puede hablar con este. Automa ofrece una API en `127.0.0.1:8787`: un programa puede pedirle que ejecute una receta o consultar el historial.

Append-only Un archivo al que solo se añaden líneas al final; nada se reescribe ni se borra. Los registros de eventos de Automa (`logs/*.jsonl`) funcionan así. *Analogía*: un cuaderno en el que solo se escribe hacia adelante.

B

Bbox · abreviatura de *bounding box*, «caja delimitadora» Un rectángulo definido por cuatro números: dónde empieza a la izquierda, dónde empieza arriba, cuánto mide de ancho y cuánto de alto. Se usa para recortar una parte de la pantalla o para señalar dónde está una palabra dentro de una imagen.

En Automa, un valor **negativo** significa «contado desde el borde opuesto». Por eso la barra de tareas se captura con `top:-48`: cuarenta y ocho puntos desde abajo.

BFS · *Breadth-First Search*, recorrido en anchura Forma de explorar un sitio web: primero se visitan todas las páginas enlazadas desde la principal, después las enlazadas desde aquellas, y así sucesivamente. Automa lo usa para hacer el mapa de un sitio, siempre con un tope de páginas y de profundidad.

C

Caso / Flow / Flujo · [propio] propio de Automa Una receta. Una carpeta dentro de `flows/` con tres archivos: la receta en sí (`manifest.json`), sus valores por defecto (`context.example.json`) y su explicación (`README.md`). Hay **27**.

Chromium El navegador que Automa arranca por su cuenta para ver páginas web. Es el mismo motor que usan Chrome y Edge. Automa lo controla con una biblioteca llamada Playwright. **Hay que instalarlo aparte**: no viene con el programa.

CI / CD · *Continuous Integration / Continuous Delivery* Los robots que revisan el código automáticamente cada vez que alguien lo modifica, y los que construyen el instalador cuando llega el momento de publicar una versión. Automa tiene seis de estos robots configurados.

Contexto · [propio] propio de Automa El cuaderno de notas de una ejecución. Empieza con los valores de configuración de la receta, y cada paso puede añadirle su resultado. El paso siguiente puede leer lo que escribió el anterior. Al terminar, todo el cuaderno se guarda en el historial.

Cron Una forma abreviada y muy antigua de escribir horarios: cinco números separados por espacios que significan minuto, hora, día del mes, mes y día de la semana. `*/15 * * * *` significa «cada quince minutos».

[!] **En Automa, 0 es lunes**, mientras que en el estándar de toda la vida 0 es domingo. Y las horas se cuentan en UTC, no en la hora local. Una expresión copiada de internet se ejecutará el día o la hora equivocados sin dar ningún error.

CSRF · *Cross-Site Request Forgery* Un ataque en el que una página web maliciosa que usted visita intenta dar órdenes a un programa que corre en su propio computador, aprovechando que su navegador ya está «dentro». Automa se defiende comprobando de dónde viene cada orden.

CVE · *Common Vulnerabilities and Exposures* El catálogo público mundial de fallos de seguridad conocidos. Cada fallo tiene un número. Automa fija versiones mínimas de sus componentes eligiendo la primera versión sin fallos conocidos.

CWE · *Common Weakness Enumeration* Catálogo de **tipos** de fallo de seguridad, no de fallos concretos. Por ejemplo, CWE-78 es «inyección de comandos». Los comentarios del código de Automa citan qué CWE cierra cada control, lo que facilita mucho auditarlo.

D

Determinista · [propio] clave en este sistema Que hace exactamente lo mismo cada vez que se ejecuta, sin sorpresas ni improvisación. **Es la propiedad central de Automa**: no hay inteligencia artificial decidiendo nada. La única excepción intencional es el caso 07, que elige un registro al azar entre los que aún no ha usado.

Degradación (elegante) · [propio] patrón de Automa Cuando falta algo que un componente necesita, en vez de reventar, avisa con claridad y devuelve un resultado vacío pero válido. Si falta el lector de texto en imágenes, el sistema responde «no disponible, instálalo así» y la receta puede seguir por otro camino.

Dry run · «ensayo en seco» · [propio] propio de Automa Modo de prueba de las recetas que mueven el teclado o el ratón. Con `dry_run` activado, el sistema dice qué **habría** hecho, sin hacerlo. Es la forma segura de probar una receta nueva.

E

Entry point · «punto de entrada» Dos significados distintos: 1. **El comando que arranca el programa**: `automa`, `automa-panel`, `automa-desktop`. 2. **El mecanismo por el que otro programa puede añadir acciones nuevas** a Automa sin tocar su código.

Una acción de un tercero **nunca puede reemplazar** a una interna: el sistema da prioridad a las suyas.

Evento · [propio] propio de Automa Cada cosa que pasa durante una ejecución queda anotada como un evento: la receta empezó, un paso arrancó, un paso falló, la receta terminó. Hay nueve tipos. Se guardan **dos veces**: en un archivo y en la base de datos.

F

Familia · [propio] propio de Automa Etiqueta que agrupa las recetas por lo que hacen. Hay cinco: `sistema` (10 recetas), `navegador` (9), `pantalla` (6), `filesystem` (1) y `documentos` (1).

Frozen · «congelado» Se dice del programa cuando se ha empaquetado en un único ejecutable, en vez de ejecutarse desde el código fuente. Automa se comporta distinto en ese modo: guarda sus datos en otra carpeta, porque la del programa es de solo lectura.

H

Hash / SHA-256 · «huella digital» Un número largo que resume un texto o un archivo. Si el contenido cambia aunque sea en una letra, la huella cambia por completo. Automa lo usa para saber si una página web cambió desde la última vez, sin tener que guardar la página entera.

Headless · «sin cabeza» Un navegador que funciona sin mostrar ventana. Automa lo usa para leer páginas web sin molestar al usuario. La única receta que **sí** muestra ventana es la 07, y lo hace a propósito: es la demostración del producto.

L

Local-first · «primero local» · [propio] filosofía del producto Todo ocurre en su computador. No hay servidor en internet, no hay cuenta de usuario, no se envía nada a ningún sitio. El programa escucha únicamente en la dirección `127.0.0.1`, que significa «este mismo equipo».

Lock · «cerrojo» · [propio] propio de Automa Marca que impide que la misma receta se ejecute dos veces a la vez.

[!] **Solo lo usa el programador de horarios.** Lanzar la misma receta dos veces desde el panel sí es posible.

LLM · *Large Language Model*, modelo de lenguaje grande Lo que popularmente se llama «una inteligencia artificial que escribe».

Automa no usa ninguno. Existe en el código un adaptador capaz de hablar con uno, pero ninguna de las 27 recetas puede llegar hasta él.

M

Manifest · «manifiesto» · [propio] propio de Automa El archivo `manifest.json` de una receta: la receta escrita. Dice qué pasos hay, en qué orden, con qué valores y qué hacer si algo falla. *Es el corazón del sistema*: añadir una funcionalidad nueva es escribir uno de estos, no programar.

Mermaid Un lenguaje para dibujar diagramas escribiendo texto. Los diagramas de esta documentación están escritos así, y se convierten en imágenes al generar los PDF.

O

OCR · *Optical Character Recognition*, reconocimiento óptico de caracteres Leer el texto que aparece dentro de una imagen. Automa lo usa para inventariar lo que se ve en el escritorio. Necesita un programa externo llamado `tesseract`; si falta, Automa lo dice con claridad en vez de fallar.

Orquestador · [propio] propio de Automa El motor que lee la receta y la ejecuta paso a paso, comprobando permisos, sustituyendo valores y anotándolo todo. Vive en `engine/orchestrator.py`.

P

Panel La pantalla desde la que se maneja todo. Tiene tres pestañas: **Ejecutar** (una tarjeta por receta), **Programadas** (horarios) e **Histórico** (todo lo que se ha ejecutado).

Paso · *step* · [propio] propio de Automa Cada instrucción de una receta. Tiene un nombre, la acción que ejecuta, sus valores, cuántas veces reintentar si falla y una condición opcional para saltárselo.

Placeholder · «marcador de posición» · [propio] propio de Automa Un hueco en la receta que se rellena al ejecutarla. Se escribe entre llaves: `{{ target_url }}` se sustituye por la dirección configurada, y `{now}` por la fecha y hora del momento.

Playwright La biblioteca que permite a Automa controlar el navegador Chromium: abrirlo, navegar, rellenar formularios y leer el contenido. **No se instala automáticamente.**

Preview · [propio] propio de Automa Marca que señala una receta como «todavía no lista». El panel la muestra con un aviso y bloquea su ejecución. **Ninguna de las 27 recetas actuales está marcada así.**

Prometheus Un formato estándar para publicar números de funcionamiento (cuántas ejecuciones, cuánto tardaron, cuántas fallaron) que otros programas de vigilancia saben leer.

pywebview La biblioteca que mete el panel dentro de una ventana normal de Windows, para que no haya que abrir un navegador.

R

Regla · *rule* · [propio] propio de Automa Una comprobación escrita dentro de la receta: «si la memoria supera el 80 %, marca esto como alerta». Se evalúan en orden y **gana la primera que se cumple**.

Las reglas soportan 6 comparadores, mientras que las condiciones de los pasos soportan 13. No son el mismo motor.

Reintento · *retry* Cuántas veces volver a intentar un paso que falló. Por defecto, ninguna.

[!] **No hay espera entre reintentos.** Tres intentos contra un servidor caído ocurren en milisegundos.

robots.txt Un archivo que los sitios web publican para indicar qué pueden y qué no pueden visitar los programas automáticos. Automa lo respeta al recorrer un sitio. Si no lo puede leer, permite el acceso **pero lo deja anotado** en el informe.

Run / Corrida · [propio] propio de Automa Una ejecución concreta de una receta, con su identificador único, su hora de inicio, su duración y su resultado. Todo queda en el historial.

S

Sandbox · «caja de arena» · [propio] propio de Automa Las restricciones que una receta se impone a sí misma: qué acciones puede usar, dónde puede escribir y cuánto puede tardar.

[!] **Es una promesa, no una jaula.** Las hace cumplir el propio programa, no el sistema operativo. *Analogía:* es un cartel de «no pasar», no una puerta cerrada con llave. Y catorce de las veintisiete recetas no llevan cartel puesto.

Scheduler · «programador de tareas» · [propio] propio de Automa El componente que revisa cada dos segundos si toca ejecutar alguna receta programada.

[!] **Si una receta programada falla, no avisa a nadie.** Reprograma la siguiente ejecución con normalidad. Solo se descubre mirando el historial.

Secreto · *secret* Una contraseña, clave o testigo de acceso. Automa los busca primero en las variables del sistema y después en un archivo. **Ese archivo no está cifrado:** lo protegen únicamente los permisos del sistema operativo.

SQLite Una base de datos que cabe en un solo archivo (**db/runs.db**) y no necesita ningún servicio corriendo aparte. Es donde vive todo el historial de Automa.

Supply chain · «cadena de suministro» Todo lo que entra en el producto sin haberlo escrito uno: bibliotecas de terceros, herramientas de construcción, robots del CI. Es el punto que más protege este proyecto, porque un cambio malicioso ahí se traduciría en control del computador de quien lo use.

T

Tesseract El programa que lee texto dentro de imágenes. **Hay que instalarlo aparte.** Si falta, Automa avisa con las instrucciones concretas para cada sistema operativo.

Tracking / Seguimiento · [propio] propio de Automa La memoria que tienen algunas recetas entre una ejecución y la siguiente. Son tres archivos: el que recuerda qué registros ya se usaron en el caso 07, y los dos que guardan cómo estaba una página la última vez que se miró (casos 23 y 26).

[!] **No son archivos temporales.** Borrarlos hace que el caso 07 repita registros y que el 23 deje de detectar el cambio que estaba vigilando.

Transición · [propio] propio de Automa La regla que dice a qué paso saltar al terminar el actual, según si salió bien o mal. Es lo que permite que una receta tenga caminos alternativos.

[!] Para que una receta se recupere de un fallo, la transición de error debe apuntar a un paso **distinto** del que venía a continuación. Si apunta al mismo, el motor no lo distingue de «no hay plan B».

U

UTC · *Coordinated Universal Time* La hora de referencia mundial, sin ajustes de zona horaria ni horario de verano. Automa guarda todas sus fechas así, **y también interpreta así los horarios programados.** Para alguien en la zona horaria de Chile continental, «las 9:00» programadas son las 6:00 de la mañana local.

V

Validador · [propio] propio de Automa El programa que revisa las 27 recetas antes de dar el visto bueno: comprueba que cumplan el formato, que todas sus acciones existan y que sus saltos apunten a pasos reales.

No comprueba que los valores de cada paso coincidan con lo que la acción espera. Ese error solo aparece al ejecutar.

W

Webhook · «gancho web» Una dirección a la que otro programa puede llamar para disparar una receta.

Está desactivado por defecto. Solo funciona si el operador define una clave de acceso.

Convenciones de nombres en este repositorio

Patrón	Significado	Ejemplo
<code>NN_nombre_descriptivo/</code>	Carpeta de una receta, numerada por orden de creación	<code>23_web_change_detector/</code>
<code>familia.accion</code>	Nombre de una acción	<code>browser.extract_content</code>
<code>snake_case</code>	Variables y funciones de Python	<code>assert_paths_allowed</code>
<code>PascalCase</code>	Clases de Python	<code>LazyActionRegistry</code>
<code>_prefijo</code>	Función interna, no pensada para usarse desde fuera	<code>_safe_folder</code>
<code>AUTOMA_*</code>	Variable de entorno del sistema	<code>AUTOMA_PANEL_TOKEN</code>
<code>flujo_*</code>	Nombre de una métrica publicada	<code>flujo_runs_total</code>
<code>AAAAMDDTHHMMSSffffffZ</code>	Identificador de una ejecución	<code>20260827T143052123456Z</code>
<code>sched_*</code>	Identificador de un cerrojo tomado por el programador	<code>sched_20260827T143052...</code>

Curiosidad histórica: el prefijo `flujo_` de las métricas es un resto del nombre anterior del proyecto. Cambiarlo ahora rompería cualquier panel de vigilancia que ya lo esté leyendo, así que se conserva a propósito.

Documentos relacionados: [01 · Descripción general](#) · [05 · Referencia técnica](#) · [17 · Resumen ejecutivo](#) · [18 · Guía para nuevo desarrollador](#)

17 · Resumen ejecutivo

Para dirección y para decisión. Sin detalle técnico innecesario. Cada afirmación está respaldada por evidencia del repositorio; el detalle está en los documentos que se enlazan.

1. Qué es

Automa es un producto de automatización de escritorio para Windows. Convierte tareas repetitivas —capturar evidencia, rellenar formularios, inventariar carpetas, vigilar páginas web, auditar el equipo— en recetas escritas que se ejecutan solas, dejando registro completo de todo lo que hicieron.

Tres decisiones lo definen:

- 1. **Todo ocurre en el equipo del usuario.** No hay servidor, no hay cuenta, no hay telemetría. Con la configuración por defecto, el sistema **no genera tráfico de red**.
- 2. **No hay inteligencia artificial en el camino de ejecución.** Las decisiones las toman reglas escritas por una persona. Verificado: ninguna de las 27 recetas puede alcanzar el adaptador de visión con IA que existe en el código.
- 3. **El sistema crece añadiendo recetas, no modificando el motor.** Las 27 recetas actuales están construidas con las 36 operaciones que ya existen.

2. Qué necesidad cubre

Un operador que trabaja sobre Windows puede escribir un script para cualquiera de estas tareas. Lo que no obtiene con un script es lo que Automa aporta:

Sin Automa	Con Automa
Cada script con su propia forma y sus propias reglas	Un contrato único, validado automáticamente
Sin registro de qué se ejecutó, cuándo ni con qué resultado	Historial completo, consultable, con duración y evidencia
Un script puede escribir donde quiera	Cada receta puede acotar qué acciones usa y dónde escribe
Programar la tarea implica salir a otra herramienta	Programador propio, con horarios o intervalos
Hay que abrir una terminal	Panel con tarjetas y atajos de teclado

3. Quién lo utiliza

Un operador local. No hay usuarios múltiples, ni roles, ni permisos: quien tiene la sesión de Windows tiene el sistema completo. Es coherente con un producto de escritorio, y conviene tenerlo presente al evaluar su despliegue.

Perfiles secundarios: quien escribe recetas nuevas (necesita entender un formato JSON, no programar), quien añade operaciones nuevas (necesita Python) y sistemas externos que disparan recetas por una llamada autenticada.

4. Capacidades principales

27 casos operativos, agrupados por lo que hacen:

Grupo	Casos	Ejemplos
Operaciones sobre Windows	10	Bloquear la sesión, abrir el Explorador, leer el portapapeles, auditar con PowerShell

Extracción y vigilancia web	9	Rellenar un formulario, mapear un sitio, detectar cambios en una página, auditar enlaces rotos, extraer tablas
Captura de pantalla	6	Escritorio limpio, ventana activa, barra de tareas, inventario de texto visible
Utilidades sobre archivos	2	Inventario de carpeta, resumen de documentos de texto

Más: panel de tres pestañas con atajos de teclado, programador con horarios tipo cron, métricas en formato estándar de la industria, instalador de Windows que no requiere permisos de administrador, y un mecanismo para que terceros publiquen operaciones nuevas.

5. Tecnologías

Python 3.10 o superior, sin framework web ni base de datos externa. SQLite para el historial. El navegador Chromium se controla con Playwright cuando una receta lo necesita. El instalador se construye con PyInstaller e Inno Setup. Licencia MIT.

Dependencias externas relevantes: nueve paquetes declarados, todos con versión mínima elegida por criterio de seguridad y con motivo escrito en el propio código.

6. Arquitectura resumida

Un archivo JSON describe la tarea. Un motor lo lee, comprueba los permisos que la propia receta declara, ejecuta cada paso y guarda todo lo que pasó en tres sitios a la vez: base de datos, registro de eventos y copia completa del estado. El panel y el programador de horarios son dos formas de disparar ese mismo motor.

Es deliberadamente **un solo proceso, sin servicios ni colas**. Para una herramienta local de un operador, es la decisión correcta.

7. Estado actual verificado

Medido ejecutando los propios controles del proyecto sobre el código publicado (commit [ff246ab](#), versión [0.3.0](#)):

Control	Resultado
Suite de pruebas	150 de 150 pasan
Cobertura de código	59 % (umbral exigido: 54 %)
Análisis de estilo	Sin hallazgos
Validación de las 27 recetas	Sin errores
Prueba de humo de extremo a extremo	Correcta
Estado del repositorio	Limpio

Los cuatro controles del proyecto están en verde.

Tamaño: 210 archivos, 6 126 líneas de código de producción y 1 525 de pruebas. Es un proyecto pequeño y manejable.

8. Fortalezas

Cinco propiedades destacan por encima de la media de proyectos de este tamaño:

Seguridad de la cadena de suministro tratada como problema de primer orden. Doce capas de defensa en el pipeline, incluyendo un verificador propio que impide introducir una dependencia sin fijar, y un escaneo de credenciales sobre los últimos cincuenta cambios del historial. El razonamiento está escrito en el propio código: este producto puede controlar el teclado del usuario, así que un cambio malicioso equivale a control del equipo.

El sistema nunca oculta lo que no hizo. Cuando recorta un resultado, lo declara. Cuando falta un componente externo, devuelve un mensaje con las instrucciones de instalación en vez de fallar de forma opaca. Es una disciplina poco común y muy valiosa para la confianza del operador.

Diseño que hace testeable lo que normalmente no lo es. La familia de recetas web alcanza **91 % de cobertura sin abrir un solo navegador**, porque la lógica está separada de la interacción. Las operaciones que mueven teclado y ratón tienen un modo de ensayo.

Las decisiones están documentadas donde importan. Los comentarios del código explican *por qué* existe cada control y qué tipo de fallo cierra, no qué hace la línea siguiente. Esto reduce drásticamente el costo de auditar y de incorporar gente nueva.

Crece por casos, no por refactor. Añadir una funcionalidad es añadir una carpeta con un JSON. El motor lleva tres versiones sin necesitar cambios estructurales.

9. Riesgos

Se identificaron **27 hallazgos**: 6 de severidad alta, 12 media y 9 baja. Detalle completo en [15 · Riesgos y deuda técnica](#). Los cuatro que exigen decisión:

#	Riesgo	Consecuencia	Esfuerzo de corrección
1	Siete recetas probablemente no funcionan en el producto instalado	La familia web completa —la novedad de la última versión— fallaría en el instalador que descarga el usuario final. Falta declarar un módulo en la configuración de empaquetado	< 1 hora
2	El historial completo es legible sin autenticación	Capturas de pantalla, contenido del portapapeles y texto de ventanas abiertas quedan accesibles desde la interfaz local sin clave, incluso teniendo una configurada. En un equipo de un solo usuario el riesgo es bajo; deja de serlo si se comparte el equipo o se expone el panel	< 1 hora
3	La mitad de las recetas corre sin restricciones declaradas	14 de 27 no acotan qué pueden hacer ni dónde escribir. Nada en el proceso automático lo exige, así que el hueco crece con cada caso nuevo	1 hora el control automático; días retroadaptar las 14
4	La línea de comandos falla en Windows	Los comandos <code>automa list</code> y <code>automa run</code> terminan con error al mostrar el resultado. La tarea sí se ejecuta correctamente; lo que falla es la impresión posterior. Se soluciona con una variable de entorno	< 1 hora

Riesgos de operación a medio plazo: no existe ninguna rutina de limpieza del historial, que crece sin límite; y una tarea programada que falle de forma sistemática no genera ninguna alerta.

10. Oportunidades de mejora

Ordenadas por relación entre esfuerzo y beneficio:

Prioridad	Acción	Esfuerzo	Beneficio
-----------	--------	----------	-----------

1	Corregir el empaquetado de la familia web	< 1 h	Recupera 7 de 27 casos en el producto distribuido
2	Exigir autenticación también en las consultas	< 1 h	Cierra la exposición del historial
3	Que el validador exija restricciones en toda receta nueva	1 h	Impide que el problema crezca
4	Arreglar la salida de la línea de comandos	< 1 h	Elimina un fallo visible
5	Sincronizar los tres listados de dependencias que hoy difieren	1 h	Instalaciones reproducibles
6	Añadir índices a la base de datos y una rutina de limpieza	1 día	Sostenibilidad a largo plazo
7	Aviso ante tareas programadas que fallan de forma repetida	1 día	Operación desatendida fiable
8	Pruebas de las tres partes hoy sin cubrir	2 días	Reduce el riesgo de regresión

Total de las cinco primeras: menos de un día de trabajo, y resuelven los cuatro riesgos altos.

11. Estado de madurez, con honestidad

Dimensión	Valoración	Fundamento
Motor y arquitectura	Sólido	87 % de cobertura en el componente central; tres versiones sin cambios estructurales
Catálogo de casos	Sólido	27 casos validados automáticamente
Pruebas de la lógica pura	Sólido	150 pruebas, ejecución en 17 segundos
Seguridad del proceso de construcción	Muy sólido	Por encima de la media del sector
Interfaz de usuario	Funcional	Cubre lo necesario; el 62 % de su código no tiene pruebas
Empaquetado y distribución	Con un fallo pendiente	Automatizado, pero con el riesgo nº 1 abierto
Seguridad en ejecución	Parcial	Las restricciones existen, funcionan y la mitad del catálogo no las usa
Operación desatendida	Incompleta	Sin alertas, sin limpieza, sin recuperación automática
Documentación	Buena y ahora completa	Doce documentos previos orientados a producto, más estos veinte de sistema

Lectura de conjunto: el sustrato está bien construido y bien probado. Los huecos están donde el sistema toca el mundo real —interfaz, empaquetado, operación continua— y todos son conocidos, están localizados y son abordables.

12. Próximos pasos recomendados

Inmediato — antes de la siguiente publicación

Los cuatro riesgos altos con corrección de menos de una hora cada uno: el empaquetado de la familia web, la autenticación de las consultas, el fallo de la línea de comandos y la sincronización de los listados de dependencias. **Menos de un día en total.**

Corto plazo — próximas semanas

- Regla automática que exija restricciones de seguridad en toda receta nueva.
- Índices en la base de datos y rutina de limpieza configurable.
- Pruebas de las tres partes hoy sin cubrir.
- Documentar las tres particularidades del programador de horarios que hoy son silenciosas y pueden hacer que una tarea se ejecute el día o la hora equivocados.

Medio plazo — decisiones de dirección

Cuatro puntos que requieren una decisión, no solo trabajo técnico:

1. **Qué hacer con el adaptador de inteligencia artificial.** Son 222 líneas capaces de enviar capturas de pantalla a un servicio externo, sin ninguna receta que las use. Contradice aparentemente el posicionamiento del producto, aunque en la práctica sea inalcanzable. Opciones: eliminarlo, documentarlo explícitamente como punto de extensión, o construir el caso que lo justifique.
2. **Si el producto debe operar desatendido.** Si la respuesta es sí, faltan alertas, limpieza automática y recuperación de tareas colgadas.
3. **Si el equipo puede ser compartido.** Si la respuesta es sí, hace falta cifrado del historial y de las capturas, que hoy quedan en claro y sobreviven a la desinstalación.
4. **Si conviene firmar el instalador.** Hoy sin firma, lo que provoca un aviso de Windows al instalar y afecta a la percepción de confianza.

Lo que no hace falta hacer

- **No hace falta refactorizar el motor.** Funciona, está probado y su diseño ha soportado tres versiones de crecimiento.
- **No hace falta cambiar de base de datos.** SQLite es adecuado; lo que falta son índices y limpieza.
- **No hace falta añadir inteligencia artificial.** El determinismo es una característica del producto, no una carencia.

13. Una frase para llevarse

Automa es un producto pequeño, bien construido y honesto sobre sus límites, con cuatro problemas concretos que se corrigen en menos de un día y que hoy afectan a un tercio de su catálogo en el producto que descarga el usuario final.

Documentos relacionados: [01 · Descripción general](#) · [11 · Seguridad](#) · [12 · Pruebas y calidad](#) · [15 · Riesgos y deuda técnica](#) · [16 · Glosario](#)

18 · Guía para un nuevo desarrollador

Itinerario progresivo de incorporación. Empieza por lo que hay que leer, sigue por cómo preparar el entorno, y termina con tareas iniciales reales del repositorio, ordenadas por dificultad.

1. Qué leer primero, y en qué orden

No lea los veinte documentos. Este es el camino corto:

Día 1 · Entender qué es (≈ 45 min)

Orden	Documento	Por qué
1	README.md del repositorio	El catálogo de 27 casos con una frase cada uno. La visión del producto
2	01 · Descripción general	Qué resuelve, qué no hace, qué son sus límites
3	16 · Glosario	Los diez términos propios que se usan en todas partes: flow, manifest, acción, contexto, sandbox, tracking
4	docs/ARQUITECTURA.md	La versión corta del diseño, escrita por el autor

Día 2 · Entender cómo funciona (≈ 2 h)

Orden	Documento	Por qué
5	03 · Arquitectura	Capas, patrones, concurrencia y manejo de errores, con diagramas
6	06 · Explicación profunda , secciones 1 a 5	El bucle del orquestador, los placeholders, el sandbox y las condiciones, línea a línea
7	docs/CREAR_FLUJOS.md	El contrato que debe cumplir un flow

Día 3 · Saber dónde tocar (≈ 1 h)

Orden	Documento	Por qué
8	04 · Mapa del código	Qué hay en cada archivo y su estado: activo, sin uso, muerto
9	15 · Riesgos	Las trampas conocidas. Léalo antes de tocar nada
10	12 · Pruebas y calidad	Qué se prueba y qué no; qué se espera de su código

Los demás documentos son de consulta, no de lectura seguida: [05](#) para firmas y endpoints, [07](#) para el esquema, [10](#) para configurar, [14](#) cuando algo falle, [19](#) para seguir una funcionalidad de punta a punta.

2. Preparar el entorno

```
git clone https://github.com/vladimiracunadev-create/automa-pc.git
cd automa-pc
```

```
uv sync --extra dev --extra schema          # camino recomendado
# o bien:
python -m venv .venv && .venv\Scripts\activate && pip install -e ".[dev,schema]"

python -m playwright install chromium        # necesario para los flows 02, 07 y 21-27
pre-commit install                          # engancha ruff, formato y markdownlint
```

En Windows, añade esto a su perfil de PowerShell **antes de usar el CLI**:

```
$env:PYTHONIOENCODING = "utf-8"
```

Sin eso, `automa list` y `automa run` terminan con `UnicodeEncodeError`. Es un defecto real del repositorio, no un problema de su máquina. Ver [15 · R-06](#).

No use `make install`. Usa `requirements.txt`, al que le faltan tres paquetes.

Comprobar que quedó bien

```
python -m pytest                          # 150 passed
python -m ruff check .                    # All checks passed!
python scripts/validate_project.py        # {"ok": true, "flows_checked": 27, ...}
```

Si los tres pasan, está listo. Tardan unos 18 segundos en total.

3. Ejecutar el sistema por primera vez

```
automa-desktop          # ventana nativa
# o:
automa-panel            # panel HTTP en http://127.0.0.1:8787
```

Recorrido de cinco minutos, sin efectos sobre su equipo:

1. Pestaña **Ejecutar** → tarjeta de `05_system_healthcheck` → clic. Verá los tres pasos en tiempo real.
2. Pestaña **Histórico** → clic en la corrida recién hecha. Verá los pasos, sus duraciones, el contexto y el JSON generado.
3. Ejecute `03_folder_inventory`. Compare el resumen legible con el JSON crudo.
4. Abra `http://127.0.0.1:8787/metrics/dashboard`.
5. Ejecute `21_web_content_extract` (necesita Chromium). Apunta a un HTML local del propio repositorio: no sale a internet.

[!] **No ejecute todavía** los flows 08 (bloquea la sesión), 13, 14 o 20: mueven teclado y ratón de verdad. Cuando quiera probarlos, ponga primero `"dry_run": true` en su `context.example.json`.

4. Cómo está organizado el repositorio

engine/	El motor. No se toca para añadir un caso
actions/	Las 36 operaciones. Aquí se añade cuando hace falta una nueva
flows/	Los 27 casos. Aquí se añade casi siempre
app/	Panel HTTP y ventana nativa
plugins/	Analizadores de imagen intercambiables
schemas/	El contrato JSON del manifest
scripts/	Validador, smoke test y generador de PDF
tests/	150 pruebas
data/	HTML de demo, dataset semilla, carpetas de entrada
db/ logs/ state/ output/ configs/ secrets/	Almacenes en ejecución (ignorados por git)

La regla de oro del proyecto, escrita en su README: *el sistema se construye agregando casos, no refactorizando el motor.*

Antes de proponer un cambio en `engine/`, pregúntese si el caso se puede resolver con las 36 acciones existentes. En 27 de 27 ocasiones, la respuesta ha sido sí.

5. Seguir un flujo completo, de la interfaz a la persistencia

El mejor ejercicio de incorporación. Tome 05_system_healthcheck y recorra esto:

Paso	Archivo	Qué pasa
1	app/server.py::do_POST	Llega POST /api/run/05_system_healthcheck
2	app/server.py::_safe_folder	Se valida el slug de la URL
3	app/server.py::_authorize_mutation	Token, o Host/Origin/Referer
4	engine/orchestrator.py::__init__	Se crea la base, se lee el manifest y el contexto
5	engine/loader.py::load_manifest	JSON → FlowDefinition
6	engine/loader.py::load_context	Se resuelve la primera fuente de contexto que exista
7	engine/sandbox.py::assert_secrets_present	Único control por corrida
8	engine/orchestrator.py::run	Empieza el bucle
9	engine/template.py::render_value	{{ ... }} y {now} se sustituyen
10	engine/sandbox.py::assert_action_allowed	Se comprueba la acción
11	engine/action_registry.py::get	Se importa el módulo de la acción, la primera vez
12	actions/system.py::snapshot_system	Trabajo efectivo con psutil
13	engine/orchestrator.py	El resultado va al contexto con save_as
14	actions/rules.py::evaluate_rules	Se evalúan las reglas del manifest
15	actions/filesystem.py::write_json	Se escribe el reporte
16	engine/database.py::insert_step + upsert_run	Persistencia tras cada paso
17	engine/logger.py::JsonlLogger.write	Evento en archivo y en la tabla events
18	engine/introspection.py::extract_existing_graphs	Se detectan los archivos bajo output/
19	app/server.py::_run_status_payload	El panel hace polling y ve el avance
20	app/server.py::render_run_detail	Se dibuja el detalle

Ponga un print() en tres de esos puntos, ejecute el flow y observe el orden real. En media hora tendrá el modelo mental completo.

6. Dónde añadir cada cosa

Un caso nuevo (lo habitual)

```
mkdir flows/28_mi_caso
```

Tres archivos: manifest.json, context.example.json y README.md. Copie uno del bloque 08–20 como plantilla: son los que traen política de sandbox declarada.

```
{
  "id": "mi_caso",
  "name": "Nombre legible del caso",
  "family": "sistema",
  "description": "Qué hace, en una o dos frases.",
  "allowed_actions": ["system.snapshot_system", "filesystem.write_json"],
  "allowed_paths": ["output/reports"],
  "max_runtime_seconds": 30,
  "steps": [
    {
      "id": "primer_paso",
      "action": "system.snapshot_system",
      "params": {},
      "save_as": "snapshot"
    },
    {
      "id": "guardar",
      "action": "filesystem.write_json",
      "params": {
        "path": "output/reports/mi_caso_{now}.json",
        "data": "{{ snapshot }}"
      },
      "save_as": "report"
    }
  ]
}
```

Después:

```
python scripts/validate_project.py # debe devolver ok: true, 28 flows
PYTHONIOENCODING=utf-8 python -m engine.runner run flows/28_mi_caso
```

Y reinicie el panel: `sync_flows` solo corre al importar `app.server`.

Declare siempre `allowed_actions`, `allowed_paths` y `max_runtime_seconds`, aunque 14 flows existentes no lo hagan. Es lo correcto y es lo que va a exigirse. [!] **Nunca escriba `"allowed_actions": []`**: una lista vacía se interpreta como «sin restricción», no como «bloquear todo».

Una acción nueva (menos frecuente)

1. Escríbala en el módulo de `actions/` que corresponda a su familia.
2. Regístrela en `engine/action_registry.py::_BUILT_IN_ACTIONS`.
3. Añádala a `pyproject.toml`, [`project.entry-points."automa.actions"`].
4. Si carga un módulo pesado, añádala a `installer/automa.spec::hiddenimports`.
5. Escriba pruebas.

Contrato de una acción:

```
def mi_accion(param_obligatorio: str, param_opcional: int = 10) -> dict[str, Any]:
    """Qué hace y por qué existe.

    Args:
        param_obligatorio: ...
    Returns:
        dict serializable a JSON.
    Raises:
        RuntimeError: si falta una dependencia externa.
    """
```

Reglas no negociables:

- Devuelve un `dict` serializable a JSON. Va a acabar en SQLite.
- Los parámetros son por nombre. El motor hace `action(**rendered_params)`.
- Si aplica una cota, decláralo en el resultado (`truncated: True`). Nunca en silencio.
- Si falta una dependencia externa, mensaje útil, no un `ImportError` pelado.
- Si toca teclado o ratón, acepta `dry_run`.

- **Separa lo puro de lo impuro.** Es lo que permite probarla sin el mundo real. Mire `actions/browser_extract.py` como referencia: **91 % de cobertura sin abrir un navegador**.
- **Si un parámetro es una ruta, que su nombre contenga `path`, `output`, `file`, `source` o `destination`.** El sandbox detecta rutas por el nombre de la clave.

Los pasos 3 y 4 se olvidan con facilidad: los dos están hoy desincronizados en el repositorio (ver [15 · R-01](#) y [R-20](#)).

Un analizador de imagen nuevo

Implemente `plugins/analyzers/base.py::AnalyzerProtocol` (un método `analyze(image_path)`) y regístrelo en `actions/vision.py::ANALYZERS`.

Un endpoint nuevo del panel

En `app/server.py`, dentro de `do_GET` o `do_POST`.

[!] **Cuidado con este archivo.** 1 753 líneas con enrutado, CSS, HTML y JavaScript mezclados, y solo 38 % de cobertura. Si añade un POST, **debe** pasar por `_authorize_mutation`. Si renderiza contenido de una corrida, **debe** pasar por `html.escape`.

7. Cómo escribir pruebas

Copie el estilo de `tests/test_browser_extract.py`: es el mejor ejemplo del repositorio.

```
def test_mi_funcion_pura():
    """Una prueba por comportamiento, con nombre que lo describa."""
    assert mi_funcion("entrada") == {"resultado": "esperado"}

def test_flow_completo(tmp_runtime, project_root):
    """tmp_runtime aísla db/, logs/, state/ y output/ en un tmp_path."""
    orch = Orchestrator(project_root / 'flows' / '05_system_healthcheck')
    state = orch.run()
    assert state['status'] == 'completed'
```

Las dos fixtures disponibles están en `tests/conftest.py`:

Fixture	Qué hace
<code>tmp_runtime</code>	Crea los directorios de trabajo en un <code>tmp_path</code> y cambia el cwd
<code>project_root</code>	Devuelve la raíz del repositorio

Para probar algo que depende del mundo exterior, no lo simule con `mock`: cree un objeto falso con la misma interfaz. `test_browser_extract.py` usa una `FakePage` que imita a Playwright y un `fetch_page` falso para el crawl. Es más legible y no se rompe al cambiar la implementación.

El gate de cobertura está en 54 % y la medición real ronda el 59 %. Menos de cinco puntos de margen: una función nueva sin pruebas puede hacer fallar la CI.

8. Partes que exigen especial cuidado

Archivo	Por qué	Antes de tocarlo
<code>engine/orchestrator.py::run</code>	139 líneas de bucle con siete puntos de salida. Todo el sistema depende de él	Lea 06 §1 entera
<code>engine/template.py</code>	Los 27 manifests dependen de su semántica exacta	Entienda la diferencia entre placeholder exacto y embebido
<code>engine/sandbox.py</code>	Es la única barrera de seguridad en ejecución	Lea 11 §4

<code>actions/browser_extract.py::normalize_text</code>	Cambiarla invalida todos los hashes ya guardados. El flow 23 reportaría cambios falsos	No la toque sin plan de migración
<code>app/server.py</code> — auth y /file	Cierran CSRF, path traversal y XSS. Los comentarios citan los CWE	No relaje ningún control
<code>actions/ui.py</code>	Mueve teclado y ratón de la máquina de quien ejecute	Respete siempre <code>dry_run</code>
<code>actions/system.py::run_powershell</code>	Ejecuta comandos en el equipo	No amplíe la allowlist por defecto
<code>engine/database.py::init_db</code>	El esquema. Sin migraciones más allá de añadir columnas	Piense en las bases existentes
<code>installer/automa.spec</code>	Un <code>hiddenimport</code> que falte rompe un flow solo en el binario, nunca en desarrollo	Ver 15 · R-01

9. Convenciones que respetar

Ámbito	Convención
Idioma	Español en comentarios, docstrings, descripciones de flow y mensajes de error. Identificadores en inglés
Estilo	<code>ruff</code> con <code>E,F,W,I,B,UP</code> ; línea de 120
Tipado	Anotaciones en toda función pública. <code>from __future__ import annotations</code> al inicio
Docstrings	Explican por qué existe el código y qué decisión contiene, no qué hace la línea siguiente
Comentarios	Solo para lo no obvio. <code>[NO] # asigna value · [OK] # value va vacío a propósito: ...</code>
Nombres de flow	<code>NN_nombre_descriptivo</code> , número correlativo
Nombres de acción	<code>familia.verbo_objeto</code>
Fechas	Siempre UTC en ISO-8601
Rutas en manifests	Relativas, con barra normal
Commits	Convencional: <code>feat(scope):</code> , <code>fix(scope):</code> , <code>docs:</code> , <code>ci(deps):</code> — mire <code>git log</code>
Antes de subir	<code>pytest</code> , <code>ruff check</code> , <code>validate_project.py</code> . Los tres en verde

10. Tareas iniciales, ordenadas por dificultad

Todas son hallazgos reales del documento 15, no ejercicios inventados.

Nivel 1 · Primer día (< 1 h cada una)

#	Tarea	Archivo	Aprende
1	Sincronizar <code>requirements.txt</code> con <code>pyproject.toml</code> (faltan 3 paquetes)	<code>requirements.txt</code>	Cómo se declaran las dependencias

2	Añadir las 5 acciones que faltan en <code>[project.entry-points."automa.actions"]</code>	<code>pyproject.toml</code>	El mecanismo de extensión
3	Añadir <code>actions.browser_extract</code> a <code>hiddenimports</code>	<code>installer/automa.spec</code>	Cómo se empaqueta el producto. Corrige el riesgo nº 1
4	Documentar en <code>docs/OPERACION.md</code> que en el cron <code>0</code> es lunes y la hora es UTC	<code>docs/OPERACION.md</code>	El programador de horarios

Nivel 2 · Primera semana (medio día cada una)

#	Tarea	Archivo	Aprende
5	Prueba que verifique que <code>allowed_actions: []</code> produce política permisiva	<code>tests/test_loader.py</code>	El sandbox y un caso límite real
6	Pruebas de <code>_pick_record</code> : no repite, reinicia al agotar, marca antes de llenar	<code>tests/, actions/browser_form.py</code>	Separar lo puro de lo impuro
7	Prueba de <code>find_text_in_image</code> con un <code>matches</code> falso	<code>tests/, actions/vision.py</code>	Cómo probar sin dependencias externas
8	Reemplazar el <code>root_dir()</code> local de <code>catalog.py</code> por el de <code>engine.paths</code>	<code>engine/catalog.py</code>	La resolución de rutas y el modo empaquetado
9	Un flow nuevo que use <code>filesystem.move_file</code> (registrada, sin ningún caso)	<code>flows/28_*/</code>	El ciclo completo de crear un caso

Nivel 3 · Primer mes (uno a tres días cada una)

#	Tarea	Aprende
10	Exigir token también en los GET cuando <code>AUTOMA_PANEL_TOKEN</code> esté definido	El modelo de autorización completo
11	Regla en <code>validate_project.py</code> que exija <code>allowed_actions</code> en todo flow	Cómo se extiende el gate del proyecto
12	Mover el arranque del scheduler de las líneas de módulo a <code>run_server()</code>	Efectos secundarios de importación y su impacto en las pruebas
13	Índices sobre <code>runs.flow_id</code> , <code>steps.run_id</code> y <code>events.run_id</code>	El esquema y sus consultas
14	Unificar los dos motores de condiciones (13 operadores frente a 6)	Duplicación con consecuencias reales
15	Ramas de <code>_smart_summary</code> para los flows web y para el 07	La capa de presentación

Lo que NO conviene hacer como tarea inicial

- Refactorizar `app/server.py`. Es tentador y es el archivo más grande, pero tiene 38 % de cobertura: cualquier cambio es arriesgado sin pruebas previas.

- Cambiar `normalize_text` o el formato de los hashes.
- Reactivar `shell=True` en `launch_process` «para que funcione un comando».
- Bajar el umbral de cobertura para que pase la CI.

11. Itinerario resumido

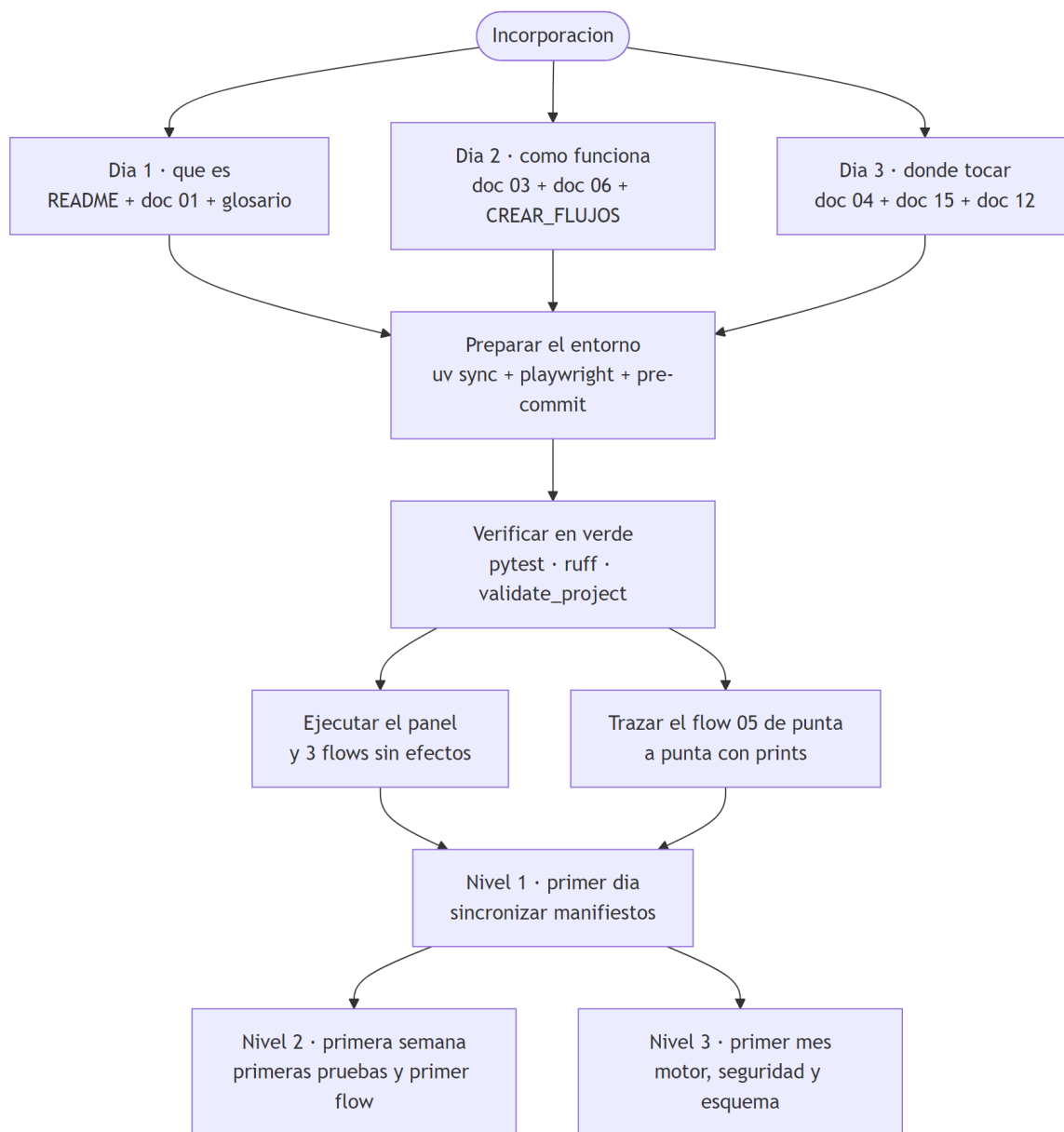


Diagrama 1 (Mermaid, renderizado)

Lo que el itinerario muestra: tres días de lectura orientada antes de tocar nada, y una progresión de tareas que va de los archivos de configuración al motor.

Lo que no muestra: que el paso más rentable es el de trazar el flow 05 con `print()`. Media hora ahí ahorra días de leer código en frío.

12. Dónde preguntar

Duda

Dónde mirar primero

«¿Qué hace esta función?»	05 · Referencia técnica
«¿Por qué está escrita así?»	06 · Explicación profunda y los comentarios del propio código
«¿Dónde se guarda esto?»	07 · Base de datos
«Me falla X»	14 · Solución de problemas
«¿Esto es un bug o es a propósito?»	15 · Riesgos — probablemente ya está registrado
«¿Cómo se prueba esto?»	12 · Pruebas y calidad
«¿Dónde toco para la funcionalidad X?»	19 · Matriz de trazabilidad
Cómo contribuir	CONTRIBUTING.md
Reportar una vulnerabilidad	SECURITY.md

Documentos relacionados: [02 · Instalación](#) · [04 · Mapa del código](#) · [12 · Pruebas y calidad](#) · [15 · Riesgos](#) · [19 · Matriz de trazabilidad](#)

19 · Matriz de trazabilidad

Cada fila permite seguir una funcionalidad **en una sola línea**, desde la interfaz hasta la persistencia y sus pruebas. Los datos proceden de la lectura de los 27 manifests, del registro de acciones y de la ejecución de la suite en el commit [ff246ab](#).

Cómo leer estas tablas

Columna	Significado
Interfaz	Cómo se dispara: tarjeta del panel, ruta HTTP, comando
Manifest	La receta que la implementa
Pasos → acciones	La cadena de acciones, en orden
Módulo · función	El código Python que hace el trabajo
Persistencia	Dónde acaban los datos
Prueba	El archivo de pruebas que la cubre, o el hueco
Doc	Dónde se explica
Estado	[OK] verificado · [!] parcial · [NO] sin cobertura · [manual] requiere validación manual

1. Los 27 casos operativos

1.1 Familia **pantalla** — 6 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-01	Capturar el escritorio y analizar brillo/RGB	Tarjeta · <code>Alt+1 · POST /api/run/01_screen_capture_analyze</code>	<code>01_screen_capture_analyze</code>	<code>capture_screen→screen.capture_screenshot; analyze_capture→vision.analyze_image; write_report→filesystem.write_json</code>	<code>actions/screen.py::capture_screenshot; plugins/analyzers/mock_image_analyzer.py::MockImageAnalyzer; actions/filesystem.py::write_json</code>	<code>output/screenshots/*.png, output/reports/*.json, runs, steps, events</code>	[NO] Sin prueba de la captura real	<code>05 §1.2, 06 §13</code>	[manual]
F-09	Minimizar todo y capturar escritorio limpio	Tarjeta · <code>POST /api/run/09_show_desktop_capture</code>	<code>09_show_desktop_capture</code>	<code>minimize_all_windows→ui.hotkey; wait_animation→system.wait_seconds; capture_desktop→screen.capture_screenshot</code>	<code>actions/ui.py::hotkey; actions/system.py::wait_seconds; actions/screen.py::capture_screenshot</code>	<code>output/screenshots/desktop_clean_*.png + histórico</code>	[!] <code>test_actions_basic.py</code> cubre hotkey en <code>dry_run</code>	<code>05 §1.5</code>	[manual]
F-12	Inventario OCR de todo el texto visible	Tarjeta · <code>POST /api/run/12_desktop_ocr_inventory</code>	<code>12_desktop_ocr_inventory</code>	<code>capture_desktop→screen.capture_screenshot; ocr_full_image→vision.ocr_image; save_inventory→filesystem.write_json</code>	<code>actions/vision.py::ocr_image; plugins/analyzers/ocr_image_analyzer.py::OCRImageAnalyzer</code>	<code>output/screenshots/, output/reports/ + histórico</code>	[NO] <code>actions/vision.py</code> al 0%; <code>plugins/</code> fuera del ámbito de cobertura	<code>09 §4.5, 06 §13</code>	[manual]

F-16	Captura solo de la ventana en foco	Tarjeta · <code>POST /api/run/16_active_window_screenshot</code>	<code>16_active_window_screenshot</code>	<code>capture_active_window→screen.capture_active_window</code>	<code>actions/screen.py::capture_active_window</code>	<code>output/screenshots/*.png</code> + histórico	[NO] Requiere ventana con foco	05 \$1.2	[manual]
F-17	Snapshot del Administrador de tareas con OCR	Tarjeta · <code>POST /api/run/17_taskmgr_snapshot</code>	<code>17_taskmgr_snapshot</code>	<code>ui.hotkey; system.wait_seconds; screen.capture_screenshot; vision.ocr_image; filesystem.write_json</code> (5 pasos)	Los cuatro módulos anteriores	<code>output/screenshots/</code> , <code>output/reports/</code> + histórico	[NO]	04 \$5	[manual]
F-19	Captura de la barra de tareas	Tarjeta · <code>POST /api/run/19_taskbar_capture</code>	<code>19_taskbar_capture</code>	<code>capture_taskbar→screen.capture_region</code> con bbox de top: -48	<code>actions/screen.py::capture_region, resolve_bbox</code>	<code>output/screenshots/taskbar_*.png</code> + histórico	[NO]	05 \$1.2	[manual]

1.2 Familia navegador — 9 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-02	Capturar el DOM renderizado de una URL	Tarjeta · <code>Alt+2</code> (pide URL)	<code>02_screen_capture_browser</code>	<code>browser.capture_page</code>	<code>actions/browser_capture.py::capture_page, _to_url</code>	<code>output/screenshots/*.png</code> + histórico	[!] 18 % : solo <code>_to_url</code>	09 \$4.1	[manual]
F-07	Rellenar un formulario de 10 campos sin repetir registro	Tarjeta · <code>Alt+7</code>	<code>07_browser_form_filler</code>	<code>fill_and_submit_form→browser.fill_form</code>	<code>actions/browser_form.py::fill_form, _pick_record, _save_used_ids</code>	<code>output/reports/form_submissions/*.json</code> ; tracking en data/seeds/used_indices.json	[NO] 0 %	05 \$1.8, 06 \$8, 08 \$10.3	[manual]
F-21	Extraer título, texto, links, metadatos y tablas	Tarjeta · <code>POST /api/run/21_web_content_extract</code>	<code>21_web_content_extract</code>	<code>browser.extract_content; browser.capture_page; filesystem.write_json</code>	<code>actions/browser_extract.py::extract_content, build_result, resolve_links, normalize_text, content_hash</code>	<code>output/reports/*.json</code> , PNG opcional + histórico	[OK] <code>test_browser_extract.py</code> — 91 % de la lógica pura	05 \$1.8, 06 \$7	[!]
F-22	Mapa de sitio con BFS acotado y robots.txt	Tarjeta · <code>POST /api/run/22_web_site_map</code>	<code>22_web_site_map</code>	<code>browser.crawl_site; filesystem.write_json</code>	<code>actions/browser_extract.py::crawl_site, crawl_pages, RobotsCache</code>	<code>output/reports/*.json</code> + histórico	[OK] <code>test_browser_extract.py</code> con <code>fetch_page</code> falso	06 \$7.5–7.7	[!]
F-23	Detectar cambios en una página y notificar	Tarjeta · <code>POST /api/run/23_web_change_detector</code>	<code>23_web_change_detector</code>	<code>extract_and_track→browser.extract_content</code> (retries: 1); <code>evaluate_change→rules.evaluate; notify_change→notify.send_condicionado; write_report→filesystem.write_json</code>	<code>browser_extract.py::apply_tracking, content_hash; actions/rules.py::evaluate_rules; actions/notify.py::send_notification</code>	<code>output/reports/</code> , <code>output/notifications/</code> <code>web_changes.log</code> ; tracking en data/web_watch/demo_page.json	[OK] <code>apply_tracking</code> y <code>content_hash</code> cubiertos; [!] el when del paso 3, en <code>test_orchestrator.py</code>	08 \$10.2, 14 \$5.3	[!]
F-24	Auditar enlaces rotos de una página	Tarjeta · <code>POST /api/run/24_web_link_audit</code>	<code>24_web_link_audit</code>	<code>extract_link→browser.extract_content; check_links→http.check_urls; evaluate_links→rules.evaluate; write_report→filesystem.write_json</code>	<code>browser_extract.py::resolve_links; actions/http_actions.py::check_urls</code>	<code>output/reports/link_audit_*.json</code> + histórico	[OK] <code>resolve_links</code> y <code>check_urls</code> en <code>test_browser_extract.py</code>	05 \$1.6	[!]

F-25	Extraer cada <table> a CSV y JSON	Tarjeta · POST /api/run/25_web_table_extract	25_web_table_extract	browser.extract_content on include_tables; filesystem.write_json	browser_extract.py::save_tables_csv, build_result	output/reports/*.json, table_NN.csv + histórico	[OK] save_tables_csv cubierto	05 \$1.8	[!]
F-26	Vigilar un valor por selector CSS y alertar	Tarjeta · POST /api/run/26_web_value_monitor	26_web_value_monitor	read_value→browser.extract_content(selector#precio); evaluate_value→rules.evaluate; notify_value→notify.send condicionado ; write_report→filesystem.write_json	browser_extract.py::parse_number, apply_tracking	output/reports/, output/notifications/value_monitor.log; tracking en data/web_watch/precio_demo.json	[OK] parse_number con formatos mixtos	06 \$7.2, 14 \$5.5	[!]
F-27	Archivar una página: Markdown + PNG + hash	Tarjeta · POST /api/run/27_web_page_archive	27_web_page_archive	extract_and_save_markdown→browser.extract_content; capture_evidence→browser.capture_page; write_metadata→filesystem.write_json	browser_extract.py::render_markdown, content_hash; browser_capture.py::capture_page	output/*.md, output/screenshots/*.png, output/reports/*.json	[OK] render_markdown cubierto	05 \$1.8	[!]

1.3 Familia sistema — 10 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-05	Snapshot de CPU/RAM/disco con reglas	Tarjeta · POST /api/run/05_system_healthcheck	05_system_healthcheck	take_snapshot→system.snapshot_system; evaluate_snapshot→rules.evaluate; write_snapshot→filesystem.write_json	actions/system.py::snapshot_system; actions/rules.py::evaluate_rules	output/reports/system_health*.json + histórico	[OK] test_actions_basic.py + smoke_test.py	06 \$14	[OK]
F-06	Top 10 procesos con alertas por umbral	Tarjeta · POST /api/run/06_process_watchdog	06_process_watchdog	top_processes→system.top_processes; watch_processes→system.watch_processes; write_report→filesystem.write_json	actions/system.py::top_processes, watch_processes	output/reports/process_watchdog*.json + histórico	[OK] test_actions_basic.py + smoke_test.py	05 \$1.4	[OK]
F-08	Bloquear la estación de trabajo	Tarjeta · POST /api/run/08_windows_lock_workstation	08_windows_lock_workstation	lock_workstation→ui.hotkey ["win", "L"]	actions/ui.py::hotkey	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]
F-10	Abrir el Explorador en una ruta	Tarjeta · POST /api/run/10_explorer_open_path	10_explorer_open_path	open_explorer→ui.launch_process	actions/ui.py::launch_process con shlex y shell=False	Histórico	[!] test_security_hardenimg.py verifica que shell=True se rechaza	11 \$2.2	[manual]
F-11	Abrir una sección de Configuración de Windows	Tarjeta · POST /api/run/11_settings_open_section	11_settings_open_section	open_settings→ui.open_url con URIs ms-settings:	actions/ui.py::open_url + webbrowser	Histórico	[NO]	09 \$6	[manual]
F-13	Nota rápida en el Bloc de notas	Tarjeta · POST /api/run/13_notepad_quick_note	13_notepad_quick_note	ui.launch_process; system.wait_seconds; ui.type_text	actions/ui.py::launch_process, type_text	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]
F-14	Ejecutar un comando desde el diálogo Ejecutar	Tarjeta · POST /api/run/14_run_dialog_command	14_run_dialog_command	ui.hotkey; system.wait_seconds; ui.type_text; ui.hotkey (4 pasos)	actions/ui.py::hotkey, type_text	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]

F-15	Capturar el portapapeles a JSON	Tarjeta · <code>POST /api/run/15_clipboard_capture</code>	<code>15_clipboard_capture</code>	<code>read_clipboard→system.read_clipboard; save_clipboard→filesystem.write_json</code>	<code>actions/system.py::read_clipboard + pyperclip</code>	<code>output/reports/clipboard_*.json + run.s.context_json</code>	[!] La rama de deegración sí, el contenido real no	08 \$10.1, 11 \$6	[manual]
F-18	Auditoría PowerShell con allowlist	Tarjeta · <code>POST /api/run/18_powershell_audit</code>	<code>18_powershell_audit</code>	<code>run_audit_command→system.run_powershell; save_audit_report→filesystem.write_json</code>	<code>actions/system.py::run_powershell, _PSS_DEFAULT_ALLOWLIST</code>	<code>output/reports/*.json + histórico</code>	[OK] <code>test_security_harden.ing.py</code> cubre los dos controles	11 \$5	[!]
F-20	Alternar el silencio del audio	Tarjeta · <code>POST /api/run/20_volume_mute_toggle</code>	<code>20_volume_mute_toggle</code>	<code>toggle_mute→ui.hotkey ["volumemute"]</code>	<code>actions/ui.py::hotkey</code>	Histórico	[!] Solo en <code>dry-run</code>	05 \$1.5	[manual]

1.4 Familias `filesystem` y `documentos` — 2 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-03	Inventario de una carpeta	Tarjeta · <code>Alt+3</code> (pide ruta)	<code>03_folder_inventory</code>	<code>scan_folder→filesystem.listdir; classify_inventory→filesystem.classify_file_inventory; write_inventory→filesystem.write_json</code>	<code>actions/filesystem.py::list_directory, classify_file_inventory, write_json</code>	<code>output/reports/folder_inventory_*.json + histórico</code>	[OK] <code>test_actions_basic.py + smoke_test.py</code>	06 \$14	[OK]
F-04	Pipeline documental de una carpeta de entrada	Tarjeta · <code>POST /api/run/04_document_drop_pipeline</code>	<code>04_document_drop_pipeline</code>	<code>scan_dropbox; classify_dropbox; summarize_texts→filesystem.summarize_text_folder; write_pipeline_report</code>	<code>actions/filesystem.py::summarize_text_folder</code>	<code>output/reports/document_drop_pipeline_*.json + histórico</code>	[OK] <code>test_actions_basic.py</code>	08 \$4	[OK]

2. Capacidades del motor

#	Capacidad	Interfaz	Módulo · función	Persistencia	Prueba	Doc	Estado
M-01	Cargar un manifest y convertirlo en objetos	Interna	<code>engine/loader.py::FlowLoader.load_manifest→engine/models.py</code>	—	[OK] <code>test_loader.py</code> — 100 %	06 \$5.1	[OK]
M-02	Resolver el contexto por precedencia	Interna	<code>engine/loader.py::load_context</code>	Lee <code>configs/, context.user.json, context.example.json</code>	[OK] <code>test_loader.py</code>	10 \$3.1	[OK]
M-03	Sustituir placeholder	Interna	<code>engine/template.py::render_value, flatten_context</code>	<code>steps.params_json</code>	[OK] <code>test_template.py</code> — 100 %	06 \$2	[OK]
M-04	Evaluar condiciones (13 operadores)	<code>when</code> de pasos y transiciones	<code>engine/conditions.py::evaluate_condition, matches</code>	<code>steps.status = skipped</code>	[OK] <code>test_conditions.py</code> — 90 %	06 \$4	[OK]
M-05	Ejecutar el bucle con transiciones	Interna	<code>engine/orchestrator.py::run, resolve_transition, default_next</code>	<code>runs, steps, events, state/, logs/</code>	[OK] <code>test_orchestrator.py</code> — 87 %	06 \$1	[OK]
M-06	Reintentar y recuperarse por transición	<code>retries</code> y <code>on: "failure"</code>	<code>engine/orchestrator.py::run</code>	<code>steps.attempt, evento step_recovered</code>	[OK] <code>test_orchestrator.py</code>	06 \$1.2	[OK]

M-07	Aplicar la política de sandbox	<code>allowed_actions, allowed_paths, required_secrets, max_runtime_seconds</code>	<code>engine/sandbox.py::SandboxPolicy</code>	<code>runs.context_json → policy, error_json.kind</code>	[OK] <code>test_sandbox.py</code> — 95 %	11 \$4	[OK]
M-08	Resolver acciones de forma perezosa	Interna	<code>engine/action_registry.py::LazyActionRegistry.get</code>	—	[OK] <code>test_action_registry.py</code> — 84 %	03 \$4	[OK]
M-09	Cargar acciones de terceros por entry point	<code>pyproject.toml</code> del tercero	<code>LazyActionRegistry._maybe_load_entry_points</code>	—	[!] Parcial	09 \$7	[!]
M-10	Detectar los archivos de salida	Interna	<code>engine/introspection.py::extract_existing_paths</code>	<code>runs.outputs_json</code>	[OK] 90 %	06 \$12	[OK]
M-11	Persistir estado, pasos y eventos	Interna	<code>engine/database.py, logger.py, state_store.py</code>	Las 7 tablas + <code>logs/</code> + <code>state/</code>	[OK] <code>test_orchestrator.py, test_metrics.py</code>	07	[OK]
M-12	Separar raíz de lectura y de escritura	<code>AUTOMA_ROOT, AUTOMA_DATA_ROOT</code>	<code>engine/paths.py::root_dir, data_dir</code>	Determina dónde vive todo	[OK] 3 pruebas en <code>test_actions_basic.py</code> — 86 %	10 \$5	[OK]
M-13	Resolver secretos (entorno, después archivo)	<code>secrets/secrets.json</code> , entorno	<code>engine/secrets.py::get_secret, set_secret, list_secret_names</code>	<code>secrets/secrets.json</code> sin cifrar	[OK] <code>test_secrets_and_notify.py</code> — 73 %	11 \$6	[OK]
M-14	Calcular la próxima ejecución de un cron	Panel · pestaña Programadas	<code>engine/cron.py::next_after, parse_cron</code>	<code>schedules.next_run_at</code>	[OK] <code>test_cron.py</code> — 87 %	06 \$9	[!]
M-15	Disparar tareas programadas	Automático cada 2 s	<code>engine/scheduler.py::SchedulerService</code>	<code>schedules.last_run_at, run_locks</code>	[!] 51–76 %, variable	06 \$10	[!]
M-16	Evitar ejecución doble por lock	Automático · solo scheduler	<code>engine/database.py::acquire_run_lock, release_run_lock</code>	<code>run_locks</code>	[OK] <code>test_run_locks.py</code>	15 · R-07	[!]
M-17	Validar manifests contra el JSON Schema	<code>automa-validate</code> , CI	<code>engine/manifest_schema.py; scripts/validate_project.py</code>	—	[OK] <code>test_manifest_schema.py</code> — 53 %	06 \$14	[OK]
M-18	Agregar métricas	<code>GET /metrics/</code> , <code>api/metrics</code>	<code>engine/metrics.py::overview, by_flow, prometheus_text</code>	Consulta <code>runs</code> y <code>steps</code>	[OK] <code>test_metrics.py</code> — 100 %	09 \$3	[OK]
M-19	Listar y ejecutar desde el CLI	<code>automa list</code> , <code>automa run</code> , <code>automa scheduler</code>	<code>engine/runner.py::main, build_parser</code>	Igual que el panel	[NO] 0 % — y ahí vive el defecto R-06	15 · R-06	[NO]

3. Capacidades del panel

#	Capacidad	Ruta	Módulo · función	Persistencia	Prueba	Doc	Estado
P-01	Panel de 3 pestañas	<code>GET /</code>	<code>app/server.py::render_home, _flow_card_run_tab, _flow_card_schedule_tab, _history_row</code>	Lee <code>flows</code> y <code>runs</code>	[!] <code>test_panel_endpoints.py</code>	09 \$1.1	[!]
P-02	Ejecutar un flow de forma asíncrona	<code>POST /api/run/<folder></code>	<code>app/server.py::do_POST + threading.Thread</code>	<code>runs, steps, events</code>	[!]	09 \$2	[!]
P-03	Ejecutar de forma síncrona desde formulario	<code>POST /run?flow=<folder></code>	<code>app/server.py::do_POST</code>	Igual	[!]	09 \$1.2	[!]
P-04	Polling del estado paso a paso	<code>GET /api/runs/<run_id>/status</code>	<code>app/server.py::run_status_payload</code>	Lee <code>runs, steps, manifest</code>	[!]	09 \$2	[!]
P-05	Detalle de una corrida	<code>GET /run/<flow_id>/<run_id></code>	<code>render_run_detail, _smart_summary, _output_thumb</code>	Lee <code>runs, steps, events</code>	[NO]	08 \$6	[NO]

P-06	Resumen legible por flow	Dentro de P-05	app/server.py: :_smart_summary	—	[NO]	15 · R-15	[NO] Solo 6 de 27 flows
P-07	Editar el contexto de un flow	GET/POST /flow/<flow_id>/config	render_flow_config; database.set_flow_config	flow_configs + configfigs/<flow_id>.json	[!]	10 \$3	[!]
P-08	Programar un flow	POST /flow/<flow_id>/schedule	database.set_schedule; engine/cron.py	schedules	[OK] Via test_cron.py y test_metrics.py	09 \$2	[!]
P-09	Histórico por flow	GET /flow/<flow_id>/history	render_flow_history	Lee runs	[NO]	09 \$1.1	[NO]
P-10	Dashboard de métricas	GET /metrics/dashboard	render_metrics_dashboard	Lee agregados	[NO]	09 \$3	[NO]
P-11	Servir archivos de salida	GET /file?path=...	app/server.py: :do_GET, 4 controles	Lee output/	[OK] test_security_hardening.py	11 \$2.3	[OK]
P-12	Webhook entrante	POST /api/hook/<folder>	_check_webhook_token; ejecución sincrona	runs, steps, events	[!]	09 \$2	[!]
P-13	Guardar un envío de formulario	POST /api/form/submit	app/server.py: :do_POST	output/reports/form_submission_panel_*.json	[NO]	09 \$1.2	[NO]
P-14	Autorizar mutaciones	Todos los POST salvo el webhook	_authorize_mutation, _check_token con hmac.compare_digest	—	[OK] test_panel_endpoints.py, test_security_hardenig.py	11 \$3.2	[OK]
P-15	Validar el slug de la URL	Las 7 rutas con <folder>	_safe_folder, _FOLDER_RE	—	[OK] test_security_hardening.py	11 \$2.2	[OK]
P-16	Marcar un flow como preview	Manifest o archivo .disabled	app/server.py: :_is_preview	Responde 409	[NO]	10 \$5	[NO]
P-17	Ventana nativa de escritorio	automa-desktop	app/desktop.py: :launch, _wait_for_server	—	[!] 48 %	02 \$6	[!]
P-18	Sonda de vida	GET /healthz	app/server.py: :do_GET	—	[OK] test_panel_endpoints.py	09 \$1.1	[OK]

4. Reglas de negocio verificables

#	Regla	Dónde se implementa	Cómo comprobarla	Prueba	Estado
RN-01	Un paso cuyo when no se cumple se registra como skipped con attempt: 0, no se omite	engine/orchestrator.py	Verifica en steps con status='skipped'	[OK] test_orchestrator.py	[OK]
RN-02	Un flow se recupera de un fallo solo si la transición on: "failure" apunta a un paso distinto del siguiente por defecto	engine/orchestrator.py	Evento step_recovered en events	[OK] test_orchestrator.py	[OK]
RN-03	La primera corrida de un flow con tracking nunca reporta cambio	browser_extract.py	first_run: true en el resultado	[OK] test_browser_extract.py	[OK]
RN-04	El flow 07 no repite registro hasta agotar los 100; entonces reinicia y lo declara	browser_form.py	piccycle_resetted en el resultado	[NO]	[NO]

RN-05	<code>rules.evaluate</code> se detiene en la primera regla que coincide	<code>actions/rules.py::evaluate</code>	<code>matched_rule</code> y <code>evaluations</code>	[OK] <code>test_actions_basic.py</code>	[OK]
RN-06	El crawl respeta <code>robots.txt</code> , y si no lo puede leer permite y lo declara	<code>browser_extract.py::crawl</code>	<code>robots_checked_hosts</code> y <code>robots_blocked</code>	[OK] <code>test_browser_extract.py</code>	[OK]
RN-07	Un enlace caído durante el crawl no aborta el recorrido	<code>browser_extract.py::crawl</code>	<code>crawl_errors</code> en el reporte	[OK] <code>test_browser_extract.py</code>	[OK]
RN-08	<code>system.run_powershell</code> rechaza encadenamiento y verbos fuera de la <code>allowlist</code>	<code>actions/system.py::run_powershell</code>	<code>ValueError</code> con el motivo	[OK] <code>test_security_hardening.py</code>	[OK]
RN-09	<code>ui.launch_process</code> prohíbe <code>shell=True</code>	<code>actions/ui.py::launch_process</code>	<code>ValueError</code> explicativo	[OK] <code>test_security_hardening.py</code>	[OK]
RN-10	El sandbox bloquea la corrida entera, no solo el paso	<code>engine/orchestrator.py::run</code>	<code>runs.status='failed'</code> , <code>error_json.kind='sanctest_sandbox.py'</code>	[OK] <code>test_orchestrator.py</code>	[OK]
RN-11	Toda cota aplicada se declara en el resultado	<code>check_urls</code> , <code>extract_content</code> , <code>read_clipboard</code> , <code>run_powershell</code> , <code>crawl_pages</code>	Campos <code>*_truncated</code>	[OK] <code>test_browser_extract.py</code>	[!] Dos excepciones: <code>summarize_text_folder</code> y <code>apply_tracking</code> (ver 15 · R-24)
RN-12	Un flow sin <code>allowed_actions</code> corre con política permisiva, y el histórico lo registra	<code>engine/sandbox.py::is_summary</code>	<code>runs.context_json</code> → <code>policy.permissive</code>	[OK] <code>test_sandbox.py</code>	[OK]
RN-13	El motor corta a <code>max_steps_per_run</code> (200) para evitar ciclos infinitos	<code>engine/orchestrator.py::run</code>	<code>FlowExecutionError</code> con el mensaje	[!]	[!]
RN-14	Una acción de terceros nunca puede sobrescribir una interna	<code>LazyActionRegistry::setdefault</code>	Registrar un <code>entry_point</code> con <code>setdefault</code> point homónimo	[NO]	[NO]
RN-15	Un placeholder exacto devuelve el valor con su tipo original	<code>engine/template.py::render</code>	<code>"{{ headless }}"</code> llega como <code>False</code> , no <code>"False"</code>	[OK] <code>test_template.py</code>	[OK]
RN-16	Solo se consideran salidas los archivos bajo <code>output/</code>	<code>engine/introspection.py::run</code>	<code>outputs_json</code>	[OK]	[OK]

5. De la persistencia hacia atrás

Camino inverso: dado un archivo o una tabla, quién lo produce.

Almacén	Escribe	Desde el flow	Se lee en
---------	---------	---------------	-----------

runs	engine/database.py::upsert_run ← Orchestrator._persist	Todos	catalog.list_runs, metrics.*, panel
steps	insert_step ← Orchestrator.run	Todos	_run_status_payload, render_run_detail, metrics
events	insert_event ← JsonLogger.write	Todos	render_run_detail
flows	sync_flows ← import de app.server, runner	—	Panel
flow_configs	set_flow_config ← POST /flow/<f>/config	—	get_flow_by_folder
schedules	set_schedule, mark_schedule_run	—	scheduler.run_pending_once
run_locks	acquire_run_lock ← solo scheduler._run_job	—	RUNBOOK.md
logs/*.jsonl	JsonLogger.write	Todos	Manual
state/*.json	StateStore.save ← _persist	Todos	Manual
output/screenshots/*.png	screen.capture_*, browser.capture_page	01, 02, 09, 12, 16, 17, 19, 21, 27	GET /file
output/reports/*.json	filesystem.write_json	16 de 27 flows	GET /file, panel
output/reports/*.csv	browser_extract.save_tables_csv	25	GET /file
output/*.md	browser_extract.render_markdown	27	GET /file
output/notifications/*.log	notify.send con backend: "file"	23, 26	Manual
data/seeds/.used_indices.json	browser_form._save_used_ids	07	El propio flow, corrida siguiente
data/web_watch/*.json	browser_extract.apply_tracking	23, 26	El propio flow, corrida siguiente
configs/<folder>.json	set_flow_config	—	FlowLoader.load_context
secrets/secrets.json	engine.secrets.set_secret	—	get_secret

6. Resumen de cobertura de trazabilidad

Categoría		Total	[OK] Verificado	[!] Parcial	[NO] Sin cobertura	[manual] Requiere validación manual
Casos operativos (F)		27	4	8	0	15
Capacidades del motor (M)		19	13	5	1	0
Capacidades del panel (P)		18	4	8	6	0
Reglas de negocio (RN)		16	11	3	2	0

Total	80	32	24	9	15
-------	----	----	----	---	----

Lectura del cuadro:

- **El motor está bien cubierto:** 13 de 19 capacidades verificadas por pruebas automáticas. Es el activo mejor protegido del proyecto.
- **Los 15 casos marcados [manual] no son un defecto de las pruebas:** dependen de un escritorio Windows interactivo, de Chromium o del binario `tesseract`. La CI no puede ejecutarlos. La mitigación existente —`dry_run` en las acciones de UI— es la correcta.
- **Los 9 sin cobertura son huecos reales y abordables:** `engine/runner.py` (donde vive el defecto R-06), seis capacidades de renderizado del panel, y las reglas RN-04 y RN-14. La propuesta priorizada está en [12 §10](#).

Documentos relacionados: [04](#) · [Mapa del código](#) · [05](#) · [Referencia técnica](#) · [07](#) · [Base de datos](#) · [12](#) · [Pruebas y calidad](#) · [15](#) · [Riesgos y deuda técnica](#)